



DesignWare Cores Mobile Storage Host Databook

DWC_mobile_storage

Copyright Notice and Proprietary Information

Copyright © 2010 Synopsys, Inc. All rights reserved. This software and documentation contain confidential and proprietary information that is the property of Synopsys, Inc. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Synopsys, Inc., or as expressly provided by the license agreement.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Registered Trademarks (®)

Synopsys, AMPS, Astro, Behavior Extracting Synthesis Technology, Cadabra, CATS, CRITIC, Certify, CHIPit, Design Compiler, DesignWare, Formality, HDL Analyst, HSIM, HSPICE, Identify, Leda, MAST, ModelTools, NanoSim, OpenVera, PathMill, Physical Compiler, PrimeTime, SCOPE, Simply Better Results, SiVL, SNUG, SolvNet, Syndicated, Synplicity, the Synplicity Logo, Synplify, Synplify Pro, Synthesis Constraints Optimization Environment, TetraMAX, UMRBus, VCS, Vera, and YIELDirector are registered trademarks of Synopsys, Inc.

Trademarks (™)

AFGen, Apollo, Astro, Astro-Rail, Astro-Xtalk, Aurora, AvanWaves, BEST, Columbia, Columbia-CE, Confirma, Cosmos, CosmosLE, CosmosScope, CRITIC, CustomExplorer, CustomSim, DC Expert, DC Professional, DC Ultra, Design Analyzer, Design Vision, DesignerHDL, DesignPower, DFTMAX, Direct Silicon Access, Discovery, Eclipse, Encore, EPIC, Galaxy, Galaxy Custom Designer, HANEX, HAPS, HapsTrak, HDL Compiler, Hercules, Hierarchical Optimization Technology, High-performance ASIC Prototyping Ssystem, HSIMplus, i-Virtual Stepper, IICE, in-Sync, iN-Tandem, Jupiter, Jupiter-DP, JupiterXT, JupiterXT-ASIC, Liberty, Libra-Passport, Library Compiler, Magellan, Mars, Mars-Rail, Mars-Xtalk, Milkyway, ModelSource, Module Compiler, MultiPoint, Physical Analyst, Planet, Planet-PL, Polaris, Power Compiler, Raphael, Saturn, Scirocco, Scirocco-i, Star-RCXT, Star-SimXT, StarRC, Taurus, TotalRecall, TSUPREM-4, VCS Express, VCSi, VHDL Compiler, VirSim, and VMC are trademarks of Synopsys, Inc.

Service Marks (SM)

MAP-in, SVP Café, and TAP-in are service marks of Synopsys, Inc.

SystemC is a trademark of the Open SystemC Initiative and is used under license.

ARM and AMBA are registered trademarks of ARM Limited.

Saber is a registered trademark of SabreMark Limited Partnership and is used under license.

PCI Express is a trademark of PCI-SIG.

All other product or company names may be trademarks of their respective owners.

Synopsys, Inc.
700 E. Middlefield Road
Mountain View, CA 94043
www.synopsys.com

Contents

Preface	9
Databook Organization	9
Web Resources	9
Reference Documentation	10
Document Revision History	10
Customer Support	10
Chapter 1	
Product Overview	13
1.1 General Product Description	13
1.1.1 DWC_mobile_storage Block Diagram	19
1.2 Features	20
1.2.1 Internal DMA Block Features	20
1.2.2 AHB Master Interface Features	20
1.2.3 Bus Interface Features	21
1.2.4 Card Interface Features	21
1.2.5 Verification Environment Features	22
1.2.6 Example Linux Demonstration Software Package Features	22
1.2.7 UHS-1 and Voltage-Switching Features	22
1.3 Standards Compliance	22
1.4 Verification Environment Overview	23
Chapter 2	
Building and Verifying Your Core	25
2.1 Setting up Your Environment	25
2.2 Starting coreConsultant	26
2.3 Checking Your Environment	27
2.4 Configuring the DWC_mobile_storage	27
2.5 Synthesizing the DWC_mobile_storage	30
2.5.1 Checking Synthesis Status and Results	33
2.5.2 Synthesis Output Files	33
2.5.3 Running Synthesis from Command Line	33
2.6 Verifying the DWC_mobile_storage	33
2.6.1 DWC_mobile_storage Testbench	34
2.6.2 Verifying the Simulation Model	35
Chapter 3	
Functional Description	43
3.1 Bus Interface Unit (BIU)	45

3.2 Internal Direct Memory Access Controller (IDMAC)	69
3.2.1 Architecture	70
3.2.2 IDMAC CSR Access	72
3.2.3 Descriptors	73
3.2.4 Initialization	77
3.2.5 FBE Scenarios	82
3.3 Card Interface Unit (CIU)	83
3.3.1 Command Path	85
3.3.2 Data Path	91
3.3.3 Non-Data Transfer Commands that Use Data Path	97
3.3.4 SDIO Interrupt Control	98
3.3.5 Clock Control	98
3.3.6 SD_MMC_CEATA Mux/Demux Unit	101
3.3.7 Error Detection	103
3.4 Clock Domains	104
3.5 Voltage Switching and UHS-1 Card Support	108
3.6 Shared Bus Mode	110
3.7 Dedicated Interrupt Pins	113
3.8 Asynchronous Interrupts	113
3.9 Back-End Power	114
3.10 Master Power Control	114
Chapter 4	
Parameters	115
4.1 Parameter Descriptions	115
Chapter 5	
Signals	119
5.1 DWC_mobile_storage Interface Diagram	119
5.2 Signal Descriptions	121
Chapter 6	
Registers	133
6.1 Register Address Map	134
6.2 Register and Field Descriptions	138
6.2.1 CTRL	138
6.2.2 PWREN	141
6.2.3 CLKDIV	142
6.2.4 CLKSRC	143
6.2.5 CLKENA	143
6.2.6 TMOUT	144
6.2.7 CTYPE	145
6.2.8 BLKSIZ	146
6.2.9 BYTCNT	146
6.2.10 INTMASK	147
6.2.11 CMDARG	148
6.2.12 CMD	149
6.2.13 RESP0	153
6.2.14 RESP1	153
6.2.15 RESP2	154

6.2.16	RESP3	154
6.2.17	MINTSTS	155
6.2.18	RINTSTS	156
6.2.19	STATUS	157
6.2.20	FIFOTH	159
6.2.21	CDETECT	161
6.2.22	WRTprt	162
6.2.23	GPIO	162
6.2.24	TCBCNT	163
6.2.25	TBBCNT	164
6.2.26	DEBNCE	164
6.2.27	USRID	165
6.2.28	VERID	165
6.2.29	HCON	166
6.2.30	UHS_REG	167
6.2.31	RST_n	168
6.2.32	BMOD	169
6.2.33	PLDMND	170
6.2.34	DBADDR	170
6.2.35	IDSTS	171
6.2.36	IDINTEN	173
6.2.37	DSCADDR	174
6.2.38	BUFADDR	174
6.2.39	CardThrCtl	175
6.2.40	Back_end_power	175

Chapter 7

Programming the DWC_mobile_storage	177
7.1 Software/Hardware Restrictions	177
7.2 Programming Sequence	179
7.2.1 Initialization	179
7.2.2 Power Control	183
7.2.3 Clock Programming	183
7.2.4 No-Data Command With or Without Response Sequence	184
7.2.5 Data Transfer Commands	185
7.2.6 Single-Block or Multiple-Block Write	187
7.2.7 Stream Read	189
7.2.8 Stream Write	189
7.2.9 Sending Stop or Abort in Middle of Transfer	189
7.2.10 Suspend or Resume Sequence	190
7.2.11 Read_Wait Sequence	192
7.2.12 CE-ATA Data Transfer Commands	192
7.2.13 Controller/DMA/FIFO Reset Usage	199
7.2.14 Card Read Threshold	199
7.2.15 Shared bus mode	205
7.2.16 Dedicated Interrupts	205
7.2.17 Asynchronous Interrupts	206
7.2.18 Back-End Power	206
7.2.19 Master Power Control	206
7.2.20 Error Handling	209

7.2.21	Transmission and Reception with Internal DMAC (IDMAC)	210
7.3	Programming DWC_mobile_storage for Boot Operation	211
7.3.1	Boot Operation	211
7.3.2	Alternative Boot Operation	216
7.4	Programming DWC_mobile_storage for Voltage Switching and DDR Operations	221
7.4.1	Voltage Switch Operation	221
7.4.2	DDR Operation	227
7.4.3	8-bit DDR Transfer	230
7.4.4	H/W Reset Operation	232

Chapter 8

Verification	235
8.1 Overview of Vera Tests	235
8.2 Verification Environment Overview	237
8.3 Testbench Structure	237
8.3.1 System-Test Architecture	238
8.4 Testharness	240
8.4.1 disableModelMsg	241
8.4.2 driveReset	241
8.4.3 enableModelMsg	242
8.4.4 initialize	242
8.4.5 setTopConfig	243
8.5 Simulation Scripts	243
8.5.1 run.scr	244
8.6 Simulation Output Files	245
8.7 Waveform Output	245
8.8 Executing Individual Tests or Batch Jobs	246
8.9 Constrained Random Testing	246
8.9.1 Random Command Data Generator	246
8.9.2 Constraint Selection for Random Simulation Runs	246
8.9.3 ConnectNewCard	247
8.9.4 setRcgMsgLevel	247
8.9.5 setBaseAddress	248
8.9.6 issueXactions	248
8.9.7 setAttrRange	248
8.9.8 setAttrWeight	249
8.9.9 Testcases Using Random Generator	249
8.10 Generic DMA BFM	261
8.10.1 checkDmaDone	261
8.10.2 ChkGeDMAErrCnt	261
8.10.3 GeDmaRead	262
8.10.4 geDmaWrite	262
8.10.5 New	263
8.10.6 setConfigParam	264
8.10.7 Resetting the generic DMA BFM	264
8.10.8 Data Buffer	264

Chapter 9

DWC_mobile_storage Design Integration Requirements	265
9.1 Instantiating DWC_mobile_storage	265

9.2 Bi-directional Buses	268
9.3 System Reset	269
9.4 Clock Requirements and Recommendations	270
9.4.1 Clock Requirements	270
9.4.2 Clock Generation Recommendations	270
9.5 Card Clock Input Structure	273
9.5.1 Clock Selection	274
9.5.2 Variable Delay Implementation	274
9.6 Host Controller Output Path Timing	275
9.6.1 Recommended Usage	276
9.7 FIFO Size Requirements	277
9.8 System Clock Topology	278
9.9 Card-Detect and Write-Protect Mechanism	278
9.10 Termination Requirement	279
9.10.1 Rcmd and Rod Calculation	279
9.11 Interfacing to SD Memory, SDIO, and MMC Card	280
9.12 Host Controller Clock Structure and Card Timing Requirements	284
9.12.1 Timing Requirements for SD Cards	284
9.12.2 Guidelines for Using Mobile Storage Host Controller in SDR104 Mode	285
9.12.3 Clock Requirements and Recommendations	286
9.12.4 Testing Results	288
Appendix A	
Timing Requirements for SD 3.0 Cards	297
A.1	Timing Requirements for SD 3.0 Cards 297
A.2 Guidelines for Using Mobile Storage Host Controller in SDR104 Mode	298
A.3 Testing Results	299
Appendix B	
Atomic VIP Command Verilog Testbench	309
B.1 AVTB Overview	309
B.2 Reset, Start, and Initialization	311
B.3 DWC Mobile Storage Data Transfer Tests	312
B.4 Using VTB	314
B.4.1 Directory Structure and Files	315
B.4.2 Tool and Setup Requirements	316
B.5 Running VCS Simulations	316
B.6 Working with Testcases	317
B.6.1 Configuring the Testbench	317
B.6.2 Running a Testcase	317
B.6.3 Changing Selected Card Types	318
B.6.4 Debugging a Testcase	318
Appendix C	
Database Description	319
C.1 Design/HDL Files	319
C.1.1 RTL-Level Files	319
C.1.2 Simulation Model Files	320
C.2 Register Map Files	320
C.3 Synthesis Files	320

C.4 Verification Reference Files321

Appendix D

Area, Speed, Power, and Quality Matrix323

Index329

Preface

Databook Organization

The chapters of this databook are organized as follows:

- ❖ Chapter 1, “[Product Overview](#)” provides a component block diagram, basic features, and an overview of the verification environment.
- ❖ Chapter 2, “[Building and Verifying Your Core](#)” provides getting started information that allows you to walk through the process of using the DWC_mobile_storage.
- ❖ Chapter 3, “[Functional Description](#)” describes the functional operation of the DWC_mobile_storage.
- ❖ Chapter 4, “[Parameters](#)” identifies the configurable parameters supported by the DWC_mobile_storage.
- ❖ Chapter 5, “[Signals](#)” provides a list and description of the DWC_mobile_storage signals.
- ❖ Chapter 6, “[Registers](#)” describes the programmable registers of the DWC_mobile_storage.
- ❖ Chapter 7, “[Programming the DWC_mobile_storage](#)” provides information needed to program the configured DWC_mobile_storage.
- ❖ Chapter 8, “[Verification](#)” provides information on verifying the configured DWC_mobile_storage.
- ❖ Chapter 9, “[DWC_mobile_storage Design Integration Requirements](#)” includes information you need to integrate the configured component into your design.
- ❖ Appendix B, “[Atomic VIP Command Verilog Testbench](#)” describes the Atomic VIP Command Verilog Testbench (AVTB) files and how to create test scenarios.
- ❖ Appendix C, “[Database Description](#)” briefly describes the structure of a coreConsultant workspace.
- ❖ Appendix D, “[Area, Speed, Power, and Quality Matrix](#)” provides synthesis results based on an industry-standard 65nm library.
- ❖ Appendix E, “[Vera Functional Coverage Report](#)” provides a Vera functional coverage summary.

Web Resources

- ❖ DesignWare IP product information: <http://www.designware.com>.
- ❖ Your custom DesignWare IP page: <http://www.mydesignware.com>.
- ❖ Documentation through SolvNet: <http://solvnet.com> (Synopsys password required).
- ❖ Synopsys Common Licensing (SCL): <http://www.synopsys.com/keys>

Reference Documentation

The following non-Synopsys documents provide additional information and are prerequisite to understanding the DWC_mobile_storage controller.

- ❖ SD Memory Card Specification, Version 3.0
- ❖ Secure Digital I/O (SDIO - version 2.0)
- ❖ Consumer Electronics Advanced Transport Architecture (CE-ATA – version 1.1)
- ❖ Multimedia Cards (MMC - version 4.41)

The SD and MMC specifications can be purchased from the appropriate organizations.

- ❖ <http://www.sdcard.org>
- ❖ <http://www.mmca.org>

You can download the *CE-ATA Digital Protocol Revision 1.0* from the following web site:

<http://www.ce-ata.org/specifications.asp>

Document Revision History

This section tracks the significant documentation changes that occur from release-to-release and during a release from version 2.11a onward.

Table 3-1 Databook Revision History

Version	Databook Date	Description
2.40a	November 2010	Added information for Card Read Threshold programming, Card Clock Input structure, Host Controller Output timing, back-end power.
2.30a	September 28, 2010	Added text to show that DWC Mobile Storage Host controller supports MMC4.41. No changes to the product were required.
2.30a	June 2010	Added support for MMC4.4/eMMC cards; added appendix for SD 3.0 timing requirements; edited hold time and timing requirements; edited list of tests.
2.20a	December 2009	Added functionalities for Voltage Switching and UHS-1 card support
2.11a	March 2009	Added information for new boot and alternative boot operations

Customer Support

To obtain support for your product, choose one of the following:

- ❖ Enter a call through SolvNet.
 - ◆ Go to <http://solvnet.synopsys.com/EnterACall> and provide the requested information, including:
 - ❖ Product: DesignWare Cores
 - ❖ Sub Product: DWC_mobile_storage
 - ❖ Tool Version: <product version number>
- ❖ Send an e-mail message to support_center@synopsys.com.
 - ◆ Include the Product name, Sub Product name, and Tool Version in your e-mail so it can be routed correctly.

- ❖ Telephone your local support center.
 - ◆ North America:
Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific time, Monday through Friday.
 - ◆ All other countries:
<http://www.synopsys.com/Support/GlobalSupportCenters>

1

Product Overview

The DWC Mobile Storage Host is a Secure Digital (SD), Multimedia Card (MMC), and CE-ATA host controller that you can configure and synthesize in order to control:

- ❖ Secure Digital memory (SD mem - version 3.0)
- ❖ Secure Digital I/O (SDIO - version 3.0)
- ❖ Consumer Electronics Advanced Transport Architecture (CE-ATA - version 1.1)
- ❖ Multimedia Cards (MMC - version 4.41)

Note that the MMC 4.4 standard has been superseded by MMC 4.41 (JESD84-A441, March 2010). The DWC Mobile Storage Host supports the changes made from MMC 4.4 to MMC 4.41.

**Note**

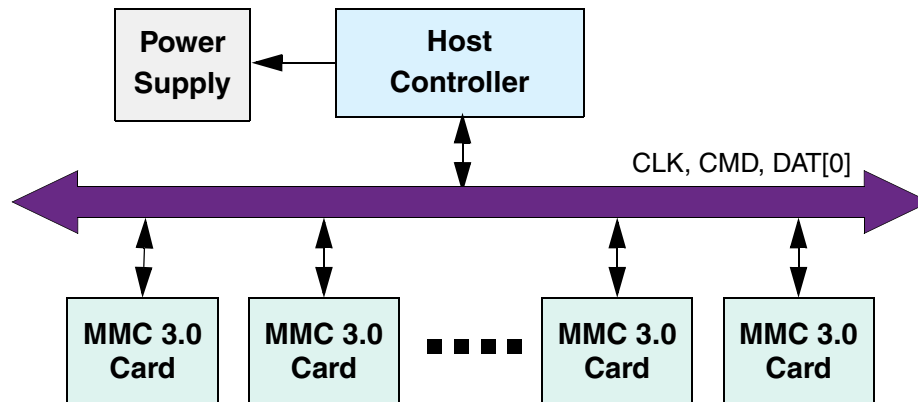
The indicated versions are only card versions. It should not be mistaken that DWC_mobile_storage host implementation is compliant to any available Standard Host specifications.

1.1 General Product Description

The DWC_mobile_storage can be configured either as a Multimedia Card-only controller or as a Secure Digital_Multimedia Card controller, which simultaneously supports Secure Digital memory (SD Mem), Secure Digital I/O (SDIO), Multimedia Cards (MMC), and Consumer Electronics Advanced Transport Architecture (CE-ATA).

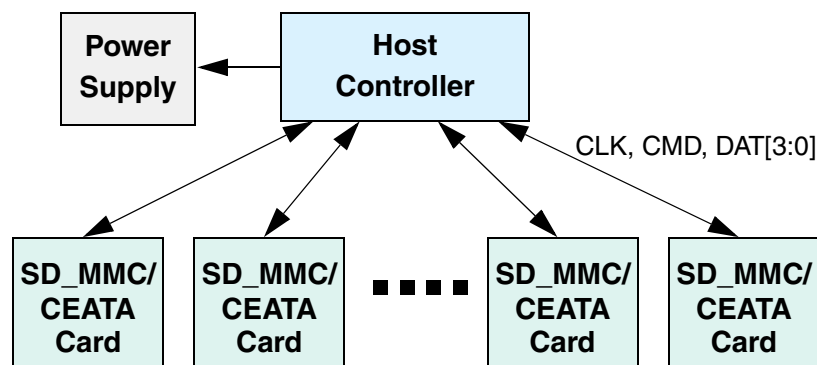
One main difference between MMC-Ver3.3-only mode and SD_MMC_CE-ATA mode is the bus topology. In MMC-Ver3.3-only mode, the MMC cards are connected in a single shared-bus topology, illustrated in Figure 1-1.

Figure 1-1 Multimedia Ver3.3 Card System – Bus Topology



In SD_MMC_CE-ATA mode, the memory cards are connected in a star topology, illustrated in Figure 1-2.

Figure 1-2 SD_MMC and CE-ATA Card System Star Topology



In MMC-Ver3.3-only mode, the DWC_mobile_storage supports a 1-bit data bus width.

In SD_MMC_CE-ATA mode, the DWC_mobile_storage supports 1-bit, 4-bit, and 8-bit data bus widths, depending on the card types (SD/SDIO, HSMMC, CE-ATA).

The DWC_mobile_storage primarily targets host-controller and card-reader applications and comes with the following deliverables:

- ❖ DWC_mobile_storage Verilog RTL source code
- ❖ coreConsultant tool for configuration, simulation, and synthesis
- ❖ Verification environment
- ❖ Example Linux demonstration software package
- ❖ Verilog testbench

An SD_MMC memory card is typically a device for FLASH mass storage. The SDIO card usually functions in I/O applications, which can also have optional FLASH memory. The CE-ATA card functions in mobile device applications. The SD_MMC_CEATA bus mode includes the following signals:

- ❖ CLK – Host-to-card clock signal
- ❖ CMD – Bidirectional command and response signal
- ❖ DATA – Bidirectional data signal (1-bit, 4-bit, or 8-bit MMC Cards; 1-bit or 4-bit in SD cards)
- ❖ VDD, VSS1, VSS2 – Power and ground

Figure 1-3 illustrates the block diagram of the SD cards and signals.

Figure 1-3 Typical SD Memory Card

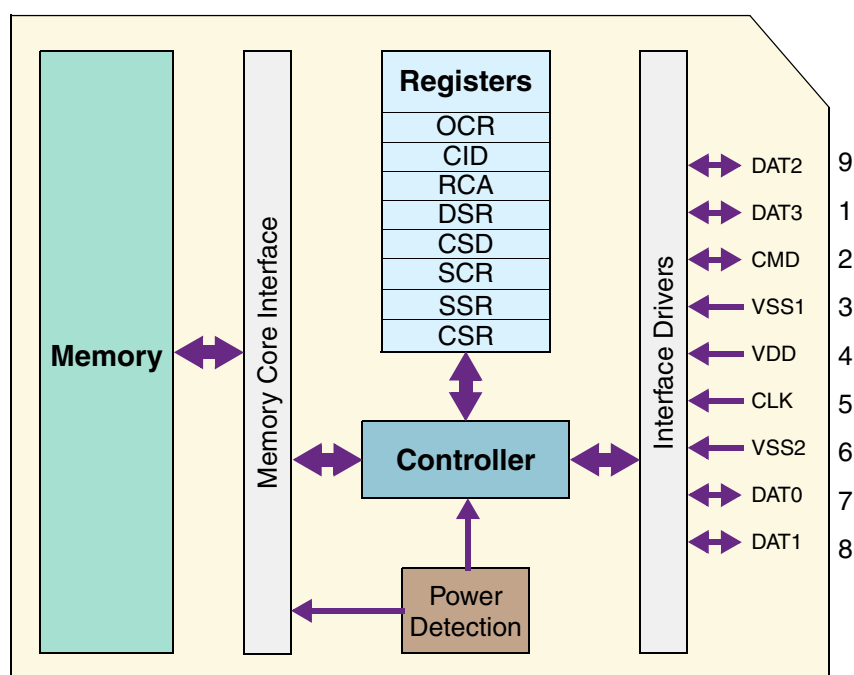


Figure 1-4 illustrates the block diagram of the MMC 3.3 card and signals.

Figure 1-4 Typical MMC 3.3 Card

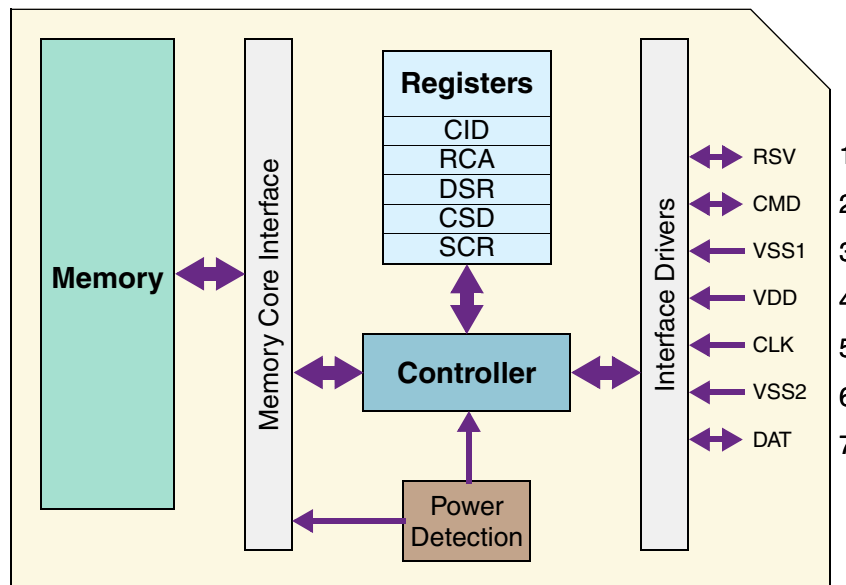
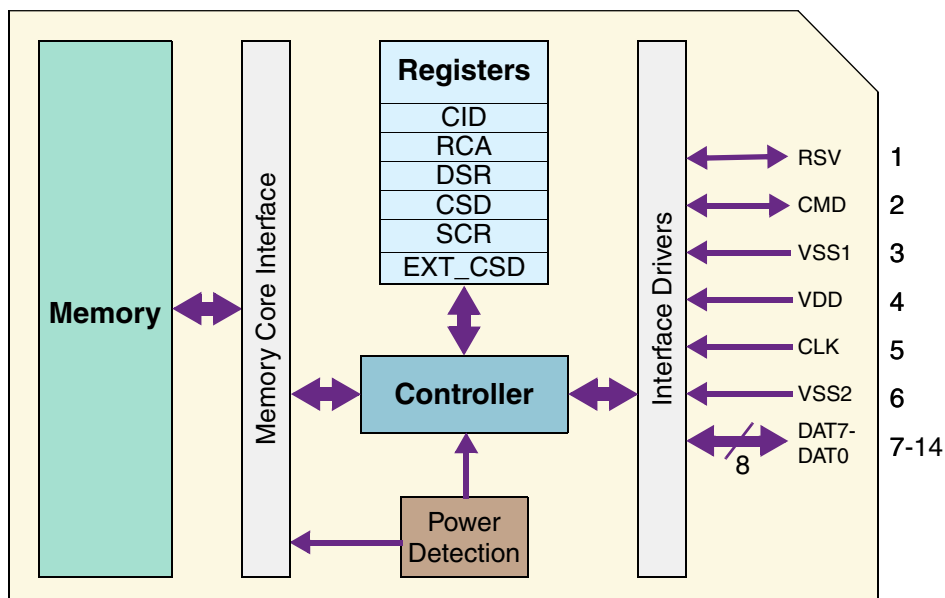


Figure 1-5 illustrates the block diagram of a high-speed MMC (HSMMC) card and signals.

Figure 1-5 Typical HSMMC Card



The SD_MMC_CEATA protocol is based on command and data bit streams that are initiated by a start bit and terminated by a stop bit. Additionally, the controller provides a reference clock and is the only master that can initiate a transaction.

- ❖ Command – A token, sent serially on the CMD line, that starts an operation.
- ❖ Response – A token that is sent from an addressed card and serially on the CMD line; not all the commands expect a response from the cards.
- ❖ Data – Can be transferred from the host to the card or vice versa. Data is transferred serially on the data line; not all commands involve data transfer.

Figure 1-6 illustrates an example multiple-block read operation; the clock is representative only and does not show the exact the number of clock cycles.

Figure 1-6 Multiple-Block Read Operation

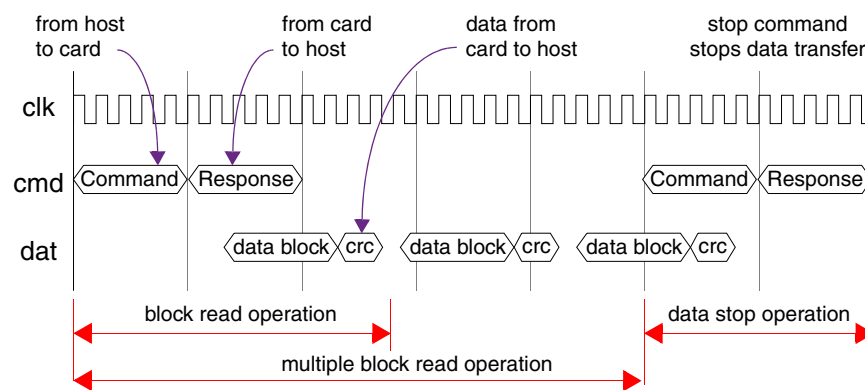


Figure 1-7 illustrates an example multiple-block write operation; again, the clock is representative only and does not show the exact number of clock cycles.

Figure 1-7 Multiple-Block Write Operation

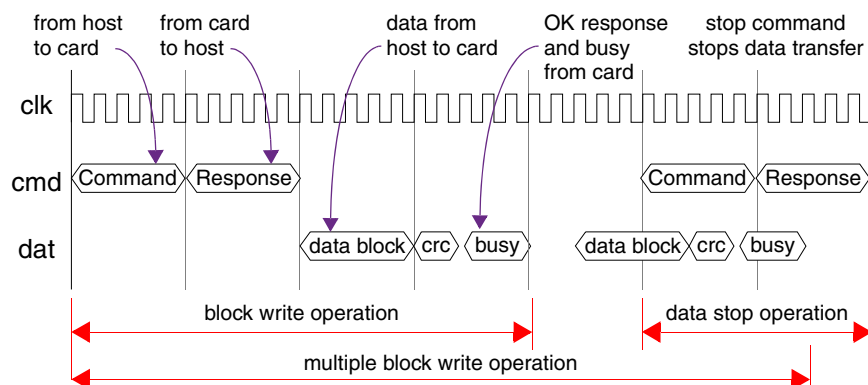


Figure 1-8 illustrates an example command token sent by the host.

Figure 1-8 Example Command Token

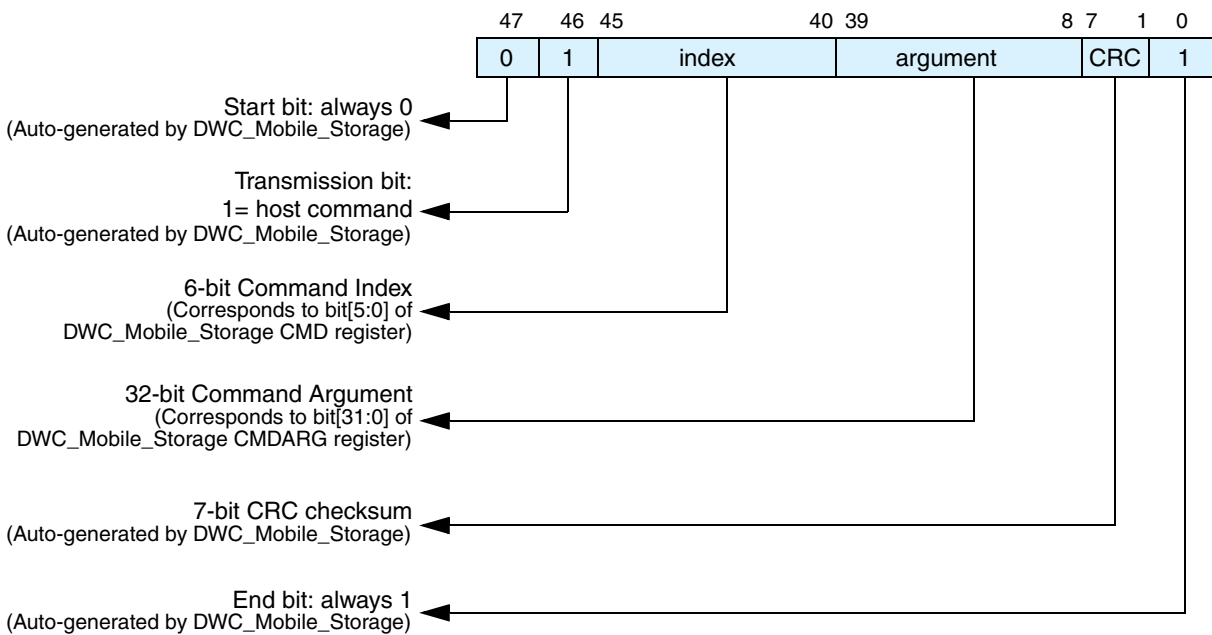
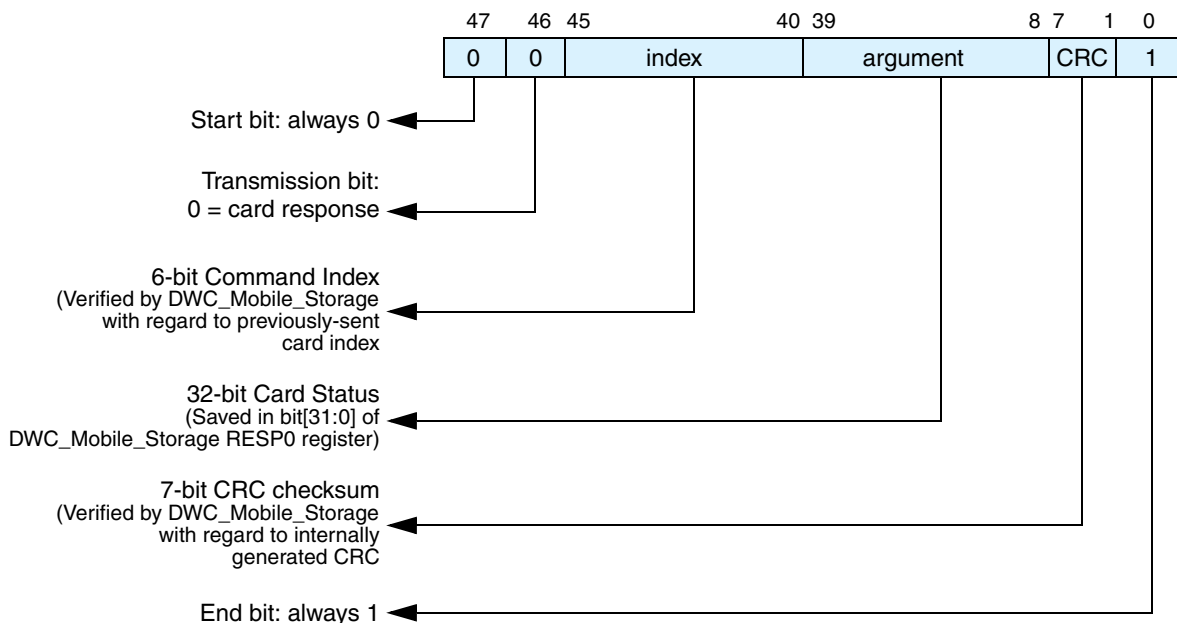


Figure 1-9 illustrates an example Short Response from a card.

Figure 1-9 Example Short Response from Card



The DWC_mobile_storage provides a flexible bus interface that enables you to integrate the DWC_mobile_storage into embedded applications for system-on-a-chip (SoC) designs. You can configure the DWC_mobile_storage in coreConsultant in order to have either an AMBA AHB or AMBA APB slave interface. In addition to the AMBA interface, the DWC_mobile_storage supports an optional internal DMA

controller and an optional external DMA controller interface for data transfers. The external DMA can be the DW_ahb_dmac, or a non-DW_ahb_dmac or generic request/acknowledge DMA interface with a separate data bus.



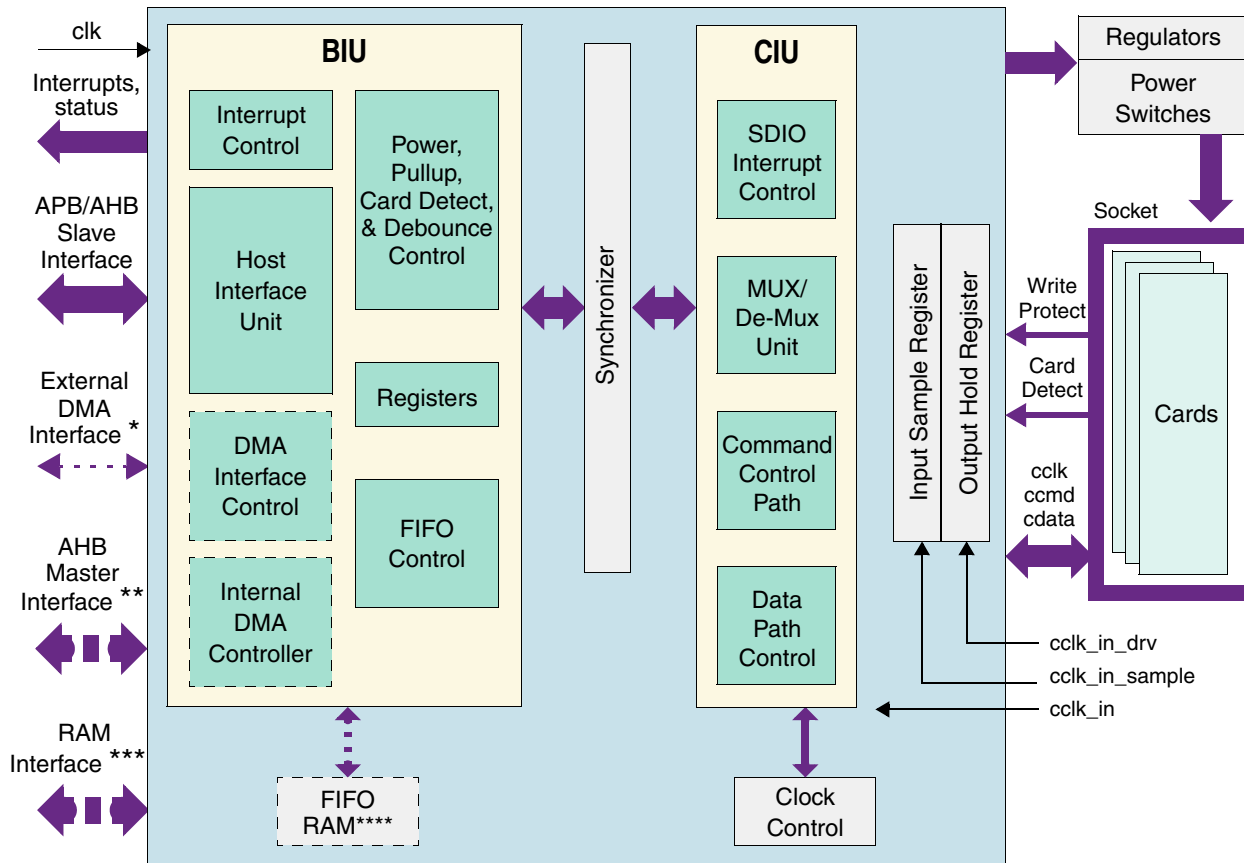
Unless otherwise noted in this manual, SD refers to both SD memory and SDIO.

1.1.1 DWC_mobile_storage Block Diagram

The DWC_mobile_storage consists of the following main functional blocks, which are illustrated in Figure 1-10.

- ❖ Bus Interface Unit (BIU) – Provides AMBA AHB/APB and DMA interfaces for register and data read/writes.
- ❖ Card Interface Unit (CIU) – Takes care of the SD_MMC_CEATA protocols and provides clock management.

Figure 1-10 Block Diagram



* Optional External DMA Interface; present only if internal DMAC is *not* present

** Optional AHB/APB Master Interface; present only if internal DMAC is present

*** Optional RAM Interface

**** FIFO RAM can be chosen as either internal or external RAM

 - optional

Note: The card_detect and write-protect signals are from the SD/MMC card socket and not from the SD/MMC card.

1.2 Features

The following are features of the DWC_mobile_storage:

- ❖ Supports Secure Digital memory protocol commands
- ❖ Supports Secure Digital I/O protocol commands
- ❖ Supports Multimedia Card protocol commands
- ❖ Supports CE-ATA digital protocol commands
- ❖ Supports Command Completion signal and interrupt to host processor
- ❖ Command Completion Signal disable feature

The following features of MMC4.41 are supported:

- ❖ DDR in 4-bit and 8-bit mode
- ❖ GO_PRE_IDLE_STATE command (CMD0 with argument F0F0F0F0)
- ❖ New EXTCSO registers
- ❖ Hardware Reset as supported by eMMC 4.41

The following features of MMC4.41 are not supported:

- ❖ Boot in DDR mode

1.2.1 Internal DMA Block Features

The following are features for the internal DMA interface:

- ❖ Supports 16/32/64-bit data transfers
- ❖ Single-channel; single engine used for Transmit and Receive, which are mutually exclusive
- ❖ Fully synchronous design operating on a single system clock
- ❖ Dual-buffer and chained descriptor linked list
- ❖ Descriptor architecture allows large blocks of data transfer with minimum CPU intervention; each descriptor can transfer up to 4KB of data in chained mode and 8KB of data in dual-buffer mode
- ❖ Comprehensive status reporting for normal operation and transfers with errors
- ❖ Programmable burst size for optimal host bus utilization
- ❖ Programmable interrupt options for different operational conditions

1.2.2 AHB Master Interface Features

The following are features for the AHB Master interface, which exists only when the internal DMA is selected:

- ❖ Supports 16/32/64-bit data
- ❖ Supports split, retry, and error AHB responses; does not support wrap
- ❖ Configurable for Little-Endian or Big-Endian mode
- ❖ Allows selection of AHB burst type through software

1.2.3 Bus Interface Features

The following are features for the Bus Interface Unit (BIU):

- ❖ Supports either AMBA AHB or APB interface (DWC_mobile_storage is an AMBA slave)
- ❖ Supports data widths of 16, 32, or 64 bits
- ❖ In addition to AMBA slave interface, supports optional external DMA controllers for data transfers. The external DMA interface is present when the internal DMAC is not present. Provides these types of DMA interfaces:
 - ◆ Interface to DW_ahb_dmac DMA controller, which shares AMBA host bus for DMA transfers
 - ◆ Generic DMA interface with dedicated data bus
 - ◆ Interface to non-DesignWare DMA controller, which shares AMBA host bus for DMA transfers
- ❖ Does not generate split, retry, or error responses on the AMBA Slave AHB bus
- ❖ Supports pin-based little-endian or big-endian modes of AHB operation
- ❖ Supports separate clocks for bus interface and card interface for ease of integration
- ❖ Supports combined single FIFO for both transmit and receive operations, which saves area and power
- ❖ Supports configurable FIFO depths of 8 to 4096
- ❖ FIFO controller shipped with a register-based single clock, dual-port synchronous read, and synchronous write RAM; can be replaced by user's FAB-dependent dual-port synchronous RAM for area-sensitive applications. In addition, has a parameter to keep FIFO RAM either inside or outside the core
- ❖ Supports FIFO over-run and under-run prevention by stopping card clock

1.2.4 Card Interface Features

The following are features for the Card Interface Unit (CIU):

- ❖ Can be configured as MMC-Ver3.3-only controller or SD_MMC controller
- ❖ Supports 1 to 30 cards in MMC-Ver3.3-only mode, and 1 to 16 SD or MMC (3.3 or 4.0) or CE-ATA devices in SD_MMC_CE-ATA mode
- ❖ Supports Command Completion Signal and interrupts to host
- ❖ Supports Command Completion Signal disable
- ❖ Supports CRC generation and error detection
- ❖ Supports programmable baud rate. Supports up to 4 clock dividers to support simultaneous operation of multiple cards with different clock speed requirements
- ❖ Provides individual clock control to selectively turn ON or OFF clock to a card
- ❖ Supports power management and power switch. Provides individual power control to selectively turn ON or OFF power to a card
- ❖ Supports host pull-up control
- ❖ Supports card detection and initialization
- ❖ Supports write protection
- ❖ Supports SDIO interrupts in 1-bit and 4-bit modes

- ❖ Supports SDIO suspend and resume operation
- ❖ Supports SDIO read wait
- ❖ Supports block size of 1 to 65,535 bytes
- ❖ Supports 4-bit and 8-bit DDR, as defined by SD3.0 and MMC4.41

1.2.5 Verification Environment Features

The following are features for the verification environment.

- ❖ AMBA, SD memory, SDIO, and MMC verification IP (VIP) and AHB verification IP (VIP), which can be used in a Verilog or Vera environment.
- ❖ SDMMC VIP supports only Vera and Verilog; support for System Verilog, VHDL and C are not provided.
- ❖ Constrained random tests and directed tests.
- ❖ Configurable and self-checking testbench and test suites in sample Verilog testbench.

1.2.6 Example Linux Demonstration Software Package Features

- ❖ DWC_mobile_storage controller-specific host-driver APIs
- ❖ DWC_mobile_storage controller-independent, SD_MMC_CEATA protocol-specific bus-driver APIs

**Note**

The software driver is available on request. Contact Synopsys support center for more information.

1.2.7 UHS-1 and Voltage-Switching Features

The following are supported UHS-1 features:

- ❖ Support for UHS 50 and UHS 104 cards with the following speeds, frequencies, and voltages, as appropriate for each card:
 - ◆ Default Speed Mode – 25 MHz, 3.3V
 - ◆ High Speed mode – 50 MHz, 3.3V
 - ◆ SDR12 – 25 MHz, 1.8V
 - ◆ SDR25 – 50 MHz, 1.8V
 - ◆ SDR50 – 100 MHz, 1.8V
 - ◆ SDR104 – 208 MHz, 1.8V
 - ◆ DDR50 – 50 MHz, 1.8V
- ❖ Voltage switching

1.3 Standards Compliance

The DWC_mobile_storage component conforms to the [AMBA Specification, Revision 2.0](#) from ARM. Readers are assumed to be familiar with this specification.

1.4 Verification Environment Overview

The DWC_mobile_storage includes an extensive verification environment, which sets up and invokes your selected simulation tool to execute tests that verify the functionality of the configured component. You can then analyze the results of the simulation.

The section titled “[Verification](#)” on page [235](#) discusses the specific procedures for verifying the DWC_mobile_storage. The appendix titled “[Atomic VIP Command Verilog Testbench](#)” on page [309](#) discusses the Verilog testbench features and usage.

2

Building and Verifying Your Core

This chapter provides an overview of the step-by-step process you use to configure, synthesize, and verify your `DWC_mobile_storage` component using the Synopsys `coreConsultant` tool. You use `coreConsultant` to create a workspace that is your working version of a component, where you configure, simulate, and synthesize your implementation of the `DWC_mobile_storage`. You can create several workspaces to experiment with different design alternatives.

2.1 Setting up Your Environment

You need to set up your environment correctly using specific environment variables, such as `DESIGNWARE_HOME`, `VERA_HOME`, `PATH`, and `SYNOPSISYS`. If you are not familiar with these requirements and the necessary licenses, refer to the section [“Setting up Your Environment”](#) in the *DesignWare DWC Mobile Storage Host Installation Guide*.

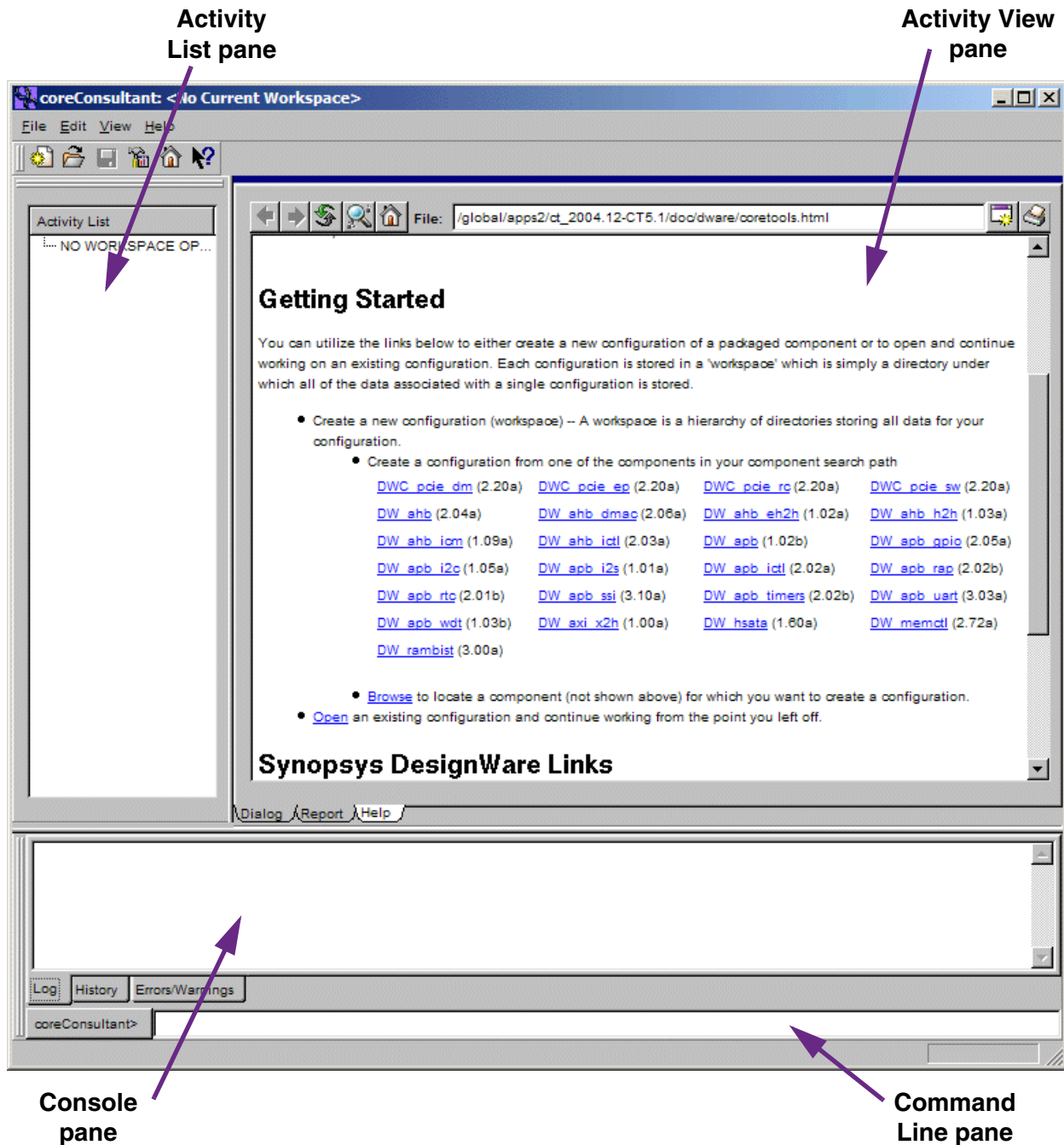
2.2 Starting coreConsultant

To invoke coreConsultant:

1. In a UNIX shell, navigate to a directory where you plan to locate your component workspace.
2. Invoke the coreConsultant GUI:

% **coreConsultant**

The welcome page is displayed, similar to the one below.



3. To create a new workspace, click on the `DWC_mobile_storage` link in the “Configuring and Using an IP block” section, or use the **File > New Workspace** pulldown menu item. In the resulting dialog box, specify the workspace name and workspace root directory, or use the defaults – a workspace name is the name of a configuration of a core; the workspace root directory is the directory in which the configuration is created. Enter the correct path to the installed coreKit path; click OK.

2.3 Checking Your Environment

Before you begin configuring your component, it is recommended that you check your environment to make sure you have the latest tool versions installed and your environment variables set up correctly.

To check your environment, use the **Help > Check Tool Environment** menu path.

An HTML report is displayed in a separate dialog. This report lists the specific tools and versions installed in your environment. It also displays errors when a specific tool is not installed or if you are using an older version. You will also see an error if your `$DESIGNWARE_HOME` environment variable has not been set up correctly.

2.4 Configuring the `DWC_mobile_storage`

This section steps you through the tasks in the coreConsultant GUI that configure your core. Complete information about the latest version of coreConsultant is available on the web in the [coreConsultant User Guide](#). To view documentation specific to your version of coreConsultant, choose the Help pulldown menu from the coreConsultant GUI.

At any time during this process you can click on the Help tab for each activity to activate the coreConsultant online help.



Note

Throughout the remaining steps in this chapter, it is best if you apply the default values so that the directions and descriptions in the chapter will coincide with your display. After you have used the `DWC_mobile_storage` in coreConsultant, you can then go back through these steps and change values in order to see how they affect the design.

1. Notice that the Set Design Prefix activity is already checked. This setting is used to make each design in your core have a unique name. This is needed only when you have two or more versions of one component, each with different values of configuration parameters.
2. Click on the Specify Configuration activity to specify the basic configuration of the `DWC_mobile_storage`.

If you have a Source license, you can choose to use DesignWare Building Block IP (DWBB) components for optimal Synthesis QoR. Alternatively, if you have an RTL source licence, you may use source code for DWBB components without a DesignWare license. If you use RTL source and also have a DesignWare key, you can choose to retain the DWBB parts.

3. Look through the basic parameters for each item. If you need help with any field in the activity pane, right-click on the field name and then left-click on the What's This box. When finished, click Apply.

Field Name	Description
Controller Type (Bus topology or Star topology)	Values: MMC_VER3.3_ONLY, SD_MMC_CEATA Default Value: SD_MMC_CEATA Description: Configures the controller either as Multimedia Card Ver3.3-only controller, or as SD/MMC controller that supports SD Memory, SDIO, MMC simultaneously. MMC_VER3.3_ONLY mode supports only MMC Version 3.3 cards. In MMC Ver 3.3 mode, cards and controller are connected by single shared bus (bus-topology). SD_MMC_CEATA mode supports 1-bit and 4-bit SD cards, and 1-bit, 4-bit, and 8-bit MMC cards. In SD_MMC mode, separate bus is provided between controller and each card in star-topology.
Maximum Number of Cards	Values: 1-30 cards Default Value: 3 Description: Configures number of cards supported by controller. In MMC_VER3.3_ONLY mode, up to 30 physical cards are supported. In SD_MMC_CEATA mode, up to 16 cards are supported.
Host Bus Type	Values: APB, AHB Default Value: AHB Description: Configures Host bus as either AMBA APB or AMBA AHB.
Host Data Bus Width	Values: 16, 32, 64 Default Value: 32 Description: Width of host data bus.
Host Address Bus Width	Values: 9-28 Default Value: 20 Description: Width of host address bus. Address 0x to 0xFF selects internal registers; address 0X200 and above selects data FIFO.
Internal DMA Controller	Values: 0, 1 Default Value: 1 Description: Determines whether an Internal DMA Controller is required.
DMA Interface Type	Values: None, DW-DMA, Generic-DMA, Non-DW-DMA Default Value: None Dependencies: Present only if internal DMA controller is not present Description: Configures type for external DMA Interface. In addition to AMBA host interface, data FIFO can be accessed by an optional DMA interface. DMA type can be: • None – No DMA interface • DW-DMA – Provides handshake signals to DW_ahb_dmac controller • Generic-DMA – Generic DMA interface, which provides simpler request/acknowledgement protocol and dedicated DMA data-bus • Non DW-DMA controller – Provides handshake signals to non-DW_ahb_dmac controller

Field Name	Description
Generic DMA Data Bus Width	Values: 16, 32, 64 Default Value: 32 Description: Width of Generic-DMA data bus. This option is enabled only when DMA_INTERFACE is set to Generic-DMA
FIFO Buffer Depth	Values: 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096 Default Value: 16 Description: Configures depth of combined transmit/receive FIFO buffer. FIFO data width is set to either 32 or 64, depending on host and DMA configuration. Typically a smaller 16 to 32 deep FIFO is sufficient when DMA interface is selected. If processor is used to move data between FIFO and system memory, then at least one block larger size of FIFO RAM is recommended.
FIFO RAM Inside or Outside the Controller	Values: Outside, Inside Default Value: Inside Description: Selects whether FIFO RAM is instantiated inside core or instantiated outside core by user. For smaller FIFO size, it is recommended to use "INSIDE" option; the RAM is automatically synthesized using registers. When "OUTSIDE" option is chosen, RAM interface ports are added to DWC_mobile_storage.v. User can then interface a vendor-specific dual-port synchronous RAM to ports.
Number of Clock Dividers	Values: 1, 2, 3, 4 Default Value: 1 Description: Configures number of clock dividers. DWC_mobile_storage supports 1 to 4 clock dividers. In MMC_VER3.3_ONLY mode, since there is only one cclk_out, only one clock divider is supported.
Power-On Value of User Identification Register	Values: h0x0 to h0xffffffff Default Value: Description: Power on value of User Identification register, which can be used as either a scratch pad or identification register by user.
Set clk to cclk_in and vice versa false path?	Values: No, Yes Default Value: Yes Description: When this parameter is set, in synthesis false path between "clk to cclk_in," "cclk_in to clk," and "reset_n to cclk_in" are set. If clk and cclk_in are two different free-running clocks, it is recommended to set false path. If cclk and cclk_in have phase relationship (derived), it is recommended not to set false path so that metastability associated with signal synchronization can be avoided. If user does not set false path, then clk and cclk_in frequencies should be integer multiple of each other.

Field Name	Description
Enable Area Optimization?	Values: No, Yes Default Value: No Description: When this parameter is set, area is optimized by removing following optional hardware features: 1. General purpose input/output ports are removed 2. User identification register (USRID) is not implemented 3. Transferred CIU card byte count (TCBCNT) register can be only read after data transfer done and not during data transfer (will return 0).
IMPLEMENT_SCAN_MUX	Parameter Name: IMPLEMENT_SCAN_MUX Legal Values: 0, 1 Default Value: 0 Description: When the parameter is set, the negative-edge-triggered flip-flops are driven by a clock that is coming out of a scan MUX, which is instantiated only if IMPLEMENT_SCAN_MUX is set to 1. The select line for the scan MUX is the scan_mode signal. The two inputs to this scan MUX are the clock and the inverted clock. When the scan_mode signal is high, the negative-edge-triggered flip-flops get the inverted version of the clock. Implementing this scan MUX enables the negative-edged-triggered flip-flops to be included in the same scan chain as the positive-edged-triggered flip-flops. However, clock balancing at the chip level can be a challenge if this MUX is implemented. Note: The IMPLEMENT_SCAN_MUX is not required if a "clock mixing" technique is used for DFT implementation. This option is provided to support wide range of DFT tools and methodologies.

When the configuration setup is complete, the Report tab is displayed, which gives you all the source files (in encrypted format if you have a DW license, and unencrypted if you have a source license), and all the parameters that have been set for this particular configuration. Reports contain useful information as you complete each step in the coreConsultant process. Familiarize yourself with the report contents before going to the next step.

For more information about the configuration parameters for DWC_mobile_storage, refer to [“Parameters”](#) on page 115.

2.5 Synthesizing the DWC_mobile_storage

To run synthesis on the component and create a gate-level netlist, step through the following tasks in the coreConsultant GUI. You need to click the check box next to each activity in order to access the specific activity dialog. At any time, you can click on the Help tab for each activity to display more information.

1. Look at the tool installation root directories in the Tool Installation Roots dialog, which is accessed from the toolbar menu through **Edit > Tool Installation Roots**, or by using the Tools button on the toolbar. You can type values directly in the data fields, or use the buttons to locate the correct directories. The tool choices are:
 - ◆ Design Compiler (dc_shell) – Specifies the location for the root directory of the Design Compiler installation, if different from the default location.
 - ◆ Primetime (pt_shell) – Enables Primetime if you plan to implement budgeting or generate timing models.

- ◆ Formality (fm_shell) – Enables Formality if you plan to formally verify the synthesized gate-level implementation of the core.

At a minimum for this exercise, dc_shell must have defined installation directories, and in order to complete the optional formal verification in this chapter, you will also need fm_shell.

2. **Specify Target Technology** – coreConsultant analyzes the target technology library and uses it to generate a synthesis strategy that is optimized for your technology library. In the Design Compiler window, a target and link library must be specified; otherwise, errors occur in coreConsultant.

Under the Specify Target Technology category in the Activity List, the title in the tabs depends on the compiler you chose in the previous step. Regardless, this screen provides fields for you to enter the search path for the specific compiler, as well as target and link library paths. If necessary, specify the search path for the tool you specified in the previous screen. Also, specify the path to the target and link libraries. Click Apply and familiarize yourself with the resultant report, which gives you the technology information.

3. **Specify Clock(s)** – In the Specify Clock(s) activity, look at the attributes associated with each of the real and virtual clocks in your design. Click Apply and familiarize yourself with the resultant report, which gives you clock information.
4. **Specify Operating Conditions and Wire Loads** – In the Specify Operating Conditions and Wire Loads activity, look at the attributes relating to the chip environment. If you do not see a value beside OperatingConditionsWorst, select an appropriate value from the drop-down list; if there is no value for this attribute, you will get an error message. Click Apply and look at the report, which gives the operating conditions and wireload information.
5. **Specify Port Constraints** – In the Specify Port Constraints activity, look at the attributes associated with input delay, drive strength, DRC constraints, output delay, and load specifications. Click Apply and look at the report, which gives the port constraint checks.
6. **Specify Synthesis Methodology** – In the Specify Synthesis Methodology activity, look at the synthesis strategy attributes. Note that these attributes are typically set by the core developer and are not required to be modified by the core integrator. If you want to add your own commands during a synthesis, you use the Advanced tab in order to provide path names to your auxiliary scripts. Also, click on the Physical Synthesis tab to familiarize yourself with those options. Click Apply and look at the report, which gives design information. For more information on adding auxiliary scripts, refer to “Advanced Synthesis Methodology Attributes” in the [coreAssembler User Guide](#).
7. **Specify Test Methodology** – In the Specify Test Methodology activity, look at the scan test attributes. Also click on the other tabs to familiarize yourself with auto-fix attributes, SoC test wrapper attributes, test wrapper integration attributes, BIST attributes, and BIST testpoint insertion attributes. Click Apply and look at the report, which gives design-for-test information.
8. **Synthesize** – Choose the Synthesize activity. Do the following:

- a. Choose the Strategy tab.
- b. Click the Options button beside DCTCL_opto_strategy and look through the strategy parameters. For example, you can use the Gate Clocks During Elaboration check box in the Clock Gating tab in order to add parameters that enable and control the use of clock gating. Click OK when you are done. For more information on clock gating and other parameters for synthesis strategies, refer to “DC(TCL)_opto_strategy” in the [coreAssembler User Guide](#).

For Design for Test, click the Options button and then select the Design for Test tab. Here you can specify whether to add the -scan option to the initial compile call (Test Read Compile)

and/or insert design for test circuitry (Insert Dft). For more information about include DFT in your synthesis run, refer to [coreAssembler User Guide](#).

- c. Choose the Options tab. Look at the values for the parameters listed below.

Field Name	Description
Execution Options	
Generate Scripts only?	Values: Enable or Disable Default Value: Disable Description: Writes the run.scr script, but it is not run when you click Apply. To run the script, go to the component workspace and run the script.
Run Style	Values: local, lsf, grd, or remote Default Value: local Description: Describes how to run the command: locally, through LSF, through GRD, or through the remote shell.
Run Style Options	Values: user-defined Default Value: none Description: Additional options for the run style options except local. For remote, specify the hostname. For LSF and GRD, specify bsub or qsub commands.
Parallel job CPU limit	Values: user-defined; minimum value is 1 Default Value: 1 Description: Specifies number of parallel compile jobs that can be run.
Send e-mail	Values: current user's name Description: E-mail is sent when the command script completes or is terminated.
Skip reading \$HOME/.synopsys_dc.setup	Values: Enable or Disable Default Value: Disable Description: Forces tools not to read .synopsys_dc.setup file from \$HOME.

- d. If it is not already set, choose the "local" Run Style option and keep the other default settings.
 - e. Look through the Licenses and Reports tabs, and ensure that you have all the licenses that are required to run this synthesis session.
 - f. Click Apply in the Synthesize Activity pane to start synthesis from coreConsultant. The current status of the synthesis run is displayed in the main window. Click the Reload Page button if you want to update the status in this screen.
9. Generate Test Vectors – This option allows you to generate ATPG test vectors with TetraMax. For more information about this, refer to "Generating Test Vectors" in the [coreAssembler User Guide](#).

2.5.1 Checking Synthesis Status and Results

To check synthesis status and results, click the Report tab for the synthesis options; coreConsultant displays a dialog that indicates:

- ❖ Your selected Run Style (local, lsf, grd, or remote)
- ❖ The full path to the HTML file that contains your synthesis results
- ❖ The name of the host on which the synthesis is running
- ❖ The process ID (Job Id) of the synthesis
- ❖ The status of the synthesis job (running or done)

The Results dialog also enables you to kill the synthesis (Kill Job) and to refresh the status display in the Results dialog (Refresh Status). The Results information includes:

- ❖ Summary of log files
- ❖ Synthesis stages that completed
- ❖ Summary of stage results

This information indicates whether the synthesis executed successfully, and lists the DWC_mobile_storage transactions that occurred during the scenario(s). Thorough analysis of the scenario execution requires detailed analysis of all synthesis log files and inspection of report summaries.

2.5.2 Synthesis Output Files

All the synthesis results and log files are created under the syn directory in your workspace. Two of the files in the workspace/syn directory are:

- ❖ run.scr – Top-level synthesis script for DWC_mobile_storage
- ❖ run.log – Synthesis log file

Your final netlist and report directories depend on the QoR effort that you chose for your synthesis (default is medium):

- ❖ low – initial
- ❖ medium – incr1
- ❖ high – incr2

For information about deliverables that are generated after synthesis is performed, refer to [“Database Description”](#) on page 319.

2.5.3 Running Synthesis from Command Line

To run synthesis from the command line prompt for the files generated by coreConsultant, enter the following command:

```
% run.scr
```

This script resides in your *workspace/syn* directory.

2.6 Verifying the DWC_mobile_storage

The DWC_mobile_storage coreKit includes a verification environment that sets up and invokes your selected simulation tool to execute specified test algorithms. You can then analyze the results of the

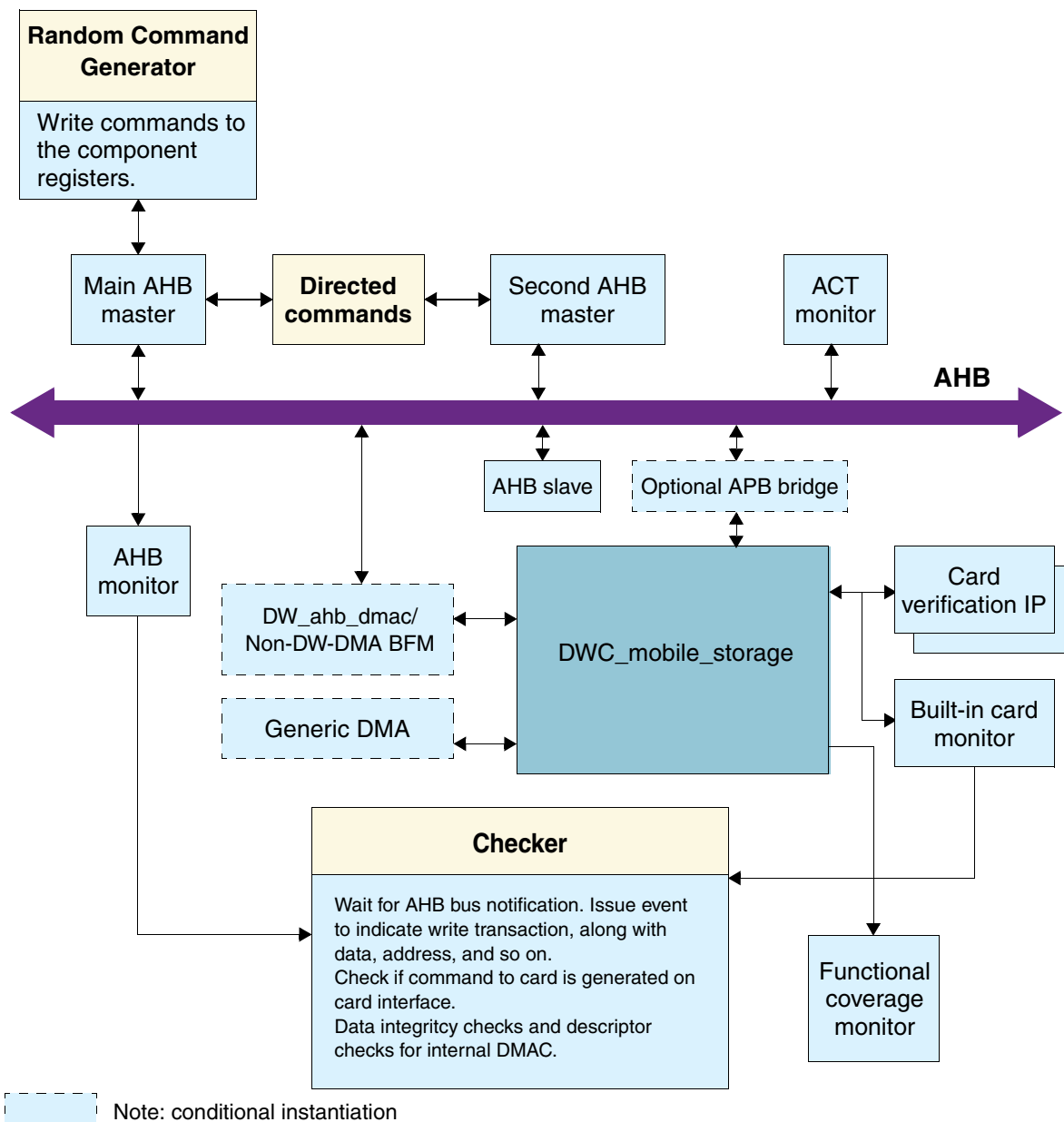
simulation. The following sections provide a brief overview of the `DWC_mobile_storage` testbench and describes how to execute the `DWC_mobile_storage` simulation activities using `coreConsultant` in GUI mode:

- ❖ “[DWC_mobile_storage Testbench](#)” on page 34 – Specifies the register base address, as well as the base addresses and block sizes of the devices involved in the design.
- ❖ “[Verifying the Simulation Model](#)” on page 35 – Simulates the selected algorithms and compares the results with the expected results. For more information about how the verification environment tests the algorithms, refer to “[Verification](#)” on page 235.

2.6.1 DWC_mobile_storage Testbench

Figure 2-1 shows the structure of the `DWC_mobile_storage` verification environment.

Figure 2-1 `DWC_mobile_storage` Verification Environment



For more detailed information about the DWC_mobile_storage verification environment, including procedures to operate the verification environment directly from the UNIX command line, refer to [“Verification”](#) on page 235.

1. Choose the Testbench Parameter Setup item in the activity list.
2. Select values according to your needs; for an initial run-through of this usage chapter, the defaults are appropriate.
3. When the Parameters for Testbench Parameter Setup activity is complete, Apply the activity.

2.6.2 Verifying the Simulation Model

To verify DWC_mobile_storage, use coreConsultant to complete the following steps:

1. Testbench Parameter Setup – Specify the endian mode you need.



The selection of Card Type using the GUI is meant for VTB tests and not for CRV tests.

2. Setup and Run Simulations – Specify the simulation by completing the Setup and Run Simulations activity:
 - a. In the Select Simulator area, click on the Simulator view list item to view available simulators (VCS is the default).
 - b. Specify an appropriate Verilog simulator from the drop-down menu.
For installation instructions and information about required tools and versions, refer to [“Setting up Your Environment”](#) in the *DesignWare DWC Mobile Storage Host Installation Guide*.
 - c. In the Simulator Setup area of the Simulator pane, look at the parameters for the simulator setup, as detailed in the following table.

Field Name	Description
Root Directory of Cadence Installation	The path to the top of the directory tree where the Cadence NC-Verilog executable is found; coreConsultant automatically detects this path. The NC-Verilog executables reside in the ./bin subdirectory.
MTI Include Path	The path to the include directory contained within your MTI simulator installation area. A valid directory includes the veriusers.h file.
Vera Install Area (\$VERA_HOME)	Path to your Vera installation. Default Value: value of your \$VERA_HOME variable
C Compiler for (Vera PLI)	Values: gcc or cc Default Value: gcc Description: Invokes the specific C compiler to create a Vera PLI for your chosen non-VCS simulator. Choose cc if you have the platform native ANSI C compiler installed. Choose gcc if you have the GNU C compiler installed.

- d. In the Waves Setup area of the Simulator pane, look at the parameters for the waves setup, as detailed below.

**Note**

For the Generate Waves File setting, enable the check box so that the simulation creates a dump file that you can use later for debugging the simulation, if you want to do so.

Field Name	Description
Generates waves file	Values: Enable or Disabled Default Value: Disabled Description: Indicates whether a wave file should be created for debugging with a wave file browser after simulation ends. Uses VPD file format for VCS and VCD format for the other supported simulators.
Depth of waves to be recorded	Values: 0 or 1 Default Value: 0 Description: Determines which signals you will see in the dump file. A depth of 1 indicates top-level signal and one below. A depth of 0 indicates all signals in the design hierarchy.

- e. Choose the Design View list choice.
- f. In the View Selection area of the View pane, look at the choice of views of the design you can simulate from the drop-down list:
- ✧ RTL – requires a source license or Synopsys VCS
 - ✧ GateLevel – required if you are using a non-VCS simulator and do not have a source license.
- g. In the Model Definition area of the View pane, indicate the following:
- ✧ Get Level Models Directory – directory containing simulation models for the target technology library of the component.
 - ✧ Get Level Model File Name – library files for the target technology library.
- h. Choose the Execution Options list choice to set the following options:

Field Name	Description
Do Not Launch Simulation	Values: Enable or Disable Default Value: Disable Description: Determines whether to execute the simulation or just generate the simulation run script. If enabled, coreConsultant generates, but does not execute, the simulation run script. You can execute the script at a later time by invoking the run script (<i>workspace/sim/run.scr</i>) directly from the UNIX command line or by repeating the Verification activity with Do Not Launch Simulation unselected.
Run Style	Values: local, lsf, grd, or remote Default Value: local Description: Describes how to run the command: locally, through LSF, through GRD, or through the remote shell.

Field Name	Description
Run Style Options	Values: user-defined Default Value: none Description: Additional options for the run style options except local. For remote, specify the hostname. For LSF and GRD, specify bsub or qsub commands.
Send e-mail	Values: current user's name Description: E-mail is sent when the command script completes or is terminated.
Parallel Simulations Using LSF	Controls how simulation jobs are submitted in LSF mode.
Parallel Simulations Using GRD	Controls how simulation jobs are submitted in GRD mode.

i. Choose Select Tests and look at the options described below:

Field Name	Description
Run test act	Values: Enable or Disable Default Value: Enable Description: Tests all AMBA AHB cycles and creates stimulus for passing AMBA Compliance Certification.
Run test fifo	Values: Enable or Disable Default Value: Enable Description: Tests the data and address accesses to the FIFO area with all combinations of AHB sizes and bursts; also tests AHB busy cycles.
Run test early_term	Values: Enable or Disable Default Value: Enable Description: Tests AHB early terminations to the register and FIFO area.
Run test readcmd17	Values: Enable or Disable Default Value: Enable Description: Tests single-block reads to SD/MMC card.
Run test readcmd18	Values: Enable or Disable Default Value: Enable Description: Tests multiple-block reads to SD/MMC card.
Run test readcmd18_clkcov	Values: Enable or Disable Default Value: Enable Description: Reads multiple blocks of data from SD/MMC cards. This test case is used to improve the functional coverage.
Run test writecmd24_clkcov	Values: Enable or Disable Default Value: Enable Description: Writes single blocks of data into SD/MMC card. This test case is used to improve the functional coverage.

Field Name	Description
Run test writecmd24	Values: Enable or Disable Default Value: Enable Description: Tests single-block writes to SD/MMC card.
Run test writecmd25	Values: Enable or Disable Default Value: Enable Description: Tests multiple-block writes to SD/MMC card.
Run test rd_blk_1, rd_blk_2	Values: Enable or Disable Default Value: Enable Description: Test block reads to SD/MMC card.
Run test wr_blk_1, wr_blk_2	Values: Enable or Disable Default Value: Enable Description: Test block writes to SD/MMC card.
Run test rd_wr_blk_1, rd_wr_blk_2	Values: Enable or Disable Default Value: Enable Description: Test block read/writes to SD/MMC card.
Run test io_rd_wr_1, io_rd_wr_2	Values: Enable or Disable Default Value: Enable Description: Test block read/writes to SDIO card.
Run test rd_wr_strm_1, rd_wr_strm_2	Values: Enable or Disable Default Value: Enable Description: Test stream read/writes to MMC card.
Run test misc_1, misc_2	Values: Enable or Disable Default Value: Enable Description: Verify other SD/MMC commands.
Run test io_rd_intr_1, io_rd_intr_2	Values: Enable or Disable Default Value: Enable Description: Test SDIO interrupts during read.
Run test io_wr_intr_1, io_wr_intr_2	Values: Enable or Disable Default Value: Enable Description: Test SDIO interrupts during write.
Run test io_rd_wait_1, io_rd_wait_2	Values: Enable or Disable Default Value: Enable Description: Test SDIO read wait operation.
Run test io_suspend_1	Values: Enable or Disable Default Value: Enable Description: Tests SDIO suspend operation.

Field Name	Description
Run test mmc_intr_1	Values: Enable or Disable Default Value: Enable Description: Tests MMC interrupt.
Run test rd_wr_blk_dat_err, rd_wr_blk_resp_err	Values: Enable or Disable Default Value: Enable Description: Tests for error conditions.
Run test basic_clock	Values: Enable or Disable Default Value: Enable Description: Verifies clock generation logic.
Run test basic_misc	Values: Enable or Disable Default Value: Enable Description: Verifies other SD/MMC commands.
Run test gedma_cardRd	Values: Enable or Disable Default Value: Enable Description: Verifies Generic DMA interface during card read.
Run test gedma_cardWr	Values: Enable or Disable Default Value: Enable Description: Verifies Generic DMA interface during card write.
Run test basic_wrdata_cov	Values: Enable or Disable Default Value: Enable Description: Write blocks of data into SD/MMC card to improve the coverage.
Run test basic_cov	Values: Enable or Disable Default Value: Enable Description: Tests to improve coverage.
Run test io_suspend_2	Values: Enable or Disable Default Value: Enable Description: Tests SDIO suspend operation.
Run test io_suspend_intr	Values: Enable or Disable Default Value: Enable Description: SDIO suspend operation with different variations like FIFO threshold, auto stop, block size, and so on.
Run test rd_wr_strm_3	Values: Enable or Disable Default Value: Enable Description: Test stream read/writes to MMC card.
Run test rd_wr_starv_1, rd_wr_starv_2	Values: Enable or Disable Default Value: Enable Description: SD/MMC Block read/write operation with data starvation.

Field Name	Description
Run test basic_cov_2	Values: Enable or Disable Default Value: Enable Description: Tests to improve coverage.
Run test basic_rd_wait	Values: Enable or Disable Default Value: Enable Description: SD/MMC Block read operation for read wait functionality.
Run test err_inject	Values: Enable or Disable Default Value: Enable Description: Test cases for error injection and detection; CRC error, CMD index error, and so on.
Run test fifo_cov	Values: Enable or Disable Default Value: Enable Description: Block read/write to improve the coverage.
Run test ceata_ccsd_col	Values: Enable or Disable Default Value: Enable Description: Host is driving the CCSD and device driving the CCS on the CMD line. Another loop tests the condition where send_ccsd is asserted on the same clock cycle of CCS sampling inside the CIU module.
Run test ceata_basic_err_inject	Values: Enable or Disable Default Value: Enable Description: Basic error injection scenarios are tested in this testcase. The error scenarios include: Response Transmit Bit error, CRC status error, No CRC status, Data CRC error, and CMD index error in the response.
Run test ceata_allcmds	Values: Enable or Disable Default Value: Enable Description: Verification for the complete functionality of the ATA commands. Each ATA command consists of RW_REG for task file, and RW_BLK for the data transfer, if any. The RW_BLK transfers are with either CCS enabled or CCS disabled.
Run test ceata_ccsd	Values: Enable or Disable Default Value: Enable Description: Randomly generated CCSD and STOP commands given during the data transfer. The STOP commands can be either internally generated auto-stop or externally generated stop.
Run test ceata_err_inject	Values: Enable or Disable Default Value: Enable Description: Randomly generated error scenarios. The error scenarios include Response timeout and ACIO timeout.

j. Click Apply to run the simulation.

When you click Apply, coreConsultant performs the following actions:

- ◆ Sets up the verification environment to match your selected DWC_mobile_storage configuration.
- ◆ Generates the simulation run script (run.scr) and writes it to your *workspace/sim* directory.
- ◆ Invokes the simulation run script, unless you enabled the Do Not Launch Simulation option.

The simulation run script, in turn, performs the following actions:

- ◆ Links the generated command files, and recompiles the testbench.
- ◆ Invokes your simulator to simulate the specified scenarios.
- ◆ Writes the simulation output files to your *workspace/sim/test_** directory.
- ◆ If an e-mail address is specified, sends the simulation completion information to that e-mail address when the simulation is complete.

For an overview of the related tests, refer to [“Verification”](#) on page 235.

3. (Optional) Formal Verification – You can run formal verification scripts using Synopsys Formality (fm_shell) to check two designs for functional equivalence. You can check the gate-level design from a selected phase of a previously executed synthesis strategy against either the RTL version of the design or the gate-level design from another stage of synthesis. To run this, choose Formal Verification under the Verify Component category and then click Apply.

2.6.2.1 Checking Simulation Status and Results

To check simulation status and results, click the Report tab for either the GTECH models or for the simulation options; coreConsultant displays a dialog that indicates:

- ❖ Your selected Run Style (local, lsf, grd, or remote)
- ❖ The full path to the HTML file that contains your simulation results
- ❖ The name of the host on which the simulation is running
- ❖ The process ID (Job Id) of the simulation
- ❖ The status of the simulation job (running or done)

If you selected the “LSF/GRD” option for the Run Style, then the status of the simulation jobs (running or complete) is incorrect. Once all the simulation jobs are submitted to the LSF/GRD queue, the status would indicate “complete.” You should use “bjobs/qstatus” to see whether all the jobs are completed.

The Results dialog also enables you to kill the simulation (Kill Job) and to refresh the status display in the Results dialog (Refresh Status). The Results information includes:

- ❖ Vera compile execution messages
- ❖ Simulation execution messages
- ❖ DWC_mobile_storage bus transactions

This information indicates whether the simulation executed successfully, and lists the DWC_mobile_storage transactions that occurred during the scenario(s).

Thorough analysis of the scenario execution requires detailed analysis of all simulation output files and inspection of simulation waveforms with a waveform viewer.

2.6.2.2 Creating a Batch Script

It sometimes helps to have a batch file that contains information about the workspace, parameters, attributes, and so on. You can then review these by looking at the file in an ASCII editor. To do this, choose

the **File > Write Batch Script** menu item and enter a name for the file. Then look at the contents to familiarize yourself with the information that you can get from this file. You can use the batch script to reproduce the workspace.

2.6.2.3 Applying Default Verification Attributes

To reset all DWC_mobile_storage verification attributes to their default values, use the Default button in the Setup and Run Simulation activity under the Verification tab.

To examine default attribute values without resetting the attribute values in your current workspace, create a new workspace; the new workspace has all the default attribute values. Alternatively, use the Default button to reset the values, and then close your current workspace without saving it.

3

Functional Description

This chapter provides an overview of the DWC_mobile_storage architecture. The primary parts of the architecture are:

- ❖ **Bus Interface Unit (BIU)** – provides the host interface to the registers and the data FIFO through the Host Interface Unit (HIU). Additionally, it provides independent data FIFO access through a DMA interface. You can configure the host interface as either an AMBA APB or an AMBA AHB slave interface.
- ❖ **Internal Direct Memory Access Controller (IDMAC)** – responsible for exchanging data between the FIFO and the host memory. A set of IDMAC registers is accessible by the host for controlling the IDMAC operation through the AMBA AHB/APB slave interface.
- ❖ **Card Interface Unit (CIU)** – controls the card-specific protocols. Within the CIU, the command path control unit and data path control unit interface the controller to the command and data ports of the SD_MMC_CEATA cards. The CIU also provides clock control.

Table 3-1 lists the DWC_mobile_storage design hierarchy.

Table 3-1 DWC_mobile_storage Design Files and Hierarchy

File Name	Block Name
DWC_mobile_storage_params.v [†]	DWC_mobile_storage Parameter Defines
DWC_mobile_storage_derived_params.v [†]	DWC_mobile_storage Derived Parameter Defines
DWC_mobile_storage [†]	Top-level Design
DWC_mobile_storage_ahb2apb @	AHB-to-APB Conversion Module
DWC_mobile_storage_dmac_if ^#	Control/Data access arbitration resolver
DWC_mobile_storage_dmac_ahb ^#	IDMAC Top level Design
DWC_mobile_storage_dmac_cntrl ^#	IDMAC FSM
DWC_mobile_storage_dmac_csr ^#	IDMAC Control & Status Register
DWC_mobile_storage_ahb_ahm ^#	AHB Master Unit
@ __Conditionally instantiated, when H_BUS_TYPE = AHB † __ Unencrypted File; & __ Conditionally instantiated, when FIFO_RAM_INSIDE = 1 ^# __Conditionally instantiated when INTERNAL_DMACH = 1	

Table 3-1 DWC_mobile_storage Design Files and Hierarchy (Cont.)

File Name	Block Name
DWC_mobile_storage_biu	Bus Interface Unit (BIU)
DWC_mobile_storage_regb	Register Bank and Interrupt Unit
DWC_mobile_storage_dma	DMA Interface Unit
DWC_mobile_storage_cdet	Card Detect Interrupt Unit
DWC_mobile_storage_2clk_fifoctl	FIFO Control Unit
DWC_mobile_storage_2clk_dssram [†] &	FIFO Dual Port Synchronous Read and Write SRAM
DWC_mobile_storage_b2c	BIU clock Domain to CIU Domain Synchronizers
DWC_mobile_storage_c2b	CIU clock Domain to BIU Domain Synchronizers
DWC_mobile_storage_ciu	Card Interface Unit (CIU)
DWC_mobile_storage_clkcntl	Card Clock Control Unit
DWC_mobile_storage_clk_mux_4x1	4:1 Mux for clock path
DWC_mobile_storage_clk_mux_2x1	2:1 Mux for clock path
DWC_mobile_storage_and	AND gate used in clock path
DWC_mobile_storage_or	OR gate used in clock path
DWC_mobile_storage_intrcntl	Card SDIO Interrupt Control Unit
DWC_mobile_storage_muxdemux	Card SD Data/Command Mux/Demux Unit
DWC_mobile_storage_clkmux_interleave	Data interleaving module
DWC_mobile_storage_c2b2clk	CIU Domain-to-BIU Domain-to-Clock Signal Extender Unit
DWC_mobile_storage_cmdpath	Card Command Path Unit
DWC_mobile_storage_crc7	Card Command CRC7 Unit
DWC_mobile_storage_datapath	Card Data Path Unit
DWC_mobile_storage_autostop	Card Auto-Stop Generation Unit
DWC_mobile_storage_crc16	Card Data CRC16 Generation Unit
DWC_mobile_storage_datarx	Card Data Receive Unit
DWC_mobile_storage_rxfifowr	Card Data Receive FIFO Write Unit
DWC_mobile_storage_datatx	Card Data Transmit Unit
DWC_mobile_storage_txliford	Card Data Transmit FIFO Read Unit

@ __Conditionally instantiated, when H_BUS_TYPE = AHB

† __ Unencrypted File; & __ Conditionally instantiated, when FIFO_RAM_INSIDE = 1

^# __Conditionally instantiated when INTERNAL_DMAC = 1

3.1 Bus Interface Unit (BIU)

The BIU provides the following functions:

- ❖ Host interface
- ❖ DMA interface
- ❖ Interrupt control
- ❖ Register access
- ❖ FIFO access
- ❖ Power/pullup control and card detection

3.1.0.1 Host Interface Unit (HIU)

The Host Interface Unit (HIU) is an AHB/ APB slave interface, which provides the interface between the DWC_mobile_storage and the host bus. You can configure the host interface in coreConsultant as either an AMBA Advanced High-performance Bus (AHB) or an AMBA Advanced Peripheral Bus (APB); it provides a glueless interface to an AMBA AHB or an AMBA APB Bus.

The native interface inside the controller is an APB interface with two additional sideband features:

- ❖ Partial read/write support through byte-enable signals (pbe); not supported in a standard APB bus
- ❖ One-clock burst support to the data FIFOs

You invoke the burst access by keeping the APB enable signal (penable) continuously active. The burst support is provided to only the data FIFO address region. The register address region is accessed through only the standard two-clock APB access. When interfacing the controller to a standard APB bus, you should tie the byte-enable (pbe) signal high; by default, an APB bridge never keeps the penable signal continuously active.

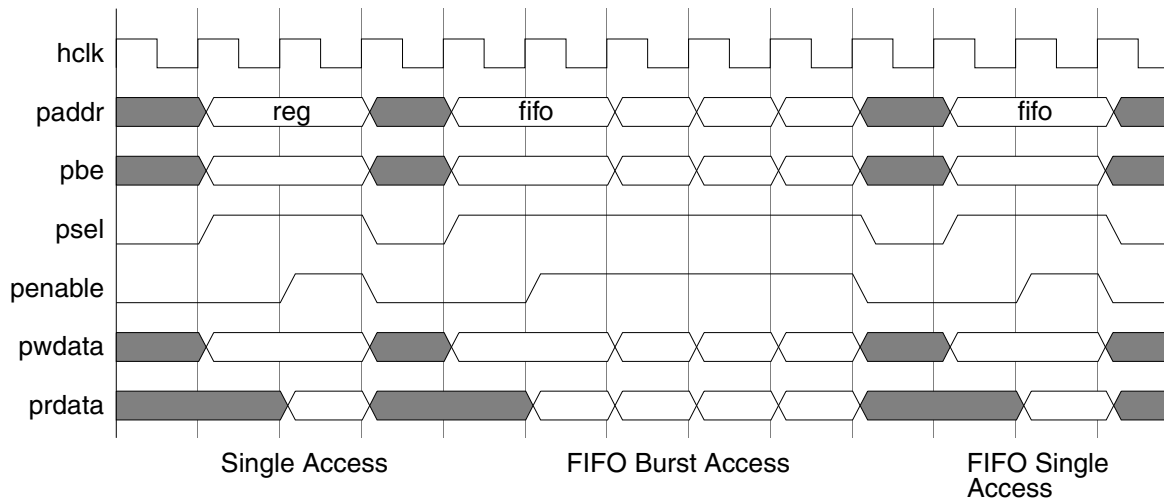
The APB interface with a sideband feature provides an interface to the controller; at the same time, it can interface to any other bus through a gasket without performance penalty or restrictions. The APB interface is always accessed in little-endian mode.

The pbe[0] bit qualifies pwwdata/prdata[7:0], pbe[1] qualifies pwwdata/prdata[15:8], and so on. In addition to the native APB with sideband interface, an AHB-to-APB gasket comes with the core. If you choose the AHB interface option, then the AHB-to-APB gasket is automatically instantiated inside the core.

Figure 3-1 shows the access to the internal registers and data FIFO using the native APB sideband feature. The register access takes two clocks, and the data FIFO can be accessed either in two-clock mode or in

one-clock-burst mode. The first transfer in a burst mode takes two clocks, and the rest of the transfers in the burst take only one clock each.

Figure 3-1 Native APB Sideband-Mode Access



The FIFO maps to an address offset that is greater than or equal to 0X200. While accessing the FIFO, you can set the address to any value 0X200 or greater; partial access to the FIFO is also supported. For example, if the AHB is 16 bits, you can access the FIFO with two 8-bit accesses. The lower address should be accessed first, and then the higher address, such as 0X200, and then 0X201.

During a read, the FIFO pop occurs only when the higher address is accessed. During a write, the lower bytes are put in a temporary register until the last higher byte arrives. When access to the higher byte is made, the temporary buffers and the active higher bytes are merged and written into the FIFO.

When the FIFO is accessed in a 16-bit AHB, address bits [7:1] are “don’t care.” Similarly, in a 32-bit AHB system, address bits [7:2] are “don’t care.”

Figure 3-2 illustrates the access to the registers and data FIFO in a standard APB system; both the register access and the data FIFO access take two clocks.

Figure 3-2 Standard APB-Mode Access (pbe inputs tied high)

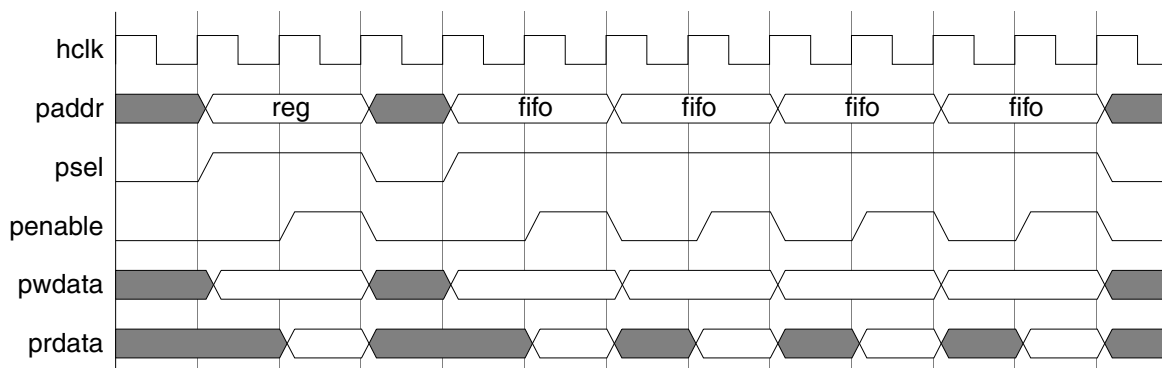
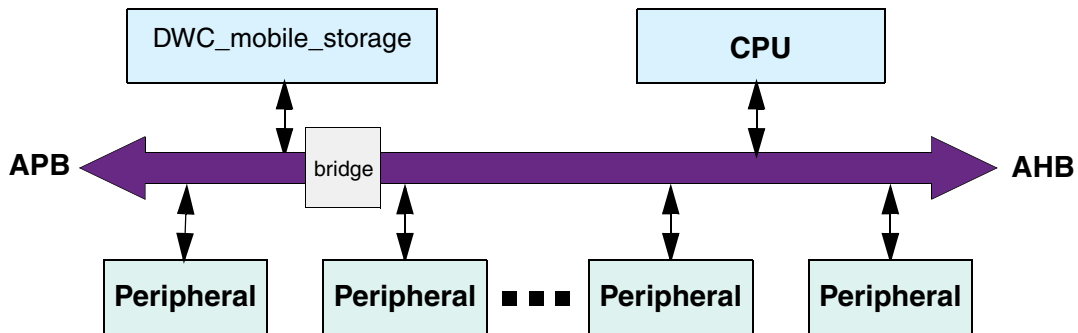


Figure 3-3 shows how the DWC_mobile_storage is used in an APB system.

Figure 3-3 DWC_mobile_storage in an APB System



In AHB accesses, a register access has one wait state. Access to the register area is broken into one-wait-state single accesses.

In burst accesses to the data FIFO area, the first data access incurs one wait state; consecutive accesses have no wait state. Figure 3-4 illustrates the AHB cycles.

Figure 3-4 AHB Access

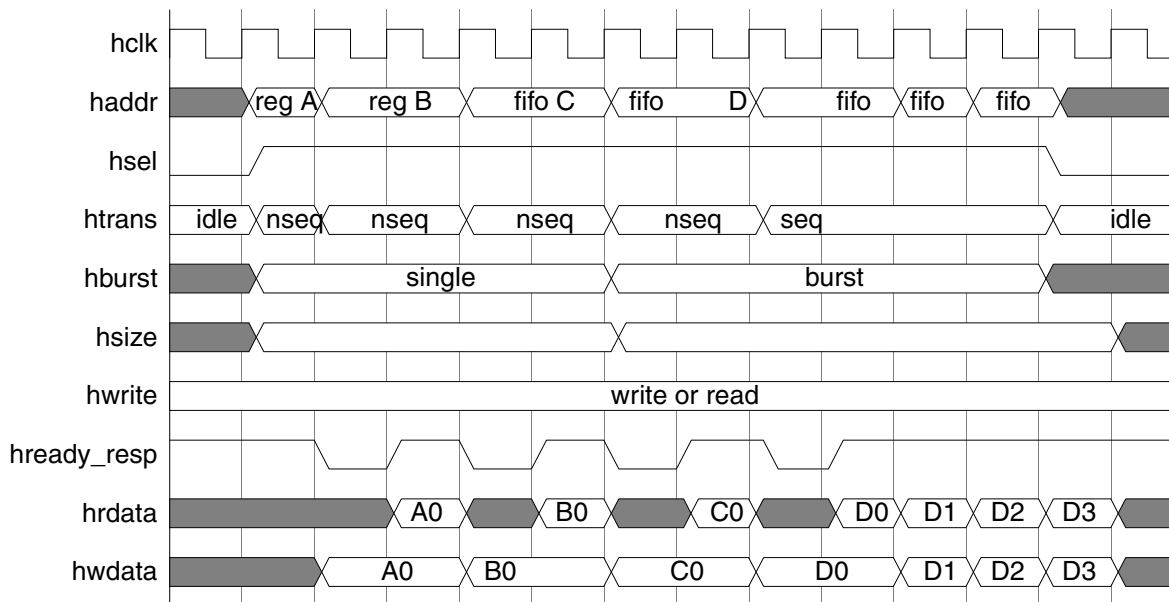
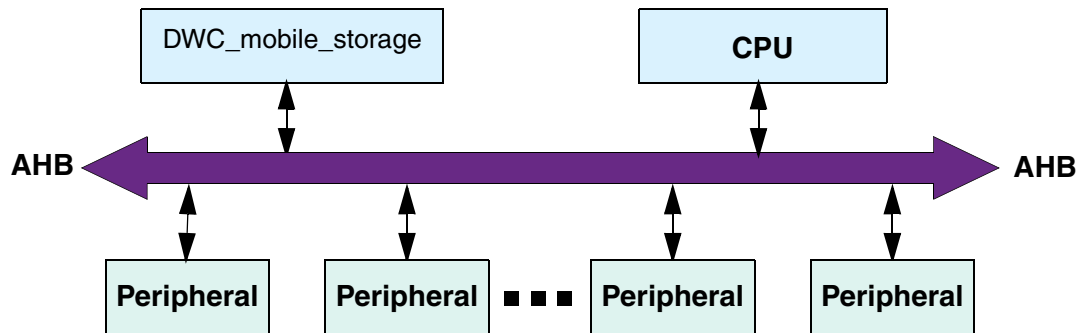


Figure 3-5 shows the DWC_mobile_storage in an AHB system.

Figure 3-5 DWC_mobile_storage in an AHB System (AHB Slave)



During partial accesses to the data FIFO area, the actual FIFO data pop or push occurs during the FIFO data width boundary access. For example, if the FIFO data width is 32, then the FIFO pop or push occurs when you do a 32-bit access, a 16-bit access with address[1] equal to 1'b1, or an 8-bit access with address[1:0] equal to 2'b11. Partial writes are stored in a temporary register until the FIFO data width boundary access. It is recommended that you do full bus-size accesses to the data FIFO. If you need to do partial bus-size accesses, you must first access the lower bytes.

When accessing the FIFO through burst access, the burst-type is “don’t care.” As long as the htrans=SEQ or back-to-back SINGLE accesses are to the FIFO, the DWC_mobile_storage does a 1-clock, 0-wait-state access.

Even though one address is enough to access the FIFO, more than one address ($\geq 0X200$) is mapped to the FIFO because:

- ❖ The AHB does not allow a burst to the same address.
- ❖ Although DMA controllers can be programmed to use one fixed address and the DWC_mobile_storage does a 0-wait-state access on a back-to-back single-FIFO access, the AHB arbiter may pull the “grant” to the DW_ahb_dmac after every single access.

The following are recommendations for using the DW_ahb_dmac:

- ❖ Map a larger memory region to the DWC_mobile_storage and program the DW_ahb_dmac to use all address ranges; use the INC address option on a burst.
- ❖ Map only a small memory region to the DWC_mobile_storage (Registers + FIFO Size) and have an option in the DW_ahb_dmac to auto preload the source/destination address after every dma-request.

You can apply a similar strategy when interfacing to a non-DesignWare DMA controller.

3.1.0.2 DMA Interface Unit

In addition to the AHB/APB host interface, you can choose one of the following types of optional DMA interfaces provided in the core.

- ❖ DW-DMA (DesignWare DW_ahb_dmac)
- ❖ Non-DW-DMA (non-DesignWare DMA)
- ❖ Generic DMA

3.1.0.2.1 DW-DMA (DesignWare DW_ahb_dmac) or Non-DW-DMA (Non-Designware DMA) Interface

DMA signals interface the DWC_mobile_storage to an external AHB/APB DMA controller to reduce the software overhead during FIFO data transfers. The DMA request/acknowledge handshake is used for only data transfers. The DMA interface provides a connection to the DesignWare DW_ahb_dmac. The DW-DMA interface is configured for Mode-B signaling, where the DW_ahb_dmac controller is the flow controller.

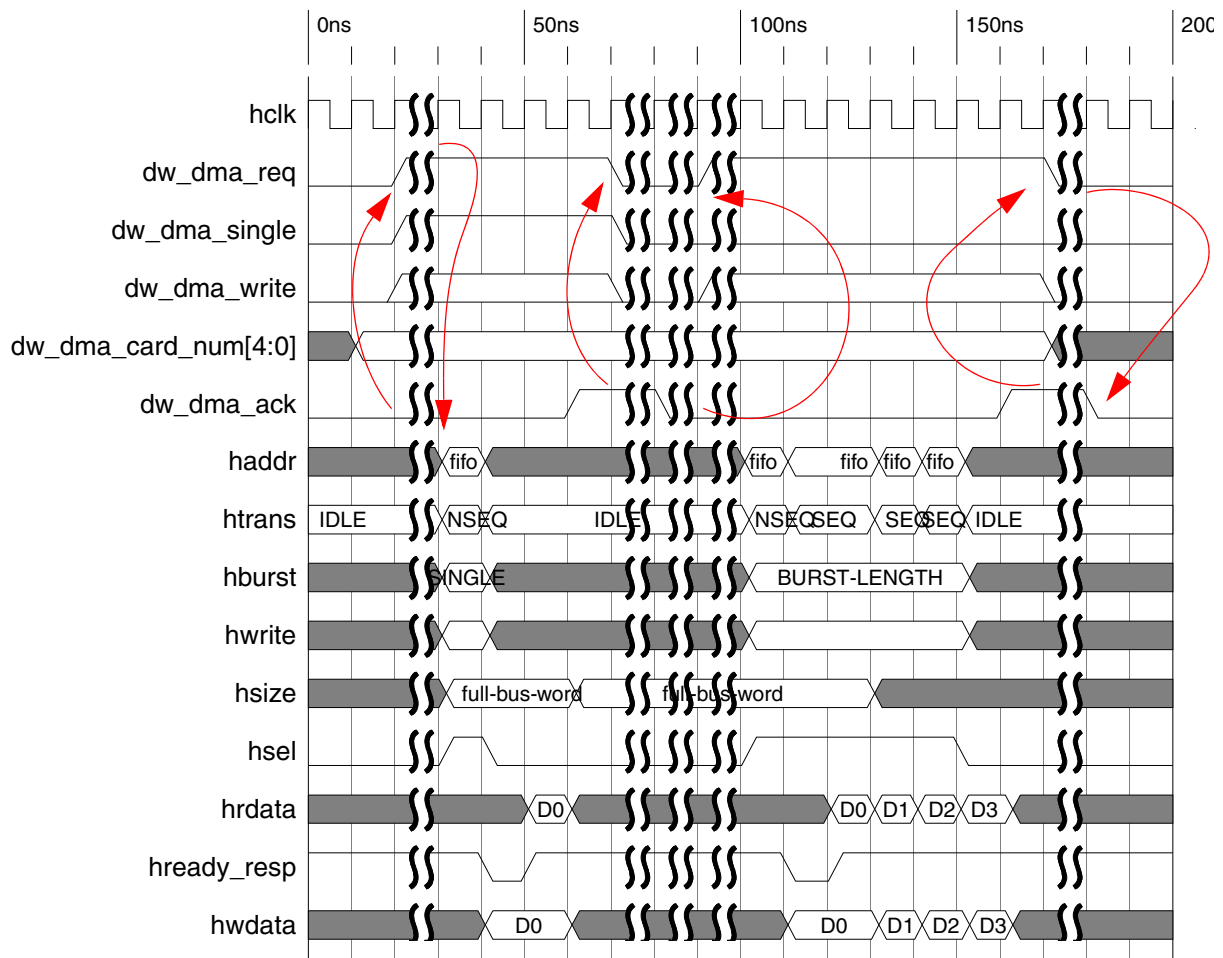
On seeing the DMA request, the DMA controller initiates accesses through the host interface to read or write into the data FIFO. The DWC_mobile_storage has FIFO transmit/receive watermark registers that you can set, depending on system latency. The DMA interface asserts the request in the following cases:

- ❖ Read from a card when the data FIFO word count exceeds the Rx-Watermark level
- ❖ Write to a card when the FIFO word count is less than or equal to the Tx-Watermark level

When the DMA interface is enabled, you can use normal host read/writes to access the data FIFOs. A 0 on the dw_dma_write signal indicates a write to the host memory, and a 1 indicates a read from the host-memory (0). The dma_single signal is always driven 0 when interfaced to the DW_ahb_dmac, since in Mode-B the DW_ahb_dmac transfers only the required number of data at the end of the transfer.

Figure 3-6 illustrates the DW-DMA/Non-DW-DMA interface cycles.

Figure 3-6 DW-DMA/Non-DW-DMA Waveform



Note: dw_dma_single is never driven high by DWC_mobile_storage when interfaced to DW_ahb_dmac controller. However, dw_dma_single is driven high for doing single transfers when interfaced to a non-DW-DMA.

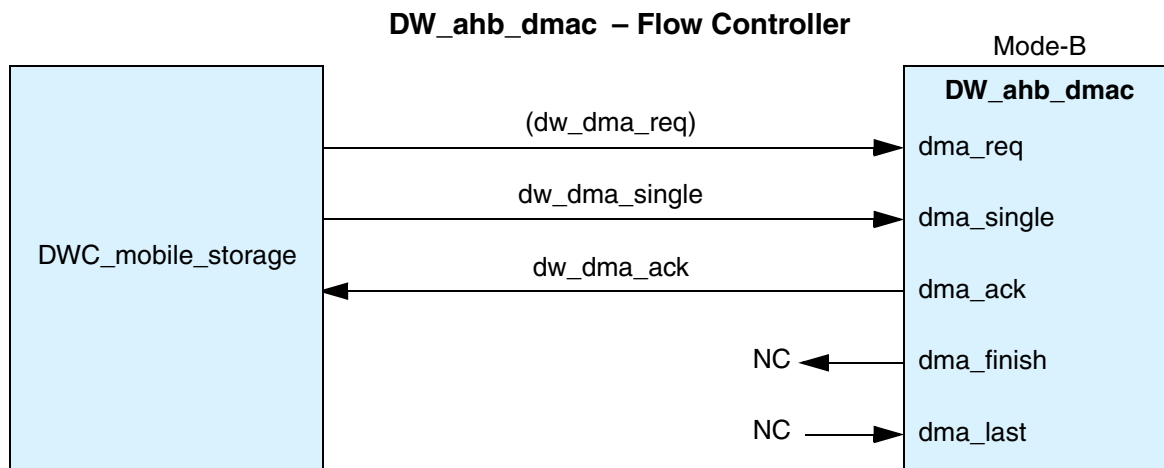
When interfacing to the DW_ahb_dmac, various equations apply, based on the situation.

❖ Interfacing to a single DMA channel:

```
assign dma_req[0] = dw_dma_req;
assign dma_single[0] = dw_dma_single;
assign dw_dma_ack = dma_ack[0];
```

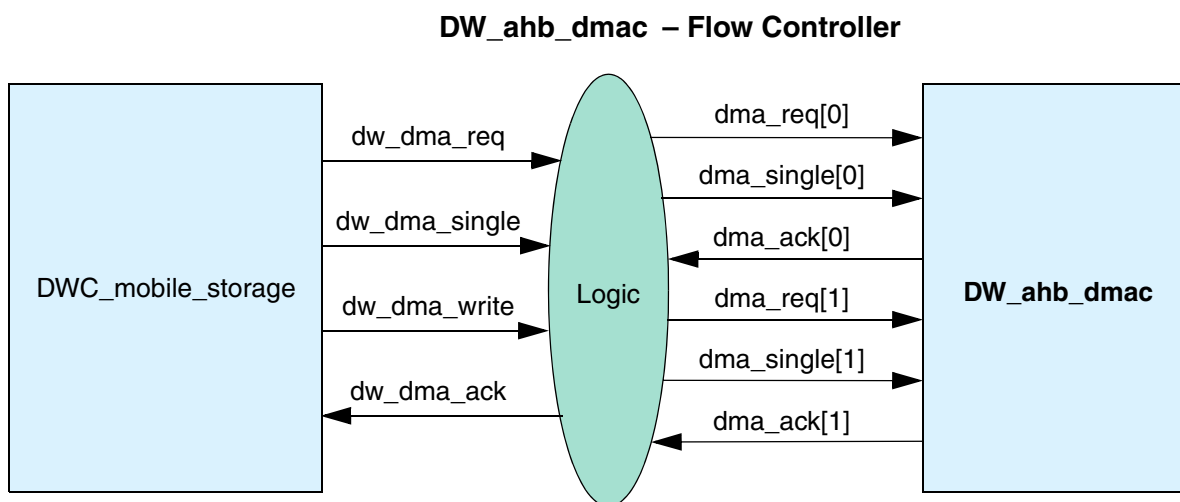
This scenario is illustrated in [Figure 3-7](#).

Figure 3-7 DWC_mobile_storage and DW_ahb_dmac Interface – Single Channel



- ❖ If separate DMA channels for read and write are used, then separate requests for read and write can be generated from **dw_dma_req** and **dw_dma_write**, as illustrated in [Figure 3-8](#).

Figure 3-8 DWC_mobile_storage and DW_ahb_dmac Interface – Two Channels

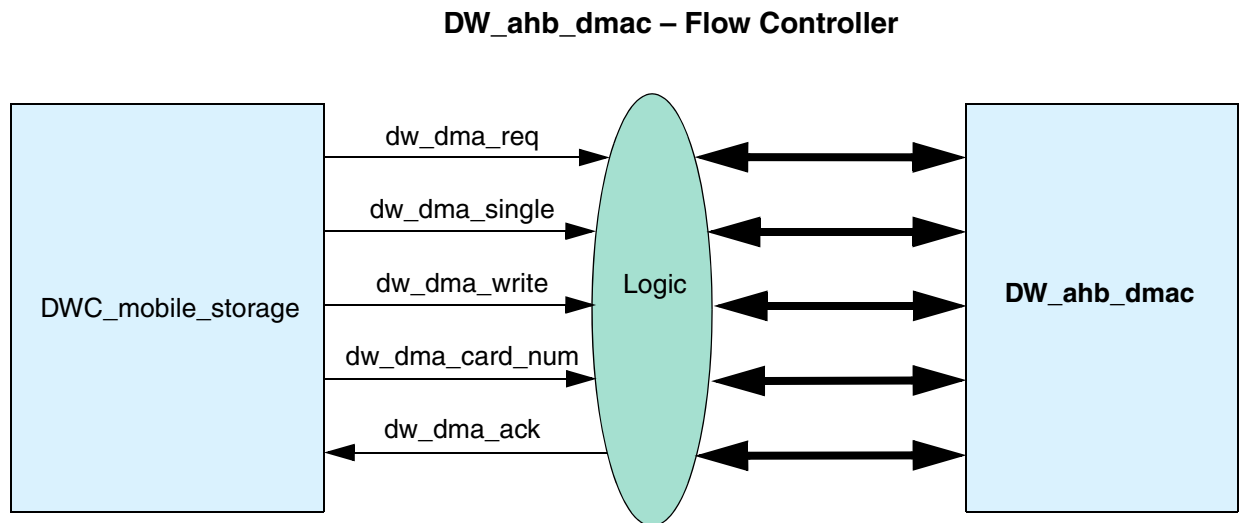


The following shows the corresponding code:

```
assign dma_req[0] = dw_dma_req & ~dw_dma_write;
assign dma_req[1] = dw_dma_req & dw_dma_write;
assign dma_single[0] = dw_dma_single & ~dw_dma_write;
assign dma_single[1] = dw_dma_single & dw_dma_write;
assign dw_dma_ack = dw_dma_write? dma_ack[1] : dma_ack[0];
```

- ❖ Similarly, if separate channels are needed for each card, the dma signals can be further qualified with the `dw_dma_card_num` signal, illustrated in [Figure 3-9](#).

Figure 3-9 DWC_mobile_storage and DW_ahb_dmac Interface – Multiple Channels



The following example shows separate channels for a two-card system:

```
assign dma_req[0] = dw_dma_req & ~dw_dma_write & (dw_dma_card_num = 0);
assign dma_req[1] = dw_dma_req & dw_dma_write & (dw_dma_card_num = 0);
assign dma_req[2] = dw_dma_req & ~dw_dma_write & (dw_dma_card_num = 1);
assign dma_req[3] = dw_dma_req & dw_dma_write & (dw_dma_card_num = 1);

assign dma_single[0] = dw_dma_single & ~dw_dma_write & (dw_dma_card_num = 0);
assign dma_single[1] = dw_dma_single & dw_dma_write & (dw_dma_card_num = 0);
assign dma_single[2] = dw_dma_single & ~dw_dma_write & (dw_dma_card_num = 1);
assign dma_single[3] = dw_dma_single & dw_dma_write & (dw_dma_card_num = 1);

assign dw_dma_ack = (dw_dma_card_num == 0) ? (dw_dma_write? dma_ack[1] :
dma_ack[0] : (dw_dma_write? dma_ack[3] : dma_ack[2]));
```

While interfacing to a DMA controller, you should choose Peripheral-to-Memory transaction and Memory-to-Peripheral under DMAC flow control as the transfer type. The `dma_single` signal is connected to the DMACSREQ of the DMA controller for a read from the `DWC_mobile_storage`.

Figure 3-10 illustrates the interface between the DWC_mobile_storage and non-DW-DMA controller.

Figure 3-10 DMAC Flow Controller

DW_mobile_storage and DMAC Interface – Single Channel

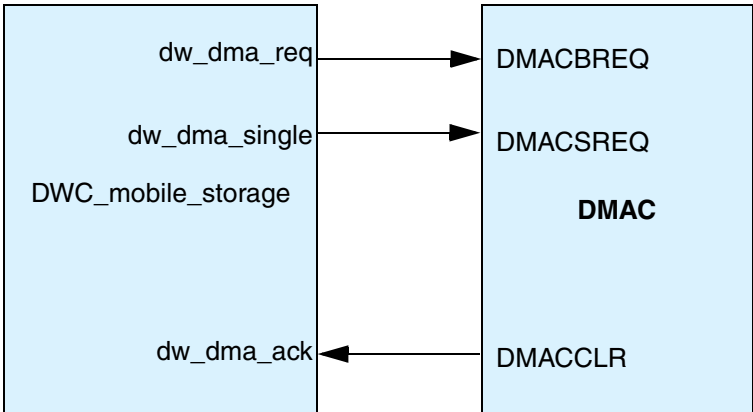


Figure 3-11 shows how the DWC_mobile_storage interfaces to a DW_ahb_dmac controller in an APB system.

Figure 3-11 DWC_mobile_storage in an APB System with DW_ahb_dmac Controller

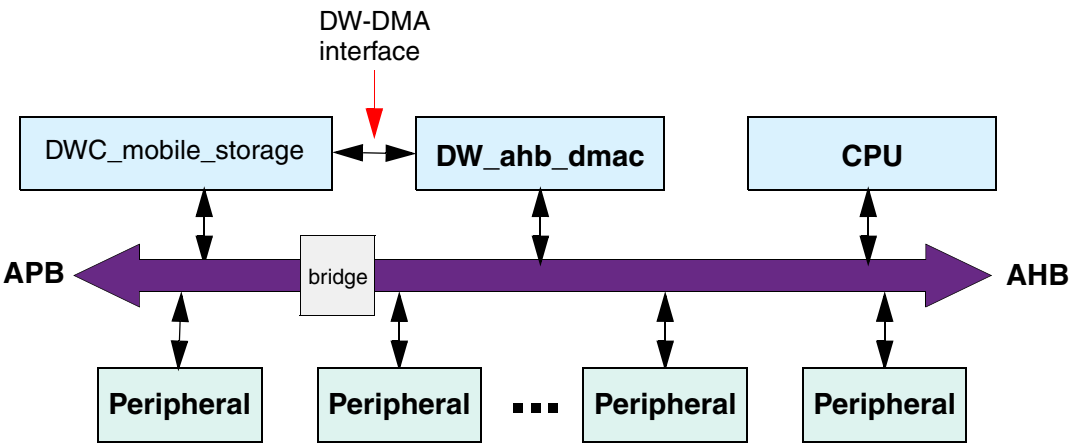
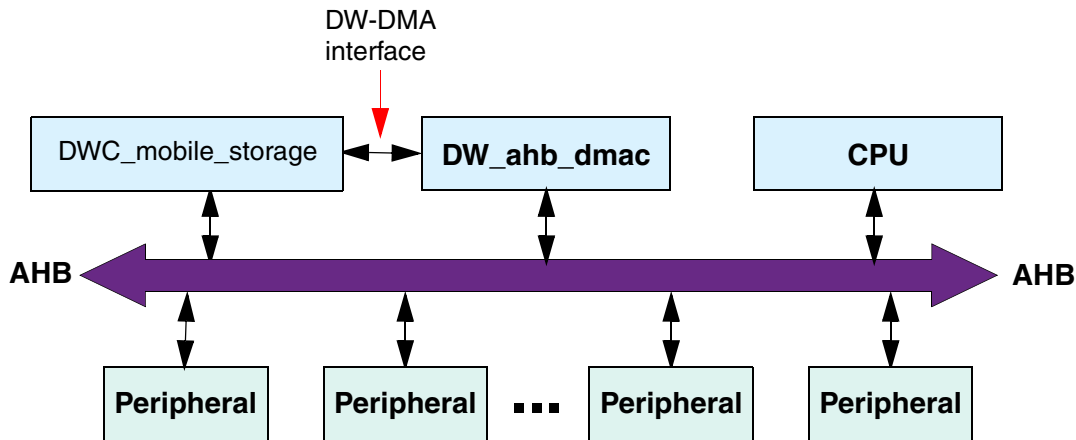


Figure 3-12 shows how the DWC_mobile_storage interfaces to a DW_ahb_dmac controller in an AHB system.

Figure 3-12 DWC_mobile_storage in an AHB System with DW_ahb_dmac Controller



Note The DWC_mobile_storage has not been verified with the DW_ahb_dmac controller and non-Synopsys DMAs. However, it has been verified with models that are functionally similar.

The following example shows how even a smaller FIFO_DEPTH can be sufficient to transfer data to or from an SD_MMC_CEATA card without interruption.

```
AHB_DATA_WIDTH = 32;
F_DATA_WIDTH = 32;
FIFO_DEPTH = 16;
Rx-Mark = 8;
Tx-Mark = 8;
DMA-BURST-LENGTH = 8;
AHB-Frequency = 100MHz;
SD-MMC-Frequency = 25MHz;
```

The following equations determine the number for the AHB clock latency that is allowed between the dma_req to dma_ack signals in order to have a no-wait-state data transfer:

$$\text{empty_fifo_locations} \times F_DATA_WIDTH \times \frac{AHB_Frequency}{SD_MMC_Frequency} \times \frac{1}{SD_Card_Data_width}$$

For a 1-bit card, the number of clock cycles is:

$$8 * 32 * 100 / 25 * 1 / 1 = 1024$$

Bus utilization by DWC_mobile_storage = $9 / 1024 * 100 = 0.88\%$; the rest is available to other components.

For a 4-bit card, the number of clock cycles is:

$$8 * 32 * 100 / 25 * 1 / 4 = 256$$

Bus utilization by DWC_mobile_storage = $9 / 256 * 100 = 3.52\%$; the rest is available for other components.

For example, when transferring data from a 4-bit SD card, as long as the dma_req to dma_ack latency is less than 256 clocks, the SD data transfer happens without interruption. Even if occasionally this condition is not met, the DWC_mobile_storage stops the card clock in order to avoid data loss; data transfer continues as soon as FIFO data is available.

Depending upon your system requirements, the FIFO_DEPTH and FIFO-Watermark levels can be picked for area-versus-performance trade-offs.

3.1.0.2.2 Generic DMA Interface

The generic DMA interface is similar to the DW-DMA interface, except that it has its own read and write data bus and the DMA data transfers do not share the host interface. This provides a simple request/acknowledge handshake interface. The data is transferred when both the request and acknowledge are valid. The `ge_dma_done` signal becomes valid for one clock after the CIU completes a write to a card during a write to an SD_MMC_CEATA card, and after the CIU completes a read from an SD_MMC_CEATA card and all the data from the FIFO are flushed to memory.

Similar to the DW-DMA request, the generic DMA request is also controlled by the FIFO transmit/receive watermark registers. This generic DMA interface provides architectural flexibility and can be used for:

- ❖ Putting the data-buffer RAMs close to the controller
- ❖ Adding DMA master capability
- ❖ Providing a card reader application

Even when the external DMA interface is enabled, the data FIFO can also be accessed through the host interface, provided that the host interface data width is smaller or equal to the FIFO interface data width. The `ge_dma_done` signal is active for one clock when the last transfer completes.

The `ge_dma_req` signal becomes active once the FIFO-Watermark condition is met. The signal stays active until all FIFO contents are emptied when writing to memory and the FIFO is full when reading from memory. During a write to the cards, `ge_dma_done` is asserted for one clock as soon as the CIU completes the transfer to the SD_MMC_CEATA cards. During a read from the cards, `ge_dma_done` is asserted active for one clock only when the CIU completes the transfer and when the FIFO becomes empty. During a data transfer, if the `dma-reset` bit is set in the control register, then `ge_dma_done` becomes active for one clock.

Figure 3-13 illustrates the generic DMA interface cycles.

Figure 3-13 Generic DMA Interface

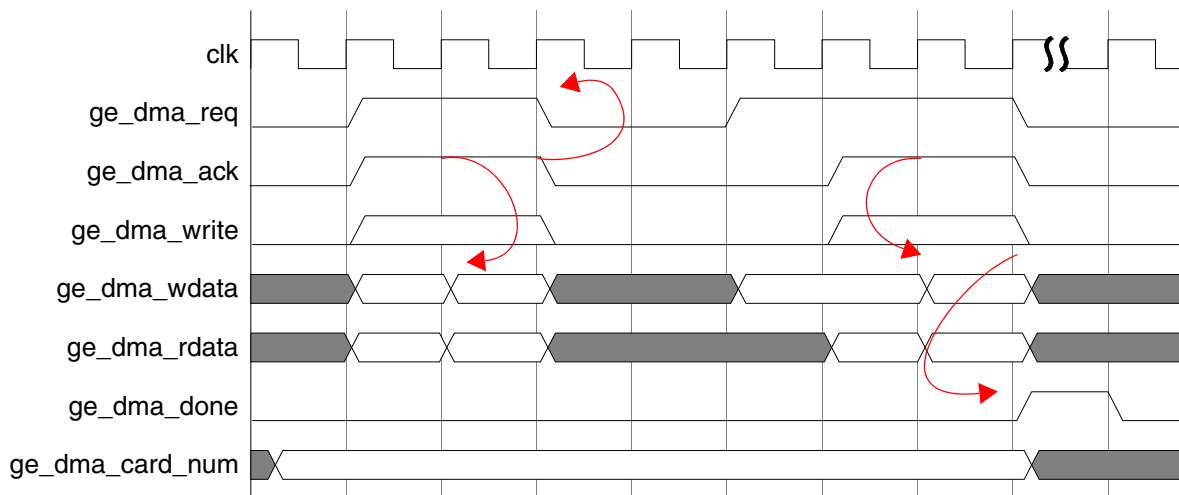


Figure 3-14 and Figure 3-15 show how the generic DMA interface can be used.

Figure 3-14 DWC_mobile_storage with Optional Generic DMA Interface

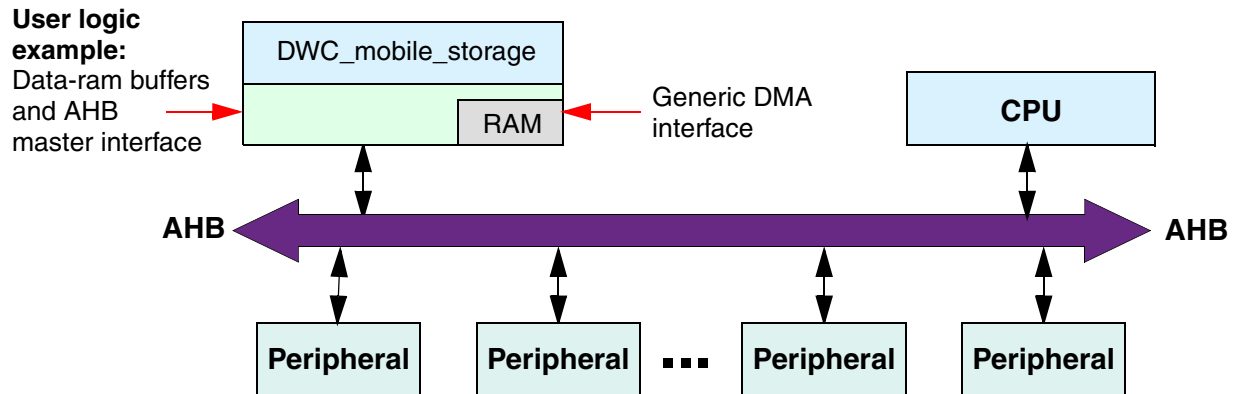
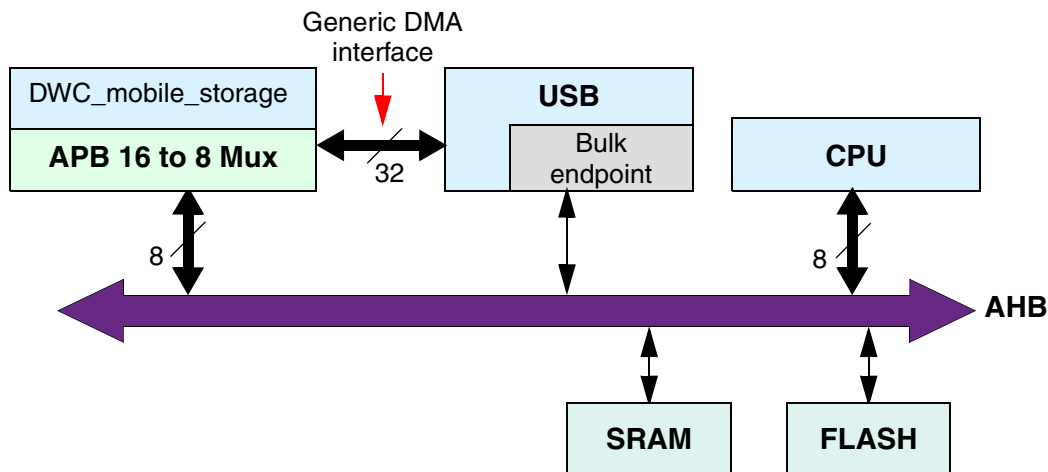


Figure 3-15 DWC_mobile_storage with USB Reader Application



3.1.0.3 Register Unit

The register unit is part of the bus interface unit (BIU); it provides read and write access to the registers. For details about each register and programming model, refer to “Registers” on page 133.

Since the DWC_mobile_storage supports up to a maximum host bus width of 64 bits, the registers are internally implemented to be 64-bits wide. Except for the TCBCNT and TBBCNT registers, all other registers can be accessed in 8, 16, 32, or 64 bits, depending on the host bus width and byte-enable control. The Transferred CIU Card Byte Count (TCBCNT) and Transferred BIU FIFO Byte Count (TBBCNT) registers are 32-bit counters. When the H_DATA_WIDTH is 32 or 64, these registers should be read either in 32-bit or 64-bit access.

For example, the read value would be 32'h0001_ffff and therefore wrong if all of these conditions exist:

- ❖ TBBCNT is 32'h0000_ffff
- ❖ TBBCNT is accessed by two 16-bit partial accesses
- ❖ Counter is updated between the two accesses

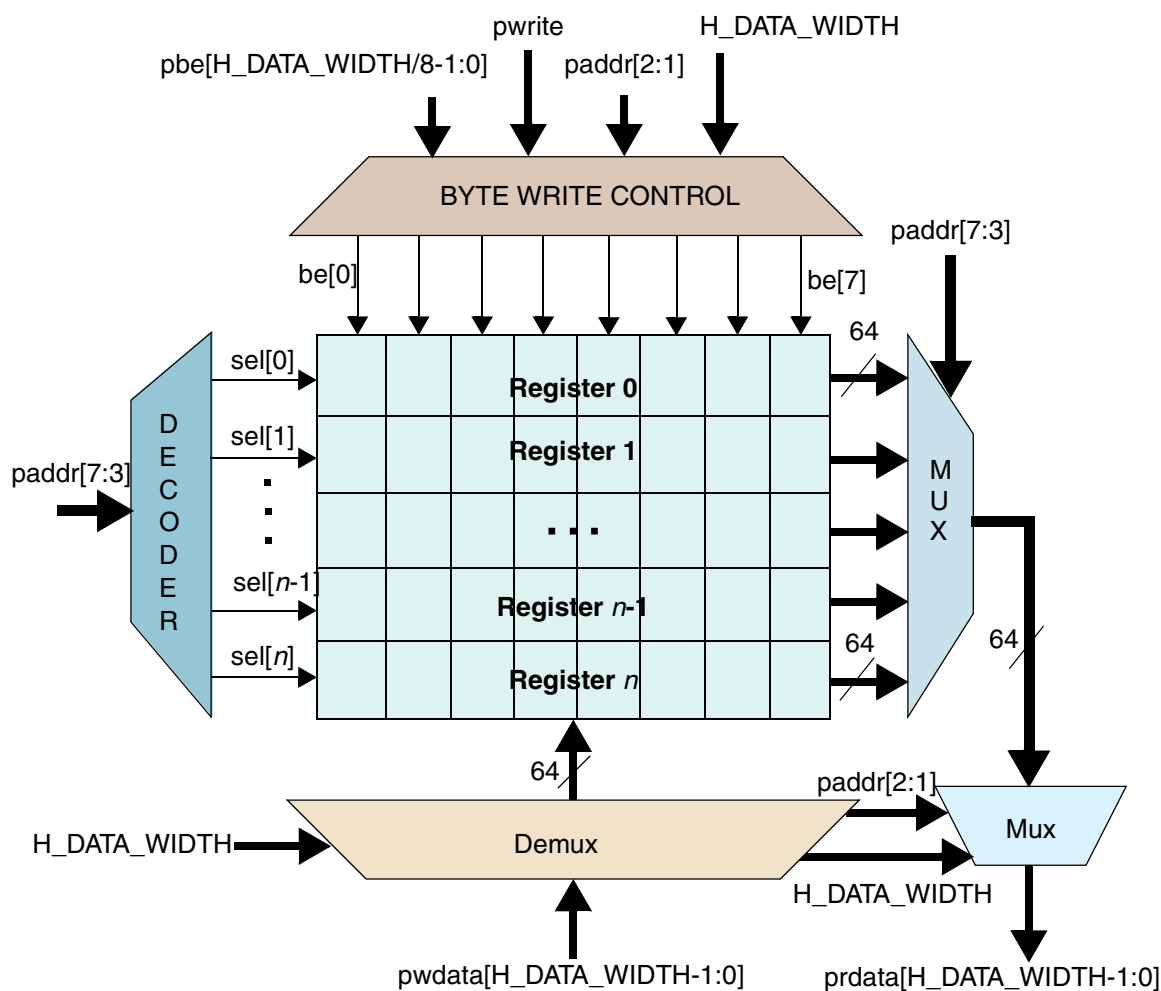
If TBBCNT is read instead by a 32-bit access, the read value would be either 32'h0000_ffff or 32'h0001_0000.

When H_DATA_WIDTH is 16, a 16-bit coherency register is implemented for read access. When a read occurs to these registers with $\text{addr}[1] = 0$, the upper 16 bits of the register is stored in the temporary coherency register. The next time a read occurs to the register with $\text{addr}[1] = 1$, the coherency register value is sent to the host bus. The software should read:

- ❖ All 16 bits at one time
- ❖ Lower 16 bits first
- ❖ Higher 16 bits next

Figure 3-16 shows how the registers are implemented inside the core.

Figure 3-16 Logic Diagram of Internal Register Implementation



Note: H_DATA_WIDTH = 16, 32, or 64

All registers reside in the BIU clock domain. When a command is sent to a card by setting the `start_bit`, which is bit[31] of the `CMD` register, all relevant registers needed for the CIU operation are transferred to the CIU block. During this time, the registers that are transferred from the BIU to the CIU should not be written. The software should wait for the hardware to clear the `start_bit` before writing to these registers again. The register unit has a hardware locking feature to prevent illegal writes to registers. The lock is necessary in order to avoid metastability violations, both because the host and card clock domains are different and to prevent illegal software operations.

Once a command start is issued by setting the start_bit of the CMD register, the following registers cannot be reprogrammed until the command is accepted by the card interface unit (CIU):

- ❖ **CMD** – Command
- ❖ **CMDARG** – Command Argument
- ❖ **BYTCNT** – Byte Count
- ❖ **BLKSIZ** – Block Size
- ❖ **CLKDIV** – Clock Divider
- ❖ **CLKENA** – Clock Enable
- ❖ **CLKSRC** – Clock Source
- ❖ **TMOUT** – Timeout
- ❖ **CTYPE** – Card Type

The hardware resets the start_bit once the CIU accepts the command. If a host write to any of these registers is attempted during this locked time, then the write is ignored and the hardware lock error bit is set in the raw interrupt status register. Additionally, if the interrupt is enabled and not masked for a hardware lock error, then an interrupt is sent to the host.

When the CIU is in an idle state, it typically takes the following number of clocks for the command handshake, where clk is the BIU clock and cclk_in is the CIU clock:

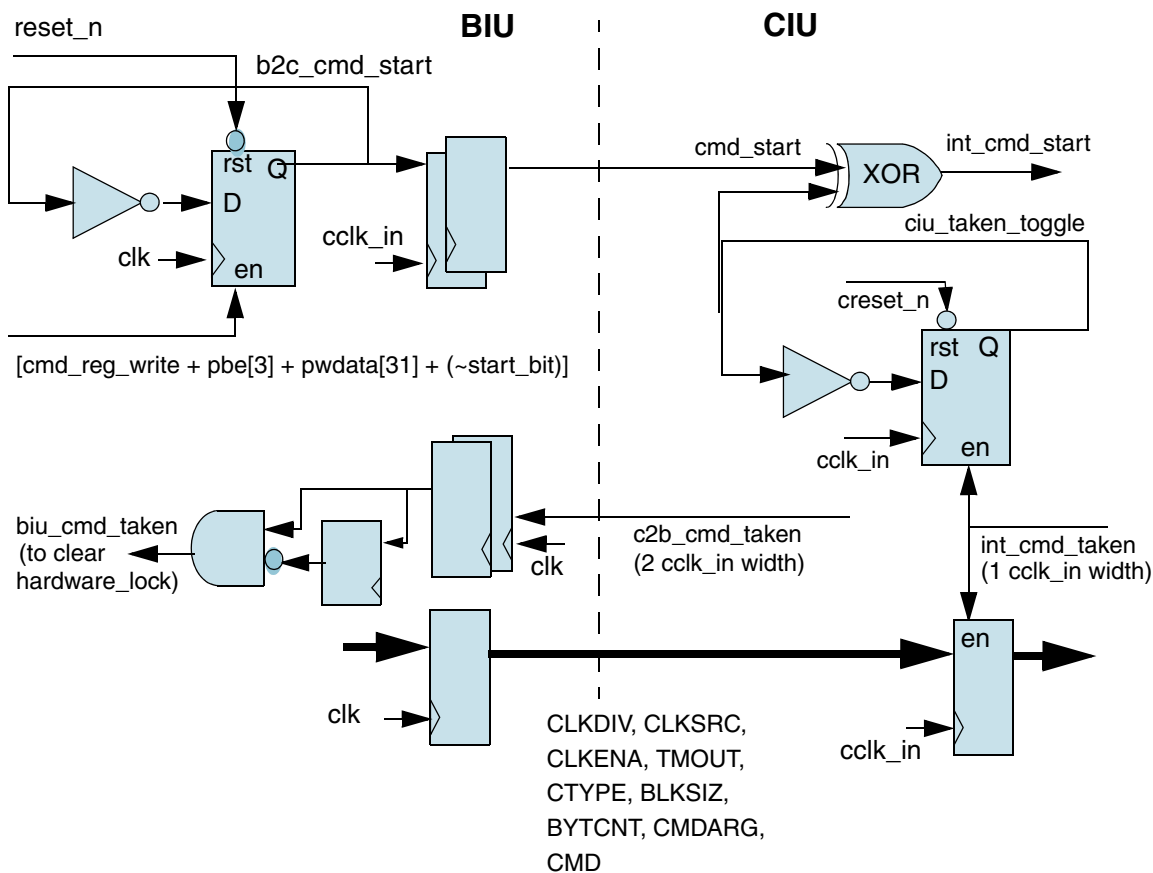
$$3 \text{ (clk)} + 3 \text{ (cclk_in)}$$

Once a command is accepted, you can send another command to the CIU – which has a one-deep command queue – under the following conditions:

- ❖ If the previous command was not a data transfer command, the new command is sent to the SD_MMC_CEATA card once the previous command completes.
- ❖ If the previous command is a data transfer command and if wait_prvdata_complete (bit[13]) of the Command register is set for the new command, the new command is sent to the SD_MMC_CEATA card only when the data transfer completes.
- ❖ If the wait_prvdata_complete is 0, then the new command is sent to the SD_MMC_CEATA card as soon as the previous command is sent. Typically, you should use this only to stop or abort a previous data transfer or query the card status in the middle of a data transfer.

Figure 3-17 illustrates the diagram of the BIU/CIU handshake for a 32-bit APB system.

Figure 3-17 BIU to CIU cmd_start/cmd_taken Handshake



```
hardware_lock_error <= start_bit + reg_write + address(CLKDIV, CLKSRC, CLKENA, TMOUT, CTYPE,
BLKSIZ, BYTCNT, CMDARG, or CMD)
```

```
if(biu_cmd_taken | ciu_reset)
    start_bit <= 1'b0;
else if(ccmd_reg_write & pbe[3] & pwwdata[31] & ~start_bit)
    start_bit <= 1'b1;
```

Figure 3-18 illustrates the timing diagram for the BIU/CIU handshake of a 32-bit APB system.

Figure 3-18 BIU to CIU Handshake

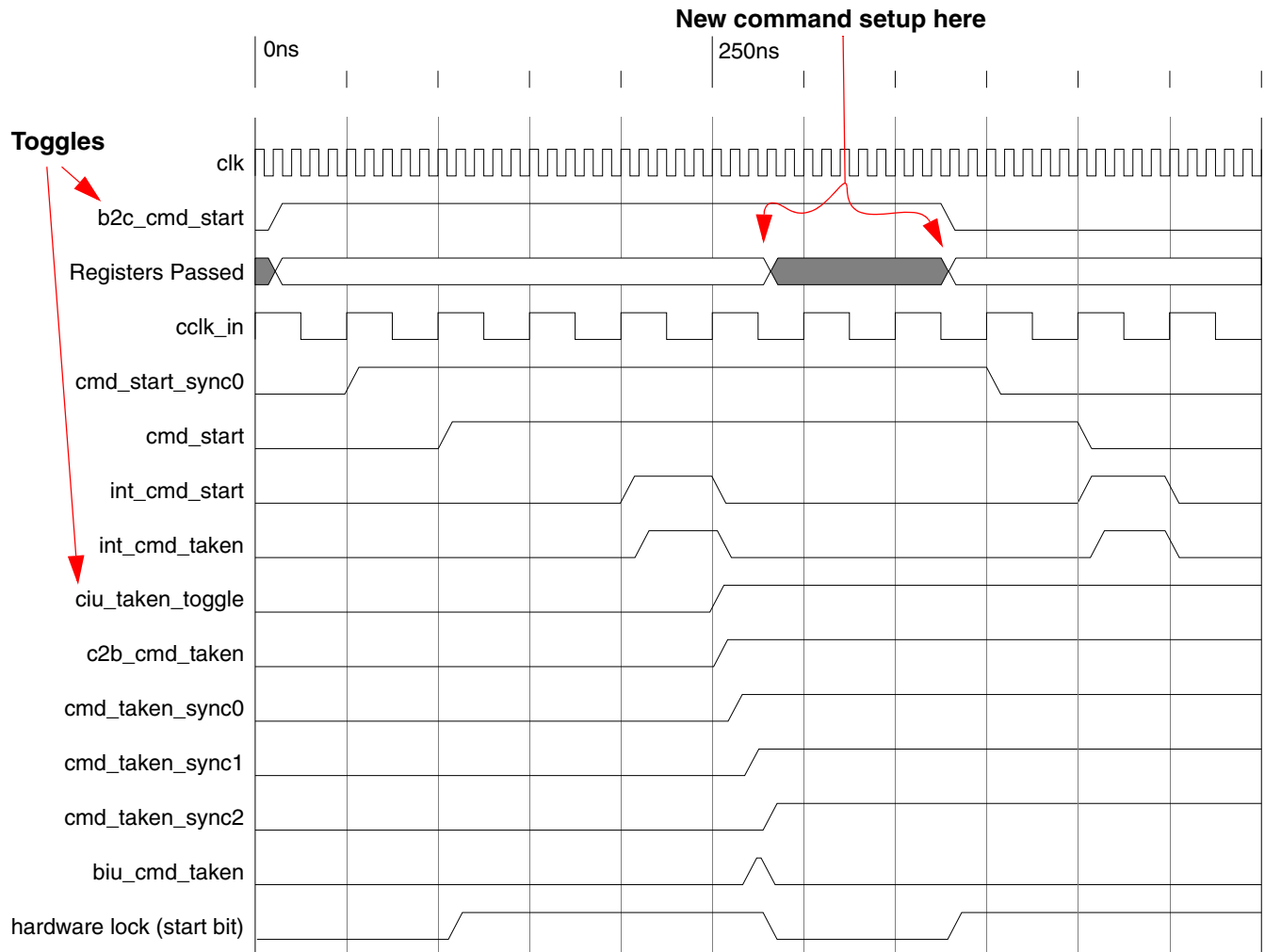
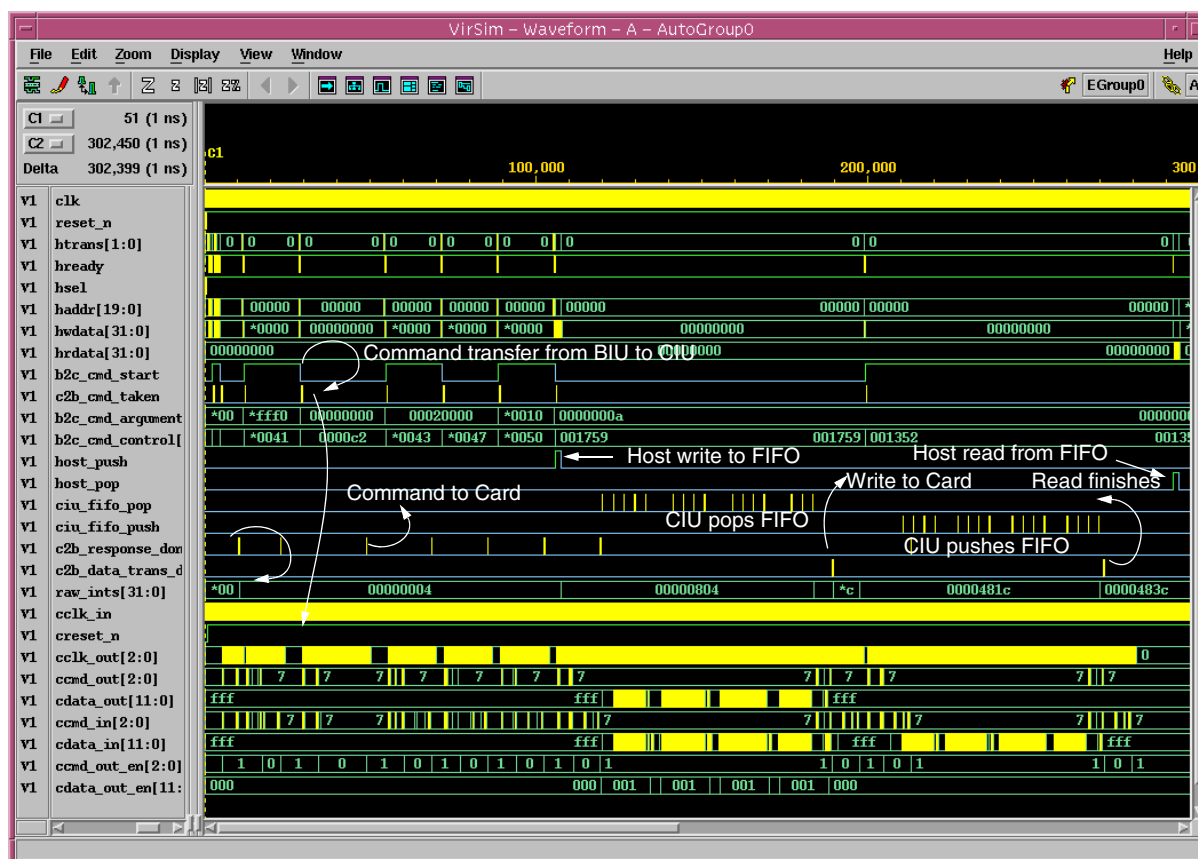


Figure 3-19 shows the host write and read to an MMC card.



In order to see the details in Figure 3-19 more clearly, you can print the waveform.

Figure 3-19 End-to-End Waveform



1. The command is transferred from the BIU to the CIU through the b2c_cmd_start/c2b_cmd_taken handshake.
2. Once the command is sent to the card and a response is received from the card, c2b_response_done is asserted by the CIU, which sets an interrupt bit.
3. During a write to a card, the host pushes data into the FIFO, and the CIU pops the data and sends it to the card. Once the data transfer is over, the CIU asserts c2b_data_trans_done, which sets another interrupt bit.
4. Similarly for a read from a card, the CIU reads the data from the card and pushes it to the FIFO, where the host pops the data from the FIFO. When the last data is pushed, the CIU asserts c2b_data_trans_done, which sets an interrupt bit.

3.1.0.4 Interrupt Controller Unit

The interrupt controller unit generates an interrupt that depends on the controller raw interrupt status, the interrupt-mask register, and the global interrupt-enable register bit. Once an interrupt condition is detected, it sets the corresponding interrupt bit in the raw interrupt status register. The raw interrupt status bit stays on until the software clears the bit by writing a 1 to the interrupt bit; a 0 leaves the bit untouched.

The interrupt port, `int`, is an active-high, level-sensitive interrupt. The interrupt port is active only when any bit in the raw interrupt status register is active, the corresponding interrupt mask bit is 1, and the global interrupt enable bit is 1. The interrupt port is registered in order to avoid any combinational glitches.

The following bits are available as top-level ports for debug purposes:

- ❖ Interrupt mask bits (`int_mask_n[31:0]`)
- ❖ Raw interrupt status bits (`raw_ints[31:0]`)
- ❖ Interrupt enable bit (`int_enable`)

The `int_enable` is reset to 0 on power-on, and the interrupt mask bits are set to 32'h0, which masks all the interrupts. The following equation shows how the interrupt is generated:

```
always @ (posedge clk or negedge reset_n)
  if (~reset_n) int <= 1'b0;
  else int <= |(raw_ints & int_mask_n) & int_enable;
```



Note

Before enabling the interrupt, it is always recommended that you write 32'hfff_fff to the raw interrupt status register in order to clear any pending unserviced interrupts. When clearing interrupts during normal operation, ensure that you clear only the interrupt bits that you serviced.

Table 3-2 describes the different interrupt bits in the Interrupt Status register.

Table 3-2 Bits in Interrupt Status Register

Bits	Interrupt	Description
31:16	SDIO Interrupts	Interrupts from SDIO cards; one bit for each card. Bit[31] corresponds to Card[15], and bit[16] corresponds to Card[0]. In MMC-Ver3.3-only mode, these bits are always 0.
15	End Bit Error (read)/Write no CRC (EBE)	Error in end-bit during read operation, or no data CRC received during write operation. • NOTE: For MMC CMD19, there may be no CRC status returned by the card. Hence, EBE is set for CMD19. The application should not treat this as an error.
14	Auto Command Done (ACD)	Stop/abort commands automatically sent by card unit and not initiated by host; similar to Command Done (CD) interrupt. • Recommendation: Software typically need not enable this for non CE-ATA accesses; Data Transfer Over (DTO) interrupt that comes after this interrupt determines whether data transfer has correctly competed. For CE-ATA accesses, if the software sets <code>send_auto_stop_ccsd</code> bit in the control register, then software should enable this bit.
13	Start Bit Error (SBE)	Error in data start bit when data is read from a card. In 4-bit mode, if all data bits do not have start bit, then this error is set.
12	Hardware Locked write Error (HLE)	During hardware-lock period, write attempted to one of locked registers.

Table 3-2 Bits in Interrupt Status Register (Cont.)

Bits	Interrupt	Description
11	FIFO Underrun/Overrun Error (FRUN)	Host tried to push data when FIFO was full, or host tried to read data when FIFO was empty. Typically this should not happen, except due to error in software. Card unit never pushes data into FIFO when FIFO is full, and pop data when FIFO is empty. If IDMAC (Internal Direct Memory Access Controller) is enabled, FIFO underrun/overflow can occur due to a programming error on MSIZE and watermark values in FIFOTH register; for more information, refer to “Internal Direct Memory Access Controller (IDMAC)” on page 69.
10	Data Starvation by Host Timeout (HTO)	To avoid data loss, card clock out (cclk_out) is stopped if FIFO is empty when writing to card, or FIFO is full when reading from card. Whenever card clock is stopped to avoid data loss, data-starvation timeout counter is started with data-timeout value. This interrupt is set if host does not fill data into FIFO during write to card, or does not read from FIFO during read from card before timeout period. Even after timeout, card clock stays in stopped state, with CIU state machines waiting. It is responsibility of host to push or pop data into FIFO upon interrupt, which automatically restarts cclk_out and card state machines. Even if host wants to send stop/abort command, it still needs to ensure it has to push or pop FIFO so that clock starts in order for stop/abort command to send on cmd signal along with data that is sent or received on data line.
9	Data Read Timeout (DRT0))/ Boot Data Start (BDS)	- In Normal functioning mode: Data read timeout (DRT0) Data timeout occurred. Data Transfer Over (DTO) also set if data timeout occurs. - In Boot Mode: Boot Data Start (BDS) When set, indicates that DWC_mobile_storage has started to receive boot data from the card. A write to this register with a value of 1 clears this interrupt.
8	Response Timeout (RTO)/ Boot Ack Received (BAR)	- In Normal functioning mode: Response timeout (RTO) Response timeout occurred. Command Done (CD) also set if response timeout occurs. If command involves data transfer and when response times out, no data transfer is attempted by DWC_mobile_storage. - In Boot Mode: Boot Ack Received (BAR) When expect_boot_ack is set, on reception of a boot acknowledge pattern—0-1-0—this interrupt is asserted. A write to this register with a value of 1 clears this interrupt.
7	Data CRC Error (DCRC)	Received Data CRC does not match with locally-generated CRC in CIU; expected when a negative CRC is received.
6	Response CRC Error (RCRC)	Response CRC does not match with locally-generated CRC in CIU.

Table 3-2 Bits in Interrupt Status Register (Cont.)

Bits	Interrupt	Description
5	Receive FIFO Data Request (RXDR)	Interrupt set during read operation from card when FIFO level is greater than Receive-Threshold level. Recommendation: In DMA modes, this interrupt should not be enabled. ISR, in non-DMA mode: <pre>pop RX_WMark + 1 data from FIFO</pre>
4	Transmit FIFO Data Request (TXDR)	Interrupt set during write operation to card when FIFO level reaches less than or equal to Transmit-Threshold level. Recommendation: In DMA modes, this interrupt should not be enabled. ISR in non-DMA mode: <pre>if (pending_bytes > \ (FIFO_DEPTH - TX_WMark)) push (FIFO_DEPTH - \ TX_WMark) data into FIFO else push pending_bytes data \ into FIFO</pre>
3	Data Transfer Over (DTO)	Data transfer completed, even if there is Start Bit Error or CRC error. This bit is also set when “read data-timeout” occurs or CCS is sampled from CE-ATA device. Recommendation: In non-DMA mode, when data is read from card, on seeing interrupt, host should read any pending data from FIFO. In DMA mode, DMA controllers guarantee FIFO is flushed before interrupt. NOTE: DTO bit is set at the end of the last data block, even if the device asserts MMC busy after the last data block.
2	Command Done (CD)	Command sent to card and got response from card, even if Response Error or CRC error occurs. Also set when response timeout occurs or CCSD sent to CE-ATA device.
1	Response Error (RE)	Error in received response set if one of following occurs: - Transmission bit != 0 - Command index mismatch - End-bit != 1
0	Card-Detect (CDT)	When one or more cards inserted or removed, this interrupt occurs. Software should read card-detect register (CDETECT, 0x50) to determine current card status. Recommendation: After power-on and before enabling interrupts, software should read card detect register and store it in memory. When interrupt occurs, it should read card detect register and compare it with value stored in memory to determine which card(s) were removed/inserted. Before exiting ISR, software should update memory with new card-detect value.



The SDIO Interrupts, Receive FIFO Data Request (RXDR), and Transmit FIFO Data Request (TXDR) are set by level-sensitive interrupt sources. Therefore, the interrupt source should be first cleared before you can clear the interrupt bit of the Raw Interrupt register. For example, on seeing the Receive FIFO Data Request (RXDR) interrupt, the FIFO should be emptied so that the “FIFO count greater than the RX-Watermark” condition, which triggers the interrupt, becomes inactive. The rest of the interrupts are triggered by a single clock-pulse-width source.

3.1.0.5 FIFO Controller Unit

The FIFO controller interfaces the internal FIFO to the host/DMA interface and the card controller unit. The FIFO depth can be configured between 4 to 4096. It is recommended that you use a smaller FIFO under these conditions:

- ❖ In a low-area design
- ❖ In a design that has a DMA
- ❖ If the host has a high bandwidth with low latency

In a non-DMA design, you should use a larger FIFO that is at least equal to a sector size in order to avoid too many FIFO watermark interrupt overheads. When buffer overrun and underrun conditions occur, the card clock stops in order to avoid data loss; it also allows interfacing to a host, where the host response time is high.

Since write and read transfers to the cards do not simultaneously occur, a single shared FIFO is used for read and write operations in order to save area. During the cmd_taken handshake, the read/write bit[10] of the Command register is sampled and stored. This internal bit defines whether the DWC_mobile_storage is in read or write mode.

The FIFO uses a two-clock synchronous read and synchronous write dual-port RAM. One of the ports is connected to the host clock, clk, and the second port is connected to the card clock, cclk_in.

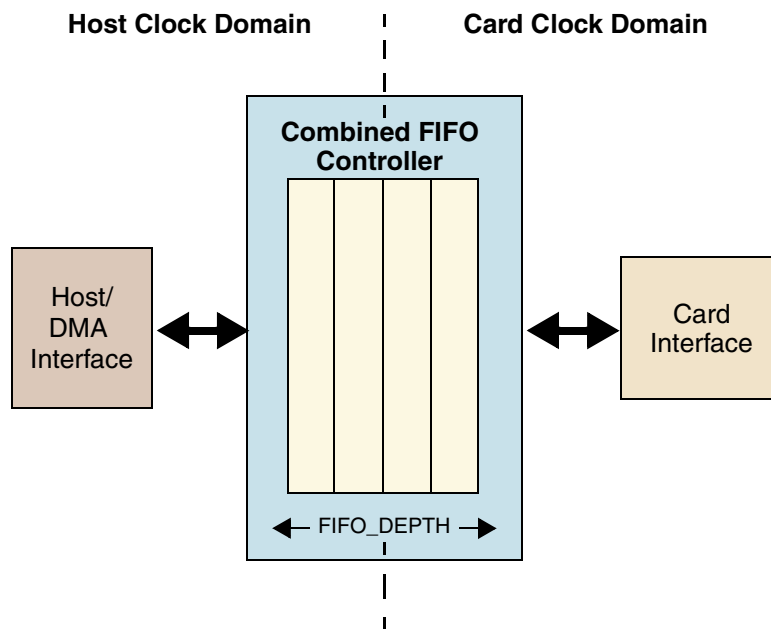
When the FIFO_RAM_INSIDE parameter is 1, the conditionally instantiated DWC_mobile_storage_dssram dual-port RAM in the top level is used. The default FIFO RAM is synthesized using registers, but can be replaced with a vendor FAB-specific dual-port RAM. When FIFO_RAM_INSIDE = 0, the RAM interface port is brought to the top level, so an external dual-port RAM can be interfaced to the DWC_mobile_storage core. For details on the dual port SSRAM, refer to the DWC_mobile_storage_2clk_dssram.v file in the *workspace/src* directory.

The FIFO data width can be either 32 bits or 64 bits, depending on your configuration. When the DW_ahb_dmac option is not chosen, the FIFO data width is 32 when H_DATA_WIDTH is either 16 or 32 bits. The FIFO data width is set to 64 when H_DATA_WIDTH is 64 bits.

When the generic DMA option is chosen, the FIFO data width is 32 when GE_DMA_DATA_WIDTH is either 16 or 32 bits, and the FIFO data width is set to 64 when GE_DMA_DATA_WIDTH is 64 bits.

Figure 3-20 illustrates the structure of a combined transmit and receive FIFO.

Figure 3-20 DWC_mobile_storage – Combined Transmit and Receive FIFO



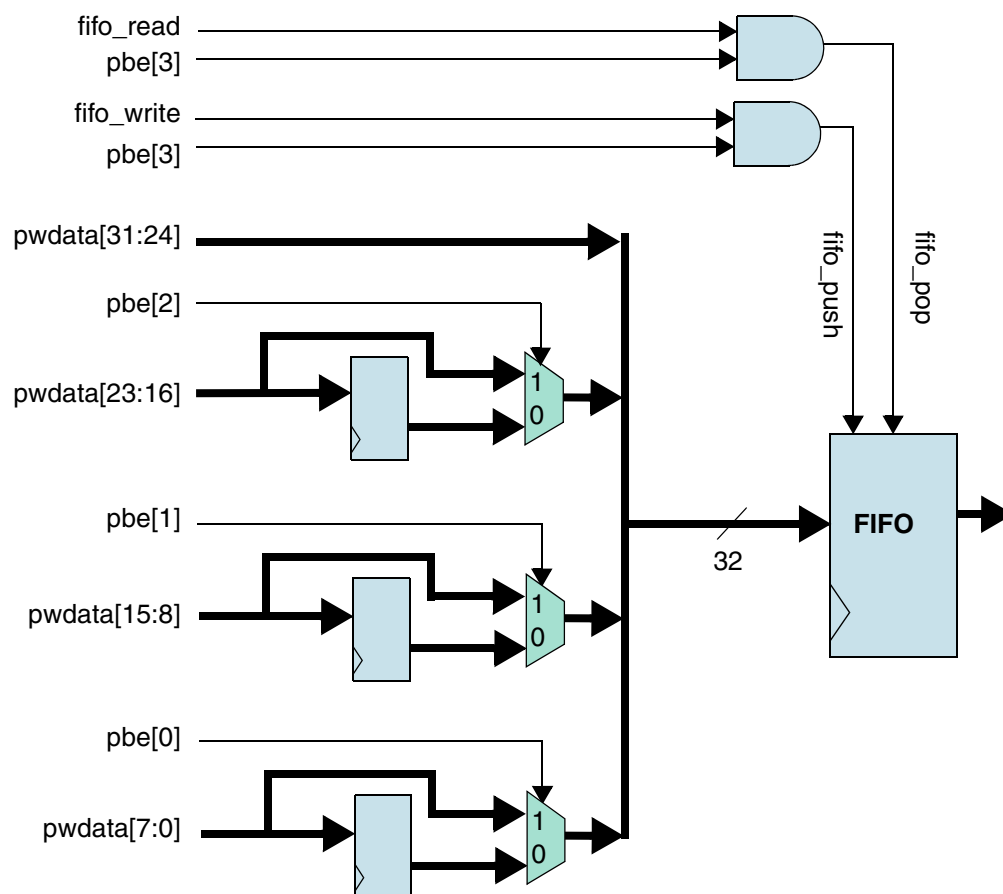
Since partial host/dw-dma bus-width accesses are permitted to the FIFO, temporary registers are implemented inside the core to store the lower bytes until the highest byte arrives.

 Note

The FIFO controller does not support simultaneous read/write access from the same port. For debugging purposes, the software may try to write into the FIFO and read back the data; results are indeterminate, since the design does not support read/write access from the same port.

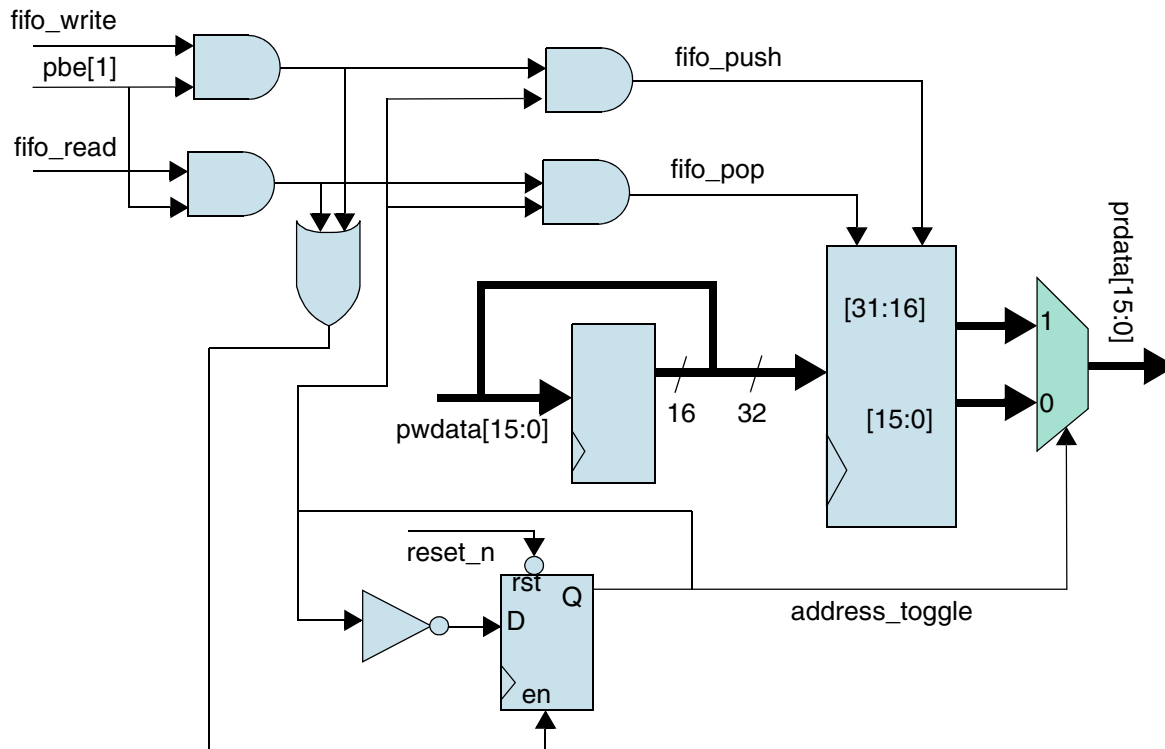
Figure 3-21 illustrates the block diagram of a 32-bit AMBA bus implementation.

Figure 3-21 Partial Bus Size Host/DW-DMA Write to FIFO



Additional Muxes and temporary registers are provided in order to support host/dw-dma access when the host data width is not equal to the FIFO data width. The block diagram in Figure 3-22 shows the details when H_DATA_WIDTH = 16 and F_DATA_WIDTH = 32.

Figure 3-22 Host/DMA Bus Width Smaller than FIFO Data Width



3.1.0.6 Power/Pullup Control and Card Detection Unit

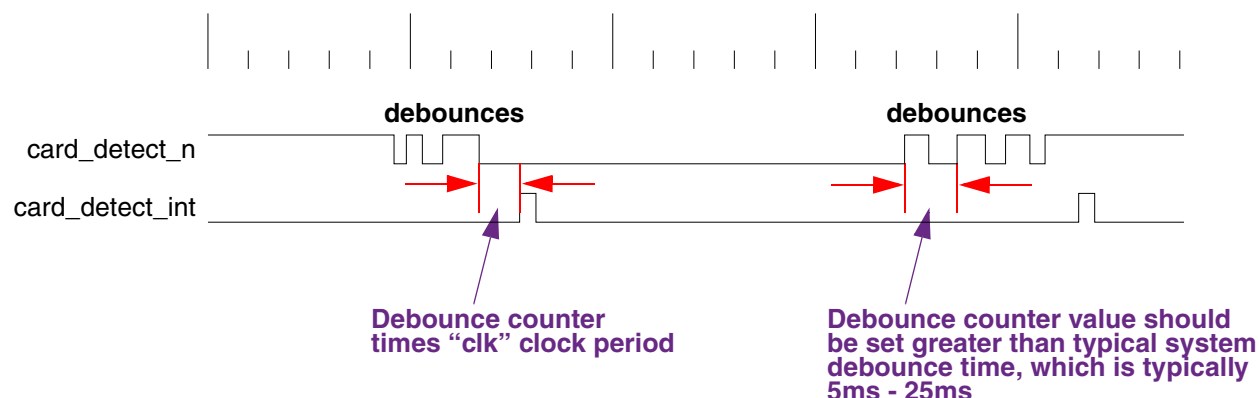
The register unit has registers that control the power and MMC open-drain pullup. Power to each card can be selectively turned on or off. Additionally, there are two 4-bit card-voltage control signals that can control the voltages of two voltage regulators. The control register has an enable_OD_pullup bit, which is used to turn on the open-drain-mode pullup during MMC initialization. Additionally, the DWC_mobile_storage provides an 8-bit general-purpose input port, gp_in, and a 16-bit general-purpose output port, gp_out.

The card detection unit looks for any changes in the card-detect signals for card insertion or card removal. It filters out the debounces associated with mechanical insertion or removal, and generates one interrupt to the host. You can program the debounce filter value.

On power-on, the controller should read in the card_detect port and store the value in the memory. Upon receiving a card-detect interrupt, it should again read the card_detect port and XOR with the previous card-detect status to find out which card has interrupted. If more than one card is simultaneously removed or inserted, there is only one card-detect interrupt; the XOR value indicates which cards have been disturbed. The memory should be updated with the new card-detect value.

Figure 3-23 illustrates the timing for card-detect signals.

Figure 3-23 Card-Detect Signals



3.2 Internal Direct Memory Access Controller (IDMAC)

The Internal Direct Memory Access Controller (IDMAC) has a Control and Status Register (CSR) and a single Transmit/Receive engine, which transfers data from host memory to the device port and vice versa. The controller utilizes a descriptor to efficiently move data from source to destination with minimal Host CPU intervention. You can program the controller to interrupt the Host CPU in situations such as data Transmit and Receive transfer completion from the card, as well as other normal or error conditions.

The IDMAC and the Host driver communicate through a single data structure. CSR addresses 0x80 to 0x98 are reserved for host programming.

The IDMAC transfers the data received from the card to the Data Buffer in the Host memory, and it transfers Transmit data from the Data Buffer in the Host memory to the DWC_mobile_storage FIFO. Descriptors that reside in the Host memory act as pointers to these buffers.

A data buffer resides in physical memory space of the Host and consists of complete data or partial data. Buffers contain only data, while buffer status is maintained in the descriptor. Data chaining refers to data that spans multiple data buffers. However, a single descriptor cannot span multiple data.

A single descriptor is used for both reception and transmission. The base address of the list is written into Descriptor List Base Address Register (DBADDR @0x88). A descriptor list is forward linked. The Last Descriptor can point back to the first entry in order to create a ring structure. The descriptor list resides in the physical memory address space of the Host. Each descriptor can point to a maximum of two data buffers.

Figure 3-24 and Figure 3-25 depict the arrangement of modules and connectivity with BIU in the absence and presence of IDMAC, respectively.

3.2.1 Architecture

Figure 3-24 illustrates the arrangement of modules and connectivity with the BIU in the absence of the IDMAC.

Figure 3-24 BIU Interface in Absence of IDMAC

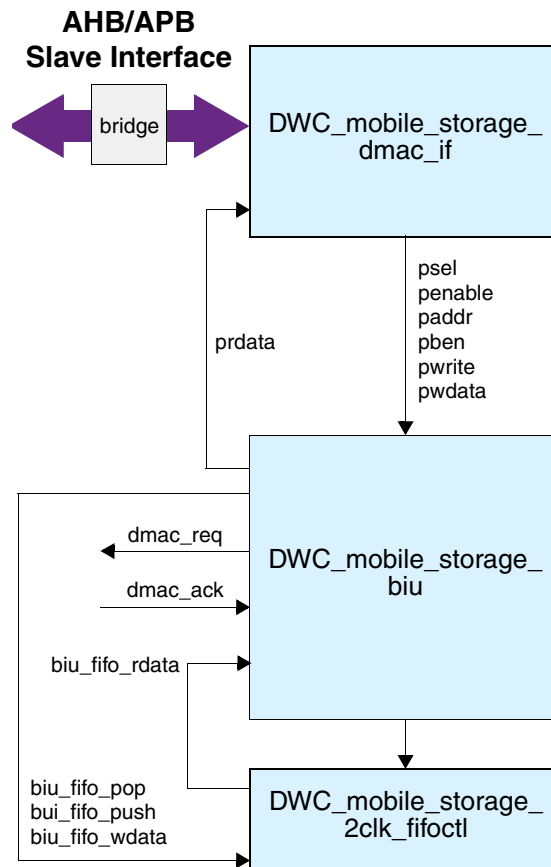
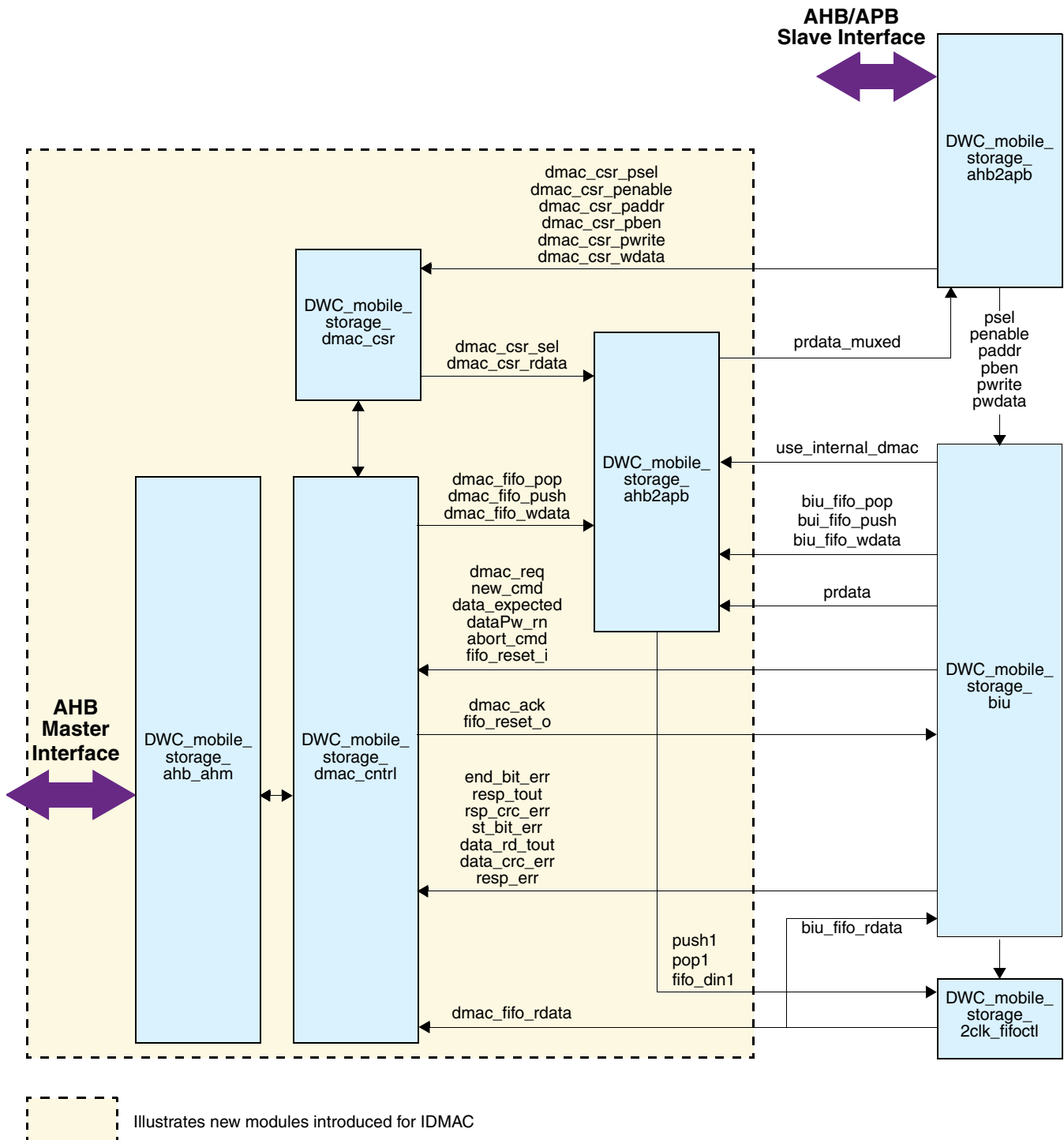


Figure 3-25 illustrates the arrangement of modules and connectivity with the BIU in the presence of the IDMAC.

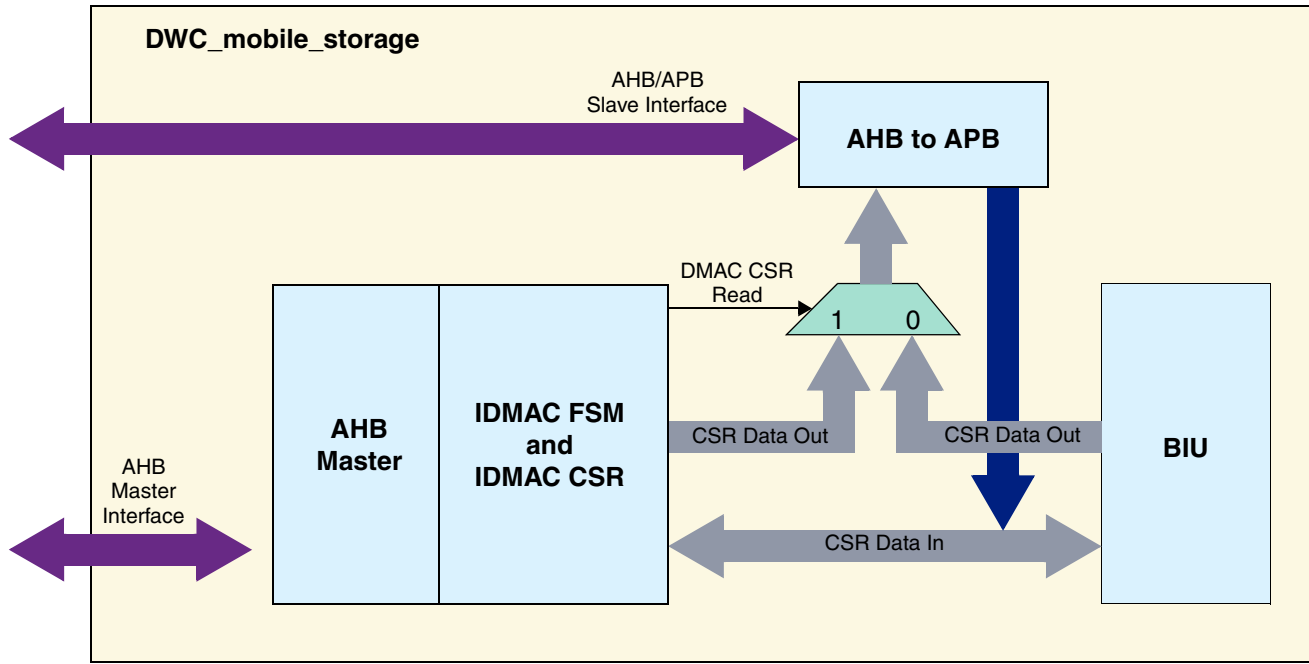
Figure 3-25 BIU Interface in Presence of IDMAC



3.2.2 IDMAC CSR Access

Figure 3-26 illustrates the arrangement of modules and connectivity of the IDMAC CSR path access.

Figure 3-26 IDMAC CSR Path Access



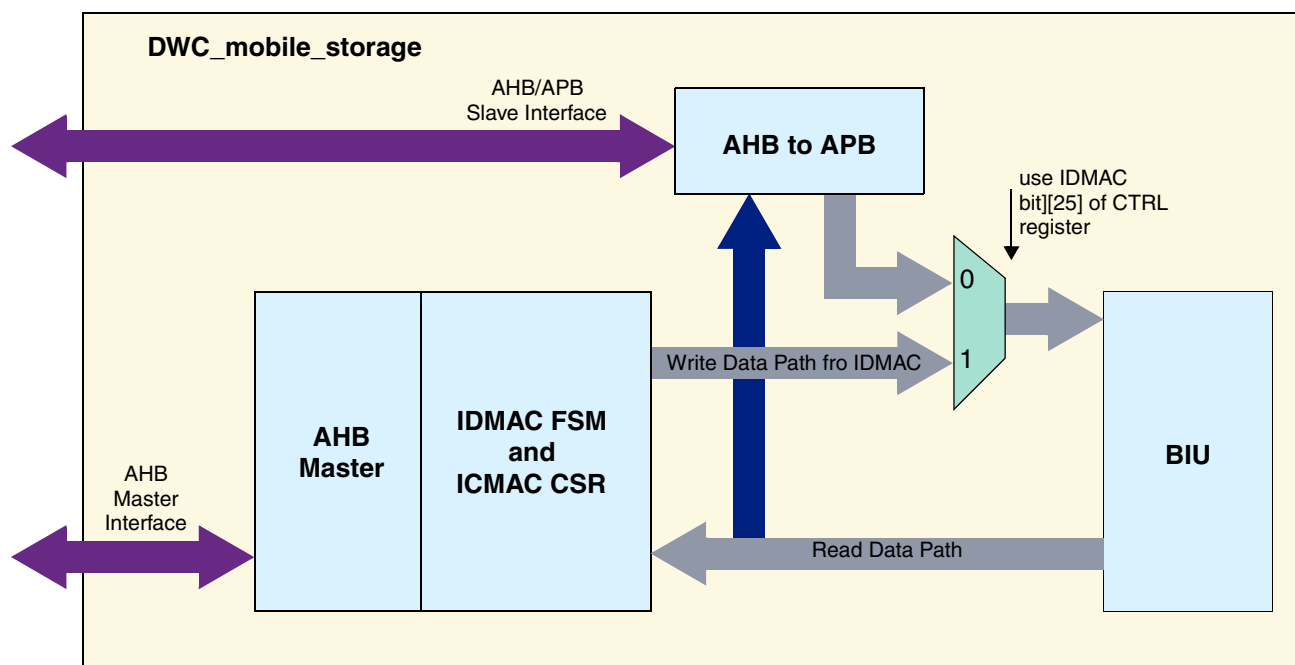
The module implementations in Figure 3-26 are:

- ❖ AHB Master – `DWC_mobile_storage_ahb_ahm`
- ❖ DMAC FSM – `DWC_mobile_storage_dmac_cntl`
- ❖ IDMAC CSR – `DWC_mobile_storage_dmac_csr`
- ❖ AHB to APB – `DWC_mobile_storage_ahb2apb`
- ❖ BIU – `DWC_mobile_storage_ahb_biu`

When an IDMAC is introduced, an additional CSR space resides in the IDMAC that controls the IDMAC functionality. The host accesses the new CSR space in addition to the existing control register set in the BIU. The IDMAC CSR primarily contains descriptor information. The AHB to APB native interface converts all incoming AHB/ APB transactions to a native APB access – similar to a standard APB access – and vice versa. For a write operation to the CSR, the respective CSR logic of the IDMAC and BIU decodes the address before accepting. For a read operation from the CSR, the appropriate CSR read path is enabled.

Figure 3-27 illustrates the arrangement of modules and connectivity of the IDMAC data path access.

Figure 3-27 IDMAC Data Path Access



You can enable or disable the IDMAC operation by programming bit[25] in the CTRL register of the BIU. This allows the data transfer by accessing the slave interface on the AMBA bus if the IDMAC is present but disabled. When IDMAC is enabled, the FIFO cannot be accessed through the slave interface.

3.2.3 Descriptors

The IDMAC uses these types of descriptor structures:

- ❖ **Dual-Buffer Structure** – The distance between two descriptors is determined by the Skip Length value programmed in the Descriptor Skip Length (DSL) field of the Bus Mode Register (BMOD @0x80).
- ❖ **Chain Structure** – Each descriptor points to a unique buffer and the next descriptor.

Figure 3-28 illustrates the IDMAC dual-buffer descriptor structure.

Figure 3-28 Dual-Buffer Descriptor Structure

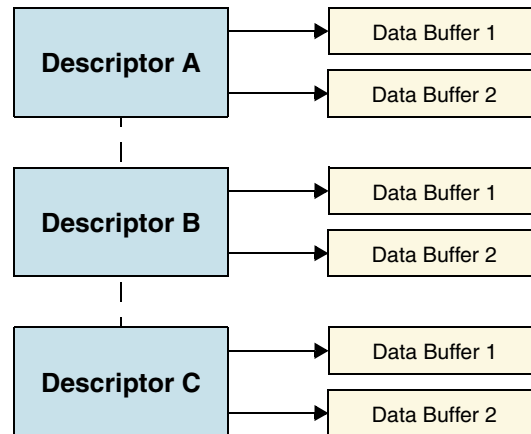


Figure 3-29 illustrates the IDMAC chain descriptor structure.

Figure 3-29 Chain Descriptor Structure

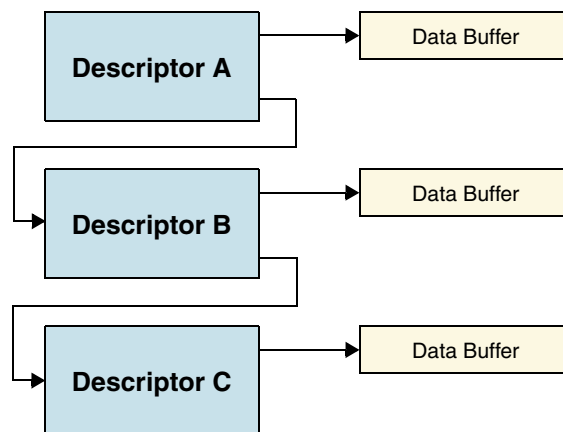
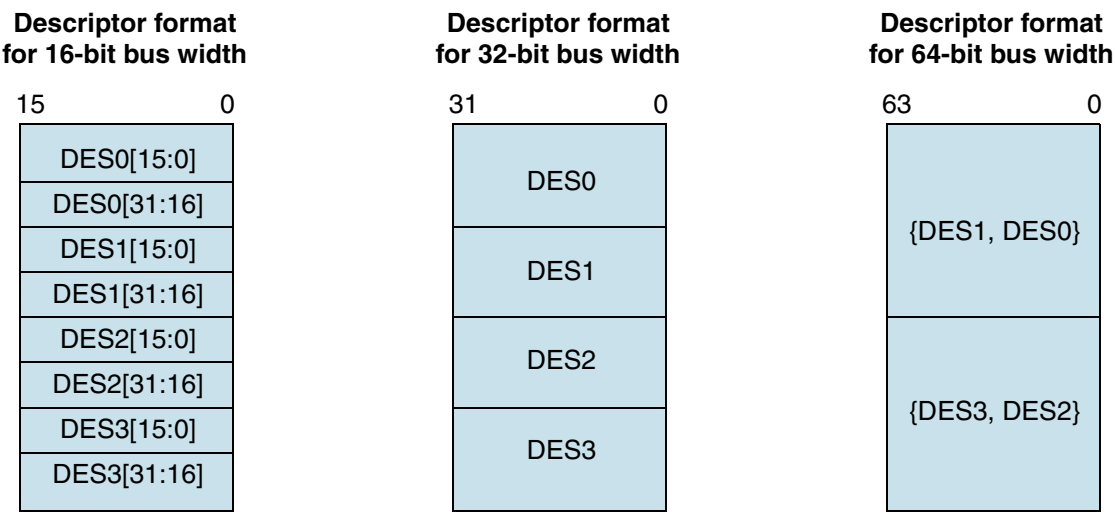


Figure 3-30 illustrates the internal formats of a descriptor. The descriptor addresses must be aligned to the bus width used for 16/32/64-bit buses. Each descriptor contains 16 bytes of control and status information.

DES0 is a notation used to denote the [31:0] bits, DES1 to denote [63:32] bits, DES2 to denote [95:64] bits, and DES3 to denote [127:96] bits in a descriptor.

Figure 3-30 Descriptor Formats for 16-Bit and 32-Bit Widths



The DES0 descriptor in the IDMAC contains control and status information; [Table 3-3](#) lists the bits in this descriptor.

Table 3-3 Bits in IDMAC DES0 Descriptor

Bits	Name	Description
31	OWN	When set, this bit indicates that the descriptor is owned by the IDMAC. When this bit is reset, it indicates that the descriptor is owned by the Host. The IDMAC clears this bit when it completes the data transfer.
30	Card Error Summary (CES)	These error bits indicate the status of the transaction to or from the card. These bits are also present in RINTSTS Indicates the logical OR of the following bits: • EBE: End Bit Error • RTO: Response Time out • RCRC: Response CRC • SBE: Start Bit Error • DRTO: Data Read Timeout • DCRC: Data CRC for Receive • RE: Response Error
29:6	Reserved	
5	End of Ring (ER)	When set, this bit indicates that the descriptor list reached its final descriptor. The IDMAC returns to the base address of the list, creating a Descriptor Ring. This is meaningful for only a dual-buffer descriptor structure.
4	Second Address Chained (CH)	When set, this bit indicates that the second address in the descriptor is the Next Descriptor address rather than the second buffer address. When this bit is set, BS2 (DES1[25:13]) should be all zeros.

Table 3-3 Bits in IDMAC DES0 Descriptor (Cont.)

Bits	Name	Description
3	First Descriptor (FS)	When set, this bit indicates that this descriptor contains the first buffer of the data. If the size of the first buffer is 0, next Descriptor contains the beginning of the data.
2	Last Descriptor (LD)	When set, this bit indicates that the buffers pointed to by this descriptor are the last buffers of the data.
1	Disable Interrupt on Completion (DIC)	When set, this bit will prevent the setting of the TI/RI bit of the IDMAC Status Register (IDSTS) for the data that ends in the buffer pointed to by this descriptor.
0	Reserved	

The DES1 descriptor contains the buffer size; [Table 3-4](#) lists the bits in this descriptor.

Table 3-4 Bits in IDMAC DES1 Descriptor

Bits	Name	Description
31:13	Reserved	
25:13	Buffer 2 Size (BS2)	These bits indicate the second data buffer byte size. The buffer size must be a multiple of 2, 4, or 8, depending upon the bus widths—16, 32, and 64, respectively. In the case where the buffer size is not a multiple of 2, 4, or 8, the resulting behavior is undefined. This field is not valid if DES0[4] is set.
12:0	Buffer 1 Size (BS1)	Indicates the data buffer byte size, which must be a multiple of 2, 4, or 8 bytes, depending upon the bus widths—16, 32, and 64, respectively. In the case where the buffer size is not a multiple of 2, 4, or 8, the resulting behavior is undefined. If this field is 0, the DMA ignores this buffer and proceeds to the next descriptor in case of a chain structure, or to the next buffer in case of a dual-buffer structure. Note: If there is only one descriptor and only one buffer to be programmed, you need to use only the Buffer 1 and not Buffer 2.

The DES2 descriptor contains the address pointer to the data buffer; [Table 3-5](#) lists the bits in this descriptor.

Table 3-5 Bits in IDMAC DES2 Descriptor

Bits	Name	Description
31:0	Buffer Address Pointer 1 (BAP1)	These bits indicate the physical address of the first data buffer. The IDMAC ignores DES2 [2/1/0:0], corresponding to the bus width of 64/32/16, internally.

The DES3 descriptor contains the address pointer to the next descriptor if the present descriptor is not the last descriptor in a chained descriptor structure or the second buffer address for a dual-buffer structure.

Table 3-6 Bits in IDMAC DES3 Descriptor

Bits	Name	Description
31:0	Buffer Address Pointer 2/ Next Descriptor Address (BAP2)	These bits indicate the physical address of the second buffer when the dual-buffer structure is used. If the Second Address Chained (DES0[4]) bit is set, then this address contains the pointer to the physical memory where the Next Descriptor is present. If this is not the last descriptor, then the Next Descriptor address pointer must be bus-width aligned (DES3[2/1/0:0] = 0 corresponding to bus-width of 64/32/16, Internally the LSBs are ignored).

3.2.4 Initialization

IDMAC initialization occurs as follows:

1. Write to IDMAC Bus Mode Register (BMOD) to set Host bus access parameters.
2. Write to IDMAC Interrupt Enable Register (IDINTEN) to mask unnecessary interrupt causes.
3. The software driver creates either the Transmit or the Receive descriptor list. Then it writes to IDMAC Descriptor List Base Address Register (DBADDR), providing the IDMAC with the starting address of the list.
4. The IDMAC engine attempts to acquire descriptors from the descriptor lists.

3.2.4.1 Host Bus Burst Access

The IDMAC attempts to execute fixed-length burst transfers on the AHB Master interface if configured using the FB bit of the IDMAC Bus Mode register. The maximum burst length is indicated and limited by the PBL field. The descriptors are always accessed in the maximum possible burst-size for the 16-bytes to be read – $16 \times 8 / \text{bus-width}$.

The IDMAC initiates a data transfer only when sufficient space to accommodate the configured burst is available in the FIFO or the number of bytes to the end of data, when less than the configured burst-length. The IDMAC indicates the start address and the number of transfers required to the AHB Master Interface. When the AHB Interface is configured for fixed-length bursts, then it transfers data using the best combination of INCR4/8/16 and SINGLE transactions. Otherwise, in no fixed-length bursts, it transfers data using INCR (undefined length) and SINGLE transactions.

3.2.4.2 Host Data Buffer Alignment

The Transmit and Receive data buffers in host memory must be aligned, depending on the data width (16/32/64).

3.2.4.3 Buffer Size Calculations

The driver knows the amount of data to transmit or receive. For transmitting to the card, the IDMAC transfers the exact number of bytes to the FIFO, indicated by the buffer size field of DES1.

If a descriptor is not marked as last – LS bit of DES0 – then the corresponding buffer(s) of the descriptor are full, and the amount of valid data in a buffer is accurately indicated by its buffer size field. If a descriptor is marked as last, then the buffer cannot be full, as indicated by the buffer size in DES1. The driver is aware of the number of locations that are valid in this case.

3.2.4.4 Transmission

IDMAC transmission occurs as follows:

1. The Host sets up the Descriptor (DES0-DES3) for transmission and sets the OWN bit (DES0[31]). The Host also prepares the data buffer.
2. The Host programs the write data command in the CMD register in BIU.
3. The Host will also program the required transmit threshold level (TX_WMark field in FIFOTH register).
4. The IDMAC determines that a write data transfer needs to be done as a consequence of step 2.
5. The IDMAC engine fetches the descriptor and checks the OWN bit. If the OWN bit is not set, it means that the host owns the descriptor. In this case the IDMAC enters suspend state and asserts the Descriptor Unable interrupt in the IDMAC status register (IDSTS). In such a case, the host needs to release the IDMAC by writing any value to the poll demand register.
6. It will then wait for Command Done (CD) bit and no errors from BIU which indicates that a transfer can be done.
7. The IDMAC engine will now wait for a DMA interface request (dw_dma_req) from BIU. This request will be generated based on the programmed transmit threshold value. For the last bytes of data which can't be accessed using a burst, SINGLE transfers are performed on AHB Master Interface.
8. The IDMAC fetches the Transmit data from the data buffer in the Host memory and transfers to the FIFO for transmission to card.
9. When data spans across multiple descriptors, the IDMAC will fetch the next descriptor and continue with its operation with the next descriptor. The Last Descriptor bit in the descriptor indicates whether the data spans multiple descriptors or not.
10. When data transmission is complete, status information is updated in IDMAC status register (IDSTS) by setting Transmit Interrupt, if enabled. Also, the OWN bit is cleared by the IDMAC by performing a write transaction to DES0.

3.2.4.5 Reception

IDMAC reception occurs as follows:

1. The Host sets up the Descriptor (DES0-DES3) for reception, sets the OWN (DES0[31]).
2. The Host programs the read data command in the CMD register in BIU.
3. The Host will program the required receive threshold level (RX_WMark field in FIFOTH register).
4. The IDMAC determines that a read data transfer needs to be done as a consequence of step 2.
5. The IDMAC engine fetches the descriptor and checks the OWN bit. If the OWN bit is not set, it means that the host owns the descriptor. In this case the IDMAC enters suspend state and asserts the Descriptor Unable interrupt in the IDMAC status register (IDSTS). In such a case, the host needs to release the IDMAC by writing any value to the poll demand register.
6. It will then wait for Command Done (CD) bit and no errors from BIU which indicates that a transfer can be done.
7. The IDMAC engine will now wait for a DMA interface request (dw_dma_req) from BIU. This request will be generated based on the programmed receive threshold value. For the last bytes of data which can't be accessed using a burst, SINGLE transfers are performed on AHB.

8. The IDMAC fetches the data from the FIFO and transfer to Host memory.
9. When data spans across multiple descriptors, the IDMAC will fetch the next descriptor and continue with its operation with the next descriptor. The Last Descriptor bit in the descriptor indicates whether the data spans multiple descriptors or not.
10. When data reception is complete, status information is updated in IDMAC status register (IDSTS) by setting Receive Interrupt, if enabled. Also, the OWN bit is cleared by the IDMAC by performing a write transaction to DES0.

3.2.4.6 Interrupts

Interrupts can be generated as a result of various events. IDMAC Status Register (IDSTS) contains all the bits that might cause an interrupt. IDMAC Interrupt Enable Register (IDINTEN) contains an Enable bit for each of the events that can cause an interrupt.

There are two groups of summary interrupts – Normal and Abnormal – as outlined in IDMAC Status Register (IDSTS). Interrupts are cleared by writing a 1 to the corresponding bit position. When all the enabled interrupts within a group are cleared, the corresponding summary bit is cleared. When both the summary bits are cleared, the interrupt signal `dmac_intr_o` is de-asserted.

Interrupts are not queued and if the interrupt event occurs before the driver has responded to it, no additional interrupts are generated. For example, Receive Interrupt (IDSTS[1]) indicates that one or more data was transferred to the Host buffer.

An interrupt is generated only once for simultaneous, multiple events. The driver must scan IDMAC Status Register for the interrupt cause.

**Note**

The final interrupt (int) signal from `DWC_mobile_storage` is a logical OR of the interrupt from BIU and IDMAC.

Figure 3-31 illustrates the IDMAC functional state machine (FSM).

Figure 3-31 IDMAC FSM

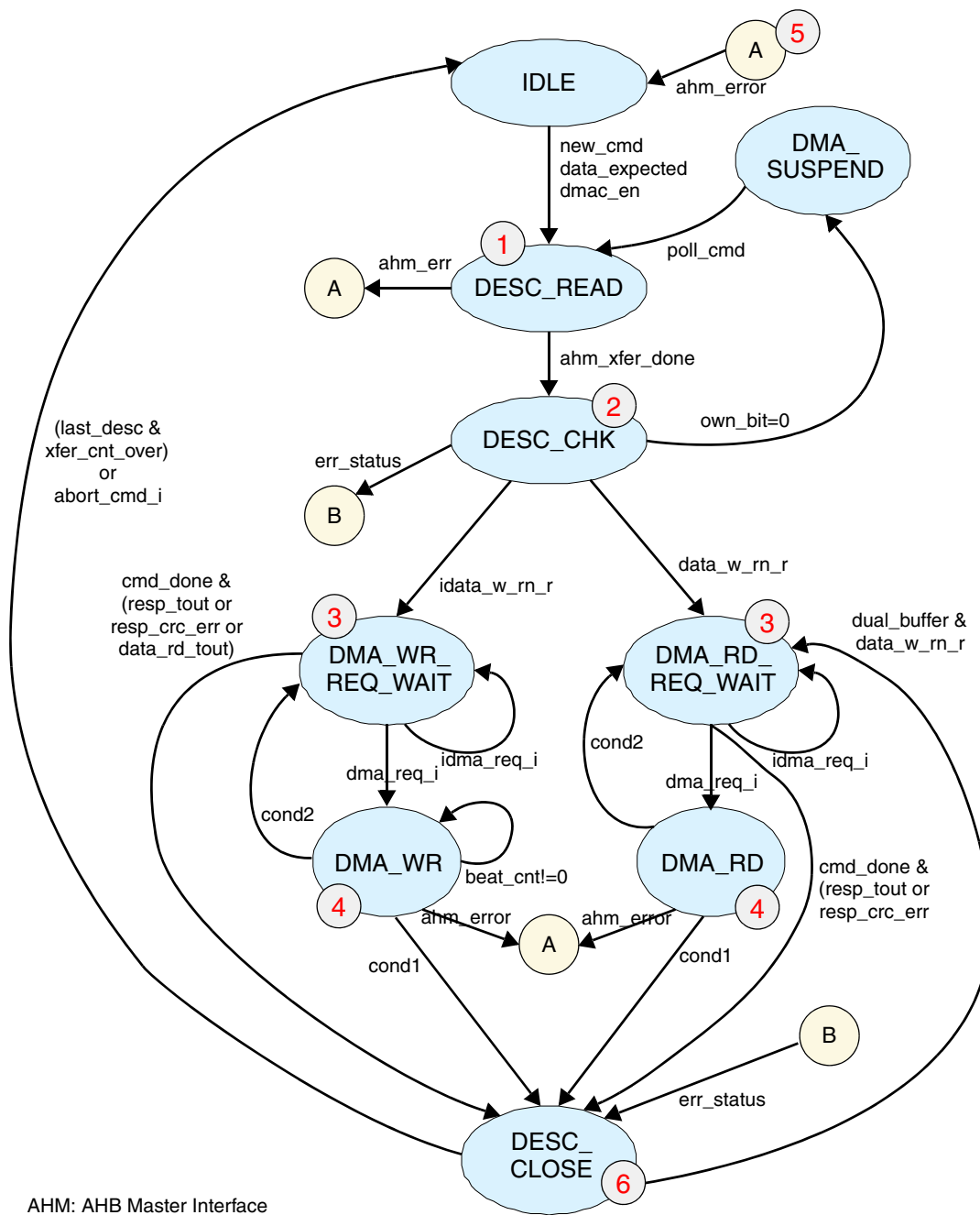


Figure 3-31 shows the following in the IDMAC FSM:

1. Performs the following accesses:
 - ◆ Sixteen accesses for HDATAWIDTH=16
 - ◆ Four accesses for HDATAWIDTH=32
 - ◆ Two accesses for HDATAWIDTH=64

2. Stores descriptors in internal register. Also issues a FIFO reset if it is first descriptor.
3. Each bit checked for correctness. In case of bit mismatches, appropriate error bit is set.
If it is first descriptor, then issues a FIFO reset and wait until FIFO reset is complete.
The err_status indicates one of the following:
 - ◆ Response timeout
 - ◆ Response CRC error
 - ◆ Data receive timeout
 - ◆ Response error
4. FSM waits in current state until dma_req is asserted, which implies that FIFO:
 - ◆ For DMA_WR_REQ_WAIT – Holds the number of data, indicated by FIFO RX watermark. In case of error due to response timeout or error, FSM goes to DESC_CLOSE state to close descriptor.
 - ◆ For DMA_RD_REQ_WAIT – Holds number of data, indicated by FIFO TX watermark. In case of error, FSM goes to DESC_CLOSE state to close descriptor.
5. FSM performs the following:
 - ◆ For DMA_WR – Requests a write on AHB by sending request to AHM. If number of beats in one transfer is greater than PBL, then one of the following occurs:
 - ◇ Burst count to AHM is PBL value
 - ◇ Single transfers are initiated
 A pop is asserted to FIFO each time data is pushed to AHM.
 - ◇ cond1 – If a chained descriptor, descriptor is closed after data transfer. If a dual buffer and the second data buffer is also accessed, or if a dual buffer and the second data buffer has not data, then descriptor is closed.
 - ◇ cond2 – If a dual buffer and second buffer is not empty.
 - ◆ For DMA_RD – Requests a read on AHB by sending request to AHM. If number of beats in one transfer is greater than PBL, then one of the following occurs:
 - ◇ AHM is PBL value
 - ◇ Single transfers initiated
 A push is asserted to FIFO each time data is pushed from AHM.
 - ◇ cond1 – If a chained descriptor, descriptor is closed after data transfer. If a dual buffer and the second data buffer is also accessed, or if a dual buffer and the second data buffer has not data, then descriptor is closed.
 - ◇ cond2 – If a dual buffer and second buffer is not empty.
6. Fatal Bus Error code for ahm_error:
 - ◆ For AHB write is 3'b010
 - ◆ For AHB read is 3'b001
7. After programmed transfer count is accessed from host memory. The OWN bit in descriptor is closed. If transfer spans more than one descriptor, FSM fetches next descriptor. If transfer ends with current descriptor, FSM goes to idle state after setting RI or TI. Depending on descriptor structure –

dual buffer or chained – appropriate starting address of descriptor is loaded. If it is second data buffer of dual buffer, descriptor is not fetched again.

3.2.4.6.1 Abort

When the host issues CMD12 when a data transfer on the card data lines is in progress, the FSM closes the present descriptor after completing the transfer of data until a DTO interrupt is asserted. Once an abort command is issued, the DMAC performs single burst transfers.

1. For a card write, the FSM keeps pushing data to the FIFO after fetching it from the host memory until a DTO interrupt is asserted. This is done in order to keep the card clock running so that CMD12 is reliably sent to the card.
2. For a card read, the FSM keeps popping data from FIFO and writes to the host memory until a DTO interrupt is generated. This is required since DTO interrupt is not generated until and unless all the FIFO data is emptied.



Note

- In case of an FBE, the current descriptor and the remaining unread descriptors are not closed by the IDMAC FSM.
- In case of a write abort, only the current descriptor during which an abort occurred is closed by the IDMAC FSM. The remaining unread descriptors are not closed by the IDMAC FSM.
- In case of a read abort, the IDMAC FSM pops the data out of the FIFO and writes them to the corresponding descriptor data buffers. The remaining unread descriptors are not closed.

3.2.5 FBE Scenarios

An FBE occurs due to an AHB error response on the AHB bus. This is a system error, so the software driver should not perform any further programming to the DWC_mobile_storage. The only recovery mechanism from such scenarios is to do one of the following:

- ❖ Issue a hard reset by asserting the reset_n signal
- ❖ Do a program controller reset by writing to the CTRL[0] register

3.2.5.1 FIFO Overflow and Underflow

During normal data transfer conditions, FIFO overflow and underflow will not occur. However if there is a programming error, then FIFO overflow/underflow can result. For example, consider the following scenarios.

For transmit:

- ❖ PBL=4
- ❖ Tx watermark = 1

For the above programming values, if the FIFO has only one location empty, it issues a dw_dma_req to IDMAC FSM. Due to PBL value=4, the IDMAC FSM performs 4 pushes into the FIFO. This will result in a FIFO overflow interrupt.

For receive:

- ❖ PBL=4
- ❖ Rx watermark = 1

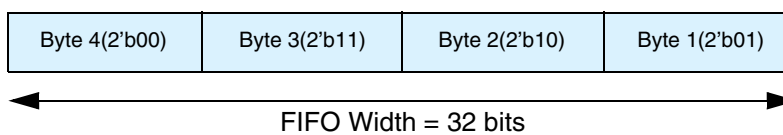
For the above programming values, if the FIFO has only one location filled, it issues a dw_dma_req to IDMAC FSM. Due to PBL value=4, the IDMAC FSM performs 4 pops to the FIFO. This will result in a FIFO underflow interrupt.

The driver should ensure that the number of bytes to be transferred as indicated in the descriptor should be a multiple of 2/4/8 bytes with respect to H_DATA_WIDTH=16/32/64. For example, if the BYTCNT = 13, the number of bytes indicated in the descriptor should be 14 for H_DATA_WIDTH=16, 16 for H_DATA_WIDTH=32 and 16 for H_DATA_WIDTH=64.

3.2.5.2 Considerations for H_DATA_WIDTH=16

Note that for H_DATA_WIDTH=64 and H_DATA_WIDTH=32, the FIFO width is 64 and 32 bits, respectively. The FIFO width for H_DATA_WIDTH=16 is still 32. Hence, the IDMAC will accumulate 4 bytes and does a push/pop operation. For H_DATA_WIDTH=16, there may be an extra push/pop required. The byte count information (BYTCNT register) is used to determine whether an extra push/pop needs to be done. If the 2 LSBits of the byte count is either 2'b01 or 2'b10 at the start of a command, an extra push/pop is performed.

Figure 3-32 Byte Arrangement



3.2.5.3 Programming of PBL and Watermark Levels

The DMAC performs data transfers depending on the programmed PBL and threshold values. [Table 3-7](#) lists the legal programming values.

Table 3-7 PBL and Watermark Levels

PBL (Number of transfers)	Tx/Rx Watermark Value
1	greater than or equal to 1
4	greater than or equal to 4
8	greater than or equal to 8
16	greater than or equal to 16
32	greater than or equal to 32
64	greater than or equal to 64
128	greater than or equal to 128
256	greater than or equal to 256

3.3 Card Interface Unit (CIU)

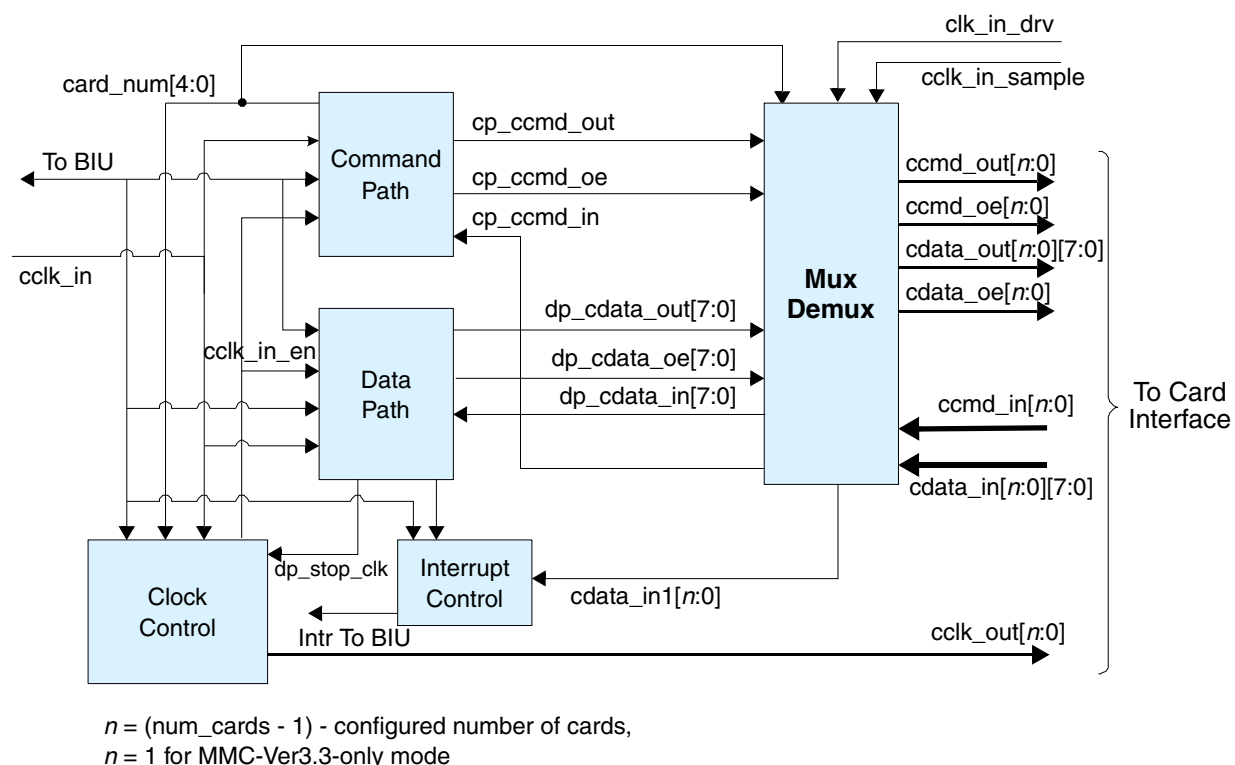
The Card Interface Unit (CIU) interfaces with the Bus Interface Unit (BIU) and the SD_MMC_CEATA cards or devices. The host writes command parameters to the DWC_mobile_storage BIU control registers, and these parameters are then passed to the CIU. Depending on control register values, the CIU generates SD_MMC_CEATA command and data traffic on a selected card bus according to SD_MMC_CEATA protocol. The control register values also decide whether the command and data traffic is destined for the CE-ATA device, and the DWC_mobile_storage accordingly controls the command and data path.

The following software restrictions should be met for proper CIU operation:

- ❖ Only one card can be selected at a time for command or data transfer. For example, when data is transferred from a card, a new command should not be sent to another card; for the same card, a new command is allowed for read status or to stop the transfer.
- ❖ Only one data transfer command can be issued at a time.
- ❖ During an open-ended card write operation, if the card clock is stopped because the FIFO is empty, the software must first fill the data into the FIFO and start the card clock. It can then issue only a stop/abort command to the card.
- ❖ During an SDIO/COMBO card transfer, if the card function is suspended and the software wants to resume the suspended transfer, it must first reset the FIFO and start the resume command as if it were a new data transfer command.
- ❖ When issuing card reset commands (CMD0, CMD15 or CMD52_reset) while a card data transfer is in progress, the software must set the stop_abort_cmd bit in the Command register so that the DWC_mobile_storage can stop the data transfer after issuing the card reset command.
- ❖ When the data end bit error is set in the RINTSTS register, the DWC_mobile_storage does not guarantee SDIO interrupts. The software should ignore the SDIO interrupts and issue the stop/abort command to the card, so that the card stops sending the read data.
- ❖ If the card clock is stopped because the FIFO is full during a card read, the software should read at least two FIFO locations to start the card clock.
- ❖ Only one CE-ATA device can be selected at a time for command or data transfer. For example, when data is transferred from a CE-ATA device, a new command should not be sent to another CE-ATA device.
- ❖ If CE-ATA device interrupts are enabled (nIEN=0), a new RW_BLK command should not be sent to the same device if there is a pending RW_BLK command in progress (the RW_BLK command used in this databook is the RW_MULTIPLE_BLOCK MMC command defined by the CE-ATA specification). Only the CCSD can be sent while waiting for the CCS.
- ❖ For the same device, a new command is allowed for reading status information, if interrupts are disabled in the CE-ATA device (nIEN=1).
- ❖ Open-ended transfers are not supported for the CE-ATA devices.
- ❖ The send_auto_stop signal is not supported (software should not set the send_auto_stop bit) for CE-ATA transfers.

Figure 3-33 illustrates the CIU block diagram, which consists of the following primary functional blocks:

- ❖ Command path
- ❖ Data path
- ❖ SDIO interrupt control
- ❖ Clock control
- ❖ Mux/demux unit

Figure 3-33 CIU Block Diagram

3.3.1 Command Path

The command path performs the following functions:

- ❖ Loads clock parameters
- ❖ Loads card command parameters
- ❖ Sends commands to card bus (ccmd_out line)
- ❖ Receives responses from card bus (ccmd_in line)
- ❖ Sends responses to BIU
- ❖ Drives the P-bit on command line

A new command is issued to the DWC_mobile_storage by programming the BIU registers and setting the start_cmd bit in the Command register. The BIU asserts start_cmd, which indicates that a new command is issued to the DWC_mobile_storage. The command path loads this new command (command, command argument, timeout) and sends an acknowledge to the BIU by asserting cmd_taken.

Once the new command is loaded, the command path state machine sends a command to the SD_MMC bus—including the internally generated CRC7—and receives a response, if any. The state machine then sends the received response and signals to the BIU that the command is done, and then waits for eight clocks before loading a new command. In CE-ATA data payload transfer (RW_MULTIPLE_BLOCK)

commands, if the device interrupts are enabled ($nIEN=0$ in CE-ATA device), the state machine does the following after receiving the response:

- ❖ Does not drive the P-bit; it waits for Command Completion Signal, decodes and goes back to idle state, and then drives the P-bit.
- ❖ If the host wants to send the Command Completion Signal Disable (CCSD) and if eight clock cycles are expired after the response, sends the CCSD pattern on the command line.

3.3.1.1 Load Command Parameters

One of the following commands or responses is loaded in the command path:

- ❖ New command from BIU – When `start_cmd` is asserted, then the `start_cmd` bit is set in the Command register.
- ❖ Internally-generated auto-stop command – When the data path ends, the stop command request is loaded.
- ❖ IRQ response with RCA 0x000 – When the command path is waiting for an IRQ response from the MMC card and a “send irq response” request is signaled by the BIU, then the `send_irq_response` bit is set in the control register.

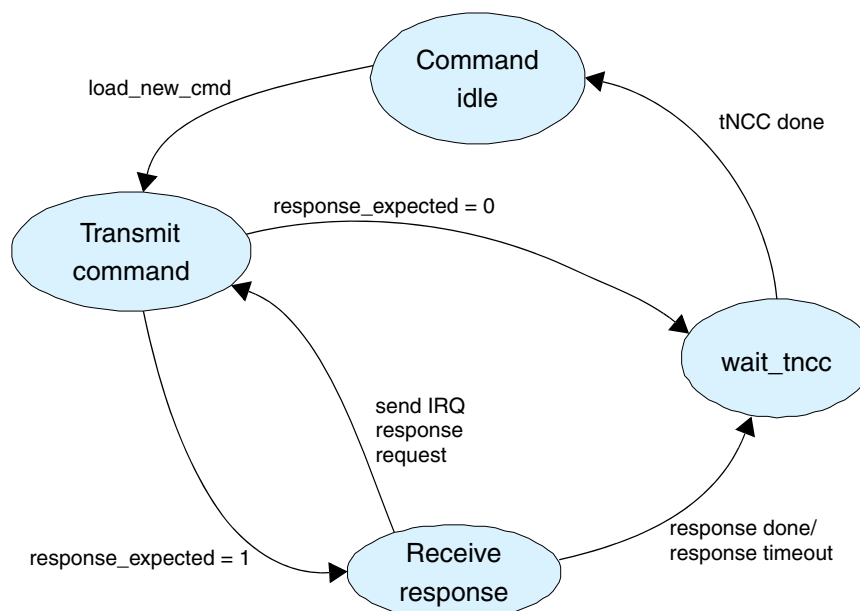
Loading a new command from the BIU in the command path depends on the following Command register bit settings:

- ❖ `update_clock_registers_only` – If this bit is set in the Command register, the command path updates only the clock enable, clock divider, and clock source registers. If this bit is not set, the command path loads the command, command argument, and timeout registers; it then starts processing the new command.
- ❖ `wait_prvdata_complete` – If this bit is set, the command path loads the new command under one of the following conditions:
 - ◆ Immediately, if the data path is free (that is, there is no data transfer in progress), or if an open-ended data transfer is in progress (`byte_count = 0`).
 - ◆ After completion of the current data transfer, if a predefined data transfer is in progress.

3.3.1.2 Send Command and Receive Response

Once a new command is loaded in the command path – `update_clock_registers_only` bit is unset – the command path state machine sends out a command on the SD_MMC bus; the command path state machine is illustrated in Figure 3-34.

Figure 3-34 SD_MMC Command Path State Machine



The command path state machine performs the following functions, according to Command register bit values:

1. `send_initialization` – Initialization sequence of 80 clocks is sent before sending the command.
2. `response_expected` – Response is expected for the command. After the command is sent out, the command path state machine receives a 48-bit or 136-bit response and sends it to the BIU. If the start bit of the card response is not received within the number of clocks programmed in the timeout register, then the response timeout and command done bit is set in the Raw Interrupt Status register as a signal to the BIU. If the response-expected bit is not set, the command path sends out a command and signals a response done to the BIU; that is, the command done bit is set in the Raw Interrupt Status register.
3. `response_length` – If this bit is set, a 136-bit response is received; if it is not set, a 48-bit response is received.
4. `check_response_crc` – If this bit is set, the command path compares CRC7 received in the response with the internally-generated CRC7. If the two do not match, the response CRC error is signaled to the BIU; that is, the response CRC error bit is set in the Raw Interrupt Status register.

3.3.1.3 Send Response to BIU

If the `response_expected` bit is set in the Command register, the received response is sent to the BIU. The Response0 register is updated for a short response, and the Response3, Response2, Response1, and Response0 registers are updated on a long response, after which the Command Done bit is set. If the response is for an `auto_stop` command sent by the CIU, the response is saved in the Response1 register, after which the Auto Command Done bit is set.

Additionally, the command path checks for the following:

- ❖ Transmission bit = 0
- ❖ Command index matches command index of the sent command
- ❖ End bit = 1 in received card response

The command index is not checked for a 136-bit response or if the `check_response_crc` bit is unset. For a 136-bit response and reserved CRC 48-bit responses, the command index is reserved — that is, 111111.

3.3.1.4 Driving P-bit on CMD Line

The command path drives a P-bit = 1 on the CMD line between two commands if a response is not expected. If a response is expected, the P-bit is driven after the response is received and before the start of the next command; this is done by asserting both `ccmd_out` and `ccmd_out_en`. While accessing a CE-ATA device, for commands that expect Command Completion Signal, the P-bit is driven after the response only if the interrupts are disabled in the CE-ATA device (`nIEN=1`); that is, the `ccs_expected` bit is cleared. If the command expects the Command Completion Signal, then the P-bit is driven only after receiving the Command Completion Signal.

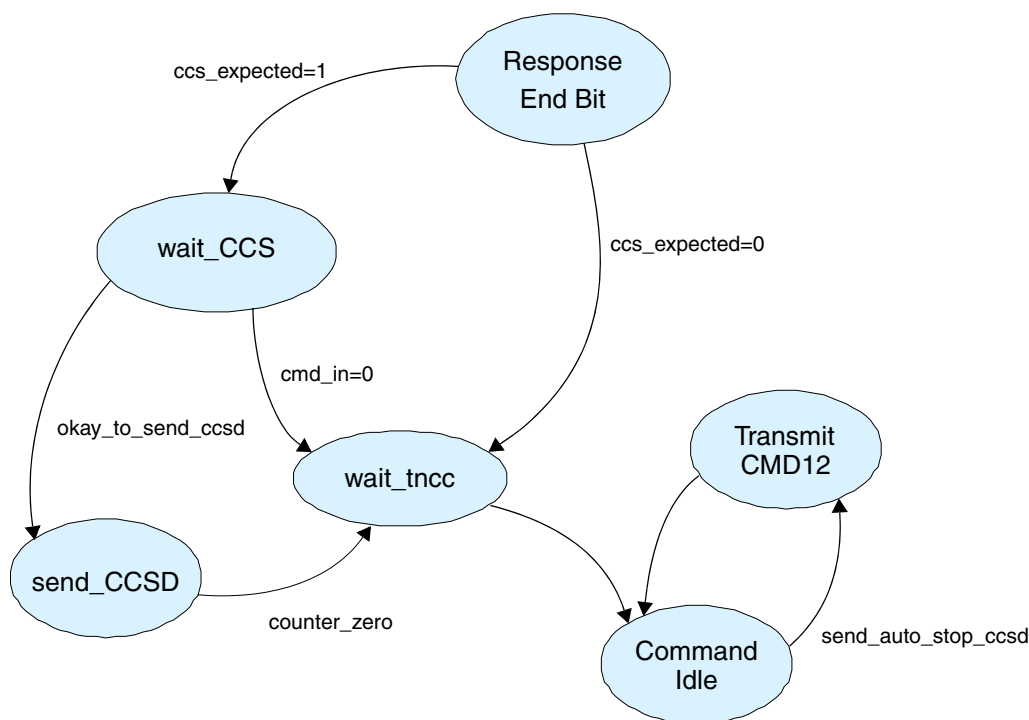
During initialization, the software should set the `ccmd_od_pullup_en` bit, which indicates an open-drain mode, during which the controller drives only a 0 or high-impedance (Z) on the command bus; a hard 1 is never driven in open-drain mode.

3.3.1.5 Polling Command Completion Signal

CE-ATA devices generate the Command Completion Signal in order to notify the host controller of the normal ATA command completion or ATA command termination. After receiving the response from the

card, the command state machine performs the functions illustrated in Figure 3-35 according to command register bit values.

Figure 3-35 CE-ATA Command Path State Machine



The following describe some of the details in Figure 3-35:

1. **Response_end_bit** – Receives the end bit of the response from the device. If `cmd_compl_expctd` bit is set, goes to `wait_CCS` state.
2. **wait_CCS** – FSM wait for the Command Completion Signal (CCS) from the CE-ATA device. While waiting for the CCS the following events can happen:
 - a. Software sets the `send_ccsd` bit, indicating not to wait for Command Completion Signal and to send the Command Completion Signal Disable pattern on the command line.
 - b. Receive the Command Completion Signal on the CMD line.
3. **send_ccsd** – Sends the Command Completion Signal Disable pattern (5'b00001) on the CMD line.

3.3.1.6 Command Completion Signal Detection and Interrupt to Host Processor

If the `ccs_expected` bit is set in the Command register, the Command Completion Signal (CCS) from the CE-ATA device is indicated by setting the Data Transfer Over (DTO) bit in the RINTSTS register. DWC_mobile_storage generates a Data Transfer Over (DTO) interrupt if this interrupt is not masked.

For the `RW_MULTIPLE_BLOCK` commands, if the CE-ATA device interrupts are disabled (`nIEN=1`) – that is, the `ccs_expected` bit is cleared in the Command register – there are no CCSs from the device. When the data transfer is over – that is, when the programmed number of bytes are transferred – the Data Transfer Over bit is set in the RINTSTS register.

3.3.1.7 Command Completion Signal Timeout

If the command expects a CCS from the device – if the `ccs_expected` bit is set in the Command register – the command state machine waits for the CCS and remains in a `wait_CCSS` state. If the CE-ATA device fails to send out the CCS, the host software should implement a timeout mechanism to free the command and data path. The `DWC_mobile_storage` does not implement a hardware timer; it is the responsibility of the host software to maintain a software timer.

In the event of a CCS timeout, the host should issue a CCSD by setting the `send_ccsd` bit in the CTRL register. The `DWC_mobile_storage` controller command state machine sends the CCSD to the CE-ATA device and exits to an idle state. After sending the CCSD, the host should also send a CMD12 to the CE-ATA device in order to abort the outstanding ATA command.

**Note**

In external DMA mode during CCS time-out, the DMA does not generate the request at the end of packet, even if remaining bytes are less than threshold. In this case, there will be some data left in the FIFO. It is the responsibility of the application to reset the FIFO after the CCS timeout.

3.3.1.8 Send Command Completion Signal Disable

If the `send_ccsd` bit is set in the CTRL register, the `DWC_mobile_storage` sends a Command Completion Signal Disable (CCSD) pattern on the CMD line. The host can send the CCSD while waiting for the CCS or after a CCS timeout happens.

After sending the CCSD pattern, the `DWC_mobile_storage` controller sets the Command Done (CD) bit in RINTSTS and also generates an interrupt to the host if the Command Done interrupt is not masked.

**Note**

Within the CIU block, if `send_ccsd` is set on the same clock cycle as CCS is sampled, the CIU block does not send a CCSD pattern on the CMD line. In this case, DTO and Command Done status are set in the RINTSTS register.

Due to asynchronous boundaries, CCS may have already happened and `send_ccsd` is set. In this case CCSD will not go to CEATA device and `send_ccsd` bit will not get cleared. The host should clear the `send_ccsd` bit before the next command is issued.

If the `send_auto_stop_ccsd` bit is set in the CTRL register, the `DWC_mobile_storage` sends an internally generated STOP command (CMD12) after sending the CCSD pattern. The `DWC_mobile_storage` controller sets the Auto Command Done bit in the RINTSTS register.

3.3.1.9 I/O transmission delay (ACIO Timeout)

The host software should maintain the timeout mechanism for handling the I/O transmission delay (N_{ACIO} cycles) time-outs while reading from the CE-ATA device. The `DWC_mobile_storage` neither maintains any timeout mechanism nor indicates that N_{ACIO} cycles are elapsed while waiting for the start bit of a data token. The I/O transmission delay is applicable for read transfers using the `RW_REG` and `RW_BLK` commands; the `RW_REG` and `RW_BLK` commands used in this databook refer to the `RW_MULTIPLE_REGISTER` and `RW_MULTIPLE_BLOCK` MMC commands defined by the CE-ATA specification.

**Note**

After the N_{ACIO} timeout, the application must abort the command by sending the CCSD and STOP commands, or the STOP command. The DRTO interrupt may be set while CMD12 is transmitted out of the `DWC_mobile_storage`, in which case the DRTO and DTO bits are set in the RINTSTS register.

3.3.2 Data Path

The data path block pops the data FIFO and transmits data on `cdata_out` during a write data transfer, or it receives data on `cdata_in` and pushes it into the FIFO during a read data transfer. The data path loads new data parameters – that is, data expected, read/write data transfer, stream/block transfer, block size, byte count, card type, timeout registers – whenever a data transfer command is not in progress.

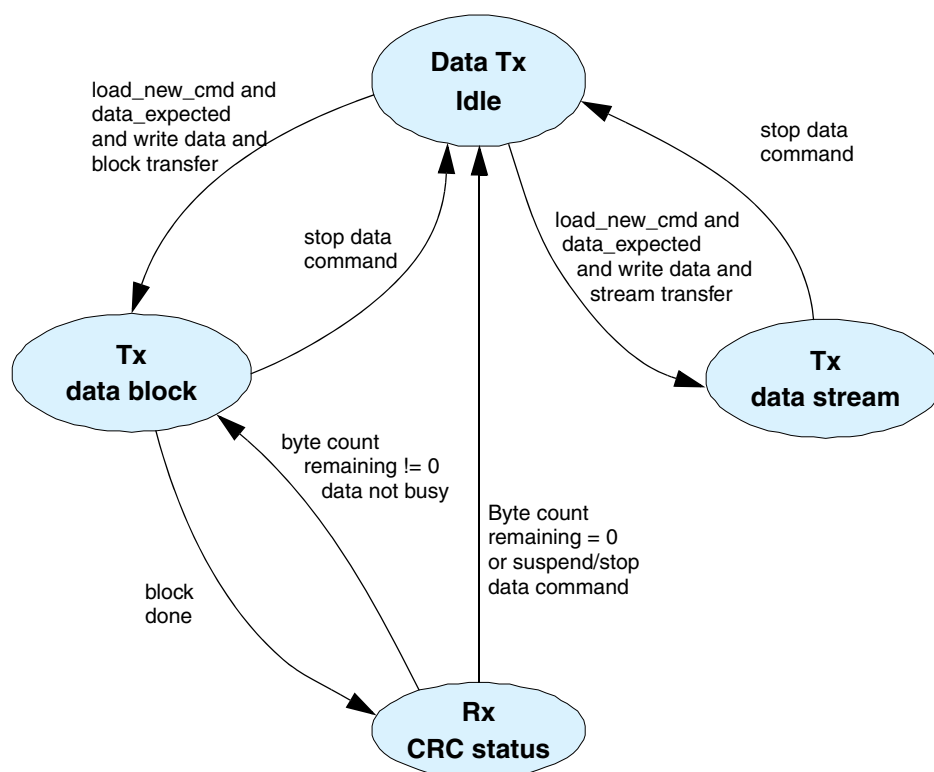
If the `data_expected` bit is set in the Command register, the new command is a data transfer command and the data path starts one of the following:

- ❖ Transmit data if the read/write bit = 1
- ❖ Data receive if read/write bit = 0

3.3.2.1 Data Transmit

The data transmit state machine, illustrated in [Figure 3-36](#), starts data transmission two clocks after a response for the data write command is received; this occurs even if the command path detects a response error or response CRC error. If a response is not received from the card because of a response timeout, data is not transmitted. Depending upon the value of the `transfer_mode` bit in the Command register, the data transmit state machine puts data on the card data bus in a stream or in block(s).

Figure 3-36 Data Transmit State Machine



3.3.2.1.1 Stream Data Transmit

If the `transfer_mode` bit in the Command register is set to 1, it is a stream-write data transfer. The data path pops the FIFO from the BIU and transmits in a stream to the card data bus. If the FIFO becomes empty, the card clock is stopped and restarted once data is available in the FIFO.

If the `byte_count` register is programmed to 0, it is an open-ended stream-write data transfer. During this data transfer, the data path continuously transmits data in a stream until the host software issues a stop command. A stream data transfer is terminated when the end bit of the stop command and end bit of the data match over two clocks.

If the `byte_count` register is programmed with a non-zero value and the `send_auto_stop` bit is set in the Command register, the stop command is internally generated and loaded in the command path when the end bit of the stop command occurs after the last byte of the stream write transfer matches. This data transfer can also terminate if the host issues a stop command before all the data bytes are transferred to the card bus.

3.3.2.1.2 Single Block Data

If the `transfer_mode` bit in the Command register is set to 0 and the `byte_count` register value is equal to the value of the `block_size` register, a single-block write-data transfer occurs. The data transmit state machine sends data in a single block, where the number of bytes equals the block size, including the internally-generated CRC16.

If the CTYPE register bit for the selected card – indicated by the `card_num` value in the Command register – is set for a 1-bit, 4-bit, or 8-bit data transfer, the data is transmitted on 1, 4, or 8 data lines, respectively, and CRC16 is separately generated and transmitted for 1, 4, or 8 data lines, respectively.

After a single data block is transmitted, the data transmit state machine receives the CRC status from the card and signals a data transfer to the BIU; this happens when the data-transfer-over bit is set in the RINTSTS register.

If a negative CRC status is received from the card, the data path signals a data CRC error to the BIU by setting the data CRC error bit in the RINTSTS register.

Additionally, if the start bit of the CRC status is not received by two clocks after the end of the data block, a CRC status start bit error is signaled to the BIU by setting the write-no-CRC bit in the RINTSTS register.

3.3.2.1.3 Multiple Block Data

A multiple-block write-data transfer occurs if the `transfer_mode` bit in the Command register is set to 0 and the value in the `byte_count` register is not equal to the value of the `block_size` register. The data transmit state machine sends data in blocks, where the number of bytes in a block equals the block size, including the internally-generated CRC16.

If the CTYPE register bit for the selected card – indicated by the `card_num` value in the Command register – is set to 1-bit, 4-bit, or 8-bit data transfer, the data is transmitted on 1, 4, or 8 data lines, respectively, and CRC16 is separately generated and transmitted on 1, 4, or 8 data lines, respectively.

After one data block is transmitted, the data transmit state machine receives the CRC status from the card. If the remaining `byte_count` becomes 0, the data path signals to the BIU that the data transfer is done; this happens when the data-transfer-over bit is set in the RINTSTS register.

If the remaining data bytes are greater than 0, the data path state machine starts to transmit another data block.

If a negative CRC status is received from the card, the data path signals a data CRC error to the BIU by setting the data CRC error bit in the RINTSTS register, and continues further data transmission until all the bytes are transmitted.

Additionally, if the CRC status start bit is not received by two clocks after the end of a data block, a CRC status start bit error is signaled to the BIU by setting the write-no-CRC bit in the RINTSTS register; further data transfer is terminated.

If the `send_auto_stop` bit is set in the Command register, the stop command is internally generated during the transfer of the last data block, where no extra bytes are transferred to the card. The end bit of the stop command may not exactly match the end bit of the CRC status in the last data block.

If the block size is less than 4, 16, or 32 for card data widths of 1 bit, 4 bits, or 8 bits, respectively, the data transmit state machine terminates the data transfer when all the data is transferred, at which time the internally generated stop command is loaded in the command path.

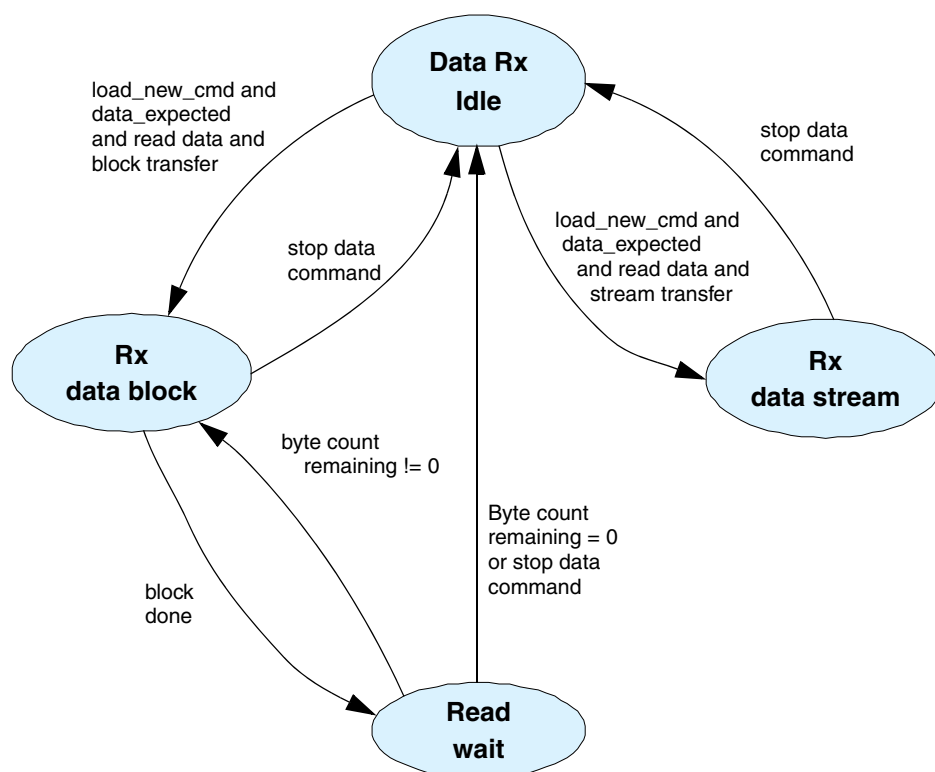
If the `byte_count` is 0 – the block size must be greater than 0 – it is an open-ended block transfer. The data transmit state machine for this type of data transfer continues the block-write data transfer until the host software issues a stop or abort command.

3.3.2.2 Data Receive

The data-receive state machine, illustrated in [Figure 3-37](#), receives data two clock cycles after the end bit of a data read command, even if the command path detects a response error or response CRC error. If a response is not received from the card because a response timeout occurs, the BIU does not receive a signal that the data transfer is complete; this happens if the command sent by the `DWC_mobile_storage` is an illegal operation for the card, which keeps the card from starting a read data transfer.

If data is not received before the data timeout, the data path signals a data timeout to the BIU and an end to the data transfer done. Based on the value of the `transfer_mode` bit in the Command register, the data-receive state machine gets data from the card data bus in a stream or block(s).

Figure 3-37 Data-Receive State Machine



3.3.2.2.1 Stream Data Read

A stream-read data transfer occurs if the `transfer_mode` bit in the Command register equals 1, at which time the data path receives data from the card and pushes it to the FIFO. If the FIFO becomes full, the card clock stops and restarts once the FIFO is no longer full.

An open-ended stream-read data transfer occurs if the `byte_count` register equals 0. During this type of data transfer, the data path continuously receives data in a stream until the host software issues a stop command. A stream data transfer terminates two clock cycles after the end bit of the stop command.

If the `byte_count` register contains a non-zero value and the `send_auto_stop` bit is set in the Command register, a stop command is internally generated and loaded into the command path, where the end bit of the stop command occurs after the last byte of the stream data transfer is received. This data transfer can terminate if the host issues a stop or abort command before all the data bytes are received from the card.

3.3.2.2.2 Single-Block Data Read

A single-block read-data transfer occurs if the `transfer_mode` bit in the Command register is set to 0 and the value of the `byte_count` register is equal to the value of the `block_size` register. When a start bit is received before the data times out, data bytes equal to the block size and CRC16 are received and checked with the internally-generated CRC16.

If the CTYPE register bit for the selected card – indicated by the `card_num` value in the Command register – is set to a 1-bit, 4-bit, or 8-bit data transfer, data is received from 1, 4, or 8 data lines, respectively, and CRC16 is separately generated and checked for 1, 4, or 8 data lines, respectively. If there is a CRC16 mismatch, the data path signals a data CRC error to the BIU. If the received end bit is not 1, the BIU receives an end-bit error.

3.3.2.2.3 Multiple-Block Data Read

If the `transfer_mode` bit in the Command register is set to 0 and the value of the `byte_count` register is not equal to the value of the `block_size` register, it is a multiple-block read-data transfer. The data-receive state machine receives data in blocks, where the number of bytes in a block is equal to the block size, including the internally-generated CRC16.

If the CTYPE register bit for the selected card – indicated by the `card_num` value in the Command register – is set to a 1-bit, 4-bit, or 8-bit data transfer, data is received from 1, 4, or 8 data lines, respectively, and CRC16 is separately generated and checked for 1, 4, or 8 data lines, respectively. After a data block is received, if the remaining `byte_count` becomes 0, the data path signals a data transfer to the BIU.

If the remaining data bytes are greater than 0, the data path state machine causes another data block to be received. If CRC16 of a received data block does not match the internally-generated CRC16, a data CRC error to the BIU and data reception continue further data transmission until all bytes are transmitted. Additionally, if the end of a received data block is not 1, data on the data path signals terminate the bit error to the CIU and the data-receive state machine terminates data reception, waits for data timeout, and signals to the BIU that the data transfer is complete.

If the `send_auto_stop` bit is set in the Command register, the stop command is internally generated when the last data block is transferred, where no extra bytes are transferred from the card; the end bit of the stop command may not exactly match the end bit of the last data block.

If the requested block size for data transfers to cards is less than 4, 16, or 32 bytes for 1-bit, 4-bit, or 8-bit data transfer modes, respectively, the data-transmit state machine terminates the data transfer when all data is transferred, at which point the internally-generated stop command is loaded in the command path. Data received from the card after that are then ignored by the data path.

If the `byte_count` is 0 — the block size must be greater than 0 — it is an open-ended block transfer. For this type of data transfer, the data-receive state machine continues the block-read data transfer until the host software issues a stop or abort command.

3.3.2.3 Auto-Stop

The `DWC_mobile_storage` internally generates a stop command and is loaded in the command path when the `send_auto_stop` bit is set in the Command register. The auto-stop command helps to send an exact number of data bytes using a stream read or write for the MMC, and a multiple-block read or write for SD memory transfer for SD cards.

The software should set the `send_auto_stop` bit according to details listed in [Table 3-8](#).

Table 3-8 Auto-Stop Generation

Card type	Transfer type	Byte Count	send_auto_stop bit set	Comments
MMC	Stream read	0	No	Open-ended stream
MMC	Stream read	> 0	Yes	Auto-stop after all bytes transfer
MMC	Stream write	0	No	Open-ended stream
MMC	Stream write	> 0	Yes	Auto-stop after all bytes transfer
MMC	Single-block read	> 0	No	Byte count = 0 is illegal
MMC	Single-block write	> 0	No	Byte count = 0 is illegal
MMC	Multiple-block read	0	No	Open-ended multiple block
MMC	Multiple-block read	> 0	Yes **	Pre-defined multiple block
MMC	Multiple-block write	0	No	Open-ended multiple block
MMC	Multiple-block write	> 0	Yes **	Pre-defined multiple block
SDMEM	Single-block read	> 0	No	Byte count = 0 is illegal
SDMEM	Single-block write	> 0	No	Byte count = 0 illegal
SDMEM	Multiple-block read	0	No	Open-ended multiple block
SDMEM	Multiple-block read	> 0	Yes	Auto-stop after all bytes transfer
SDMEM	Multiple-block write	0	No	Open-ended multiple block
SDMEM	Multiple-block write	> 0	Yes	Auto-stop after all bytes transfer
SDIO	Single-block read	> 0	No	Byte count = 0 is illegal

** The condition under which the transfer mode is set to block transfer and `byte_count` is equal to block size is treated as a single-block data transfer command for both MMC and SD cards. If `byte_count = n * block_size` ($n = 2, 3, \dots$), the condition is treated as a predefined multiple-block data transfer command. In the case of an MMC card, the host software can perform a predefined data transfer in two ways: 1) Issue the CMD23 command before issuing CMD18/CMD25 commands to the card – in this case, issue CMD18/CMD25 commands without setting the `send_auto_stop` bit. 2) Issue CMD18/CMD25 commands without issuing CMD23 command to the card, with the `send_auto_stop` bit set. In this case, the multiple-block data transfer is terminated by an internally-generated auto-stop command after the programmed byte count.

Table 3-8 Auto-Stop Generation (Cont.)

Card type	Transfer type	Byte Count	send_auto_stop bit set	Comments
SDIO	Single-block write	> 0	No	Byte count = 0 illegal
SDIO	Multiple-block read	0	No	Open-ended multiple block
SDIO	Multiple-block read	> 0	No	Pre-defined multiple block
SDIO	Multiple-block write	0	No	Open-ended multiple block
SDIO	Multiple-block write	> 0	No	Per-defined multiple block

** The condition under which the transfer mode is set to block transfer and *byte_count* is equal to block size is treated as a single-block data transfer command for both MMC and SD cards. If *byte_count* = *n* * *block_size* (*n* = 2, 3, ...), the condition is treated as a predefined multiple-block data transfer command. In the case of an MMC card, the host software can perform a predefined data transfer in two ways: 1) Issue the CMD23 command before issuing CMD18/CMD25 commands to the card – in this case, issue CMD18/CMD25 commands without setting the send_auto_stop bit. 2) Issue CMD18/CMD25 commands without issuing CMD23 command to the card, with the send_auto_stop bit set. In this case, the multiple-block data transfer is terminated by an internally-generated auto-stop command after the programmed byte count.

The following list conditions for the auto-stop command.

- ❖ Stream read for MMC card with byte count greater than 0 – The DWC_mobile_storage generates an internal stop command and loads it into the command path so that the end bit of the stop command is sent out when the last byte of data is read from the card and no extra data byte is received. If the byte count is less than 6 (48 bits), a few extra data bytes are received from the card before the end bit of the stop command is sent.
- ❖ Stream write for MMC card with byte count greater than 0 – The DWC_mobile_storage generates an internal stop command and loads it into the command path so that the end bit of the stop command is sent when the last byte of data is transmitted on the card bus and no extra data byte is transmitted. If the byte count is less than 6 (48 bits), the data path transmits the data last in order to meet the above condition.
- ❖ Multiple-block read memory for SD card with byte count greater than 0 – If the block size is less than 4 (single-bit data bus), 16 (4-bit data bus), or 32 (8-bit data bus), the auto-stop command is loaded in the command path after all the bytes are read. Otherwise, the stop command is loaded in the command path so that the end bit of the stop command is sent after the last data block is received.
- ❖ Multiple-block write memory for SD card with byte count greater than 0 – If the block size is less than 3 (single-bit data bus), 12 (4-bit data bus), or 24 (8-bit data bus), the auto-stop command is loaded in the command path after all data blocks are transmitted. Otherwise, the stop command is loaded in the command path so that the end bit of the stop command is sent after the end bit of the CRC status is received.
- ❖ Precaution for host software during auto-stop – Whenever an auto-stop command is issued, the host software should not issue a new command to the DWC_mobile_storage until the auto-stop is sent by the DWC_mobile_storage and the data transfer is complete. If the host issues a new command during a data transfer with the auto-stop in progress, an auto-stop command may be sent after the new command is sent and its response is received; this can delay sending the stop command, which transfers extra data bytes. For a stream write, extra data bytes are erroneous data that can corrupt the

card data. If the host wants to terminate the data transfer before the data transfer is complete, it can issue a stop or abort command, in which case the DWC_mobile_storage does not generate an auto-stop command.

3.3.3 Non-Data Transfer Commands that Use Data Path

Some non-data transfer commands (non-read/write commands) also use the data path. [Table 3-9](#) lists the commands and register programming requirements for them.

Table 3-9 Non-Data Transfer Commands and Requirements

	CMD27	CMD30	CMD42	ACMD13	ACMD22	ACMD51
Command register programming						
Cmd_index	6'h1B	6'h1E	6'h2A	6'h0D	6'h16	6'h33
Response_expect	1	1	1	1	1	1
Response_length	0	0	0	0	0	0
Check_response_crc	1	1	1	1	1	1
Data_expected	1	1	1	1	1	1
Read/write	1	0	1	0	0	0
Transfer_mode	0	0	0	0	0	0
Send_auto_stop	0	0	0	0	0	0
Wait_prevdata_complete	0	0	0	0	0	0
Stop_abort_cmd	0	0	0	0	0	0
Command Argument register programming						
	Stuff bits	32-bit write protect data address	Stuff bits	Stuff bits	Stuff bits	Stuff bits
Block Size register programming						
	16	4	Num_bytes*	64	4	8
Byte Count register programming						
	16	4	Num_bytes*	64	4	8
*: Num_bytes = No. of bytes specified as per the lock card data structure (Refer to the SD specification and the MMC specification).						

3.3.4 SDIO Interrupt Control

Interrupts for SD cards are reported to the BIU by asserting an interrupt signal for two clock cycles. SDIO cards signal an interrupt by asserting `cdata_in` low during the interrupt period; an interrupt period for the selected card is determined by the interrupt control state machine. An interrupt period is always valid for non-active or non-selected cards, and 1-bit data mode for the selected card. An interrupt period for a wide-bus active or selected card is valid for the following conditions:

- ❖ Card is idle
- ❖ Non-data transfer command in progress
- ❖ Third clock after end bit of data block between two data blocks
- ❖ From two clocks after end bit of last data until end bit of next data transfer command

Bear in mind that, in the following situations, the `DWC_mobile_storage` does not sample the SDIO interrupt of the selected card when the card data width is 4 bits. Since the SDIO interrupt is level-triggered, it is sampled in a further interrupt period and the host does not lose any SDIO interrupt from the card.

1. Read/Write Resume – The CIU treats the resume command as a normal data transfer command. SDIO interrupts during the resume command are handled similarly to other data commands.

According to the SDIO specification, for the normal data command the interrupt period ends after the command end bit of the data command; for the resume command, it ends after the response end bit. In the case of the resume command, the `DWC_mobile_storage` stops the interrupt sampling period after the resume command end bit, instead of stopping after the response end bit of the resume command.

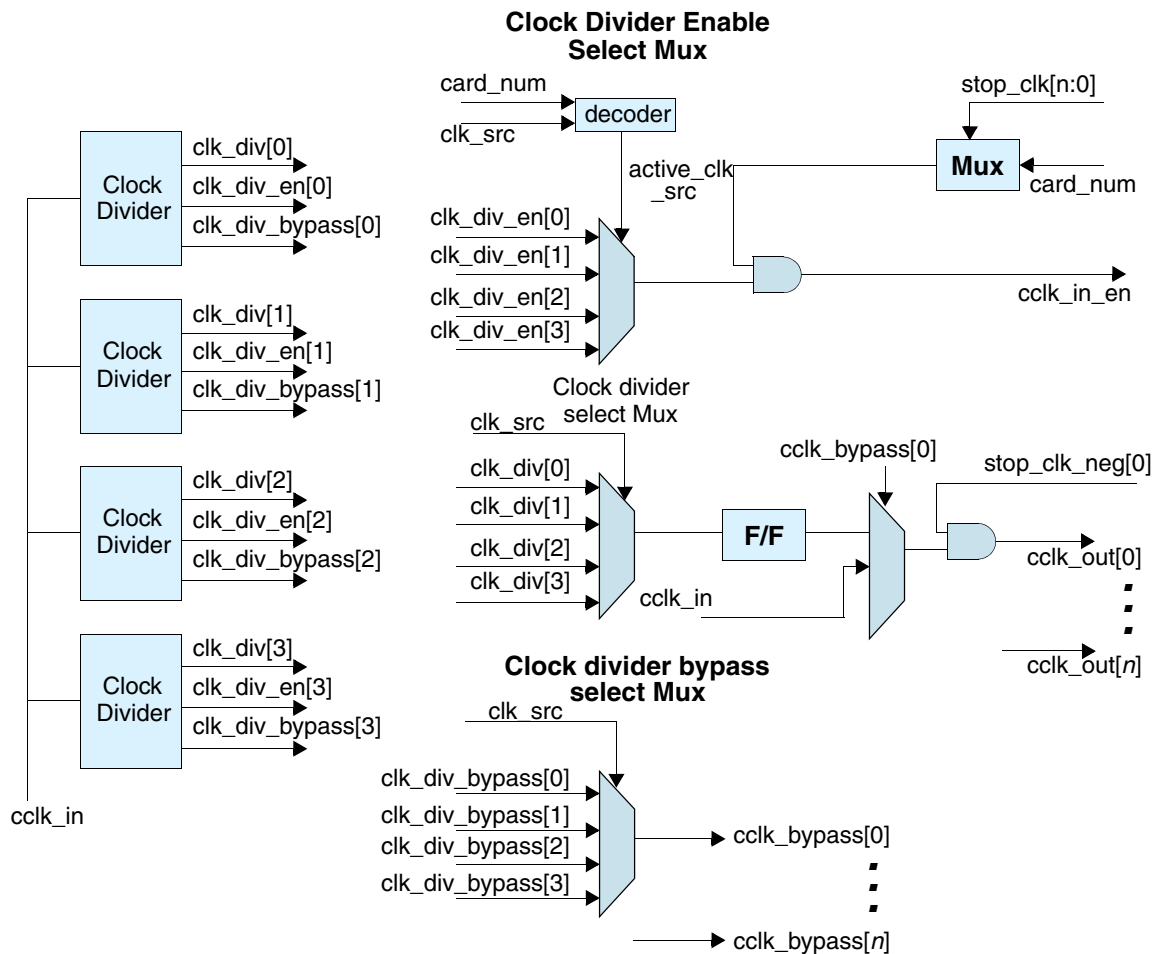
2. Suspend during read transfer – If the read data transfer is suspended by the host, the host sets the `abort_read_data` bit in the `DWC_mobile_storage` to reset the data state machine. In the CIU, the SDIO interrupts are handled such that the interrupt sampling starts after the `abort_read_data` bit is set by the host. In this case the `DWC_mobile_storage` does not sample SDIO interrupts between the period from response of the suspend command to setting the `abort_read_data` bit, and starts sampling after setting the `abort_read_data` bit.

3.3.5 Clock Control

The clock control block provides different clock frequencies required for `SD_MMC_CEATA` cards. The `cclk_in` signal is the source clock (`cclk_in` $\geq$ card max operating frequency) for clock dividers of the clock control block. This source clock (`cclk_in`) is used to generate different card clock frequencies (`cclk_out`[`NUM_CARD_BUS`-1:0]). Each card clock can have different clock frequencies, since the SD card can be a low-speed SD card or a full-speed SD card. The `DWC_mobile_storage` provides one clock signal (`cclk_out`) per card, which allows each card to operate at different clock frequencies.

Figure 3-38 illustrates the clock control logic.

Figure 3-38 Clock Control Logic



The clock frequency of a card depends on the following clock control registers:

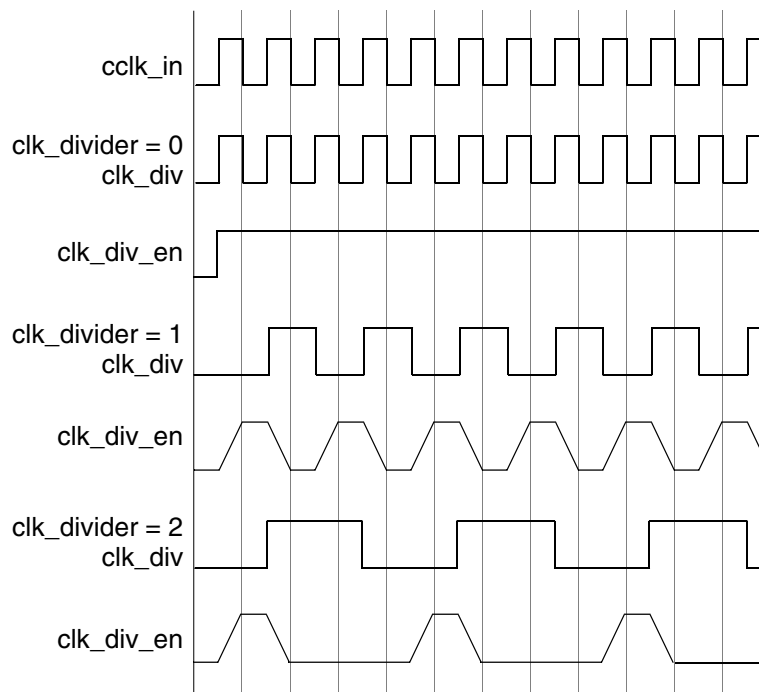
- ❖ **Clock Divider register** – Internal clock dividers are used to generate different clock frequencies required for cards; a configuration parameter determines the number of clock dividers (1-4). The division factor for each clock divider can be programmed by writing to the Clock Divider register. The clock divider is an 8-bit value that provides a clock division factor from 1 to 510; a value of 0 represents a clock-divider bypass, a value of 1 represents a divide by 2, a value of 2 represents a divide by 4, and so on.
- ❖ **Clock Source register** – One of the divided clocks from four clock dividers is selected for a card `cclk_out` by programming the Clock Source register.
- ❖ **Clock Control register** – `cclk_out` can be enabled or disabled for each card under the following conditions:
 - ◆ `clk_enable` – `cclk_out` for a card is enabled if the `clk_enable` bit for a card in the Clock Control register is programmed (set to 1) or disabled (set to 0).
 - ◆ **Low-power mode** – Low-power mode of a card can be enabled by setting the low-power mode bit of the Clock Control register to 1. If low-power mode is enabled to save card power, the

cclk_out signal is disabled when the card is idle for at least 8 card clock cycles. It is enabled when a new command is loaded and the command path goes to a non-idle state.

Additionally, cclk_out of a selected or active card is disabled when an internal FIFO is full – card read (no more data can be received from card) – or when the FIFO is empty – card write (no data is available for transmission). This helps to avoid FIFO overrun and underrun conditions.

The clock control unit also provides a clock divide enable signal, shown in [Figure 3-39](#). It is used by the command and data path to qualify cclk_in for driving outputs and sampling inputs at the programmed clock frequency for the selected card, according to the Clock Divider and Clock Source register values.

Figure 3-39 Clock Divider Waveform



Under the following conditions, the card clock is stopped or disabled, along with the active clk_en, for the selected card:

- ❖ Clock can be disabled by writing to Clock Enable register (clk_en bit = 1).
- ❖ If low-power mode is selected and card is idle, or not selected for 8 clocks.
- ❖ FIFO is full and data path cannot accept more data from the card and data transfer is incomplete – to avoid FIFO overrun.
- ❖ FIFO is empty and data path cannot transmit more data to the card and data transfer is incomplete – to avoid FIFO underrun.



Note

Care should be taken by the host firmware while changing the Clock Divider register and Clock Source register values. The card clock must be disabled through the Clock Control register before changing the values of the Clock Divider and Clock Source registers.

3.3.6 SD_MMC_CEATA Mux/Demux Unit

In MMC-only mode, there is a single shared bus connected between the `DWC_mobile_storage` and all the cards. In `SD_MMC_CEATA` mode, a separate bus runs between the `DWC_mobile_storage` and each card. In this mode, the demux logic sends the command or data to only the selected card when commands or data are sent from the controller. The unselected cards see those on their command or data path; a card is selected by the `card_number` value set in the Command register. Similarly, the response or data input from the selected card is multiplexed and sent to the `DWC_mobile_storage`.

The `cclk_in` signal registers all outputs to `SD_MMC_CEATA` cards in this block. If the `use_hold_reg` programmable bit is set, then all outputs are again registered by the `cclk_in_drv` clock, which is a delayed version of `cclk_in` and is used to guarantee high hold-time requirements of the `SD_MMC_CEATA` cards operating in SDR12 and SDR25 speed modes.

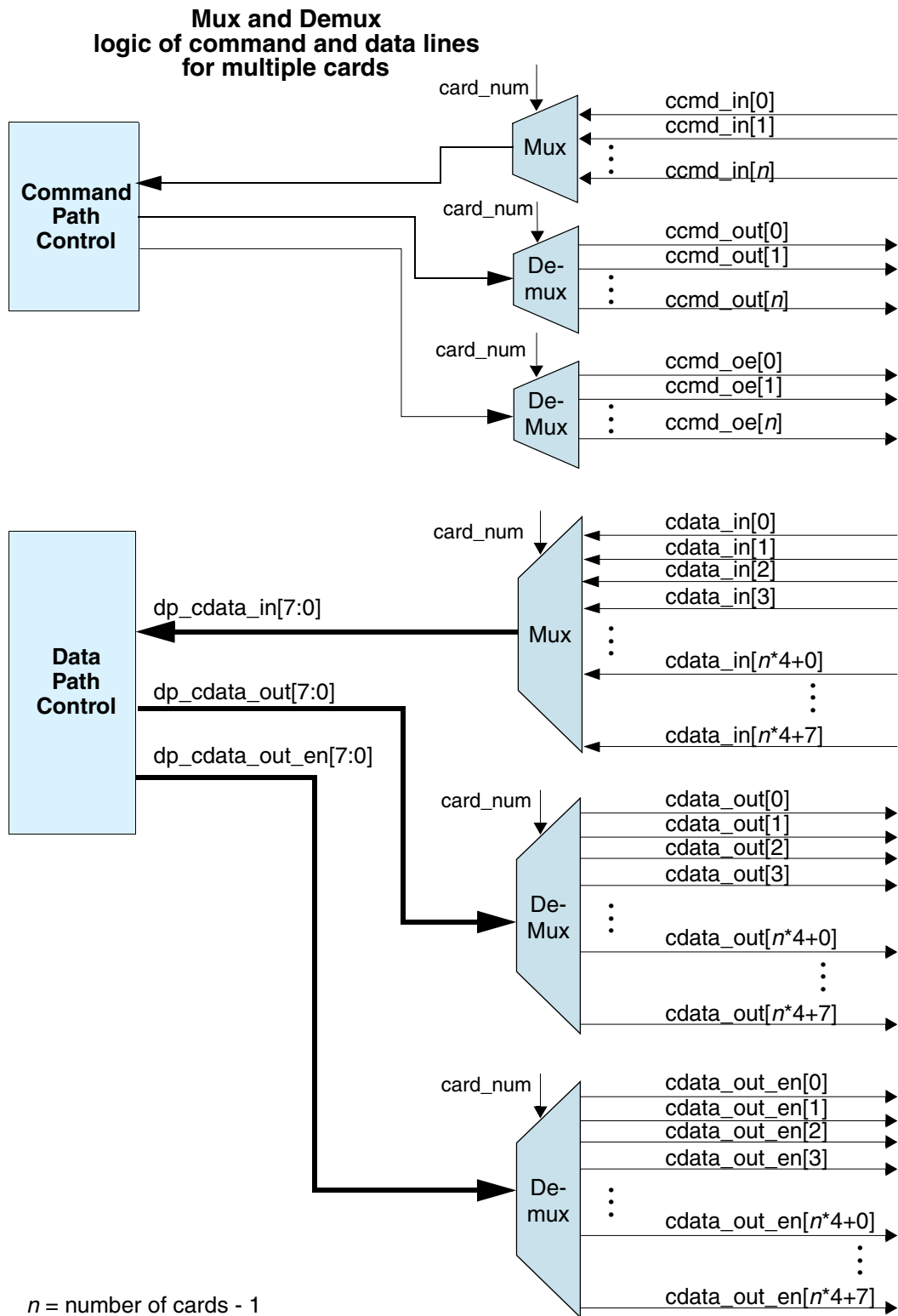
For DDR50, SDR50, and SDR104 speed modes, `use_hold_reg` bit may be programmed with 0 or 1. If the `cclk_in_drv` is phase-shifted by zero degrees, then the `use_hold_reg` bit is not set and the outputs to the `SD_MMC_CEATA` cards are not registered by the `cclk_in_drv` clock. If the `cclk_in_drv` is phase-shifted by non-zero degrees, then the `use_hold_reg` bit is set and the outputs to the `SD_MMC_CEATA` cards are registered by the `cclk_in_drv` clock.

To guarantee that the hold-time requirements are met when the card is operating in these modes, a buffers or delay lines need to be added to all the card outputs — `ccmd_out`, `ccmd_out_en`, `cdata_out`, and `cdata_out_en` — in order to meet the hold-time requirement.

For details that require attention during implementation, refer to the Clock Input Structure and the Output Path Implementation Structure specified in the [“Host Controller Clock Structure and Card Timing Requirements”](#) on page 284.

The `cclk_in_sample` signal samples all inputs from `SD_MMC_CEATA` cards in this block; `cclk_in_sample` is a delayed version of `cclk_in` and matches `cclk_out` so that the signals driven by `SD_MMC_CEATA` cards will meet setup and hold requirements when sampled. The sampling is further enabled with the `cclk_out_en_neg[*]` internal signal so that the input from a card is sampled only when the `cclk_out` that corresponds to that card has a positive-edge transition.

Figure 3-40 Mux/Demux Logic



3.3.7 Error Detection

❖ Response

- ◆ Response timeout – Response expected with response start bit is not received within programmed number of clocks in timeout register.
- ◆ Response CRC error – Response is expected and check response CRC requested; response CRC7 does not match with the internally-generated CRC7.
- ◆ Response error – Response transmission bit is not 0, command index does not match with the command index of the send command, or response end bit is not 1.

❖ Data transmit

- ◆ No CRC status – During a write data transfer, if the CRC status start bit is not received two clocks after the end bit of the data block is sent out, the data path does the following:
 - ❖ Signals no CRC status error to the BIU
 - ❖ Terminates further data transfer
 - ❖ Signals data transfer done to the BIU
- ◆ Negative CRC – If the CRC status received after the write data block is negative (that is, not 010), a data CRC error is signaled to the BIU and further data transfer is continued.
- ◆ Data starvation due to empty FIFO – If the FIFO becomes empty during a write data transmission, or if the card clock is stopped and the FIFO remains empty for data timeout clocks, then a data-starvation error is signaled to the BIU and the data path continues to wait for data in the FIFO.

❖ Data receive

- ◆ Data timeout – During a read-data transfer, if the data start bit is not received before the number of clocks that were programmed in the timeout register, the data path does the following:
 - ❖ Signals data-timeout error to the BIU
 - ❖ Terminates further data transfer
 - ❖ Signals data transfer done to BIU
- ◆ Data start bit error – During a 4-bit or 8-bit read-data transfer, if the all-bit data line does not have a start bit, the data path signals a data start bit error to the BIU and waits for a data timeout, after which it signals that the data transfer is done.
- ◆ Data CRC error – During a read-data-block transfer, if the CRC16 received does not match with the internally generated CRC16, the data path signals a data CRC error to the BIU and continues further data transfer.
- ◆ Data end-bit error – During a read-data transfer, if the end bit of the received data is not 1, the data path signals an end-bit error to the BIU, terminates further data transfer, and signals to the BIU that the data transfer is done.
- ◆ Data starvation due to FIFO full – During a read data transmission and when the FIFO becomes full, the card clock is stopped. If the FIFO remains full for data timeout clocks, a data starvation error is signaled to the BIU (Data Starvation by Host Timeout bit is set in RINTSTS Register) and the data path continues to wait for the FIFO to start to empty.

3.4 Clock Domains

There are four clocks and five clock domains used in the DWC_mobile_storage core. Both the positive and negative edges of cclk_in are used. The clocks are:

- ❖ clk, posedge – System/host clock; BIU block clock.
- ❖ cclk_in, posedge – Card clock; clk and cclk_in frequencies should meet a “clk $\geq$ 1/10 cclk_in” requirement, since a back-to-back CIU-to-BIU c2b_response can happen once in every 32 cclk_in clocks.
- ❖ cclk_in_drv – Delayed cclk_in to drive the card outputs – hold register clock; for details on timing requirements, refer to [“Timing Requirements for SD 3.0 Cards”](#) on page 297.

The registers clocked by cclk_in_drv (sampling signals generated by cclk_in positive edge) when use_hold_reg bit is programmed to 1'b1 are:

- ◆ U_DWC_mobile_storage_ciu/U_DWC_mobile_storage_muxdemux/ccmd_out_en[*]¹
- ◆ U_DWC_mobile_storage_ciu/U_DWC_mobile_storage_muxdemux/ccmd_out[*]
- ◆ U_DWC_mobile_storage_ciu/U_DWC_mobile_storage_muxdemux/cdata_out_en[*]
- ◆ U_DWC_mobile_storage_ciu/U_DWC_mobile_storage_muxdemux/cdata_out[*]
- ❖ cclk_in_sample, posedge – Delayed cclk_in to sample card inputs. All card inputs to the core (driven by the memory cards) are guaranteed to meet only setup and hold times with regard to cclk_out on the PAD. Since there is a large skew between cclk_in and cclk_out (cclk_out generation delay + routing delay + cclk_out PAD delay), the card inputs cannot be directly sampled by cclk_in.

A delayed sample clock is needed in order to sample the inputs first, and the sampled outputs are used inside the core. The cclk_in_sample should match cclk_out on PAD; that is, delay + ccmd/cdata input PAD delay.

The registers clocked by cclk_in_sample (ccmd_in and cdata_in core input signals) are:

- ◆ U_DWC_mobile_storage_ciu/U_DWC_mobile_storage_muxdemux/ccmd_in_sample[*]
- ◆ U_DWC_mobile_storage_ciu/U_DWC_mobile_storage_muxdemux/cdata_in_sample[*]
- ❖ cclk_in, negedge – The clock-gating signals that gate cclk_out are generated from the falling edge of cclk_in in order to avoid clock-gating glitches on the cclk_out ports. The cclk_out is gated when a card is idle and when low-power mode is selected. Additionally, during data transfers the cclk_out is gated in order to avoid data loss when the host does not push or pop data to or from the FIFO at the rate the data is sent or received from the cards; that is, when the FIFO is full and a read from a card occurs, or the FIFO is empty and a write to a card occurs.

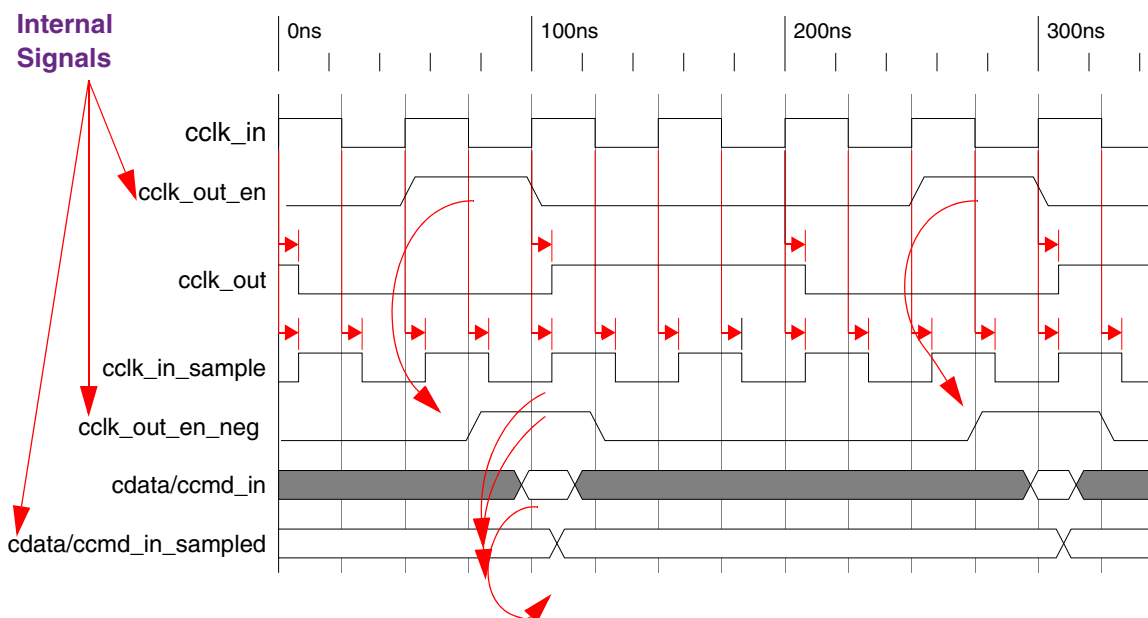
Additionally, the cclk_in posedge-triggered card clock enable signals – generated by the clock-divider unit – are registered by the negedge of cclk_in before passing them to the cclk_in_sample domain in order to guarantee setup and hold times. Since each card cclk_out frequency depends on the baud-rate, the card inputs – cdata_in and ccmd_in – are sampled only when there is a posedge on the corresponding cclk_out.

Since the cclk_in_sample is a free-running undivided clock, the input sampling is qualified with regard to clock_enable signals generated by the baud-rate logic. [Figure 3-41](#) illustrates the

1. After synthesis (by design compiler), these registers have an “_reg” extension.

relationship between `cclk_in`, `cclk_out`, `cclk_in_sample`, `cdata_in`, `ccmd_in`, and the internal signals `cclk_out_en` and `cclk_out_en_neg`.

Figure 3-41 cclk_in, cclk_in_sample, and cclk_out Relationship



The registers clocked by the `cclk_in` negedge – that is, signals generated by `cclk_in` posedge – are:

- ◆ `U_DWC_mobile_storage_ciu/U_DWC_mobile_storage_clkcntl/stop_clk_neg[*]`
- ◆ `U_DWC_mobile_storage_ciu/U_DWC_mobile_storage_muxdemux/cclk_out_en_neg[*]`
- ❖ The following are the first stage of the double synchronization registers used when signals cross the BIU/CIU domains:
 - ◆ `U_DWC_mobile_storage_b2c/b2c_stage0_creset_n`
 - ◆ `U_DWC_mobile_storage_b2c/b2c_stage0`
 - ◆ `U_DWC_mobile_storage_c2b/c2b_stage0`
- ❖ The following are the first stage of the double synchronization registers for BIU/CIU domains for the FIFO control logic:
 - ◆ `U_DWC_mobile_storage_fifectl/wr_ptr_gry_from_clk1_d`
 - ◆ `U_DWC_mobile_storage_fifectl/rd_ptr_gry_from_clk2_d`
 - ◆ `U_DWC_mobile_storage_fifectl/rd_ptr_gry_from_clk1_d`
 - ◆ `U_DWC_mobile_storage_fifectl/wr_ptr_gry_from_clk2_d`

When doing gate-level simulations, timing checks should be disabled for these registers.

In addition, the following signals cross from the BIU to the CIU without any synchronization registers:

- ◆ `b2c_cmd_control`
- ◆ `b2c_cmd_argument`
- ◆ `b2c_cclk_enable`

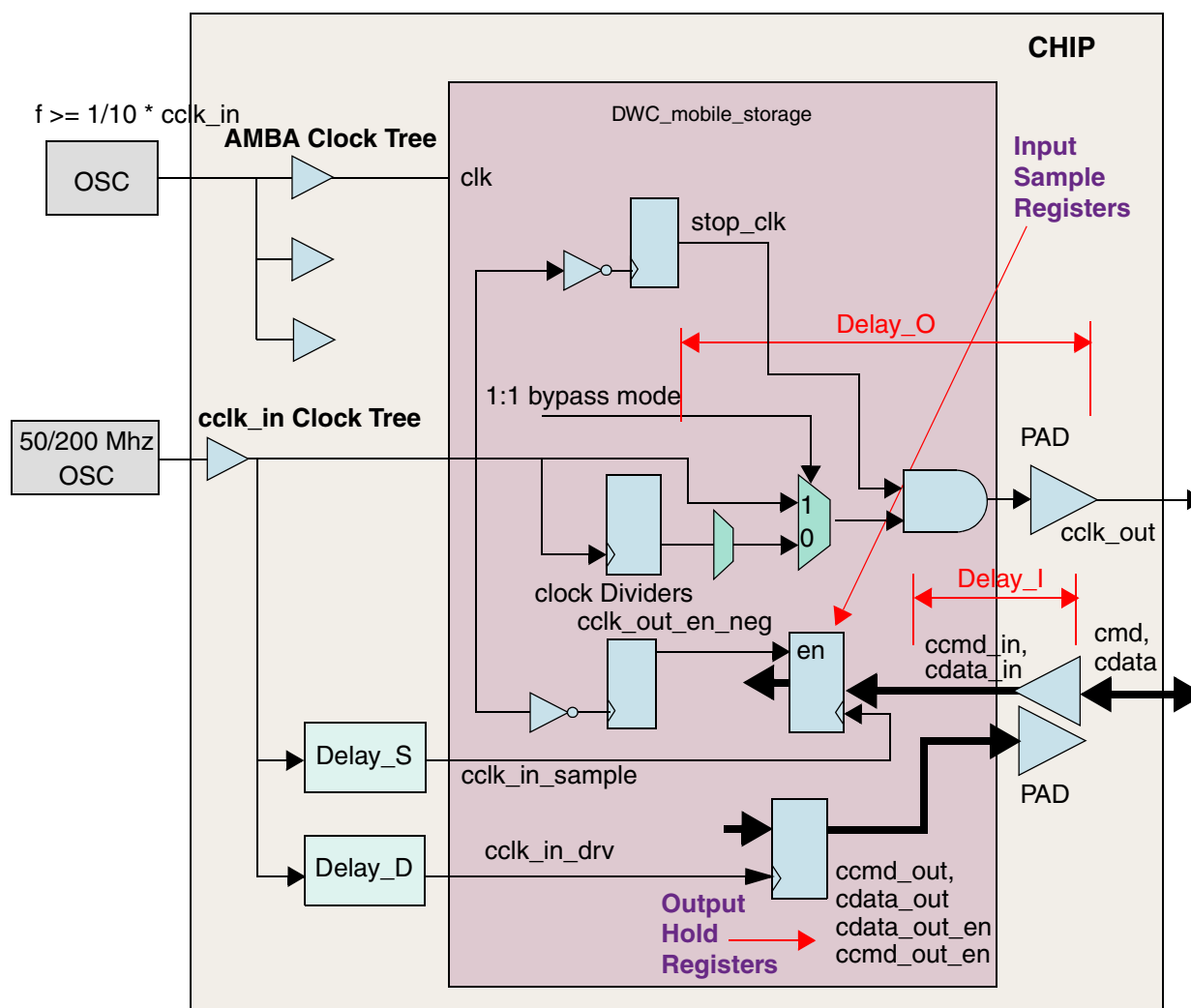
- ◆ b2c_cclk_low_power
- ◆ b2c_card_width
- ◆ b2c_card_type data

Similarly, the following signals cross from the CIU to the BIU without any synchronization registers:

- ◆ c2b_response_data
- ◆ c2b_response_addr
- ◆ c2b_trans_bytes_bin data

Figure 3-42 on page 107 and Figure 3-43 on page 108 show the `cclk_in`, `cclk_out`, `cclk_in_sampled`, and `cclk_in_drv` relationships.

Figure 3-42 Different Clocks



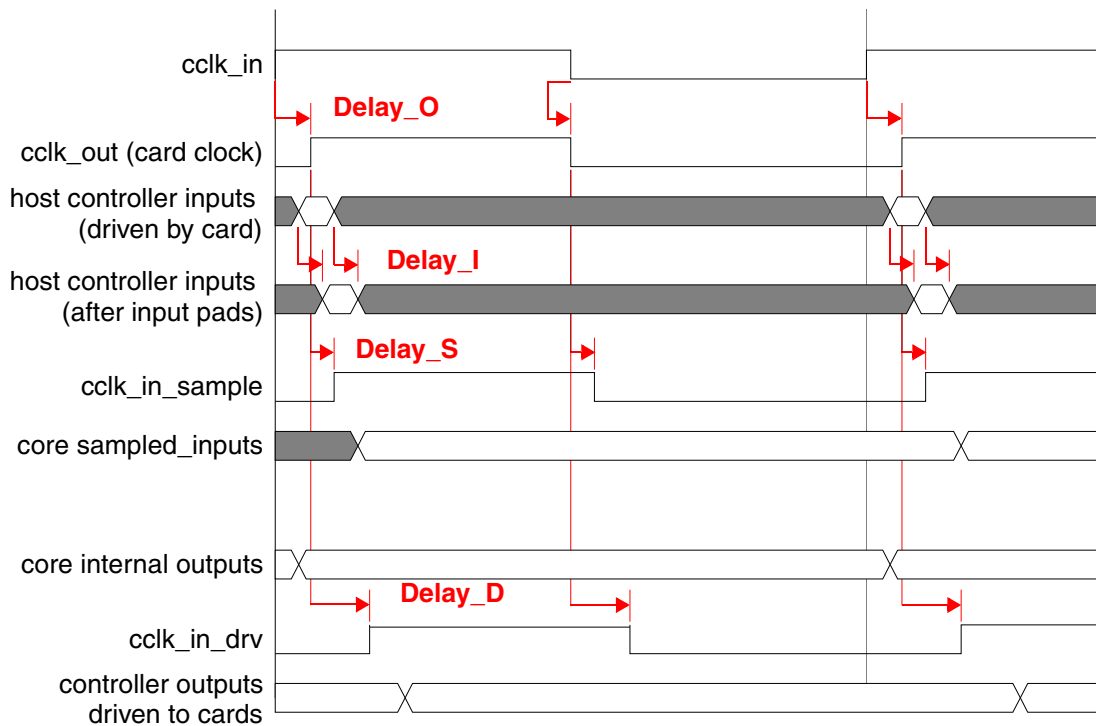
Delay_O = `cclk_in` to `cclk_out` delay (including PAD)

Delay_I = Input PAD delay + routing delay to input register

Delay_S = Delay_O + Delay_I

Delay_D = Delay_O + *hold_time*

- It is recommended that you use an internal PAD identical to the “`cclk_out`” PAD to match part of the delays of “Delay_S” and “Delay_D”. The “`cclk_in_sample`” timing is critical. The MMC cards are required to provide only a 6ns sampling window (3ns setup and 3ns hold) with regard to `cclk_out`.
- The above timings are applicable for only SD2.0 specifications. For detailed timing requirements for SD3.0 cards, refer to “[Timing Requirements for SD 3.0 Cards](#)” on page 297.

Figure 3-43 Clock Relationship**Note**

1. Because tests that were run on several SD (version 1.01) and MMC (version 3.3) cards show that the cards drive their outputs on the falling edge of the clock, it is recommended that you generate clocks using the following guidelines:
 - Connect a 50/50 duty-cycle clock to `cclk_in`
 - Connect inverted "`cclk_in`" to `cclk_in_drv`
2. Send `cclk_in` through an output PAD and bring it back with an input PAD; connect it to `cclk_in_sample` (and additional delay-matching to correctly sample the data/cmd driven out by the cards).
3. Delay values `delay_S` and `delay_D` should not cross half the clock cycle period of `cclk_in`.

3.5 Voltage Switching and UHS-1 Card Support

The DWC_mobile_storage supports SD 3.0 Ultra High Speed (UHS-1) and is capable of voltage switching in SD-mode, which can be applied to SD High-Capacity (SDHC) and SD eXtended Capacity (SDXC) cards.

**Note**

UHS-1 supports only 4-bit mode.

SD 3.0 UHS-1 supports the following transfer speed modes for UHS-50 and/or UHS-104 cards:

- ◆ DS – default-speed up to 25MHz, 3.3V signaling
- ◆ HS – high-speed up to 50MHz, 3.3V signaling
- ◆ SDR12 – SDR up to 25MHz, 1.8V signaling
- ◆ SDR25 – SDR up to 50MHz, 1.8V signaling
- ◆ SDR50 – SDR up to 100MHz, 1.8V signaling

- ◆ DDR50 – DDR up to 50MHz, 1.8V signaling

Table 3-10 lists the frequency and voltage levels appropriate for the UHS-50 and UHS-104 cards.

Table 3-10 Frequency and Voltage Levels for UHS-1 Speed Modes

Speed	Frequency	Signaling Mode	UHS Cards	
			UHS-50	UHS-104
DS	25 MHz	3.3V	✓	✓
HS	50 MHz	3.3V	✓	✓
SDR12	25 MHz	1.8V	✓	✓
SDR25	50 MHz	1.8V	✓	✓
SDR50	100 MHz	1.8V	✓	✓
SDR104	208 MHz	1.8V	✓	✓
DDR50	50 MHz	1.8V		✓

Voltage selection can be done in only SD mode. The first CMD0 selects the bus mode—either SD mode or SPI mode. The card must be in SD mode in order for 1.8V signaling mode to apply, during which time the card cannot be switched to SPI mode or 3.3V signaling without a power cycle.

If the System BIOS in an embedded system already knows that it is connected to an SD 3.0 card, then the driver programs the DWC_mobile_storage to initiate ACMD41. The software knows from the response of ACMD41 whether or not the card supports voltage switching to 1.8V.

- ❖ If bit 32 of ACMD41 response is 1'b1 – card supports voltage switching and next command – CMD11 – invokes voltage switching sequence. After CMD11 is started, the software must program the VOLT_REG register in the CSR space with the appropriate card number.
- ❖ If bit 32 of ACMD41 response is 1'b0 – card does not support voltage switching and CMD11 should not be started.

If the card and host controller accept voltage switching, then they support UHS-1 modes of data transfer. After the voltage switch to 1.8V, SDR12 is the default speed.

Since the UHS-1 can be used in only 4-bit mode, the software must start ACMD6 and change the card data width to 4-bit mode; ACMD6 is driven in any of the UHS-1 speeds.

If the host wants to select the DDR mode of data transfer, then the software must program the DDR_REG register in the CSR space with the appropriate card number.

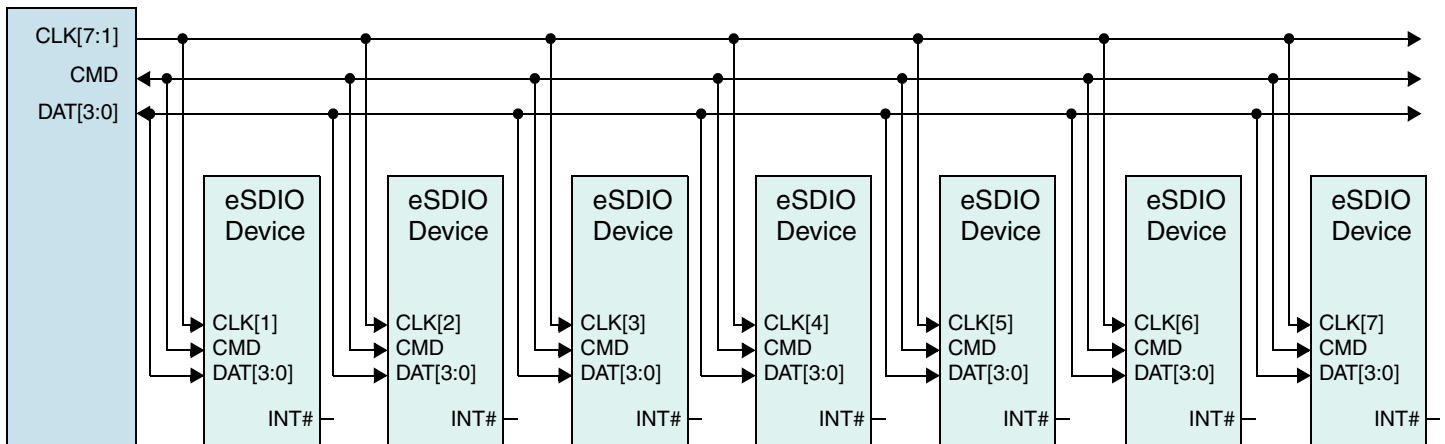
To choose from any of the SDR or DDR modes, appropriate values should be programmed in the CLKDIV register.

For information on VOLT_REG and DDR_REG registers, refer to “UHS_REG” on page 167.

3.6 Shared Bus Mode

SDIO 3.0 supports shared bus mode, which enables multiple devices/cards to be placed on the same bus. Figure 3-44 illustrates the shared bus mode.

Figure 3-44 Shared Bus Mode



There can be a maximum of seven devices on the same CMD and DAT lines; each device has a separate clock. A DWC_mobile_storage configuration can contain sixteen devices. The following shows just some of the possible eSDIO configurations:

- ❖ Three shared buses:
 - ◆ Two with seven eSDIO devices each
 - ◆ One with two eSDIO devices
- ❖ Two shared buses:
 - ◆ Two with seven eSDIO devices each and two SD devices
- ❖ Eight shared buses:
 - ◆ Seven with two eSDIO devices each
 - ◆ One with two eSDIO devices
- ❖ Seven shared buses:
 - ◆ Seven with two eSDIO devices each and two SD devices

Figure 3-45 illustrates the DWC_mobile_storage implementation of the shared bus mode and shows two I/O buffers connected to the same bus; these connections are external to the DWC_mobile_storage.

Figure 3-45 Shared Bus Mode Implementation

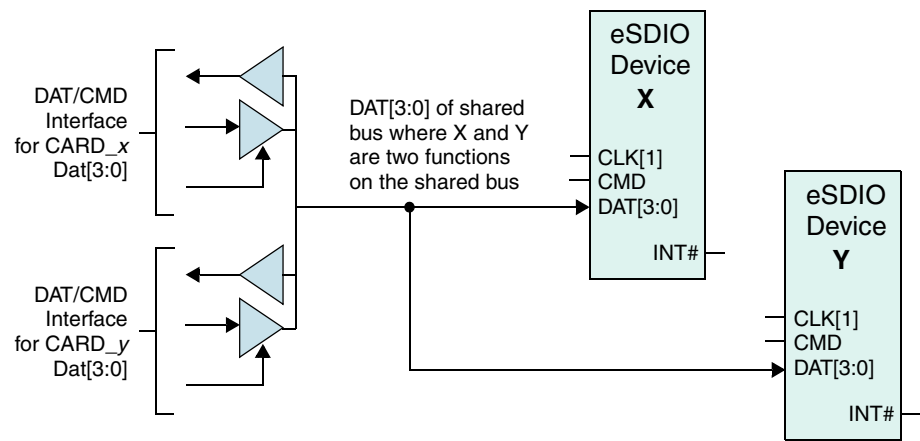
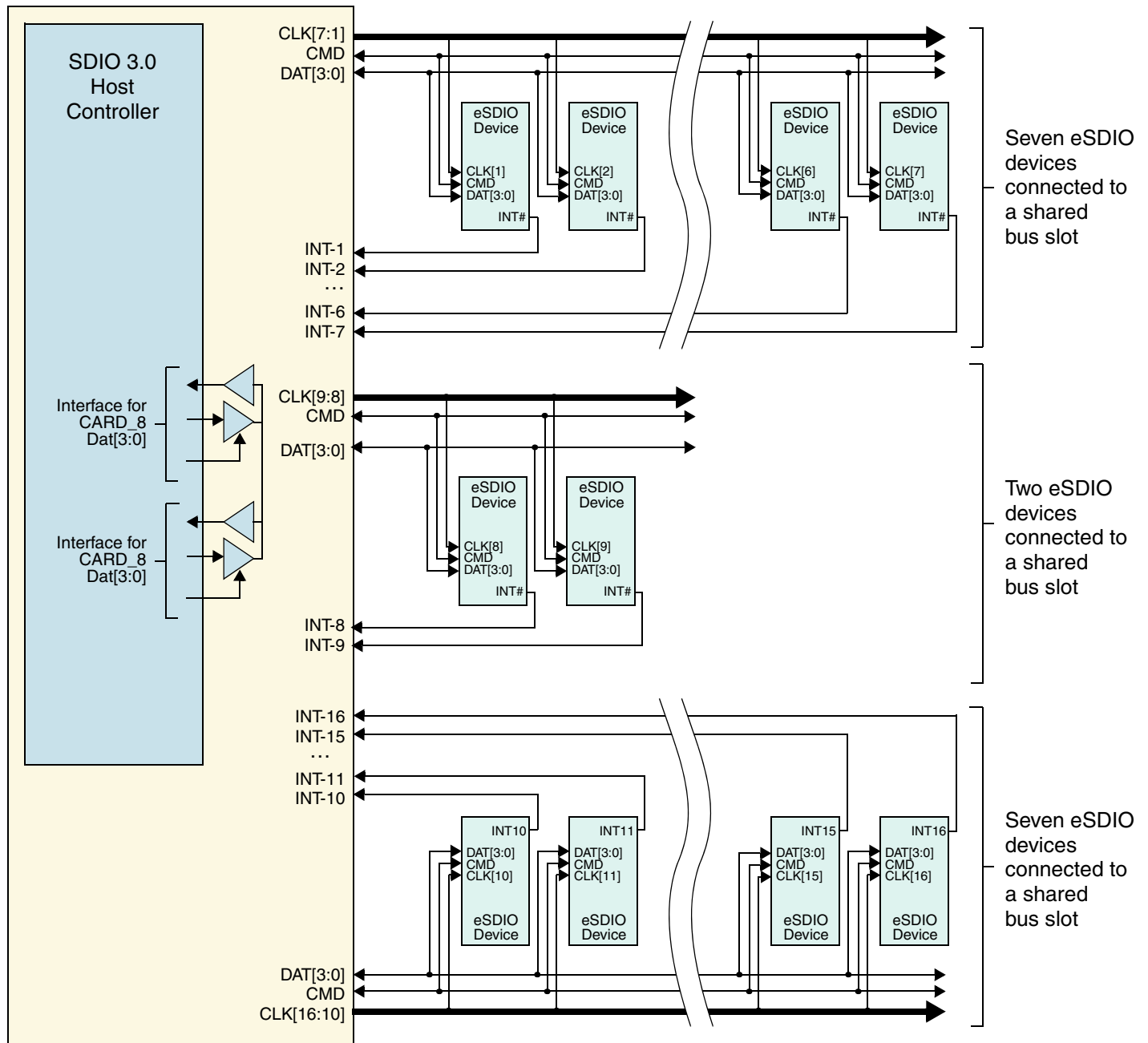


Figure 3-46 illustrates how the DWC_mobile_storage can be used in an actual implementation.

Figure 3-46 Shared Bus Mode SOC Diagram



Note INT-0 to INT-15 in Figure 3-46 are the same as `card_int_n` in the DWC_mobile_storage interface diagram on page 120.

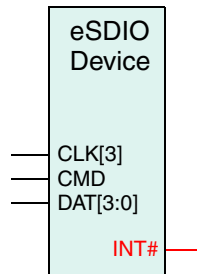
For information on verification with respect to shared buses, refer to “Shared Bus Mode” on page 110.

3.7 Dedicated Interrupt Pins

Interrupt lines are defined for only eSDIO devices, each of which can have separate interrupt lines (INT #), which are connected to the host controller interrupt lines. These interrupt lines can operate even when the card clock is switched off and can be used only during an asynchronous interrupt period.

Figure 3-47 illustrates how the DWC_mobile_storage can be used in an actual implementation.

Figure 3-47 eSDIO Device



The DWC_mobile_storage has sixteen interrupt lines on the host controller that you can use in one of the following ways:

- ❖ All sixteen interrupts can be brought out of SoC and be used as one for each function.
- ❖ Only one interrupt port can be implemented by ORing the INT# lines. In this case, the application must read all functions connected to the interrupt in order to detect which one generated the interrupt.
- ❖ More than one, but less than sixteen interrupt ports can be implemented by ORing the INT# lines. In this case, the application needs to read all functions connected to the interrupt that flagged an interrupt in order to detect which one generated the interrupt.

For information on verification with respect to dedicated interrupts, refer to “Dedicated Interrupts” on page 205.

3.8 Asynchronous Interrupts

The interrupt period timing is described as synchronous to the SD clock at the connector timing. However when operating in more than 25MB/sec data rate in SDR50, SDR104 and DDR50, the host should treat the interrupt signal as an asynchronous input considering delay from host to card.

Figure 3-48 illustrates interrupt timing for SDIO 2.0.

Figure 3-48 Interrupt timing for SDIO 2.0

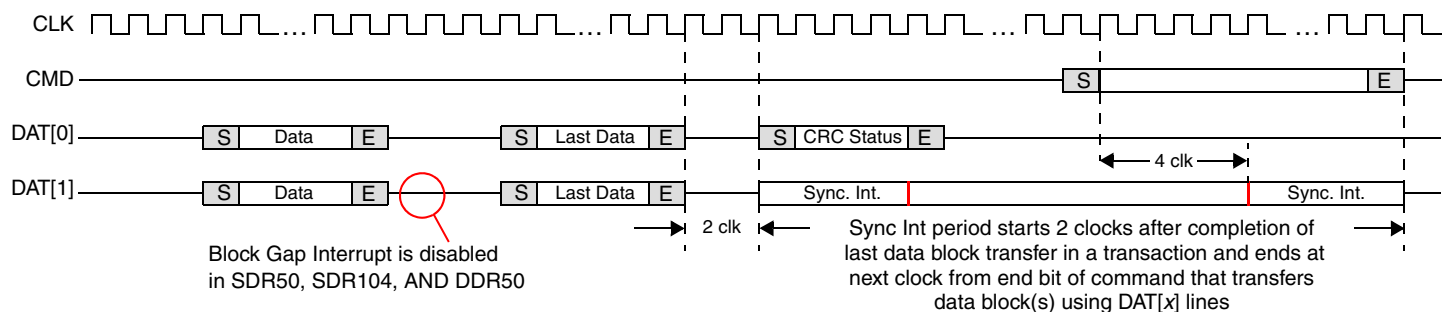
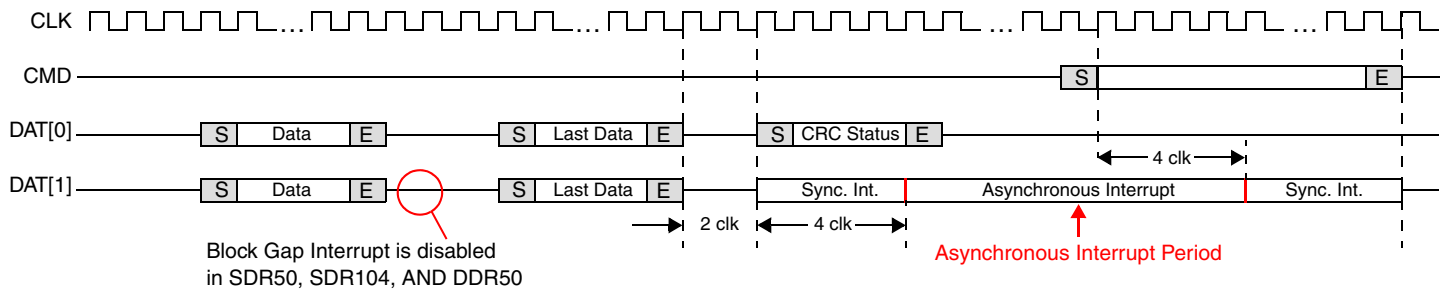


Figure 3-49 illustrates interrupt timing for SDIO 3.0.

Figure 3-49 New Timing for SDIO 3.0



The difference between timing for SDIO 2.0 and 3.0 is that during an asynchronous interrupt, the device clocks do not need to be on for an interrupt. These asynchronous interrupts occur on the DAT[1] lines and are used only when the cards are connected using the non-shared bus mode; in this mode the cards flag an interrupt using the dedicated interrupt pin.

For information on verification with respect to asynchronous interrupts, refer to [“Asynchronous Interrupts”](#) on page 206.

3.9 Back-End Power

Each device needs one bit to control the back-end power supply for an embedded device; this bit does not control the VDDH of the host controller. A `back_end_power` register enables software programming for back-end power. The value on this register is output to the `back_end_power` signal, which can be used to switch power on and off the embedded device.

For information on verification with respect to back-end power, refer to [“Back-End Power”](#) on page 206.

3.10 Master Power Control

SDIO cards prior to the 1.10 specification required a maximum power of $200\text{mA} \times 3.3\text{V} = 720\text{ mW}$, regardless of the number of functions in each card. Newer cards have increased requirements greater than 720 mW.

For information on verification with respect to these power requirements, refer to [“Master Power Control”](#) on page 206.

4

Parameters

This chapter describes the configuration parameters used by the DWC_mobile_storage. The settings of the configuration parameters determine the I/O signal list of the DWC_mobile_storage peripheral.

4.1 Parameter Descriptions

The DWC_mobile_storage provides configurable parameters that you can modify in order to tailor the card controller to your requirements. [Table 4-1](#) lists the configurable parameters that you can modify before synthesizing the core.

Table 4-1 Compile-Time Parameters

Field Label	Parameter Definition
CARD_TYPE	Parameter Name: CARD_TYPE Legal Values: 0, 1 Default Value: 1 Description: Card type supported: 0 – MMC-Ver3.3-only controller 1 – SD_MMC controller (supports SD memory, SDIO, and MMC, and CE-ATA)
NUM_CARDS	Parameter Name: NUM_CARDS Legal Values: 1-30 Default Value: 3 Description: Number of cards supported: 1-30 in MMC-Ver3.3-only mode 1-16 in SD_MMC_CE-ATA mode Note: When a single card is selected, some of the signals in the core are declared as [0:0] because of the following code: output [^NUM_CARD_BUS-1:0] cclk_out; When NUM_CARD_BUS = 1, the code becomes: output [0:0] cclk_out; This situation sometimes causes tools to give lint warnings, which can be ignored.

Table 4-1 Compile-Time Parameters (Cont.)

Field Label	Parameter Definition
H_BUS_TYPE	Parameter Name: H_BUS_TYPE Legal Values: 0, 1 Default Value: 1 Description: Host bus type: 0 – APB 1 – AHB
H_DATA_WIDTH	Parameter Name: H_DATA_WIDTH Legal Values: 16, 32, 64 Default Value: 32 Description: Host data bus width
H_ADDR_WIDTH	Parameter Name: H_ADDR_WIDTH Legal Values: 9-28 Default Value: 20 Description: Host address bus width. Addresses 0x00 - 0X1FF mapped to internal registers. Addresses 0X200 and above mapped to data FIFO. Ensure address width exactly equals address space allotted for DWC_mobile_storage. For example, in AHB system, DWC_mobile_storage registers are selected only when hsel is active and all higher address bits are 0 – (haddr[H_ADDR-WIDTH-1:9] == 0).
INTERNAL_DMAC	Parameter Name: INTERNAL_DMAC Legal Values: 0, 1 Default Value: 1 Description: 0 – No Internal DMAC present 1 – Internal DMAC present
DMA_INTERFACE	Parameter Name: DMA_INTERFACE Legal Values: 0-3 Default Value: 0 Description: 0- No DMA Interface 1- DesignWare DMA Interface 2- Generic DMA Interface 3- Non DW DMA Interface In DesignWare DMA mode, request/acknowledge protocol meets DW_ahb_dmac controller protocol. In this mode, host data bus is also used for DMA transfers. Generic DMA-type interface has simpler request/acknowledge handshake and has dedicated read/write data bus for DMA transfers. Non DW DMAC interface uses dw_dma_single interface in addition to the existing interface and uses host data bus for DMA transfers. This is configurable only if INTERNAL_DMAC=0.

Table 4-1 **Compile-Time Parameters (Cont.)**

Field Label	Parameter Definition
GE_DMA_DATA_WIDTH	Parameter Name: GE_DMA_DATA_WIDTH Legal Values: 16, 32, 64 Default Value: 32 Description: Generic DMA interface data width; valid only if DMA_INTERFACE = 2 (generic DMA), where separate data bus is provided for DMA data transfers.
FIFO_DEPTH	Parameter Name: FIFO_DEPTH Legal Values: 8-4096 Default Value: 16 Description: Depth of internal FIFO; default value is 8.
FIFO_RAM_INSIDE	Parameter Name: FIFO_RAM_INSIDE Legal Values: 0, 1 Default Value: 1 Description: 1 – Instantiate dual port FIFO RAM inside core. 0 – Dual port FIFO RAM outside core; only top-level ports are provided.
NUM_CLK_DIVIDERS	Parameter Name: NUM_CLK_DIVIDERS Legal Values: 1-4 Default Value: 1 Description: Number of clock dividers. In MMC-Ver3.3-only mode, this parameter fixed to 1 because DWC_mobile_storage provides only one clock output.
UID_REG	Parameter Name: UID_REG Legal Values: 0x0 - 0xffffffff Default Value: 0x7967797 Description: Default value of user ID register.
SET_CLK_FALSE_PATH	Parameter Name: SET_CLK_FALSE_PATH Legal Values: 0, 1 Default Value: 0 Description: When set, false path between “clk to cclk_in,” “cclk_in to clk,” and “reset_n to cclk_in” are set in synthesis. If clk and cclk_in are two different free-running clocks, then it is recommended to set false path. If cclk and cclk_in have phase relationship (derived), then it is recommended to not set false path so that metastability associated with signal synchronization can be avoided. If you are not setting false path, then clk and cclk_in frequencies should be integer multiples of each other.

Table 4-1 Compile-Time Parameters (Cont.)

Field Label	Parameter Definition
AREA_OPTIMIZED	Parameter Name: AREA_OPTIMIZED Legal Values: 0, 1 Default Value: 0 Description: When this parameter is enabled, area is optimized by the following optional hardware features: * General Purpose Input/Output Ports removed * User Identification Register (USRID) not Implemented * Transferred CIU Card Byte Count Register (TCBCNT) can be read only after data transfer has completed and not during data transfer (returns 0). Typically, software does not have to access this register during data transfer. Area optimization saves approximately 1K gates. It is not recommended to use this feature unless it is an area-sensitive design.
IMPLEMENT_SCAN_MUX	Parameter Name: IMPLEMENT_SCAN_MUX Legal Values: 0, 1 Default Value: 0 Description: When the parameter is set, the negative-edge-triggered flip-flops are driven by a clock that is coming out of a scan MUX, which is instantiated only if IMPLEMENT_SCAN_MUX is set to 1. The select line for the scan MUX is the scan_mode signal. The two inputs to this scan MUX are the clock and the inverted clock. When the scan_mode signal is high, the negative-edge-triggered flip-flops get the inverted version of the clock. Implementing this scan MUX enables the negative-edged-triggered flip-flops to be included in the same scan chain as the positive-edged-triggered flip-flops. However, clock balancing at the chip level can be a challenge if this MUX is implemented. Note: The IMPLEMENT_SCAN_MUX is not required if a "clock mixing" technique is used for DFT implementation. This option is provided to support wide range of DFT tools and methodologies.



5

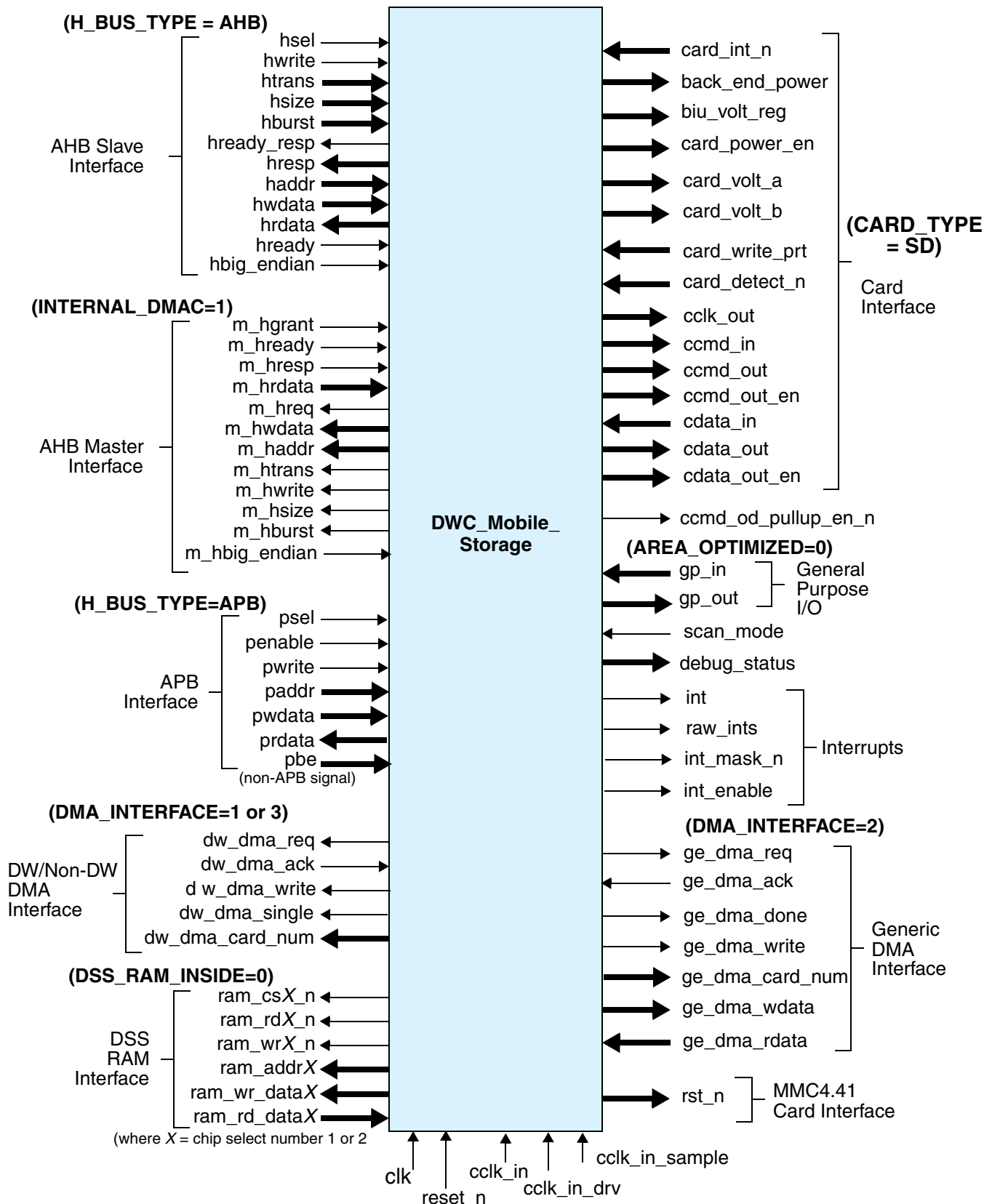
Signals

This chapter describes the DWC_mobile_storage I/O signals.

5.1 DWC_mobile_storage Interface Diagram

[Figure 5-1](#) shows the I/O signal diagram for the DWC_mobile_storage.

Figure 5-1 DWC_mobile_storage Signals



5.2 Signal Descriptions

Table 5-1 identifies the signals that are associated with each configuration. The DWC_mobile_storage signal instantiations and sizes depend on these configuration parameters:

- ❖ CARD_TYPE
- ❖ NUM_CARDS
- ❖ H_BUS_TYPE
- ❖ DMA_INTERFACE
- ❖ INTERNAL_DMACH
- ❖ FIFO_RAM_INSIDE
- ❖ H_DATA_WIDTH, GE_DMA_DATA_WIDTH, H_ADDR_WIDTH, and FIFO_DEPTH
- ❖ AREA_OPTIMIZED

Table 5-1 DWC_mobile_storage Signal Description

Name	Width	I/O	Description
Clock and reset Signals			
clk	1	Input	System clock. Clock frequency requirement: clk >= 1/10 cclk_in (frequency) Only posedge used.
biu_volt_reg	NUM_CARD_BUS	Output	Control signal to select between 3.3V and 1.8 V used in voltage switching as defined by SD3.0. Reflects VOLT_REG bits of UHS_REG register
reset_n	1	Input	System active-low reset pin; synchronous to clk. Should be kept active at least 2 clocks of clk or cclk_in, whichever is lower frequency.
rst_n	NUM_CARD_BUS	Output	Hardware reset (H/W Reset) for MMC4.41 mode. 0 – Reset 1 – Active mode

$R_ADDR_WIDTH = \log_2(FIFO_DEPTH - 2)$
 $F_DATA_WIDTH = (DMA_INTERFACE == 2) ? GE_DMA_DATA_WIDTH == 64 ? 64 : 32 : (H_DATA_WIDTH == 64) ? 64 : 32;$
The NUM_CARD_BUS = 1 if MMC-Ver3.0-only mode is selected; else in SD_MMC_CE-ATA mode,
NUM_CARD_BUS = NUM_CARDS

Table 5-1 **DWC_mobile_storage Signal Description (Cont.)**

Name	Width	I/O	Description
cclk_in	1	Input	Card input clock. The condition is $cclk_in \geq$ card maximum operating frequency—since $cclk_in$ is divided and supplied to the card, it should be greater than or equal to the card maximum operating frequency. Maximum operating frequencies: • MMC-Ver3.3 card – 20 MHz • SD card – 25 Mhz or 50MHz • MMC-Ver4.0 card – 26 MHz or 52 MHz). Both posedge and negedge are used. CIU usually uses posedge of $cclk_in$. Some registers clocked with negedge of $cclk_in$ to pass signals from $cclk_in$ domain to $cclk_in_sample$ domain. Also, signals that gate $cclk_out$ are also generated from negedge of $cclk_in$. For details, refer to “Clock Domains” on page 104.
cclk_in_sample	1	Input	Card input sampling clock; delayed version of card input clock, used for clocking card inputs. Since card output clock, clk_out, is generated by core, skew exists between $cclk_in$ and $cclk_out$ due to clock divider and $cclk_out$ PAD. Since card inputs are guaranteed to meet only setup-and-hold time with regard to card output clock on PAD, delayed version of $cclk_in$ is needed to sample all card inputs. The $cclk_in$ to $cclk_in_sample$ delay should match $cclk_in$ to $cclk_out$ delay of core, plus $cclk_out$ PAD delay, plus data/cmd pad input PAD delay. (MMC cards required to provide setup and hold on inputs; SD cards required to provide setup and hold on all inputs). For details of hold setup time and clock implementations, refer to “Timing Requirements for SD 3.0 Cards” on page 297. Since $cclk_out_en_neg$ signals generated by falling edge of $cclk_in$ are sampled by rising edge of $cclk_in_sample$, delay between them should be less than $cclk_in$ high time. Only posedge used.

$R_ADDR_WIDTH = \log_2(FIFO_DEPTH - 2)$

$F_DATA_WIDTH = (DMA_INTERFACE == 2) ? GE_DMA_DATA_WIDTH == 64 ? 64 : 32 : (H_DATA_WIDTH == 64) ? 64 : 32;$

The $NUM_CARD_BUS = 1$ if MMC-Ver3.0-only mode is selected; else in SD_MMC_CE-ATA mode,

$NUM_CARD_BUS = NUM_CARDS$

Table 5-1 DWC_mobile_storage Signal Description (Cont.)

Name	Width	I/O	Description
cclk_in_drv	1	Input	Card outputs driving clock; delayed version of card input clock, used for clocking optional hold-time registers. Should be delayed version of cclk_out. (Cards need hold on all inputs, so all outputs clocked out of cclk_in are re-clocked by cclk_in_drv to meet card hold-time requirement.) For details of hold setup time and clock implementations, refer to “Timing Requirements for SD 3.0 Cards” on page 297. Both posedge and negedge used.
AHB Interface Signals (Valid only if AMBA_BUS_TYPE = AHB) System Clock Domain “clk”			
hsel	1	Input	AHB device select signal for DWC_mobile_storage
hwrite	1	Input	AHB write control signal: 0 – read 1 – write
htrans	2	Input	AHB bus transfer type: 00 – IDLE 01 – BUSY 10 – NOSEQ 11 – SEQ
hsize	3	Input	AHB bus transfer size: 000 – 8 bits 001 – 16 bits 010 – 32 bits 011 – 64 bits 100 – 128 bits 101 – 256 bits 110 – 512 bits 111 – 1024 bits
$R_ADDR_WIDTH = \log_2(FIFO_DEPTH - 2)$ $F_DATA_WIDTH = (DMA_INTERFACE == 2) ? GE_DMA_DATA_WIDTH == 64 ? 64 : 32 : (H_DATA_WIDTH == 64) ? 64 : 32;$ The NUM_CARD_BUS = 1 if MMC-Ver3.0-only mode is selected; else in SD_MMC_CE-ATA mode, NUM_CARD_BUS = NUM_CARDS			

Table 5-1 DWC_mobile_storage Signal Description (Cont.)

Name	Width	I/O	Description
hburst	3	Input	AHB bus burst type: 000 – SINGLE 001 – INCR 010 – WRAP4 011 – INCR4 100 – WRAP8 101 – INCR8 110 – WRAP16 111 – INCR16 Note: hburst is unused; although DWC_mobile_storage supports all AHB bursts, only htrans=NSEQ or htrans=SEQ information is used by DWC_mobile_storage.
hready_resp	1	Output	AHB bus data ready response
hready	1	Input	AHB bus data ready input
hresp	2	Output	AHB bus transfer response: 00 – OKAY 01 – ERROR 10 – RETRY 11 – SPLIT Note: Always drives OKAY response.
haddr	H_ADDR_WIDTH	Input	AHB bus interface address bus. Lower 8-bits used for accessing registers. Address above or equal to 0x200 selects FIFO data.
hwddata	H_DATA_WIDTH	Input	AHB bus interface write data
hrdata	H_DATA_WIDTH	Output	AHB bus interface read data
hbig_endian	1	Input	AHB bus interface endianness: 1 – Big-endian AHB bus interface 0 – Little-endian AHB bus interface
APB Interface Signals (Valid only if AMBA_BUS_TYPE = APB) System Clock Domain “clk”			
psel	1	Input	APB peripheral select signal for DWC_mobile_storage. In standard APB system, signal asserted for at least two clocks.
R_ADDR_WIDTH = log2(FIFO_DEPTH - 2) F_DATA_WIDTH = (DMA_INTERFACE==2)? GE_DMA_DATA_WIDTH==64? 64:32 : (H_DATA_WIDTH==64)? 64 : 32; The NUM_CARD_BUS = 1 if MMC-Ver3.0-only mode is selected; else in SD_MMC_CE-ATA mode, NUM_CARD_BUS = NUM_CARDS			

Table 5-1 DWC_mobile_storage Signal Description (Cont.)

Name	Width	I/O	Description
penable	1	Input	APB enable control. Asserted for single clock and used for timing read/write operations. Signal typically active every other clock in back-to-back APB cycles. In non-APB sideband mode of DWC_mobile_storage, signal can stay continuously active for one clock FIFO data burst when accessing FIFO region.
pwrite	1	Input	APB write control signal: 0 – read 1 – write
paddr	H_ADDR_WIDTH	Input	APB bus interface address bus. Lower 8-bits used for accessing register. Address above or equal to 0x200 selects FIFO data.
pwrdata	H_DATA_WIDTH	Input	APB bus interface write data
prdata	H_DATA_WIDTH	Output	APB bus interface read data
pbe	H_DATA_WIDTH/8	Input	Non-APB sideband byte-enable signal, which is sideband signal added to DWC_mobile_storage APB bus for supporting individual byte read and write. In standard APB system that does not support byte operation, bits should be tied high.
AHB Master Interface Signals (Valid only if INTERNAL_DMAC =1) System Clock Domain “clk”			
m_hgrant	1	Input	AHB Grant. Indicates that the AHB Master is currently the highest priority Master. Ownership of address and control signals changes at the end of the transfer, when m_hready is high.
m_hready	1	Input	AHB Slave Ready. Indicates that a transfer has finished on the bus. It may be driven low by the Slave being addressed to extend a transfer.
m_hresp	2	Input	AHB Slave Response. Provides information on the transfer status. 00: OKAY – Transfer completed OK 01: ERROR – Error in current Transfer 10: RETRY – Slave is busy and wants Master to retry the transfer 11: SPLIT – Slave accepted request, but Master shall back off from bus until the Slave is ready to serve
R_ADDR_WIDTH = log2(FIFO_DEPTH - 2) F_DATA_WIDTH = (DMA_INTERFACE==2)? GE_DMA_DATA_WIDTH==64? 64:32 : (H_DATA_WIDTH==64)? 64 : 32; The NUM_CARD_BUS = 1 if MMC-Ver3.0-only mode is selected; else in SD_MMC_CE-ATA mode, NUM_CARD_BUS = NUM_CARDS			

Table 5-1 DWc_mobile_storage Signal Description (Cont.)

Name	Width	I/O	Description
m_hrdata	H_DATA_WIDTH	Input	AHB Slave Read Data. Transfer data from the AHB slaves to IDMAC during read operations. The width of the data bus is configurable to 16/32/64 bits.
m_hbig_endian	1	Input	When set, indicates big endian format for data transfer. Make this value same as hbig_endian.
m_hreq	1	Output	AHB Request. Indicates that the AHB Master requires the bus; processed by the arbiter to grant the bus.
m_haddr	H_ADDR_WIDTH	Output	AHB 32-bit address for the current transaction.
m_htrans	2	Output	AHB Transfer Type. The AHB Master Interface uses the following values: 00 – IDLE 10 – Non Sequential 11 – Sequential All other encodings are not used.
m_hwrite	1	Output	AHB Read/Write. Set to high, indicates a write transfer; set to low indicates a read transfer.
m_hsize	3	Output	AHB Data Transfer Size. Typically used values are: 01 – 16 bits 10 – 32 bits 11 – 64 bits
m_hburst	3	Output	AHB Burst Length. Indicates the transfer part of an AHB burst. 000: SINGLE – Single Transfer 001: INCR – Incrementing burst of undefined length 010: WRAP4 – 4-beat wrapping burst 011: INCR4 – 4-beat incrementing burst 100: WRAP8 – 8-beat wrapping burst 101: INCR8 – 8-beat incrementing burst 110: WRAP16 – 16-beat wrapping burst 111: INCR16 – 16-beat incrementing burst)
m_hwdata	H_DATA_WIDTH	Output	AHB Write Data. AHB write data input to the Slave port. The data bus width is configurable to 16, 32, or 64 bits.
DW-DMA/Non DW-DMA Interface Signals (Valid only if DMA_INTERFACE = 1 or DMA_INTERFACE = 3) System Clock Domain “clk”			
dw_dma_req	1	Output	DW-DMA request signal to DMA controller.
$R_ADDR_WIDTH = \log_2(FIFO_DEPTH - 2)$ $F_DATA_WIDTH = (DMA_INTERFACE == 2)? GE_DMA_DATA_WIDTH == 64? 64:32 : (H_DATA_WIDTH == 64)? 64 : 32;$ The NUM_CARD_BUS = 1 if MMC-Ver3.0-only mode is selected; else in SD_MMC_CE-ATA mode, NUM_CARD_BUS = NUM_CARDS			

Table 5-1 DWC_mobile_storage Signal Description (Cont.)

Name	Width	I/O	Description
dw_dma_ack	1	Input	DW-DMA acknowledgement from DMA controller
dw_dma_single	1	Output	DW-DMA transfer type: 0 – burst transfer 1 – single transfer
 Note In the default configuration, this signal is unused. Use this signal with DMA that do not maintain a count of remaining data in a burst transaction.			
dw_dma_write	1	Output	DW-DMA write signal: 0 – DWC_mobile_storage read from host memory (transmit to card) 1 – DWC_mobile_storage write-to-host memory (receive from card) Inverted and registered version of read/write (bit[10]) of CMD register.
dw_dma_card_num	5	Output	Physical card involved in current data transfer. Registered version of card_number bits of CMD register.
Generic-DMA Interface Signals (Valid only if DMA_INTERFACE = 2) System Clock Domain “clk”			
ge_dma_req	1	Output	Generic-DMA request signal. Valid data transferred when both ge_dma_req and ge_dma_ack simultaneously active.
ge_dma_ack	1	Input	Generic-DMA acknowledgement
ge_dma_done	1	Output	Generic-DMA last DMA transfer completed
ge_dma_write	1	Output	Generic-DMA write signal: 0 – DWC_mobile_storage read-from-host memory (transmit to card) 1 – DWC_mobile_storage write-to-host memory (receive from card) Inverted and registered version of read/write (bit[10]) of CMD register.
ge_dma_card_num	5	Output	Physical card involved in current data transfer. Registered version of card_number bits of CMD register.
ge_dma_wdata	GE_DMA_DATA_WIDTH	Output	Generic-DMA interface write data output
$R_ADDR_WIDTH = \log_2(FIFO_DEPTH - 2)$ $F_DATA_WIDTH = (DMA_INTERFACE == 2) ? GE_DMA_DATA_WIDTH == 64 ? 64 : 32 : (H_DATA_WIDTH == 64) ? 64 : 32;$ The NUM_CARD_BUS = 1 if MMC-Ver3.0-only mode is selected; else in SD_MMC_CE-ATA mode, NUM_CARD_BUS = NUM_CARDS			

Table 5-1 DWC_mobile_storage Signal Description (Cont.)

Name	Width	I/O	Description
ge_dma_rdata	GE_DMA_DATA_WIDTH	Input	Generic-DMA interface read data input
Dual Port DSSRAM Interface Signals System Clock Domain "clk"			
ram_rd_data1	F_DATA_WIDTH	Input	DSSRAM read output data-1
ram_cs1_n	1	Output	DSSRAM active low chipselect-1
ram_rd1_n	1	Output	DSSRAM active low read-1
ram_wr1_n	1	Output	DSSRAM active low write-1
ram_addr1	R_ADDR_WIDTH	Output	DSSRAM address-1
ram_wr_data1	F_DATA_WIDTH	Output	DSSRAM write input data-1
Dual Port DSSRAM Interface Signals Card Clock Domain "cclk_in"			
ram_rd_data2	F_DATA_WIDTH	Input	DSSRAM read output data-2
ram_cs2_n	1	Output	DSSRAM active low chipselect-2
ram_rd2_n	1	Output	DSSRAM active low read-2
ram_wr2_n	1	Output	DSSRAM active low write-2
ram_addr2	R_ADDR_WIDTH	Output	DSSRAM address-2
ram_wr_data2	F_DATA_WIDTH	Output	DSSRAM write input data-2
Card Interface Signals Driven By System Clock "clk"			
card_power_en	NUM_CARDS	Output	Card power-enable control signal; one bit for each card. Could also be used as general-purpose output. Reflects power_enable bits of PWREN register.
card_volt_a	4	Output	Card voltage regulator-A control. Could also be used as general-purpose outputs if voltage regulation is not needed. Reflects card_voltage_a bits of CTRL register.
card_volt_b	4	Output	Card voltage regulator-B control. Could also be used as general-purpose outputs if voltage regulation is not needed. Reflects card_voltage_b bits of CTRL register.

$R_ADDR_WIDTH = \log_2(FIFO_DEPTH - 2)$

$F_DATA_WIDTH = (DMA_INTERFACE == 2) ? GE_DMA_DATA_WIDTH == 64 ? 64 : 32 : (H_DATA_WIDTH == 64) ? 64 : 32;$

The NUM_CARD_BUS = 1 if MMC-Ver3.0-only mode is selected; else in SD_MMC_CE-ATA mode,

NUM_CARD_BUS = NUM_CARDS

Table 5-1 DWC_mobile_storage Signal Description (Cont.)

Name	Width	I/O	Description
ccmd_od_pullup_en_n	1	Output	Card command open-drain pull-up enable; used in MMC mode. Reflects inverted value of enable_OD_pullup bit of CTRL register.
Card Interface Signals Asynchronous inputs, sampled by System Clock “clk” with double-synchronization registers			
card_detect_n	NUM_CARDS	Input	Card detect signals. A 0 represents presence of card. Bits double-synchronized with clk and passed to CDETECT register for read. In addition, any change in signal causes card-detect interrupt, if enabled.
card_write_prt	NUM_CARDS	Input	Card write protect signals. A 1 represents write is protected. Bits double-synchronized by clk and passed to WRTPRT register for read. Software <i>must</i> honor write-protect. Conditional signal; present only when CARD_TYPE = SD (1)
Card Interface Signals Card Clock Domain “cclk_in.” ccmd_in and cdata_in are sample by “cclk_in_sample” and then passed to cclk_in domain			
cclk_out	NUM_CARD_BUS ^c	Output	Card clocks. Output from internal clock dividers. In case of MMC-Ver3.3-only mode, only one clock output present. In SD_MMC_CE-ATA mode, each card receives separate clock. Has register delay, plus Mux delay from cclk_in. This is input clock to SD_MMC_CEATA cards after it goes out of chip through PAD. Driven out by cclk_in.
ccmd_in	NUM_CARD_BUS	Input	Card Command input. Sampled by cclk_in_sample.
ccmd_out	NUM_CARD_BUS	Output	Card Command output. Driven out by cclk_in_drv.
ccmd_out_en	NUM_CARD_BUS	Output	Card command; active-high output enable. Driven out by cclk_in_drv.
cdata_in	NUM_CARD_BUS*8	Input	Card data input. Sampled by cclk_in_sample.
R_ADDR_WIDTH = log2(FIFO_DEPTH - 2) F_DATA_WIDTH = (DMA_INTERFACE==2)? GE_DMA_DATA_WIDTH==64? 64:32 : (H_DATA_WIDTH==64)? 64 : 32; The NUM_CARD_BUS = 1 if MMC-Ver3.0-only mode is selected; else in SD_MMC_CE-ATA mode, NUM_CARD_BUS = NUM_CARDS			

Table 5-1 DWC_mobile_storage Signal Description (Cont.)

Name	Width	I/O	Description
cdata_out	NUM_CARD_BUS*8	Output	Card data output. Driven out by cclk_in_drv.
cdata_out_en	NUM_CARD_BUS*8	Output	Card data output enable bit; active-high output enable. One output enable control bit for each cdata_out bit. Driven out by cclk_in_drv.
card_int_n	NUM_CARD_BUS	Input	Card interrupt lines. These interrupt lines are connected to the eSDIO card interrupt lines; they are defined for only eSDIO. These pins are used to indicate a card interrupt, which is sampled even when the clock to the card is switched off.
back_end_power	NUM_CARD_BUS	Output	Back-end power supply for embedded device. One bit needed for each device to control back-end power supply for an embedded device; this bit does not control the VDDH of the host controller. A register bit enables software programming. The value on this register controls switching on and off of power to embedded device.
Interrupt Signals System Clock Domain "clk"			
int	1	Output	Combined active-high, level-sensitive host interrupt.
raw_ints	32	Output	Raw interrupt status register output. For debug purposes or for user to generate own interrupt logic. Reflects RINTSTS register.
int_mask_n	32	Output	Interrupt mask register output. For debug purposes or for user to generate own interrupt logic. Value of 0 represents an interrupt is masked; value of 1 represents an interrupt is enabled. Reflects INTMASK register.
int_enable	1	Output	Global interrupt enable bit. For debug purposes or for user to generate own interrupt logic. Reflects int_enable bit of CTRL register.
R_ADDR_WIDTH = log2(FIFO_DEPTH - 2) F_DATA_WIDTH = (DMA_INTERFACE==2)? GE_DMA_DATA_WIDTH==64? 64:32 : (H_DATA_WIDTH==64)? 64 : 32; The NUM_CARD_BUS = 1 if MMC-Ver3.0-only mode is selected; else in SD_MMC_CE-ATA mode, NUM_CARD_BUS = NUM_CARDS			

Table 5-1 **DWC_mobile_storage Signal Description (Cont.)**

Name	Width	I/O	Description
Miscellaneous Signals System Clock Domain “clk”			
gp_out	16	Output	General-purpose outputs. Reflects gpo bits of GPIO register. Conditional signal; present only when AREA_OPTIMIZED = 0.
gp_in	8	Input	General-purpose inputs. Double-synchronized by cclk and passed to gpi bits of GPIO register for read. Conditional signal; present only when AREA_OPTIMIZED = 0.
scan_mode	1	Input	Scan mode bypass pin for coverage improvement; signal should be tied low during normal operation.
debug_status	963	Output	Internal status signals brought out to top of core for easy debugging. bit[1122:931] - IDMAC registers bit[930] - CIU to BIU – data transfer done bit[929] - CIU to BIU – command taken bit[928] - BIU to CIU – command start bit[927:0] – Registers bit[31:0] – CTRL register bit[927:896] – HCON register
R_ADDR_WIDTH = log2(FIFO_DEPTH - 2) F_DATA_WIDTH = (DMA_INTERFACE==2)? GE_DMA_DATA_WIDTH==64? 64:32 : (H_DATA_WIDTH==64)? 64 : 32; The NUM_CARD_BUS = 1 if MMC-Ver3.0-only mode is selected; else in SD_MMC_CE-ATA mode, NUM_CARD_BUS = NUM_CARDS			

6

Registers

This chapter describes the programmable registers of the DWC_mobile_storage.

All logical register definitions are 32 bits in size. You can access the registers in 8, 16, 32, or 64 bits, depending upon your host bus-size and byte-enable control; the exception is TCBCNT and TBBCNT – for details, refer to the “[Register Unit](#)” on page 56.

When accessed in 64 bits, two consecutive registers are used. In a standard APB system, the registers should be accessed in only the APB bus width, since the APB bus does not support partial bus-size access, even though the DWC_mobile_storage supports it.



Note

Non-implemented registers and register bits are reserved for future use. To be compatible with future versions of the DWC_mobile_storage, non-implemented bits always should be programmed to 0. The read values of non-implemented register bits should be treated as “don’t care.”

6.1 Register Address Map

Figure 6-1 illustrates the DWC_mobile_storage memory map.

Figure 6-1 Illustration of Memory Map

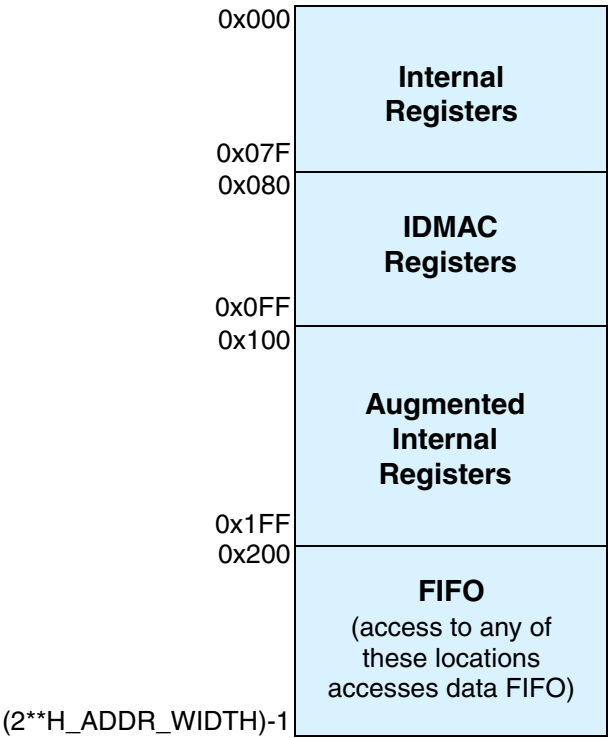


Table 6-1 lists the memory map for the DWC_mobile_storage.

Table 6-1 Memory Map

Register Name	Address Offset	Width	Description
CTRL	0x00	32 bits	Control register Reset Value: 0x0
PWREN	0x04	32 bits	Power-enable register Reset Value: 0x0
CLKDIV	0x08	32 bits	Clock-divider register Reset Value: 0x0
CLKSRC	0x0C	32 bits	Clock-source register Reset Value: 0x0

DWC_mobile_storage controller uses address bits haddr[8:0] (paddr[8:0]) to access internal registers. Address 0x200 and above are mapped to data FIFO. More than one address is mapped to data FIFO so that FIFO can be accessed using AMBA bursts. Base address depends on where the DWC_mobile_storage is mapped by system address decoder (hsel/psel generation). Registers from address 0x80 to 0x98 are present only for Internal DMAC Configuration or else they are reserved.

Table 6-1 Memory Map (Cont.)

Register Name	Address Offset	Width	Description
CLKENA	0x10	32 bits	Clock-enable register Reset Value: 0x0
TMOUT	0x14	32 bits	Time-out register (number of card clock output clocks – cclk_out) Reset Value: 0xFFFFFFFF40
CTYPE	0x18	32 bits	Card-type register Reset Value: 0x0
BLKSIZ	0x1C	32 bits	Block-size register Reset Value: 0x200
BYTCNT	0x20	32 bits	Byte-count register Reset Value: 0x200
INTMASK	0x24	32 bits	Interrupt-mask register Reset Value: 0x0
CMDARG	0x28	32 bits	Command-argument register Reset Value: 0x0
CMD	0x2C	32 bits	Command register Reset Value: 0x0
RESP0	0x30	32 bits	Response-0 register Reset Value: 0x0
RESP1	0x34	32 bits	Response-1 register Reset Value: 0x0
RESP2	0x38	32 bits	Response-2 register Reset Value: 0x0
RESP3	0x3C	32 bits	Response-3 register Reset Value: 0x0
MINTSTS	0x40	32 bits	Masked interrupt-status register Reset Value: 0x0
RINTSTS	0x44	32 bits	Raw interrupt-status register Reset Value: 0x0
STATUS	0x48	32 bits	Status register; mainly for debug purposes Reset Value: {20'h00000, 4'b00xx, 8'h06}

DWC_mobile_storage controller uses address bits haddr[8:0] (paddr[8:0]) to access internal registers. Address 0x200 and above are mapped to data FIFO. More than one address is mapped to data FIFO so that FIFO can be accessed using AMBA bursts. Base address depends on where the DWC_mobile_storage is mapped by system address decoder (hsel/psel generation). Registers from address 0x80 to 0x98 are present only for Internal DMAC Configuration or else they are reserved.

Table 6-1 Memory Map (Cont.)

Register Name	Address Offset	Width	Description
FIFOTH	0x4C	32 bits	FIFO threshold register Reset Value: {4'h0, bits[27:16] = FIFO_DEPTH - 1 bits[15:0] = 0}
CDETECT	0x50	32 bits	Card-detect register Reset Value: Value in card_detect signal
WRTprt	0x54	32 bits	Write-protect register Reset Value: Value in card_write_prt signal
GPIO	0x58	32 bits	GPIO register Reset Value: Value in bits[31:8] = 24'h0 bits[7:0] gp_in signal
TCBCNT	0x5C	32 bits	Transferred CIU card byte count Reset Value: 0x0
TBBCNT	0x60	32 bits	Transferred host/DMA to/from BIU-FIFO byte count Reset Value: 0x0
DEBNCE	0x64	32 bits	Card detect debounce register (number of host clocks – clk) Reset Value: 0x00ffffff
USRID	0x68	32 bits	User ID register Reset Value: UID_REG configuration parameter
VERID	0x6C	32 bits	Synopsys version ID register Reset Value: 0x5342_230a
HCON	0x70	32 bits	Hardware configuration register Reset Value: Dependent on user-defined values in configuration parameters; refer to “Parameters” on page 115.
UHS_REG	0x74	32 bits	UHS-1 register Reset Value: 0x0
RST_n	0x78	32 bits	Hardware reset register Reset Value: 0x0000_FFFF
BMOD	0x80	32 bits	Bus Mode Register; controls the Host Interface Mode. Reset Value: 0x0
PLDMND	0x84	32 bits	Poll Demand Register. Used by the host to instruct the IDMAC to poll the Descriptor List while in suspend. Reset Value: 0x0

DWC_mobile_storage controller uses address bits haddr[8:0] (paddr[8:0]) to access internal registers. Address 0x200 and above are mapped to data FIFO. More than one address is mapped to data FIFO so that FIFO can be accessed using AMBA bursts. Base address depends on where the DWC_mobile_storage is mapped by system address decoder (hsel/psel generation). Registers from address 0x80 to 0x98 are present only for Internal DMAC Configuration or else they are reserved.

Table 6-1 Memory Map (Cont.)

Register Name	Address Offset	Width	Description
DBADDR	0x88	32 bits	Descriptor List Base Address Register. Points the IDMAC to the start of the Descriptor list. Reset Value: 0x0
IDSTS	0x8C	32 bits	Internal DMAC Status Register. Software driver application reads this register during interrupt service routine or polling to determine the status of the IDMAC. Reset Value: 0x0
IDINTEN	0x90	32 bits	Internal DMAC Interrupt Enable Register. Enables the interrupts reported by the IDMAC Status Register (IDSTS). Reset Value: 0x0
DSCADDR	0x94	32 bits	Current Host Descriptor Address Register. Points to the start of current Descriptor read by the IDMAC. Reset Value: 0x0
BUFADDR	0x98	32 bits	Current Host Buffer Address Register. Points to the current Buffer address accessed by the IDMAC. Reset Value: 0x0
Reserved	0x9C – 0xFF		Reserved; although AHB/APB access to this region completes, write data is ignored and read data is 32'hx.
CARDTHRCTL	0X100	32 bits	Card Read Threshold Enable (CardRdThrEn) • 1'b0 – Card Read Threshold disabled • 1'b0 – Card Read Threshold enabled. Host Controller initiates Read Transfer only if CardRdThreshold amount of space is available in Receive FIFO. For more information, refer to “ Card Read Threshold ” on page 199. Reset Value: 0x0
BACK_END_POWER	0X104	16 bits	Back-end Power • 0 – Off; Reset • 1 – Back-end Power is supplied Reset Value: 0x0
DATA	>= 0x200	R/W	Data FIFO read/write; if address is equal or greater than 0x200, then FIFO is selected as long as device is selected (hsel/psel active) Reset Value: 32'hx

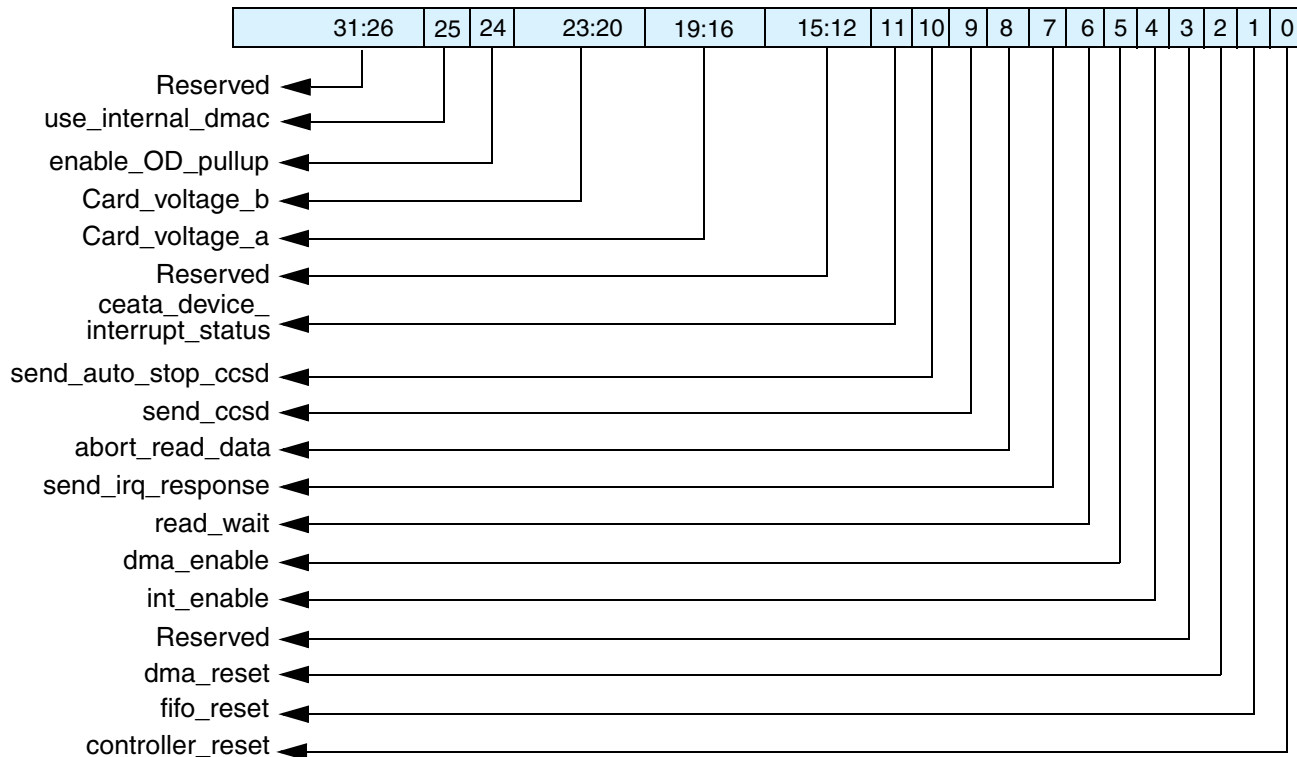
DWC_mobile_storage controller uses address bits haddr[8:0] (paddr[8:0]) to access internal registers. Address 0x200 and above are mapped to data FIFO. More than one address is mapped to data FIFO so that FIFO can be accessed using AMBA bursts. Base address depends on where the DWC_mobile_storage is mapped by system address decoder (hsel/psel generation). Registers from address 0x80 to 0x98 are present only for Internal DMAC Configuration or else they are reserved.

6.2 Register and Field Descriptions

The following sections contain the memory diagrams and field descriptions for the individual registers.

6.2.1 CTRL

- ❖ Name: Control Register
- ❖ Size: 32 bits
- ❖ Address offset: 0x00
- ❖ Read/write access: read/write



Bits	Name	Default	Description
31:26	Reserved		
25	use_internal_dmac	0	Present only for the Internal DMAC configuration; else, it is reserved. 0 – The host performs data transfers through the slave interface 1 – Internal DMAC used for data transfer
24	enable_OD_pullup	1	External open-drain pullup: 0 – Disable 1 – Enable Inverted value of this bit is output to ccmd_od_pullup_en_n port. When bit is set, command output always driven in open-drive mode; that is, DWC_mobile_storage drives either 0 or high impedance, and does not drive hard 1.

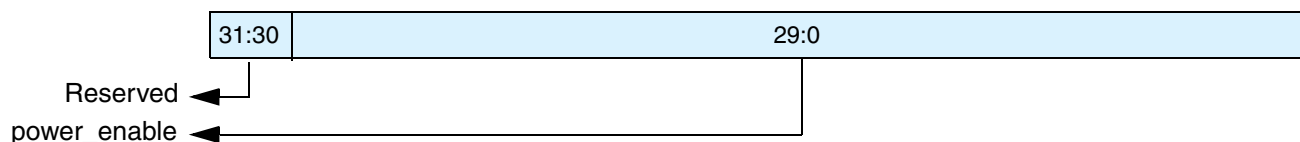
Bits	Name	Default	Description
23:20	Card_voltage_b	0	Card regulator-B voltage setting; output to card_volt_b port. Optional feature; ports can be used as general-purpose outputs.
19:16	Card_voltage_a	0	Card regulator-A voltage setting; output to card_volt_a port. Optional feature; ports can be used as general-purpose outputs.
15:12	Reserved		
11	ceata_device_interrupt_status	0	0 – Interrupts not enabled in CE-ATA device (nIEN = 1 in ATA control register) 1 – Interrupts are enabled in CE-ATA device (nIEN = 0 in ATA control register) Software should appropriately write to this bit after power-on reset or any other reset to CE-ATA device. After reset, usually CE-ATA device interrupt is disabled (nIEN = 1). If the host enables CE-ATA device interrupt, then software should set this bit.
10	send_auto_stop_ccsd	0	0 – Clear bit if DWC_mobile_storage does not reset the bit. 1 – Send internally generated STOP after sending CCSD to CE-ATA device. NOTE: Always set send_auto_stop_ccsd and send_ccsd bits together; send_auto_stop_ccsd should not be set independent of send_ccsd. When set, DWC_Mobile_Storage automatically sends internally-generated STOP command (CMD12) to CE-ATA device. After sending internally-generated STOP command, Auto Command Done (ACD) bit in RINTSTS is set and generates interrupt to host if Auto Command Done interrupt is not masked. After sending the CCSD, DWC_mobile_storage automatically clears send_auto_stop_ccsd bit.
9	send_ccsd	0	0 – Clear bit if DWC_mobile_storage does not reset the bit. 1 – Send Command Completion Signal Disable (CCSD) to CE-ATA device When set, DWC_mobile_storage sends CCSD to CE-ATA device. Software sets this bit only if current command is expecting CCS (that is, RW_BLK) and interrupts are enabled in CE-ATA device. Once the CCSD pattern is sent to device, DWC_mobile_storage automatically clears send_ccsd bit. It also sets Command Done (CD) bit in RINTSTS register and generates interrupt to host if Command Done interrupt is not masked. NOTE: Once send_ccsd bit is set, it takes two card clock cycles to drive the CCSD on the CMD line. Due to this, during the boundary conditions it may happen that CCSD is sent to the CE-ATA device, even if the device signalled CCS.
8	abort_read_data	0	0 – No change 1 – After suspend command is issued during read-transfer, software polls card to find when suspend happened. Once suspend occurs, software sets bit to reset data state-machine, which is waiting for next block of data. Bit automatically clears once data state-machine resets to idle. Used in SDIO card suspend sequence.

Bits	Name	Default	Description
7	send_irq_response	0	0 – No change 1 – Send auto IRQ response Bit automatically clears once response is sent. To wait for MMC card interrupts, host issues CMD40, and DWC_mobile_storage waits for interrupt response from MMC card(s). In meantime, if host wants DWC_mobile_storage to exit waiting for interrupt state, it can set this bit, at which time DWC_mobile_storage command state-machine sends CMD40 response on bus and returns to idle state.
6	read_wait	0	0 – Clear read wait 1 – Assert read wait For sending read-wait to SDIO cards.
5	dma_enable	0	0 – Disable DMA transfer mode 1 – Enable DMA transfer mode Valid only if DWC_mobile_storage configured for External DMA interface. Even when DMA mode is enabled, host can still push/pop data into or from FIFO; this should not happen during the normal operation. If there is simultaneous FIFO access from host/DMA, the data coherency is lost. Also, there is no arbitration inside DWC_mobile_storage to prioritize simultaneous host/DMA access.
4	int_enable	0	Global interrupt enable/disable bit: 0 – Disable interrupts 1 – Enable interrupts The int port is 1 only when this bit is 1 and one or more unmasked interrupts are set.
3	Reserved		
2	dma_reset	0	0 – No change 1 – Reset internal DMA interface control logic To reset DMA interface, firmware should set bit to 1. This bit is auto-cleared after two AHB clocks.
1	fifo_reset	0	0 – No change 1 – Reset to data FIFO To reset FIFO pointers To reset FIFO, firmware should set bit to 1. This bit is auto-cleared after completion of reset operation.

Bits	Name	Default	Description
0	controller_reset	0	0 – No change 1 – Reset DWC_mobile_storage controller To reset controller, firmware should set bit to 1. This bit is auto-cleared after two AHB and two cclk_in clock cycles. This resets: * BIU/CIU interface * CIU and state machines * abort_read_data, send_irq_response, and read_wait bits of Control register * start_cmd bit of Command register Does not affect any registers or DMA interface, or FIFO or host interrupts

6.2.2 PWREN

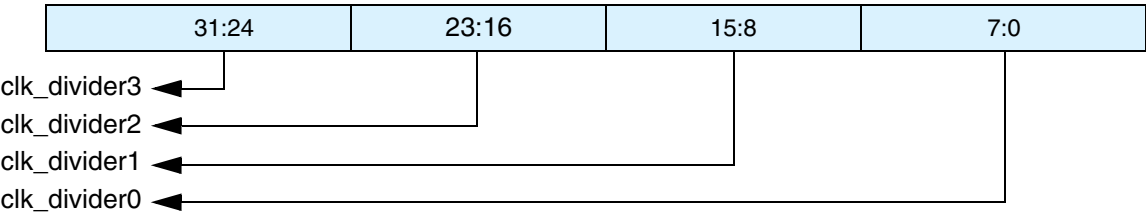
- ❖ Name: Power Enable Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x04
- ❖ Read/write access: write/read



Bits	Name	Default	Description
31:30	Reserved		
29:0	power_enable	0	Power on/off switch for up to 16 cards; for example, bit[0] controls card 0. Once power is turned on, firmware should wait for regulator/switch ramp-up time before trying to initialize card. 0 – power off 1 – power on Only NUM_CARDS number of bits are implemented. Bit values output to card_power_en port. Optional feature; ports can be used as general-purpose outputs.

6.2.3 CLKDIV

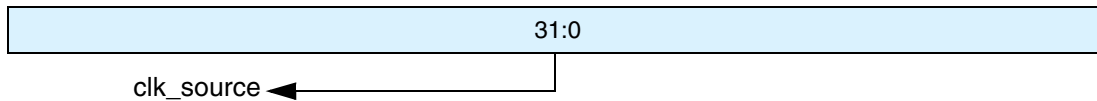
- ❖ Name: Clock Divider Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x08
- ❖ Read/write access: read/write



Bits	Name	Default	Description
31:24	clk_divider3	0	Clock divider-3 value. Clock division is 2^n . For example, value of 0 means divide by $2^0 = 1$ (no division, bypass), a value of 1 means divide by $2^1 = 2$, a value of "ff" means divide by $2^{255} = 510$, and so on. In MMC-Ver3.3-only mode, bits not implemented because only one clock divider is supported.
23:16	clk_divider2	0	Clock divider-2 value. Clock division is 2^n . For example, value of 0 means divide by $2^0 = 1$ (no division, bypass), value of 1 means divide by $2^1 = 2$, value of "ff" means divide by $2^{255} = 510$, and so on. In MMC-Ver3.3-only mode, bits not implemented because only one clock divider is supported.
15:8	clk_divider1	0	Clock divider-1 value. Clock division is 2^n . For example, value of 0 means divide by $2^0 = 1$ (no division, bypass), value of 1 means divide by $2^1 = 2$, value of "ff" means divide by $2^{255} = 510$, and so on. In MMC-Ver3.3-only mode, bits not implemented because only one clock divider is supported.
7:0	clk_divider0	0	Clock divider-0 value. Clock division is 2^n . For example, value of 0 means divide by $2^0 = 1$ (no division, bypass), value of 1 means divide by $2^1 = 2$, value of "ff" means divide by $2^{255} = 510$, and so on.

6.2.4 CLKSRC

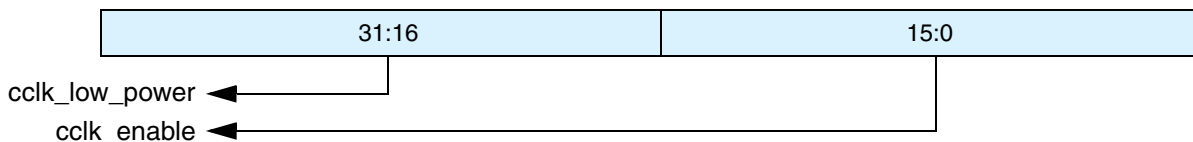
- ❖ Name: SD Clock Source Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x0C
- ❖ Read/write access: write/read



Bits	Name	Default	Description
31:0	clk_source	0	Clock divider source for up to 16 SD cards supported. Each card has two bits assigned to it. For example, bits[1:0] assigned for card-0, which maps and internally routes clock divider[3:0] outputs to cclk_out[15:0] pins, depending on bit value. 00 – Clock divider 0 01 – Clock divider 1 10 – Clock divider 2 11 – Clock divider 3 In MMC-Ver3.3-only controller, only one clock divider supported. The cclk_out is always from clock divider 0, and this register is not implemented.

6.2.5 CLKENA

- ❖ Name: Clock Enable Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x10
- ❖ Read/write access: write/read

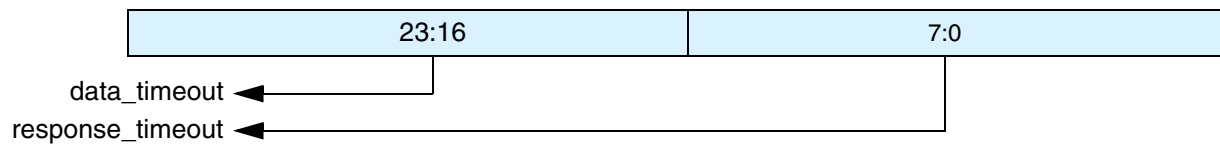


Bits	Name	Default	Description
31:16	cclk_low_power	0	Low-power control for up to 16 SD card clocks and one MMC card clock supported. 0 – Non-low-power mode 1 – Low-power mode; stop clock when card in IDLE (should be normally set to only MMC and SD memory cards; for SDIO cards, if interrupts must be detected, clock should not be stopped). In MMC-Ver3.3-only mode, since there is only one cclk_out, only cclk_low_power[0] is used.

Bits	Name	Default	Description
15:0	cclk_enable	0	Clock-enable control for up to 16 SD card clocks and one MMC card clock supported. 0 – Clock disabled 1 – Clock enabled In MMC-Ver3.3-only mode, since there is only one cclk_out, only cclk_enable[0] is used.

6.2.6 TMOUT

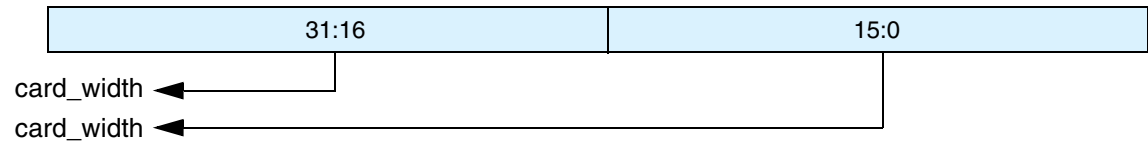
- ❖ Name: Timeout Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x14
- ❖ Read/write access: write/read



Bits	Name	Default	Description
31:8	data_timeout	0xffffffff	Value for card Data Read Timeout; same value also used for Data Starvation by Host timeout. Value is in number of card output clocks – cclk_out of selected card.
7:0	response_timeout	0x40	Response timeout value. Value is in number of card output clocks – cclk_out.

6.2.7 CTYPE

- ❖ Name: Card Type Register
- ❖ Size: 32 bits
- ❖ Address offset: 0x18
- ❖ Read/write access: read/write



Bits	Name	Default	Description
31:16	card_width	0	One bit per card indicates if card is 8-bit: 0 – Non 8-bit mode 1 – 8-bit mode Bit[31] corresponds to card[15]; bit[16] corresponds to card[0].
15:0	card_width	0	One bit per card indicates if card is 1-bit or 4-bit: 0 – 1-bit mode 1 – 4-bit mode Bit[15] corresponds to card[15], bit[0] corresponds to card[0]. Only NUM_CARDS*2 number of bits are implemented.

The following examples use values for CTYPE[16]:

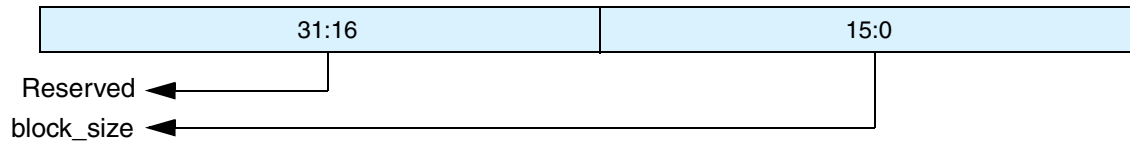
- ❖ If CTYPE[16] = 1, the card at port0 is in 8-bit mode. Note that the CTYPE[0] value is ignored; it is recommended to keep this set to 0.
- ❖ If CTYPE[16] = 0, the card at port0 is in either 1-bit or 4-bit mode, depending upon the value of CTYPE[0]; that is, if CTYPE[0] = 1 - 4-bit, CTYPE[0] = 0 - 1-bit.

Similarly, the following examples use values for CTYPE[17]:

- ❖ If CTYPE[17] = 1, the card at port1 is in 8-bit mode. Note that the CTYPE[1] value is ignored; it is recommended to keep this set to 0.
- ❖ If CTYPE[17] = 0, the card at port1 is in either 1-bit or 4-bit mode, depending upon the value of CTYPE[1]; that is, if CTYPE[1] = 1 - 4-bit, CTYPE[1] = 0 - 1-bit.

6.2.8 BLKSIZ

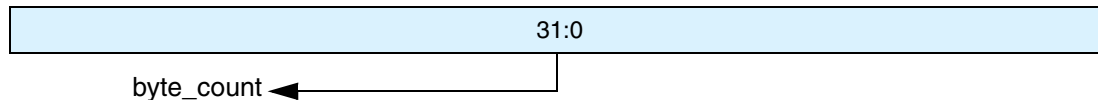
- ❖ Name: Block Size Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x1C
- ❖ Read/write access: write/read



Bits	Name	Default	Description
31:16	Reserved		
15:0	block_size	0x200	Block size

6.2.9 BYTCNT

- ❖ Name: Byte Count Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x20
- ❖ Read/write access: write/read



Bits	Name	Default	Description
31:0	byte_count	0x200	Number of bytes to be transferred; should be integer multiple of Block Size for block transfers. For undefined number of byte transfers, byte count should be set to 0. When byte count is set to 0, it is responsibility of host to explicitly send stop/abort command to terminate data transfer.

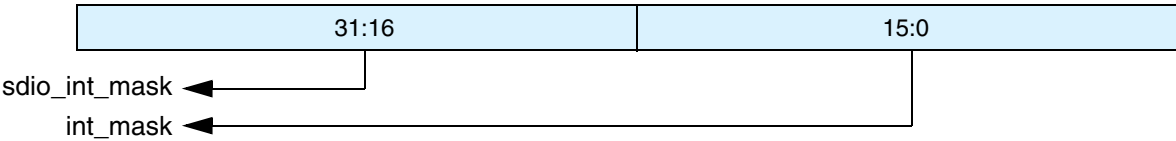


Note

In SDIO mode, if a single transfer is greater than 4 bytes and non-DWORD-aligned, the transfer should be broken where only the last transfer is non-DWORD-aligned and less than 4 bytes. For example, if a transfer of 129 bytes must occur, then the driver should start at least two transfers; one with 128 bytes and the other with 1 byte.

6.2.10 INTMASK

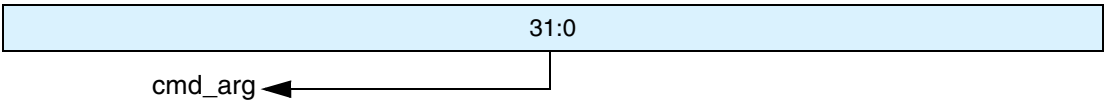
- ❖ Name: Interrupt Mask Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x24
- ❖ Read/write access: write/read



Bits	Name	Default	Description
31:16	sdio_int_mask	0x0	Mask SDIO interrupts One bit for each card. Bit[31] corresponds to card[15], and bit[16] corresponds to card[0]. When masked, SDIO interrupt detection for that card is disabled. A 0 masks an interrupt, and 1 enables an interrupt. In MMC-Ver3.3-only mode, these bits are always 0.
15:0	int_mask	0x0	Bits used to mask unwanted interrupts. Value of 0 masks interrupt; value of 1 enables interrupt. bit 15 – End-bit error (read)/Write no CRC (EBE) bit 14 – Auto command done (ACD) bit 13 – Start-bit error (SBE) bit 12 – Hardware locked write error (HLE) bit 11 – FIFO underrun/overrun error (FRUN) bit 10 – Data starvation-by-host timeout (HTO) /Volt_switch_int bit 9 – Data read timeout (DRTO) bit 8 – Response timeout (RTO) bit 7 – Data CRC error (DCRC) bit 6 – Response CRC error (RCRC) bit 5 – Receive FIFO data request (RXDR) bit 4 – Transmit FIFO data request (TXDR) bit 3 – Data transfer over (DTO) bit 2 – Command done (CD) bit 1 – Response error (RE) bit 0 – Card detect (CD)

6.2.11 CMDARG

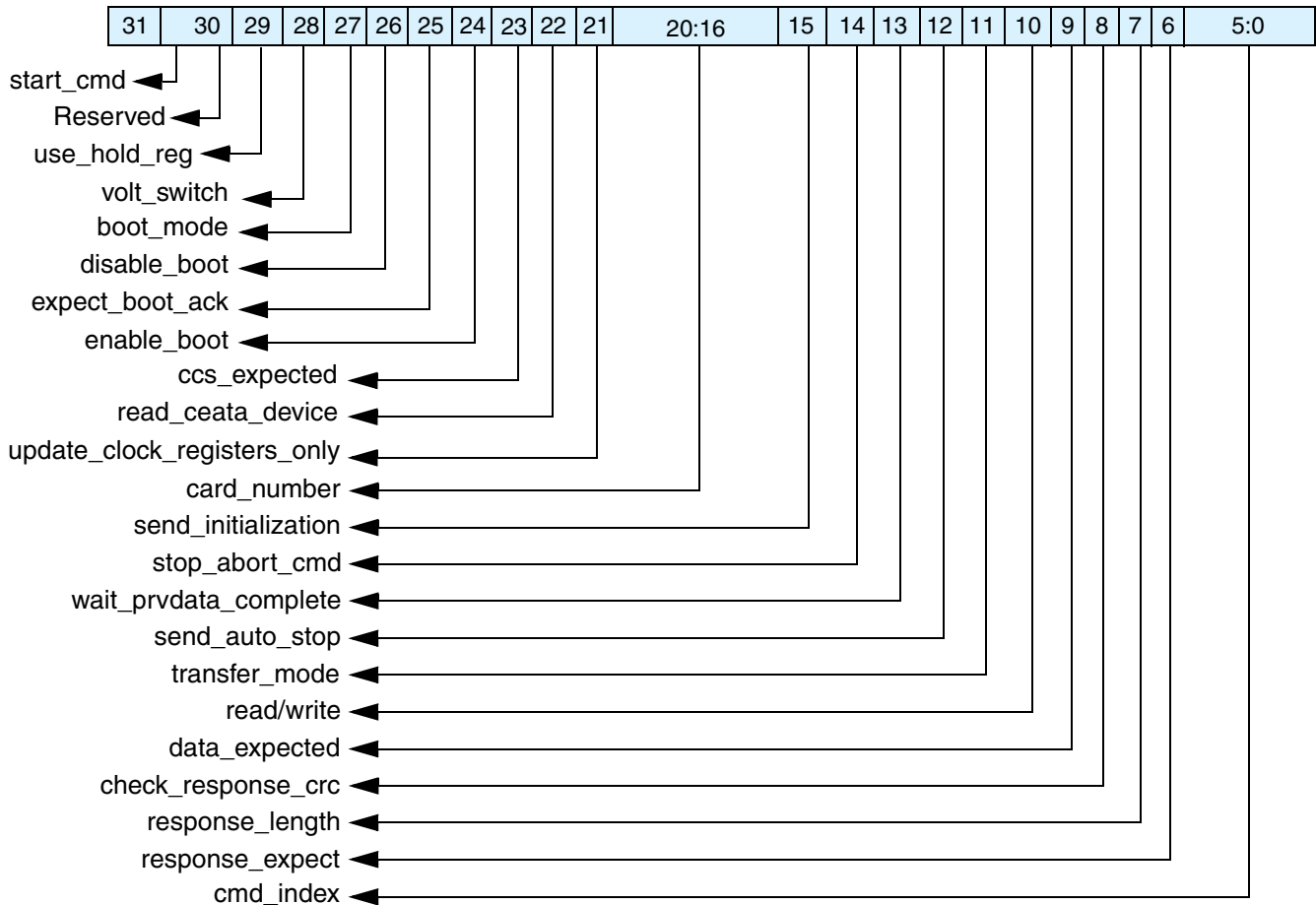
- ❖ Name: Command Argument Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x28
- ❖ Read/write access: write/read



Bits	Name	Default	Description
31:0	cmd_arg	0	Value indicates command argument to be passed to card.

6.2.12 CMD

- ❖ Name: Command Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x2C
- ❖ Read/write access: write/read



Bits	Name	Default	Description
31	start_cmd	0	Start command. Once command is taken by CIU, bit is cleared. When bit is set, host should not attempt to write to any command registers. If write is attempted, hardware lock error is set in raw interrupt register. Once command is sent and response is received from SD_MMC_CEATA cards, Command Done bit is set in raw interrupt register.
30	Reserved		

Bits	Name	Default	Description
29	use_hold_reg	0	Use Hold Register 0 - CMD and DATA sent to card bypassing HOLD Register 1 - CMD and DATA sent to card through the HOLD Register Note: - Set to 1'b1 for SDR12 and SDR25 (with non-zero phase-shifted cclk_in_drv); zero phase shift is not allowed in these modes. - Set to 1'b0 for SDR50, SDR104, and DDR50 (with zero phase-shifted cclk_in_drv) - Set to 1'b1 for SDR50, SDR104, and DDR50 (with non-zero phase-shifted cclk_in_drv) For more details on using use_hold_reg and the implementation requirements for meeting the Card input hold time, refer to “Recommended Usage” and Table 9-2 on page 276
28	volt_switch	0	Voltage switch bit 0 - No voltage switching 1 - Voltage switching enabled; must be set for CMD11 only
27	boot_mode	0	Boot Mode 0 - Mandatory Boot operation 1 - Alternate Boot operation
26	disable_boot	0	Disable Boot. When software sets this bit along with start_cmd, CIU terminates the boot operation. Do NOT set disable_boot and enable_boot together.
25	expect_boot_ack	0	Expect Boot Acknowledge. When Software sets this bit along with enable_boot, CIU expects a boot acknowledge start pattern of 0-1-0 from the selected card.
24	enable_boot	0	Enable Boot—<i>this bit should be set only for mandatory boot mode.</i> When Software sets this bit along with start_cmd, CIU starts the boot sequence for the corresponding card by asserting the CMD line low. Do NOT set disable_boot and enable_boot together.
23	ccs_expected	0	0 – Interrupts are not enabled in CE-ATA device (nIEN = 1 in ATA control register), or command does not expect CCS from device 1 – Interrupts are enabled in CE-ATA device (nIEN = 0), and RW_BLK command expects command completion signal from CE-ATA device If the command expects Command Completion Signal (CCS) from the CE-ATA device, the software should set this control bit. DWC_mobile_storage sets Data Transfer Over (DTO) bit in RINTSTS register and generates interrupt to host if Data Transfer Over interrupt is not masked.

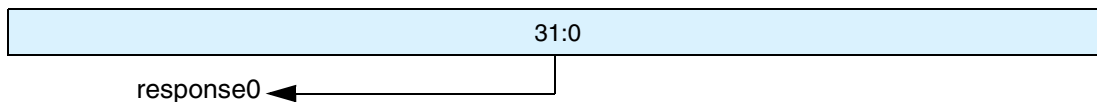
Bits	Name	Default	Description
22	read_ceata_device	0	0 – Host is not performing read access (RW_REG or RW_BLK) towards CE-ATA device 1 – Host is performing read access (RW_REG or RW_BLK) towards CE-ATA device Software should set this bit to indicate that CE-ATA device is being accessed for read transfer. This bit is used to disable read data timeout indication while performing CE-ATA read transfers. Maximum value of I/O transmission delay can be no less than 10 seconds. DWC_mobile_storage should not indicate read data timeout while waiting for data from CE-ATA device.
21	update_clock_registers_only	0	0 – Normal command sequence 1 – Do not send commands, just update clock register value into card clock domain Following register values transferred into card clock domain: CLKDIV, CLRSRC, CLKENA. Changes card clocks (change frequency, truncate off or on, and set low-frequency mode); provided in order to change clock frequency or stop clock without having to send command to cards. During normal command sequence, when update_clock_registers_only = 0, following control registers are transferred from BIU to CIU: CMD, CMDARG, TMOUT, CTYPE, BLKSIZ, BYTCNT. CIU uses new register values for new command sequence to card(s). When bit is set, there are no Command Done interrupts because no command is sent to SD_MMC_CEATA cards.
20:16	card_number	0	Card number in use. Represents physical slot number of card being accessed. In MMC-Ver3.3-only mode, up to 30 cards are supported; in SD-only mode, up to 16 cards are supported. Registered version of this is reflected on dw_dma_card_num and ge_dma_card_num ports, which can be used to create separate DMA requests, if needed. In addition, in SD mode this is used to mux or demux signals from selected card because each card is interfaced to DWC_mobile_storage by separate bus.
15	send_initialization	0	0 – Do not send initialization sequence (80 clocks of 1) before sending this command 1 – Send initialization sequence before sending this command After power on, 80 clocks must be sent to card for initialization before sending any commands to card. Bit should be set while sending first command to card so that controller will initialize clocks before sending command to card. This bit should not be set for either of the boot modes (alternate or mandatory).

Bits	Name	Default	Description
14	stop_abort_cmd	0	0 – Neither stop nor abort command to stop current data transfer in progress. If abort is sent to function-number currently selected or not in data-transfer mode, then bit should be set to 0. 1 – Stop or abort command intended to stop current data transfer in progress. When open-ended or predefined data transfer is in progress, and host issues stop or abort command to stop data transfer, bit should be set so that command/data state-machines of CIU can return correctly to idle state. This is also applicable for Boot mode transfers. To Abort boot mode, this bit should be set along with CMD[26] = disable_boot.
13	wait_prvdata_complete	0	0 – Send command at once, even if previous data transfer has not completed 1 – Wait for previous data transfer completion before sending command The wait_prvdata_complete = 0 option typically used to query status of card during data transfer or to stop current data transfer; card_number should be same as in previous command.
12	send_auto_stop	0	0 – No stop command sent at end of data transfer 1 – Send stop command at end of data transfer When set, DWC_mobile_storage sends stop command to SD_MMC_CEATA cards at end of data transfer. Refer to Table 3-8 on page 95 to determine: * when send_auto_stop bit should be set, since some data transfers do not need explicit stop commands * open-ended transfers that software should explicitly send to stop command Additionally, when “resume” is sent to resume – suspended memory access of SD-Combo card – bit should be set correctly if suspended data transfer needs send_auto_stop. Don't care if no data expected from card.
11	transfer_mode	0	0 – Block data transfer command 1 – Stream data transfer command Don't care if no data expected.
10	read/write	0	0 – Read from card 1 – Write to card Don't care if no data expected from card.
9	data_expected	0	0 – No data transfer expected (read/write) 1 – Data transfer expected (read/write)
8	check_response_crc	0	0 – Do not check response CRC 1 – Check response CRC Some of command responses do not return valid CRC bits. Software should disable CRC checks for those commands in order to disable CRC checking by controller.

Bits	Name	Default	Description
7	response_length	0	0 – Short response expected from card 1 – Long response expected from card
6	response_expect	0	0 – No response expected from card 1 – Response expected from card
5:0	cmd_index	0	Command index

6.2.13 RESP0

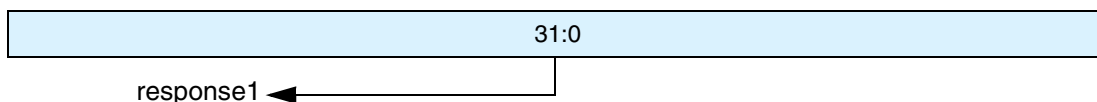
- ❖ Name: Response Register 0
- ❖ Size: 32 bits
- ❖ Address Offset: 0x30
- ❖ Read/write access: read-only



Bits	Name	Default	Description
31:0	response 0	0	Bit[31:0] of response

6.2.14 RESP1

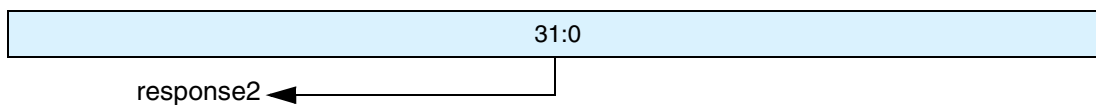
- ❖ Name: Response Register 1
- ❖ Size: 32 bits
- ❖ Address Offset: 0x34
- ❖ Read/write access: read-only



Bits	Name	Default	Description
31:0	response 1	0	Register represents bit[63:32] of long response. When CIU sends auto-stop command, then response is saved in register. Response for previous command sent by host is still preserved in Response 0 register. Additional auto-stop issued only for data transfer commands, and response type is always “short” for them. For information on when CIU sends auto-stop commands, refer to “Auto-Stop” on page 95.

6.2.15 RESP2

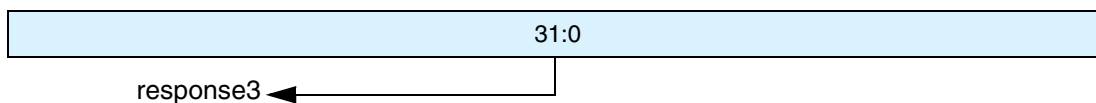
- ❖ Name: Response Register 2
- ❖ Size: 32 bits
- ❖ Address Offset: 0x38
- ❖ Read/write access: read-only



Bits	Name	Default	Description
31:0	response 2	0	Bit[95:64] of long response

6.2.16 RESP3

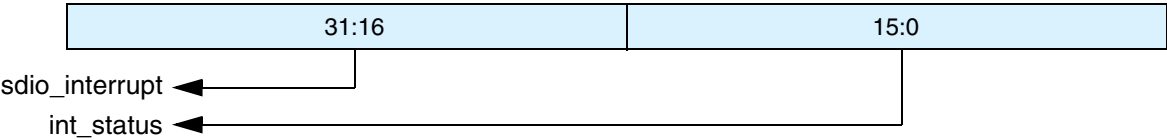
- ❖ Name: Response Register 3
- ❖ Size: 32 bits
- ❖ Address Offset: 0x3C
- ❖ Read/write access: read-only



Bits	Name	Default	Description
31:0	response 3	0	Bit[127:96] of long response

6.2.17 MINTSTS

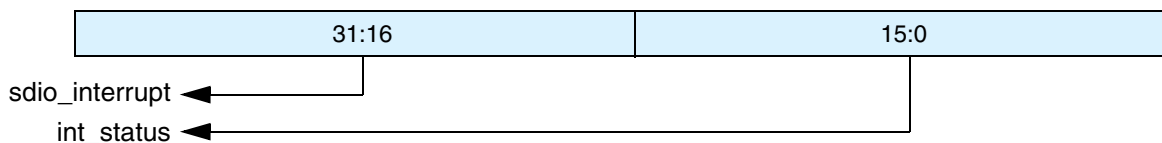
- ❖ Name: Masked Interrupt Status Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x40
- ❖ Read/write access: read
- ❖ MISTATS = RIINTSTS and INTMASK



Bits	Name	Default	Description
31:16	sdio_interrupt	0	Interrupt from SDIO card; one bit for each card. Bit[31] corresponds to Card[15], and bit[16] is for Card[0]. SDIO interrupt for card enabled only if corresponding sdio_int_mask bit is set in Interrupt mask register (mask bit 1 enables interrupt; 0 masks interrupt). 0 – No SDIO interrupt from card 1 – SDIO interrupt from card In MMC-Ver3.3-only mode, bits always 0.
15:0	int_status	0	Interrupt enabled only if corresponding bit in interrupt mask register is set. bit 15 – End-bit error (read)/write no CRC (EBE) bit 14 – Auto command done (ACD) bit 13 – Start-bit error (SBE) bit 12 – Hardware locked write error (HLE) bit 11 – FIFO underrun/overrun error (FRUN) bit 10 – Data starvation by host timeout (HTO)/Volt_switch_int bit 9 – Data read timeout (DRT0) bit 8 – Response timeout (RTO) bit 7 – Data CRC error (DCRC) bit 6 – Response CRC error (RCRC) bit 5 – Receive FIFO data request (RXDR) bit 4 – Transmit FIFO data request (TXDR) bit 3 – Data transfer over (DTO) bit 2 – Command done (CD) bit 1 – Response error (RE) bit 0 – Card detect (CD)

6.2.18 RINTSTS

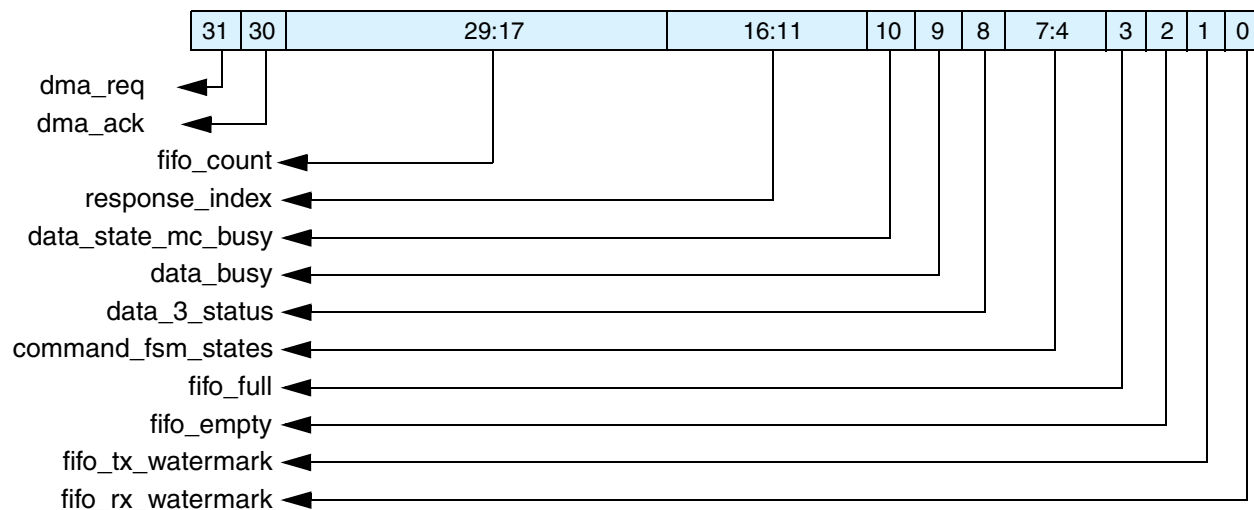
- ❖ Name: Raw Interrupt Status Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x44
- ❖ Read/write access: write/read



Bits	Name	Default	Description
31:16	sdio_interrupt	0	Interrupt from SDIO card; one bit for each card. Bit[31] corresponds to Card[15], and bit[16] is for Card[0]. Writes to these bits clear them. Value of 1 clears bit and 0 leaves bit intact. 0 – No SDIO interrupt from card 1 – SDIO interrupt from card In MMC-Ver3.3-only mode, bits always 0. Bits are logged regardless of interrupt-mask status.
15:0	int_status	0	Writes to bits clear status bit. Value of 1 clears status bit, and value of 0 leaves bit intact. Bits are logged regardless of interrupt mask status. bit 15 – End-bit error (read)/write no CRC (EBE) bit 14 – Auto command done (ACD) bit 13 – Start-bit error (SBE) bit 12 – Hardware locked write error (HLE) bit 11 – FIFO underrun/overflow error (FRUN) bit 10 – Data starvation-by-host timeout (HTO) /Volt_switch_int bit 9 – Data read timeout (DRTO)/Boot Data Start (BDS) bit 8 – Response timeout (RTO)/Boot Ack Received (BAR) bit 7 – Data CRC error (DCRC) bit 6 – Response CRC error (RCRC) bit 5 – Receive FIFO data request (RXDR) bit 4 – Transmit FIFO data request (TXDR) bit 3 – Data transfer over (DTO) bit 2 – Command done (CD) bit 1 – Response error (RE) bit 0 – Card detect (CD)

6.2.19 STATUS

- ❖ Name: Status Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x48
- ❖ Read/write access: read

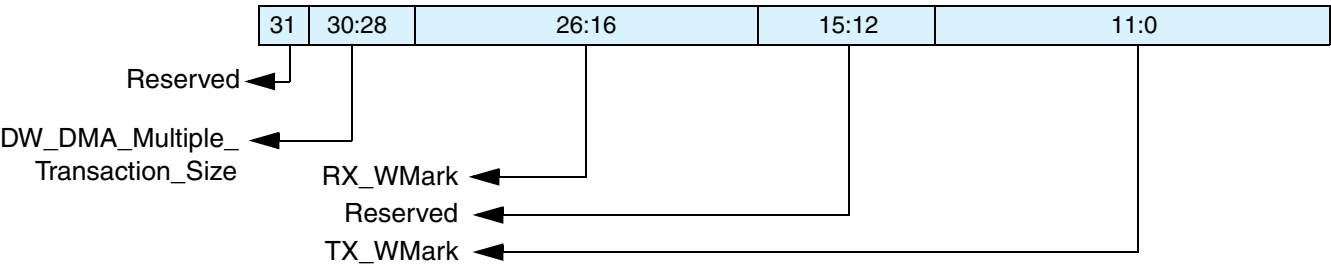


Bits	Name	Default	Description
31	dma_req	0	DMA request signal state; either dw_dma_req or ge_dma_req, depending on DW-DMA or Generic-DMA selection.
30	dma_ack	0	DMA acknowledge signal state; either dw_dma_ack or ge_dma_ack, depending on DW-DMA or Generic-DMA selection.
29:17	fifo_count	0	FIFO count – Number of filled locations in FIFO
16:11	response_index	0	Index of previous response, including any auto-stop sent by core
10	data_state_mc_busy	1	Data transmit or receive state-machine is busy
9	data_busy	1 or 0; depends on cdata_in	Inverted version of raw selected card_data[0] 0 – card data not busy 1 – card data busy
8	data_3_status	1 or 0; depends on cdata_in	Raw selected card_data[3]; checks whether card is present 0 – card not present 1 – card present

Bits	Name	Default	Description
7:4	command fsm states	0	Command FSM states: 0 – Idle 1 – Send init sequence 2 – Tx cmd start bit 3 – Tx cmd tx bit 4 – Tx cmd index + arg 5 – Tx cmd crc7 6 – Tx cmd end bit 7 – Rx resp start bit 8 – Rx resp IRQ response 9 – Rx resp tx bit 10 – Rx resp cmd idx 11 – Rx resp data 12 – Rx resp crc7 13 – Rx resp end bit 14 – Cmd path wait NCC 15 – Wait; CMD-to-response turnaround NOTE: The command FSM state is represented using 19 bits. The STATUS Register(7:4) has 4 bits to represent the command FSM states. Using these 4 bits, only 16 states can be represented. Thus three states cannot be represented in the STATUS(7:4) register. The three states that are not represented in the STATUS Register(7:4) are: * Bit 16 – Wait for CCS * Bit 17 – Send CCSD * Bit 18 – Boot Mode Due to this, while command FSM is in “Wait for CCS state” or “Send CCSD” or “Boot Mode”, the Status register indicates status as 0 for the bit field 7:4.
3	fifo_full	0	FIFO is full status
2	fifo_empty	1	FIFO is empty status
1	fifo_tx_watermark	1	FIFO reached Transmit watermark level; not qualified with data transfer.
0	fifo_rx_watermark	0	FIFO reached Receive watermark level; not qualified with data transfer.

6.2.20 FIFOTH

- ❖ Name: FIFO Threshold Watermark Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x4C
- ❖ Read/write access: write/read



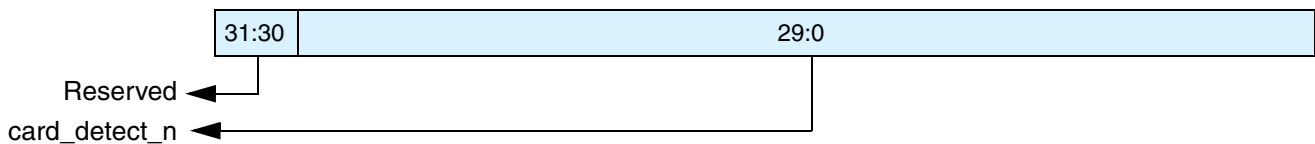
Bits	Name	Default	Description
31	Reserved		
30:28	DW_DMA_Mutiple_Transaction_Size	3'b000	Burst size of multiple transaction; should be programmed same as DW-DMA controller multiple-transaction-size SRC/DEST_MSIZ. 000 – 1 transfers 001 – 4 010 – 8 011 – 16 100 – 32 101 – 64 110 – 128 111 – 256 The units for transfers is the H_DATA_WIDTH parameter. A single transfer (dw_dma_single assertion in case of Non DW DMA interface) would be signalled based on this value. Value should be sub-multiple of (RX_WMark + 1)* (F_DATA_WIDTH/H_DATA_WIDTH) and (FIFO_DEPTH - TX_WMark)* (F_DATA_WIDTH/H_DATA_WIDTH) For example, if FIFO_DEPTH = 16, FDATA_WIDTH == H_DATA_WIDTH Allowed combinations for MSize and TX_WMark are: MSize = 1, TX_WMARK = 1-15 MSize = 4, TX_WMark = 8 MSize = 4, TX_WMark = 4 MSize = 4, TX_WMark = 12 MSize = 8, TX_WMark = 8 MSize = 8, TX_WMark = 4

Bits	Name	Default	Description
30:28 (cont.)			Allowed combinations for MSize and RX_WMark are: MSize = 1, RX_WMARK = 0-14 MSize = 4, RX_WMark = 3 MSize = 4, RX_WMark = 7 MSize = 4, RX_WMark = 11 MSize = 8, RX_WMark = 7 Recommended: MSize = 8, TX_WMark = 8, RX_WMark = 7
27:16	RX_WMark	FIFO_DEPTH - 1	FIFO threshold watermark level when receiving data to card. When FIFO data count reaches greater than this number, DMA/FIFO request is raised. During end of packet, request is generated regardless of threshold programming in order to complete any remaining data. In non-DMA mode, when receiver FIFO threshold (RXDR) interrupt is enabled, then interrupt is generated instead of DMA request. During end of packet, interrupt is not generated if threshold programming is larger than any remaining data. It is responsibility of host to read remaining bytes on seeing Data Transfer Done interrupt. In DMA mode, at end of packet, even if remaining bytes are less than threshold, DMA request does single transfers to flush out any remaining bytes before Data Transfer Done interrupt is set. 12 bits – 1 bit less than FIFO-count of status register, which is 13 bits. Limitation: $RX_WMark \leq FIFO_DEPTH-2$ Recommended: $(FIFO_DEPTH/2) - 1$; (means greater than $(FIFO_DEPTH/2) - 1$) NOTE: In DMA mode during CCS time-out, the DMA does not generate the request at the end of packet, even if remaining bytes are less than threshold. In this case, there will be some data left in the FIFO. It is the responsibility of the application to reset the FIFO after the CCS timeout.

Bits	Name	Default	Description
15:12	Reserved		
11:0	TX_WMark	0	FIFO threshold watermark level when transmitting data to card. When FIFO data count is less than or equal to this number, DMA/FIFO request is raised. If Interrupt is enabled, then interrupt occurs. During end of packet, request or interrupt is generated, regardless of threshold programming. In non-DMA mode, when transmit FIFO threshold (TXDR) interrupt is enabled, then interrupt is generated instead of DMA request. During end of packet, on last interrupt, host is responsible for filling FIFO with only required remaining bytes (not before FIFO is full or after CIU completes data transfers, because FIFO may not be empty). In DMA mode, at end of packet, if last transfer is less than burst size, DMA controller does single cycles until required bytes are transferred. 12 bits – 1 bit less than FIFO-count of status register, which is 13 bits. Limitation: TX_WMark $\geq$ 1; Recommended: FIFO_DEPTH/2; (means less than or equal to FIFO_DEPTH/2)

6.2.21 CDETECT

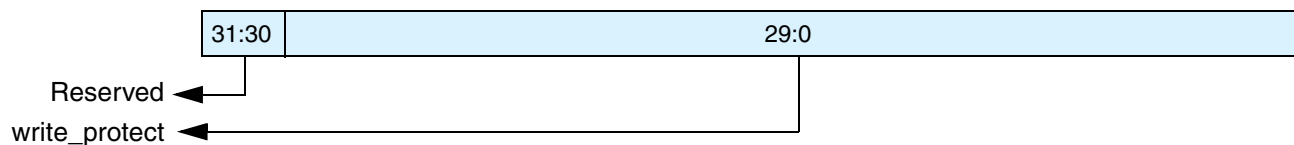
- ❖ Name: Card Detect Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x50
- ❖ Read/write access: write/read and read-only



Bits	Name	Default	Description
31:30	Reserved		
29:0	card_detect_n	card_detect_n inputs	Value on card_detect_n input ports (1 bit per card); read-only bits. 0 represents presence of card. Only NUM_CARDS number of bits are implemented.

6.2.22 WRTPT

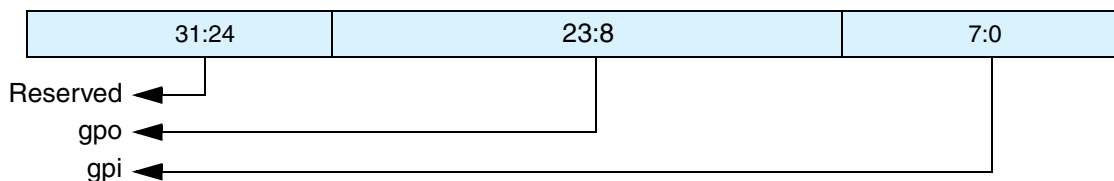
- ❖ Name: Write Protect Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x54
- ❖ Read/write access: read-only



Bits	Name	Default	Description
31:30	Reserved		
29:0	write_protect	card_write_prt inputs	Value on card_write_prt input ports (1 bit per card). 1 represents write protection. Only NUM_CARDS number of bits are implemented.

6.2.23 GPIO

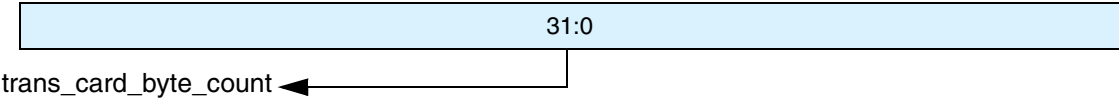
- ❖ Name: General Purpose Input/Output Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x58
- ❖ Read/write access: Partly write/read and partly read-only



Bits	Name	Default	Description
31:24	Reserved		
23:8	gpo	0	Value needed to be driven to gpo pins; this portion of register is read/write. Valid only when AREA_OPTIMIZED parameter is 0.
7:0	gpi	gpi input	Value on gpi input ports; this portion of register is read-only. Valid only when AREA_OPTIMIZED parameter is 0.

6.2.24 TCBCNT

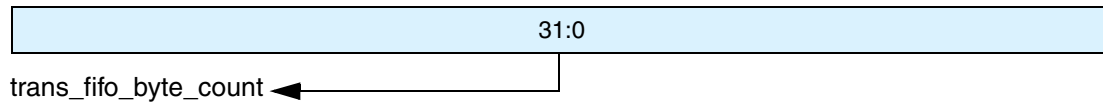
- ❖ Name: Transferred CIU Card Byte Count Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x5C
- ❖ Read/write access: read



Bits	Name	Default	Description
31:0	trans_card_byte_count	0	Number of bytes transferred by CIU unit to card. In 32-bit or 64-bit AMBA data-bus-width modes, register should be accessed in full to avoid read-coherency problems. In 16-bit AMBA data-bus-width mode, internal 16-bit coherency register is implemented. User should first read lower 16 bits and then higher 16 bits. When reading lower 16 bits, higher 16 bits of counter are stored in temporary register. When higher 16 bits are read, data from temporary register is supplied. Both TCBCNT and TBBCNT share same coherency register. When AREA_OPTIMIZED parameter is 1, register should be read only after data transfer completes; during data transfer, register returns 0.

6.2.25 TBBCNT

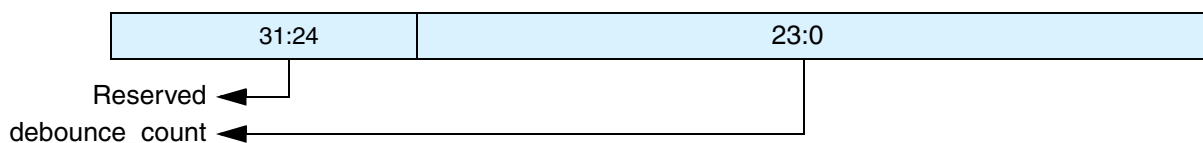
- ❖ Name: Transferred Host to BIU-FIFO Byte Count Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x60
- ❖ Read/write access: read



Bits	Name	Default	Description
31:0	trans_fifo_byte_count	0	Number of bytes transferred between Host/DMA memory and BIU FIFO. In 32-bit or 64-bit AMBA data-bus-width modes, register should be accessed in full to avoid read-coherency problems. In 16-bit AMBA data-bus-width mode, internal 16-bit coherency register is implemented. User should first read lower 16 bits and then higher 16 bits. When reading lower 16 bits, higher 16 bits of counter are stored in temporary register. When higher 16 bits are read, data from temporary register is supplied. Both TCBCNT and TBBCNT share same coherency register.

6.2.26 DEBNCE

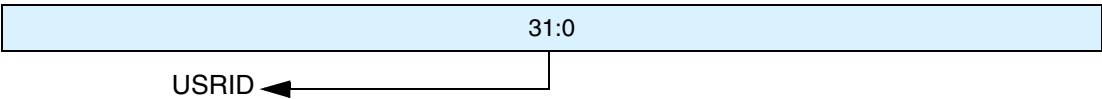
- ❖ Name: Debounce Count Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x64
- ❖ Read/write access: write/read



Bits	Name	Default	Description
31:24	Reserved		
23:0	debounce_count	24'hff_fff	Number of host clocks (clk) used by debounce filter logic; typical debounce time is 5-25 ms.

6.2.27 **USRID**

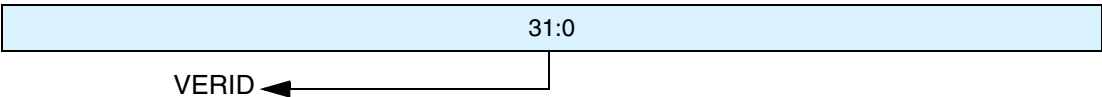
- ❖ Name: User ID Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x68
- ❖ Read/write access: write/read



Bits	Name	Default	Description
31:0	USRID	Configuration Value	User identification register; value set by user. Default reset value can be picked by user while configuring core before synthesis. Can also be used as scratch pad register by user.

6.2.28 **VERID**

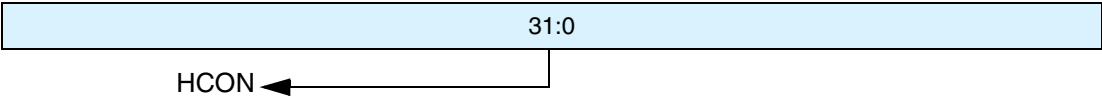
- ❖ Name: Version ID Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x6C
- ❖ Read/write access: read



Bits	Name	Default	Description
31:0	VERID	32'h5342_240a	Synopsys version identification register; register value is hard-wired. Can be read by firmware to support different versions of core.

6.2.29 HCON

- ❖ Name: Hardware Configuration Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x70
- ❖ Read/Write access: read

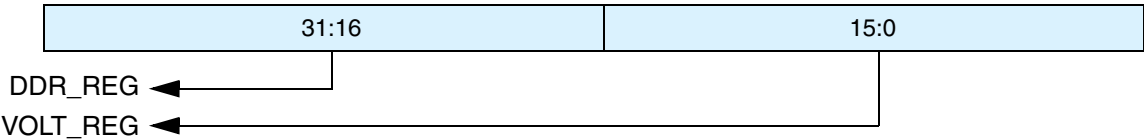


Bits	Name	Default	Description
31:0	HCON	Configuration Dependent	Hardware configurations selected by user before synthesizing core. Register values can be used to develop configuration-independent software drivers. bit 0 – CARD_TYPE 0 – MMC_ONLY 1 – SD_MMC bit [5:1] – NUM_CARDS - 1 bit 6 – H_BUS_TYPE 0 – APB 1 – AHB bit [9:7] H_DATA_WIDTH 000 – 16 bits 001 – 32 bits 010 – 64 bits others – reserved bit [15:10] – H_ADDR_WIDTH 0 to 7 – reserved 8 – 9 bits 9 – 10 bits ... 31 – 32 bits 32 to 63 – reserved bit[17:16] – DMA_INTERFACE 00 – none 01 – DW_DMA 10 – GENERIC_DMA 11 – NON-DW-DMA bit [20:18] GE_DMA_DATA_WIDTH 000 – 16 bits 001 – 32 bits 010 – 64 bits others – reserved

Bits	Name	Default	Description
			bit 21 FIFO_RAM_INSIDE 0 – outside 1 – inside
			bit 22 – IMPLEMENT_HOLD_REG 0 – no hold register 1 – hold register
			bit 23 – SET_CLK_FALSE_PATH 0 – no false path 1 – false path set
			bit [25:24] – NUM_CLK_DIVIDER-1
			bit 26 - AREA_OPTIMIZED 0 – no area optimization 1 – Area optimization
			bit [31:27] – reserved (0) For FIFO_DEPTH parameter, power-on value of RX_WMark value of FIFO Threshold Watermark Register represents FIFO_DEPTH - 1. For details on configuration parameters, refer to “Parameters” on page 115.

6.2.30 UHS_REG

- ❖ Name: UHS-1 Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x74
- ❖ Read/write access: write/read

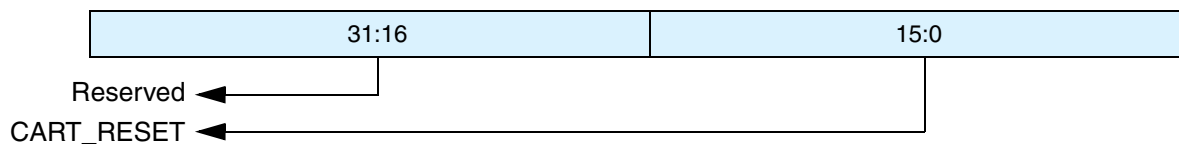


Bits	Name	Default	Description
31:16	DDR_REG	0	DDR mode. Determines the voltage fed to the buffers by an external voltage regulator. 0 – Non-DDR mode 1 – DDR mode UHS_REG [16] should be set for card number 0, UHS_REG [17] for card number 1 and so on.

Bits	Name	Default	Description
15:0	VOLT_REG	0	High Voltage mode. Determines the voltage fed to the buffers by an external voltage regulator. 0 – Buffers supplied with 3.3V Vdd 1 – Buffers supplied with 1.8V Vdd These bits function as the output of the host controller and are fed to an external voltage regulator. The voltage regulator must switch the voltage of the buffers of a particular card to either 3.3V or 1.8V, depending on the value programmed in the register. VOLT_REG[0] should be set to 1'b1 for card number 0 in order to make it operate for 1.8V.

6.2.31 RST_n

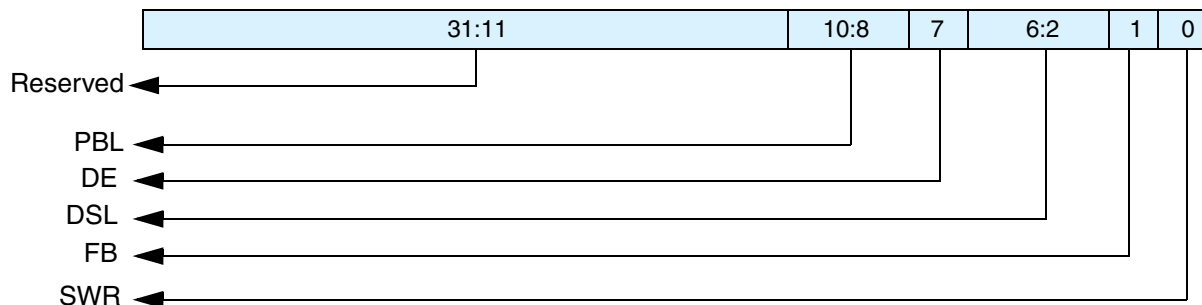
- ❖ Name: H/W Reset
- ❖ Size: 32 bits
- ❖ Address Offset: 0x78
- ❖ Read/write access: write/read



Bits	Name	Default	Description
31:16	Reserved		
15:0	CARD_RESET	1	Hardware reset. 1 – Active mode 0 – Reset These bits cause the cards to enter pre-idle state, which requires them to be re-initialized. CARD_RESET[0] should be set to 1'b1 to reset card number 0, and CARD_RESET[15] should be set to reset card number 15. The number of bits implemented is restricted to <i>NUM_CARDS</i>.

6.2.32 BMOD

- ❖ Name: Bus Mode Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x80
- ❖ Read/Write access: read/write

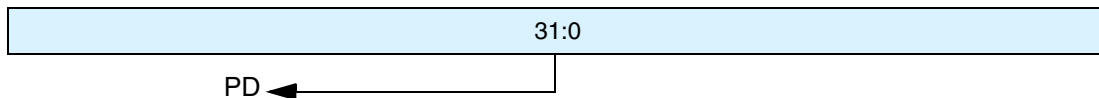


Bits	Name	Default	Description
31:11	Reserved		
10:8	PBL	32'h0	Programmable Burst Length. These bits indicate the maximum number of beats to be performed in one IDMAC transaction. The IDMAC will always attempt to burst as specified in PBL each time it starts a Burst transfer on the host bus. The permissible values are 1, 4, 8, 16, 32, 64, 128 and 256. This value is the mirror of MSIZE of FIFOTH register. In order to change this value, write the required value to FIFOTH register. This is an encode value as follows. 000 – 1 transfers 001 – 4 transfers 010 – 8 transfers 011 – 16 transfers 100 – 32 transfers 101 – 64 transfers 110 – 128 transfers 111 – 256 transfers Transfer unit is either 16, 32, or 64 bits, based on HDATA_WIDTH. PBL is a read-only value.
7	DE	0	IDMAC Enable. When set, the IDMAC is enabled. DE is read/write.
6:2	DSL	32'h00	Descriptor Skip Length. Specifies the number of HWord/Word/Dword (depending on 16/32/64-bit bus) to skip between two unchained descriptors. This is applicable only for dual buffer structure. DSL is read/write.
1	FB	0	Fixed Burst. Controls whether the AHB Master interface performs fixed burst transfers or not. When set, the AHB will use only SINGLE, INCR4, INCR8 or INCR16 during start of normal burst transfers. When reset, the AHB will use SINGLE and INCR burst transfer operations. FB is read/write.

Bits	Name	Default	Description
0	SWR	0	Software Reset. When set, the DMA Controller resets all its internal registers. SWR is read/write. It is automatically cleared after 1 clock cycle.

6.2.33 PLDMND

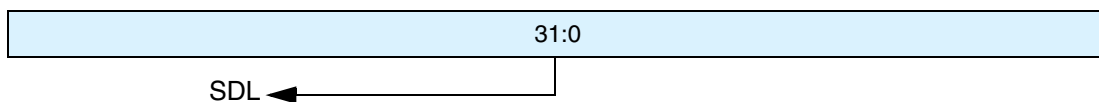
- ❖ Name: Poll Demand Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x84
- ❖ Read/write access: write



Bits	Name	Default	Description
31:0	PD	32'h0000_0000	Poll Demand. If the OWN bit of a descriptor is not set, the FSM goes to the Suspend state. The host needs to write any value into this register for the IDMAC FSM to resume normal descriptor fetch operation. This is a write only register. PD bit is write-only.

6.2.34 DBADDR

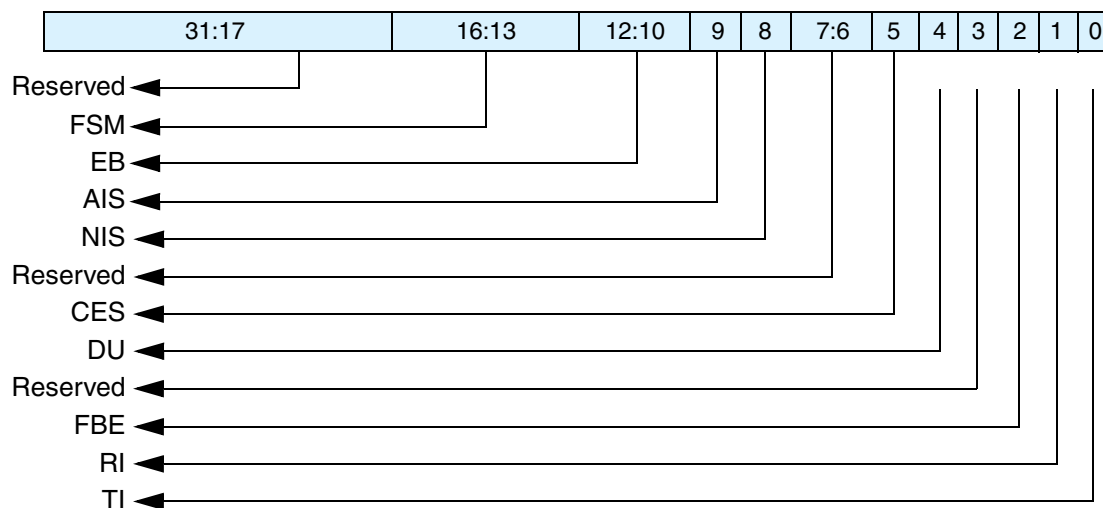
- ❖ Name: Descriptor List Base Address Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x88
- ❖ Read/write access: read/write



Bits	Name	Default	Description
31:0	SDL	32'h0000_0000	Start of Descriptor List. Contains the base address of the First Descriptor. The LSB bits [0/1/2:0] for 16/32/64-bit bus-width) are ignored and taken as all-zero by the IDMAC internally. Hence these LSB bits are read-only.

6.2.35 IDSTS

- ❖ Name: Internal DMAC Status Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x8C
- ❖ Read/write access: read/write

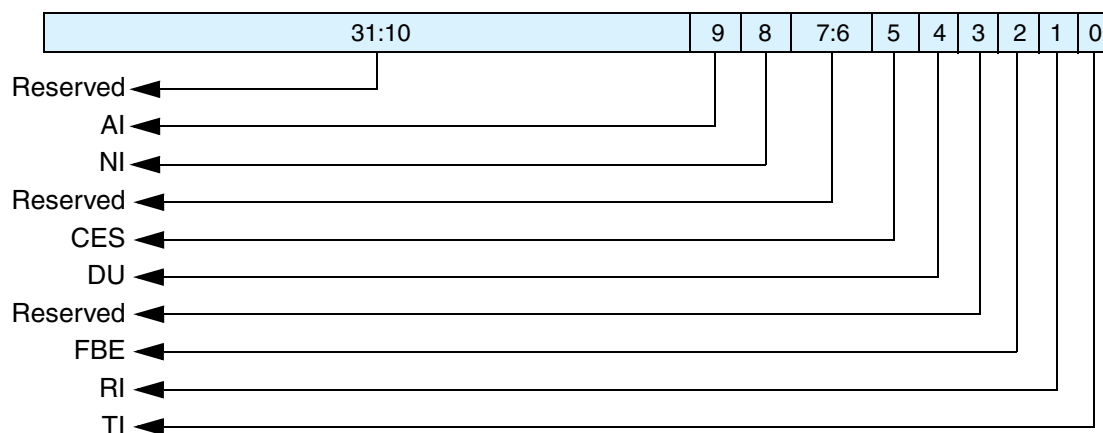


Bits	Name	Default	Description
31:17	Reserved		
16:13	FSM	32'h0	DMAC FSM present state. 0 – DMA_IDLE 1 – DMA_SUSPEND 2 – DESC_RD 3 – DESC_CHK 4 – DMA_RD_REQ_WAIT 5 – DMA_WR_REQ_WAIT 6 – DMA_RD 7 – DMA_WR 8 – DESC_CLOSE This bit is read-only.
12:10	EB	3'b000	Error Bits. Indicates the type of error that caused a Bus Error. Valid only with Fatal Bus Error bit (IDSTS[2]) set. This field does not generate an interrupt. 3'b001 – Host Abort received during transmission 3'b010 – Host Abort received during reception Others: Reserved EB is read-only.

Bits	Name	Default	Description
9	AIS	0	Abnormal Interrupt Summary. Logical OR of the following: - IDSTS[2] – Fatal Bus Interrupt - IDSTS[4] – DU bit Interrupt - IDSTS[5] – Card Error Summary Interrupt Only unmasked bits affect this bit. This is a sticky bit and must be cleared each time a corresponding bit that causes AIS to be set is cleared. Writing a 1 clears this bit.
8	NIS	0	Normal Interrupt Summary. Logical OR of the following: - IDSTS[0] – Transmit Interrupt - IDSTS[1] – Receive Interrupt Only unmasked bits affect this bit. This is a sticky bit and must be cleared each time a corresponding bit that causes NIS to be set is cleared. Writing a 1 clears this bit.
7:6	Reserved		
5	CES	0	Card Error Summary. Indicates the status of the transaction to/from the card; also present in RINTSTS. Indicates the logical OR of the following bits: - EBE – End Bit Error - RTO – Response Timeout/Boot Ack Timeout - RCRC – Response CRC - SBE – Start Bit Error - DRTO – Data Read Timeout/BDS timeout - DCRC – Data CRC for Receive - RE – Response Error Writing a 1 clears this bit.
4	DU	0	Descriptor Unavailable Interrupt. This bit is set when the descriptor is unavailable due to OWN bit = 0 (DES0[31] = 0). Writing a 1 clears this bit.
3	Reserved		
2	FBE	0	Fatal Bus Error Interrupt. Indicates that a Bus Error occurred (IDSTS[12:10]). When this bit is set, the DMA disables all its bus accesses. Writing a 1 clears this bit.
1	RI	0	Receive Interrupt. Indicates the completion of data reception for a descriptor. Writing a 1 clears this bit.
0	TI	0	Transmit Interrupt. Indicates that data transmission is finished for a descriptor. Writing a '1' clears this bit.

6.2.36 IDINTEN

- ❖ Name: Internal DMAC Interrupt Enable Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x90
- ❖ Read/write access: read/write

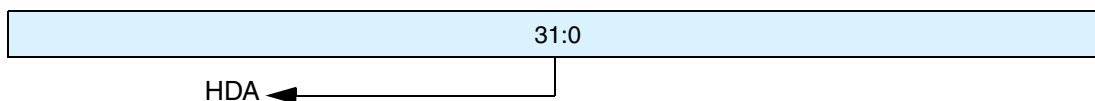


Bits	Name	Default	Description
31:10	Reserved		
9	AI	3'b000	Abnormal Interrupt Summary Enable. When set, an abnormal interrupt is enabled. This bit enables the following bits: IDINTEN[2] – Fatal Bus Error Interrupt IDINTEN[4] – DU Interrupt IDINTEN[5] – Card Error Summary Interrupt
8	NI	0	Normal Interrupt Summary Enable. When set, a normal interrupt is enabled. When reset, a normal interrupt is disabled. This bit enables the following bits: IDINTEN[0] – Transmit Interrupt IDINTEN[1] – Receive Interrupt
7:6	Reserved		
5	CES	0	Card Error summary Interrupt Enable. When set, it enables the Card Interrupt summary.
4	DU	0	Descriptor Unavailable Interrupt. When set along with Abnormal Interrupt Summary Enable, the DU interrupt is enabled.
3	Reserved		
2	FBE	0	Fatal Bus Error Enable. When set with Abnormal Interrupt Summary Enable, the Fatal Bus Error Interrupt is enabled. When reset, Fatal Bus Error Enable Interrupt is disabled.
1	RI	0	Receive Interrupt Enable. When set with Normal Interrupt Summary Enable, Receive Interrupt is enabled. When reset, Receive Interrupt is disabled.

Bits	Name	Default	Description
0	TI	0	Transmit Interrupt Enable. When set with Normal Interrupt Summary Enable, Transmit Interrupt is enabled. When reset, Transmit Interrupt is disabled.

6.2.37 DSCADDR

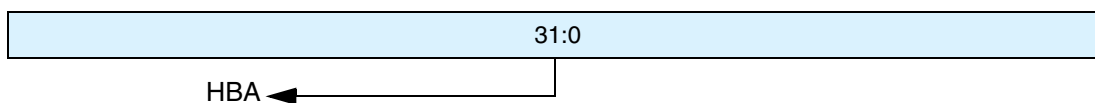
- ❖ Name: Current Host Descriptor Address Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x94
- ❖ Read/write access: read-only



Bits	Name	Default	Description
31:0	HDA	32'h0000_0000	Host Descriptor Address Pointer. Cleared on reset. Pointer updated by IDMAC during operation. This register points to the start address of the current descriptor read by the IDMAC.

6.2.38 BUFADDR

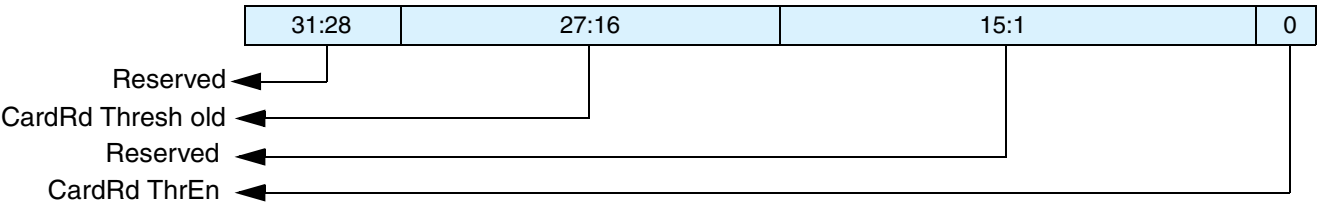
- ❖ Name: Current Buffer Descriptor Address Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0x98
- ❖ Read/write access: read-only



Bits	Name	Default	Description
31:0	HBA	32'h0000_0000	Host Buffer Address Pointer. Cleared on Reset. Pointer updated by IDMAC during operation. This register points to the current Data Buffer Address being accessed by the IDMAC.

6.2.39 CardThrCtl

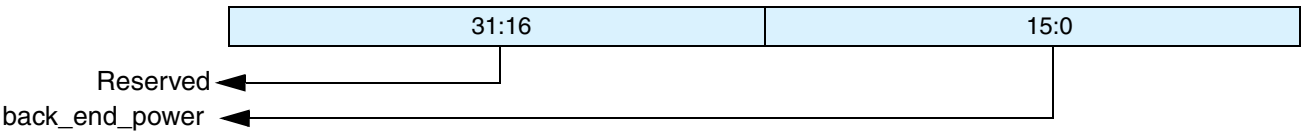
- ❖ Name: Card Threshold Control Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0100h
- ❖ Read/write access: read/write



Bits	Name	Default	Description
31:28	Reserved		
27:16	CardRd Thresh old	12'h000	Card Read Threshold size
15:1	Reserved		
0	CardRd ThrEn	1'h0	Card Read Threshold Enable 1'b0 - Card Read Threshold disabled 1'b1 - Card Read Threshold enabled. Host Controller initiates Read Transfer only if CardRdThreshold amount of space is available in receive FIFO. For more information, refer to “Card Read Threshold” on page 199.

6.2.40 Back_end_power

- ❖ Name: Back-end Power Register
- ❖ Size: 32 bits
- ❖ Address Offset: 0104h
- ❖ Read/write access: read/write



Bits	Name	Default	Description
31:16	Reserved		
15:0	Back End Power	16'h0000	Back end power 1'b0 – Off; Reset 1'b1 – Back-end Power supplied to card application; one pin per card

7

Programming the DWC_mobile_storage

This chapter contains details about programming the DWC_mobile_storage.

7.1 Software/Hardware Restrictions

Only one memory card should be selected at a time for command or data transfer. For example, when a data transfer from a card occurs, a new command should not be sent to another card; within the same card, a new command is allowed to read the status, or to stop or abort the current data transfer. Only one data transfer command should be issued at one time. For CE-ATA devices, if CE-ATA device interrupts are enabled ($nIEN=0$), only one RW_MULTIPLE_BLOCK command (RW_BLK) should be issued; no other commands (including a new RW_BLK) should be issued before the Data Transfer Over status is set for the outstanding RW_BLK.

Before issuing a new data transfer command, the software should ensure that the card is not busy due to any previous data transfer command. Before changing the card clock frequency, the software must ensure that there are no data or command transfers in progress.

If the card is enumerated in SDR50, SDR104, or DDR50 mode, then the application must program the use_hold_reg bit[29] in the CMD register to 1'b0 (phase shift of $cclk_in_drv = 0$) or 1'b1 (phase shift of $cclk_in_drv > 0$). If the card is enumerated in SDR12 or SDR25 mode, the application must program the use_hold_reg bit[29] in the CMD register to 1'b1.

This programming should be done for all data transfer commands and non-data commands that are sent to the card. When the use_hold_reg bit is programmed to 1'b0, the DWC_mobile_storage bypasses the Hold Registers in the transmit path. The value of this bit should not be changed when a Command or Data Transfer is in progress. For more details on using use_hold_reg and the implementation requirements for meeting the Card input hold time, refer to “[Recommended Usage](#)” and [Table 9-2](#) on page 276.

To avoid glitches in the card clock outputs ($cclk_out$), the software should use the following steps when changing the card clock frequency:

1. Before disabling the clocks, ensure that the card is not busy due to any previous data command. To determine this, check for 0 in bit 9 of the STATUS register.

2. Update the Clock Enable register to disable all clocks. To ensure completion of any previous command before this update, send a command to the CIU to update the clock registers by setting:
 - ◆ start_cmd bit
 - ◆ “update clock registers only” bits
 - ◆ “wait_previous data complete” bit

Wait for the CIU to take the command by polling for 0 on the start_cmd bit.

3. Set the start_cmd bit to update the Clock Divider and/or Clock Source registers, and send a command to the CIU in order to update the clock registers; wait for the CIU to take the command.
4. Set start_cmd to update the Clock Enable register in order to enable the required clocks and send a command to the CIU to update the clock registers; wait for the CIU to take the command.

In non-DMA mode, while reading from a card, the Data Transfer Over (RINTSTS[3]) interrupt occurs as soon as the data transfer from the card is over. There still could be some data left in the FIFO, and the RX_WMark interrupt may or may not occur, depending on the remaining bytes in the FIFO. Software should read any remaining bytes upon seeing the Data Transfer Over (DTO) interrupt. While using the external DMA interface for reading from a card, the DTO interrupt occurs only after all the data is flushed to memory by the DMA Interface unit.

While writing to a card in external DMA mode, if an undefined-length transfer is selected by setting the Byte Count register to 0, the DMA logic will likely request more data than it will send to the card, since it has no way of knowing at which point the software will stop the transfer. The DMA request stops as soon as the DTO is set by the CIU.

If the software issues a controller_reset command by setting control register bit[0] to 1, all the CIU state machines are reset; the FIFO is not cleared. The DMA sends all remaining bytes to the host. In addition to a card-reset, if a FIFO reset is also issued, then:

- ❖ Any pending DMA transfer on the bus completes correctly
- ❖ DMA data read is ignored
- ❖ Write data is unknown (x)

Additionally, if dma_reset is also issued, any pending DMA transfer is abruptly terminated. When the DW-DMA/Non-DW-DMA is used, the DMA controller channel should also be reset and reprogrammed.

If any of the previous data commands do not properly terminate, then the software should issue the FIFO reset in order to remove any residual data, if any, in the FIFO. After asserting the FIFO reset, you should wait until this bit is cleared.

One data-transfer requirement between the FIFO and host is that the number of transfers should be a multiple of the FIFO data width (F_DATA_WIDTH). For example, if F_DATA_WIDTH = 32 and you want to write only 15 bytes to an SD_MMC_CEATA card (BYTCNT), the host should write 16 bytes to the FIFO or program the DMA to do 16-byte transfers, if external DMA mode is enabled. The software can still program the Byte Count register to only 15, at which point only 15 bytes will be transferred to the card. Similarly, when 15 bytes are read from a card, the host should still read all 16 bytes from the FIFO.

It is recommended that you not change the FIFO threshold register in the middle of data transfers when DW-DMA/Non-DW-DMA mode is chosen.

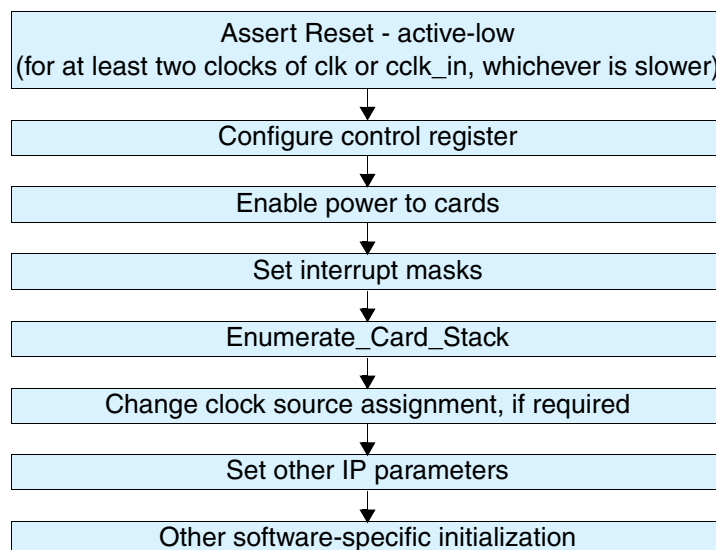
7.2 Programming Sequence

The following sections discuss the programming sequence.

7.2.1 Initialization

Figure 7-1 illustrates the initialization flow.

Figure 7-1 Initialization Sequence



Once the power and clocks are stable, reset_n should be asserted (active-low) for at least two clocks of clk or cclk_in, whichever is slower. The reset initializes the registers, ports, FIFO-pointers, DMA interface controls, and state-machines in the design. After power-on reset, the software should do the following:

1. Configure control register – For MMC-Ver3.3-only mode, enable the open-drain pullup by setting enable_OD_pullup (bit 24) in the control register.
2. Enable power to cards – Before enabling the power, confirm that the voltage setting to the voltage regulator(s) is correct. Enable power to the connected cards by setting the corresponding bit to 1 in the Power Enable register at @0x04. To enable maximum cards in MMC-Ver3.3-only mode – that is, 30 cards – write 0x03ffffff in the Power Enable register. To enable maximum cards in SD_MMC_CEATA mode – that is, 16 cards – write 0x0000ffff in the Power Enable register. Wait for the power ramp-up time.
3. Set masks for interrupts by clearing appropriate bits in the Interrupt Mask register @0x024. Set the global int_enable bit of the Control register @0x00. It is recommended that you write 0xffff_ffff to the Raw Interrupt register @0x044 in order to clear any pending interrupts before setting the int_enable bit.
4. Enumerate card stack – Each card is enumerated according to card type; for details, refer to “Enumerated Card Stack” on page 180. For enumeration, you should restrict the clock frequency to 400 KHz in accordance with SD_MMC_CEATA standards.
5. Changing clock source assignment – Required in only SD_MMC_CEATA mode. Set the card frequency using the clock-divider and lock-source registers; for details, refer to “Clock Programming” on page 183. For MMC-Ver3.3-only mode, only clock divider0 is used. MMC cards

operate at a maximum of 20 MHz (at maximum of 52 MHz in high-speed mode). SD_MMC_CEATA mode operates at a maximum of 25 MHz (at maximum of 50 MHz in high-speed mode).

6. Set other IP parameters, which normally do not need to be changed with every command, with a typical value such as timeout values in `cclk_out` according to SD_MMC_CEATA specifications.
 - ◆ `ResponseTimeOut` = 0x64
 - ◆ `DataTimeOut` = highest of one of the following:
 - ◇ $(10 * ((TAAC * Fop) + (100 * NSAC)))$
 - ◇ Host FIFO read/write latency from FIFO empty/full
 - a. Set the debounce value to 25 ms (default: 0x0fffff) in host clock (`clk`) cycle units in the `DEBNCE` register @0x064.
 - ◆ FIFO threshold value in bytes in the `FIFOTH` register @0x04C. Typically, the threshold value can be set to half the FIFO depth; that is:
 - ◇ `RX_WMark` = $(FIFO_DEPTH / 2) - 1$;
 - ◇ `TX_WMark` = $FIFO_DEPTH / 2$
7. According to MMC standards, the open-drain pullup resistor is required during only the enumeration phase. Therefore for MMC-Ver3.3-only mode, disable the open-drain pullup by clearing the `enable_OD_pullup` (bit 24) in the Control register.
8. If the software decides to handle the interrupts provided by the IP core, you should create another thread to handle interrupts.

7.2.1.0.1 Enumerated Card Stack

The card stack does the following:

- ❖ Enumerates all connected cards
- ❖ Sets the RCA for the connected cards
- ❖ Reads card-specific information
- ❖ Stores card-specific information locally

Enumeration depends on the operating mode of the `DWC_mobile_storage`; that is, MMC-Ver3.3-only or SD_MMC_CEATA mode. For SD_MMC_CEATA mode, the card type is first identified and the appropriate card enumeration routine is called.

- ❖ `Enumerate_MMC_Card_Stack` – Enumerates all the MMC cards connected on the MMC bus, up to the maximum of 30 cards. In MMC-Ver3.3-only mode, a single shared bus and clock are used to interface the `DWC_mobile_storage` to all the MMC cards. Since there is only one clock, only `divider0` is used for clock-frequency adjustments. The steps involved in this process are:
 - a. Clear the card type register to set the card width as a single bit. In MMC-Ver3.3-only mode, only one MMC card can be used at a time.
 - b. Set clock frequency to $Fod = 400 \text{ KHz}$, maximum – Program clock `divider0` (bits 0-7 in the `CLKDIV` register) value to one-half of the `cclk_in` frequency divided by 400KHz. For example, if `cclk_in` is 20 MHz, then the value is $20,000 / (2 * 400) = 25$.
 - c. Send `CMD0` with an initialization sequence of 80 cycles – This sequence is required to accommodate ramp-up time after power-on to all cards. To achieve this, set `send_initialization` (bit 15) in the `CMD` register.

- d. Send CMD1 with required voltage range – Cards that do not support the required range go to an inactive state. A common voltage range that all cards can accept is received in response to this command.
 - e. Repeat CMD1 until the card busy bit (bit 31) of the received response is set. If this bit is low, it indicates that the card is still not ready for communication.
 - f. Send CMD2 (ALL_SEND_CID) – In response to this command, all cards will try to send the CID. Ultimately, a single card wins the bus and enters an identification state.
 - g. Send CMD3 (SET_RELATIVE_ADDRESS) – Assign a relative address to the card. Start from the card address 0x01; address 0x0 is reserved for the host.
 - h. Repeat steps [f](#) and [g](#) until all the cards are finished – This state is identified when no response is received by the CMD2 command. To check if the card is connected or not, use the CDETECT register @0x050.
- ❖ Enumerate_SD_Card_Stack – Enumerates all the cards connected on the SD bus. This mode supports a maximum of 16 cards, with one card connected to one port. Each port can have an MMC, CE-ATA, SD, or SDIO type of card. All types of SDIO cards are supported; that is, SDIO_IO_ONLY, SDIO_MEM_ONLY, and SDIO_COMBO cards.

Each card is enumerated separately. The enumeration sequence includes the following steps:

- a. Start from port0.
- b. Check if the card is connected.
- c. For the given card number, clear the corresponding bits in the card_type register. Clear the register bit for a 1-bit, 4-bit, or 8-bit bus width. For example, for card number = 1, clear bit 1 and bit 17 of the card_type register @0x18.
- d. Identify the card type; that is, SD, MMC, or SDIO.
 - ❖ Send CMD5 first. If a response is received, then the card is SDIO
 - ❖ If not, send CMD8 with the following Argument:


```
Bit[31:12] = 20'h0; // Reserved bits
Bit[11:8] = 4'b0001 // VHS value
Bit[7:0] = 8'b10101010 // Preferred Check Pattern by SD2.0
```
 - ❖ If Response is received, the card supports High Capacity SD2.0; send ACMD41 with the following Argument:


```
Bit[31] = 1'b0; // Reserved bits
Bit[30] = 1'b1; // High Capacity Status
Bit[29:25] = 6'h0; // Reserved bits
Bit[24] = 1'b1; // S18R --Supports voltage switching for 1.8V
Bit[23:0] = Supported Voltage Range
```
 - ❖ If Response is received for ACMD41, then the card is SD; otherwise the card is MMC or CEATA.
 - ❖ If Bit[24] is set to 1'b1 in the response of ACMD41, then the Host can choose to switch the voltage to 1.8V, as the card supports 1.8V. Voltage Switching can be done by sending CMD11, which is explained in detail in [“Programming DWC_mobile_storage for Voltage Switching and DDR Operations”](#) on page 221.

- ✧ If Response is not received for initial CMD8, then card does not support High Capacity SD2.0; issue CMD0 followed by ACMD41 with the following Argument:

```

Bit[31] = 1'b0; // Reserved bits
Bit[30] = 1'b0; // High Capacity Status
Bit[29:24] = 6'h0; // Reserved bits
Bit[23:0] = Supported Voltage Range

```

- ✧ If Response is received for ACMD41 then card is SD otherwise card is MMC or CEATA.



Note

- It is mandatory to issue CMD8 prior to first ACMD41 to initialize the High Capacity SD Memory Card.
 - Card returns busy as a response to ACMD41 when:
 - * Card executes internal initialization process
 - * If card is high capacity SD card, then two cases are there:
 - * CMD8 was not issued before ACMD41
 - * ACMD41 is issued with HCS to 0.
- e. Enumerate the card according to the card type.
 - f. Use a clock source with a frequency = Fod (that is, 400 KHz) and use the following enumeration command sequence:
 - ✧ SD card – Send CMD0, CMD8, ACMD41, CMD2, CMD3.
 - ✧ SDIO – Send CMD5; if the function count is valid, CMD3. For the SDIO memory section, follow the same commands as for the SD card.
 - ✧ MMC – Send CMD0, CMD1, CMD2, CMD3.
 - g. Identify the MMC/CE-ATA device.
 - ✧ Selecting ATA mode for a CE-ATA v1.1 device.
 - i. Host should query the byte 504 (S_CMD_SET) of EXT_CSD register by sending CMD8. If bit 4 is set to “1,” then the device supports ATA mode.
 - ii. If ATA mode is supported, the host should select the ATA mode by setting the ATA bit (bit 4) of the EXT_CSD register slice 191(CMD_SET) to activate the ATA command set for use. The host selects the command set using the SWITCH (CMD6) command.
 - iii. The current mode selected is shown in byte 191 of the EXT_CSD register.
 - ✧ If the device does not support ATA mode, then the device can be an MMC device or a CE-ATA v1.0 device.
 - ✧ Send RW_REG; if a response is received and the response data contains CE-ATA signature, the device is a CE-ATA device.
 - ✧ Otherwise the device is an MMC card.
 - h. You can change the card clock frequency after enumeration. When cclk_in is 25 MHz:
 - ✧ SD memory card is typically set at 25 MHz.
 - ✧ MMC is set to 12.5 MHz; since the maximum frequency of an MMC card is 20 MHz, a clock divider factor of 2 is necessary.
 - ✧ Full-speed SDIO is set to 25 MHz.
 - ✧ Low-speed SDIO is set to 400 KHz.

7.2.2 Power Control

You can implement power control using the following registers, along with external circuitry:

- ❖ Control register bits `card_voltage_a` (16-19) and `card_voltage_b` (20-23) – Status of these bits is reflected at the IO pins. The bits can be used to generate or control the supply voltage that the memory cards require.
- ❖ Power enable register @0x04 – Status of these bits is reflected at the IO pins. The pins can control power to individual cards.

Programming these two registers depends on the implemented external circuitry. While turning on or off the power enable, you should confirm that power supply settings are correct. Power to all cards usually should be disabled while switching off the power.

7.2.3 Clock Programming

The `DWC_mobile_storage` supports four clock sources, each of which can be programmed with a different frequency; software can select the clock source for each card. The clock to an individual card can be enabled or disabled. Registers that support this are:

- ❖ `CLKDIV` @0x08 – Programs individual clock source frequency.
- ❖ `CLKSRC` @0x0C – Assigns clock source for each card.
- ❖ `CLKENA` @0x10 – Enables or disables clock for individual card and enables low-power mode, which automatically stops the clock to a card when the card is idle for more than 8 clocks.

The `DWC_mobile_storage` loads each of these registers only when the `start_cmd` bit and the `Update_clk_regs_only` bit in the `CMD` register are set. When a command is successfully loaded, the `DWC_mobile_storage` clears this bit, unless the `DWC_mobile_storage` already has another command in the queue, at which point it gives an HLE (Hardware Locked Error); for details on HLEs, refer to [“Error Handling”](#) on page 209.

Software should look for the `start_cmd` and the `Update_clk_regs_only` bits, and should also set the `wait_prvdata_complete` bit to ensure that clock parameters do not change during data transfer. Note that even though `start_cmd` is set for updating clock registers, the `DWC_mobile_storage` does not raise a `command_done` signal upon command completion.

For MMC-Ver3.3-only mode, only clock divider0 is used. Therefore, the `CLKSRC` register cannot be used in MMC-Ver3.3-only mode. The following shows how to program these registers:

1. Confirm that no card is engaged in any transaction; if there is a transaction, wait until it finishes.
2. Stop all clocks by writing `xxxx0000` to the `CLKENA` register. Set the `start_cmd`, `Update_clk_regs_only`, and `wait_prvdata_complete` bits in the `CMD` register. Wait until `start_cmd` is cleared or an HLE is set; in case of an HLE, repeat the command.
3. Program the `CLKDIV` and `CLKSRC` registers, as required. Set the `start_cmd`, `Update_clk_regs_only`, and `wait_prvdata_complete` bits in the `CMD` register. Wait until `start_cmd` is cleared or an HLE is set; in case of an HLE, repeat the command.
4. Re-enable all clocks by programming the `CLKENA` register. Set the `start_cmd`, `Update_clk_regs_only`, and `wait_prvdata_complete` bits in the `CMD` register. Wait until `start_cmd` is cleared or an HLE is set; in case of an HLE, repeat the command.

7.2.4 No-Data Command With or Without Response Sequence

To send any non-data command, the software needs to program the CMD register @0x2C and the CMDARG register @0x28 with appropriate parameters. Using these two registers, the DWC_mobile_storage forms the command and sends it to the command bus. The DWC_mobile_storage reflects the errors in the command response through the error bits of the RINTSTS register.

When a response is received – either erroneous or valid – the DWC_mobile_storage sets the command_done bit in the RINTSTS register. A short response is copied in Response Register0, while a long response is copied to all four response registers @0x30, 0x34, 0x38, and 0x3C. The Response3 register bit 31 represents the MSB, and the Response0 register bit 0 represents the LSB of a long response.

For basic commands or non-data commands, follow these steps:

1. Program the Command register @0x28 with the appropriate command argument parameter.
2. Program the Command register @0x2C with the settings in [Table 7-1](#).

Table 7-1 Command Register Settings for No-Data Command

Parameter	Value	Comment
Default		
start_cmd	1	–
use_hold_reg	1/0	Choose value based on speed mode being used; refer to “ use_hold_reg ” on page 150 for values
Update_clk_regs_only	0	No clock parameters update command
data_expected	0	No data command
card_number	nCardNo	Actual card number
cmd_index	command index	–
send_initialization	0	Can be 1, but only for card reset commands, such as CMD0
stop_abort_cmd	0	Can be 1 for commands to stop data transfer, such as CMD12
response_length	0	Can be 1 for R2 (long) response
response_expect	1	Can be 0 for commands with no response; for example, CMD0, CMD4, CMD15, and so on
User-selectable		
wait_prvdata_complete	1	Before sending command on command line, host should wait for completion of any data command in process, if any (recommended to always set this bit, unless the current command is to query status or stop data transfer when transfer is in progress)
check_response_crc	1	If host should crosscheck CRC of response received

3. Wait for command acceptance by host. The following happens when the command is loaded into the DWC_mobile_storage:
 - ◆ DWC_mobile_storage accepts the command for execution and clears the start_cmd bit in the CMD register, unless one command is in process, at which point the DWC_mobile_storage can load and keep the second command in the buffer.
 - ◆ If the DWC_mobile_storage is unable to load the command – that is, a command is already in progress, a second command is in the buffer, and a third command is attempted – then it generates an HLE (hardware-locked error).
4. Check if there is an HLE.
5. Wait for command execution to complete. After receiving either a response from a card or response timeout, the DWC_mobile_storage sets the command_done bit in the RINTSTS register. Software can either poll for this bit or respond to a generated interrupt.
6. Check if response_timeout error, response_CRC error, or response error is set. This can be done either by responding to an interrupt raised by these errors or by polling bits 1, 6, and 8 from the RINTSTS register @0x44. If no response error is received, then the response is valid. If required, the software can copy the response from the response registers @0x30-0x3C.

Software should not modify clock parameters while a command is being executed.

7.2.5 Data Transfer Commands

Data transfer commands transfer data between the memory card and the DWC_mobile_storage. To send a data command, the DWC_mobile_storage needs a command argument, total data size, and block size. Software can receive or send data through the FIFO.

Before a data transfer command, software should confirm that the card is not busy and is in a transfer state, which can be done using the CMD13 and CMD7 commands, respectively.



Attention

During CE-ATA RW_BLK write transfers, the MMC busy may be asserted after the last block. If the CE-ATA device interrupt is disabled (nIEN = 1), the DTO bit is set in the RINTSTS, even though the device asserts MMC BUSY. The host should not issue CMD60 to check the ATA busy status after CMD61. Instead, the host should do one of the following:

- Issue CMD13 and check the MMC Busy status before issuing a new CMD60
- Issue CMD39 and check the ATA Busy status before issuing a new CMD60

For the data transfer commands, it is important that the same bus width that is programmed in the card should be set in the card type register @0x18. Therefore, in order to change the bus width, you should always use the following supplied APIs as appropriate for the type of card:

- ❖ Set_SD_Mode() – SD/SDIO card
- ❖ Set_HSmodeSettings() – HSMMC card

The DWC_mobile_storage generates an interrupt for different conditions during data transfer, which are reflected in the RINTSTS register @0x44 as:

1. Data_Transfer_Over (bit 3) – When data transfer is over or terminated. If there is a response timeout error, then the DWC_mobile_storage does not attempt any data transfer and the “Data Transfer Over” bit is never set.
2. Transmit_FIFO_Data_request (bit 4) – FIFO threshold for transmitting data was reached; software is expected to write data, if available, in FIFO.

3. Receive_FIFO_Data_request (bit 5) – FIFO threshold for receiving data was reached; software is expected to read data from FIFO.
4. Data starvation by Host timeout (bit 10) – FIFO is empty during transmission or is full during reception. Unless software writes data for empty condition or reads data for full condition, the DWC_mobile_storage cannot continue with data transfer. The clock to the card has been stopped.
5. Data read timeout error (bit 9) – Card has not sent data within the timeout period.
6. Data CRC error (bit 7) – CRC error occurred during data reception.
7. Start bit error (bit 13) – Start bit was not received during data reception.
8. End bit error (bit 15) – End bit was not received during data reception or for a write operation; a CRC error is indicated by the card.

Conditions 6, 7, and 8 indicate that the received data may have errors. If there was a response timeout, then no data transfer occurred.

7.2.5.1 Single-Block or Multiple-Block Read

Steps involved in a single-block or multiple-block read are:

1. Write the data size in bytes in the BYTCNT register @0x20.
2. Write the block size in bytes in the BLKSIZ register @0x1C. The DWC_mobile_storage expects data from the card in blocks of size BLKSIZ each.
3. For cards that have a round trip delay of more than 0.5 cclk_in period, program the Card Read Threshold to ensure that the card clock does not stop in the middle of a block of data being transferred from the card to the Host; for programming guidelines, refer to “[Card Read Threshold](#)” on page 199.

If the Card Read Threshold feature is not enabled for such cards, then the Host system should ensure that the Rx FIFO does not become full during a Read Data transfer by ensuring that the Rx FIFO is drained out at a rate faster than that at which data is pushed into the FIFO; alternatively, a sufficiently large FIFO can be provided. For more details, refer to “[FIFO Size Requirements](#)” on page 277.

4. Program the CMDARG register @0x28 with the data address of the beginning of a data read.

Program the Command register with the parameters listed in [Table 7-2](#). For SD and MMC cards, use CMD17 for a single-block read and CMD18 for a multiple-block read. For SDIO cards, use CMD53 for both single-block and multiple-block transfers.

Table 7-2 Command Register Settings for Single-Block or Multiple-Block Read

Parameter	Value	Comment
Default		
start_cmd	1	–
use_hold_reg	1/0	Choose value based on speed mode being used – refer to “ use_hold_reg ” on page 150 for values
Update_clk_regs_only	0	No clock parameters update command
card_number	nCardNo	Actual card number
send_initialization	0	Can be 1, but only for card reset commands, such as CMD0

Table 7-2 Command Register Settings for Single-Block or Multiple-Block Read (Cont.)

Parameter	Value	Comment
stop_abort_cmd	0	Can be 1 for commands to stop data transfer, such as CMD12
send_auto_stop	0 or 1	Set according to Table 3-8 on page 95
transfer_mode	0	Block transfer
read_write	0	Read from card
data_expected	1	Data command
response_length	0	Can be 1 for R2 (long) response
response_expect	1	Can be 0 for commands with no response; for example, CMD0, CMD4, CMD15, and so on
User-selectable		
cmd_index	command index	–
wait_prvdata_complete	1	0 – sends command immediately 1 – sends command after previous data transfer ends
check_response_crc	1	0 – DWC_mobile_storage should not check response CRC 1 – DWC_mobile_storage should check response CRC

After writing to the CMD register, the DWC_mobile_storage starts executing the command; when the command is sent to the bus, the command_done interrupt is generated.

- Software should look for data error interrupts; that is, bits 7, 9, 13, and 15 of the RINTSTS register. If required, software can terminate the data transfer by sending a STOP command.
- Software should look for Receive_FIFO_Data_request and/or data starvation by host timeout conditions. In both cases, the software should read data from the FIFO and make space in the FIFO for receiving more data.
- When a Data_Transfer_Over interrupt is received, the software should read the remaining data from the FIFO.

7.2.6 Single-Block or Multiple-Block Write

Steps involved in a single-block or multiple-block write are:

- Write the data size in bytes in the BYTCNT register @0x20.
- Write the block size in bytes in the BLKSIZ register @0x1C; the DWC_mobile_storage sends data in blocks of size BLKSIZ each.
- Program CMDARG register @0x28 with the data address to which data should be written.
- Write data in the FIFO; it is usually best to start filling data the full depth of the FIFO.

5. Program the Command register with the parameters listed in [Table 7-3](#). For SD and MMC cards, use CMD24 for a single-block write and CMD25 for a multiple-block write. For SDIO cards, use CMD53 for both single-block and multiple-block transfers.

Table 7-3 Command Register Settings for Single-Block or Multiple-Block Write

Parameter	Value	Comment
Default		
start_cmd	1	–
use_hold_reg	1 or 0	Choose value based on speed mode; refer to “ use_hold_reg ” on page 150 for values
Update_clk_regs_only	0	No clock parameters update command
card_number	<i>nCardNo</i>	Actual card number
send_initialization	0	Can be 1, but only for card reset commands, such as CMD0
stop_abort_cmd	0	Can be 1 for commands to stop data transfer, such as CMD12
send_auto_stop	0 or 1	Set according to Table 3-8 on page 95
transfer_mode	0	Block transfer
read_write	1	Write to card
data_expected	1	Data command
response_length	0	Can be 1 for R2 (long) response
response_expect	1	Can be 0 for commands with no response; for example, CMD0, CMD4, CMD15, and so on
User-selectable		
cmd_index	command index	–
wait_prvdata_complete	1	0 – sends command immediately 1 – sends command after previous data transfer ends
check_response_crc	1	0 – DWC_mobile_storage should not check response CRC 1 – DWC_mobile_storage should check response CRC

After writing to the CMD register, DWC_mobile_storage starts executing a command; when the command is sent to the bus, a command_done interrupt is generated.

6. Software should look for data error interrupts; that is, for bits 7, 9, and 15 of the RINTSTS register. If required, software can terminate the data transfer by sending the STOP command.
7. Software should look for Transmit_FIFO_Data_request and/or timeout conditions from data starvation by the host. In both cases, the software should write data into the FIFO.
8. When a Data_Transfer_Over interrupt is received, the data command is over. For an open-ended block transfer, if the byte count is 0, the software must send the STOP command. If the byte count is not 0, then upon completion of a transfer of a given number of bytes, the DWC_mobile_storage

should send the STOP command, if necessary. Completion of the AUTO-STOP command is reflected by the Auto_command_done interrupt – bit 14 of the RINTSTS register. A response to AUTO_STOP is stored in RESP1 @0x34.

7.2.7 Stream Read

A stream read is like the block read mentioned in [“Single-Block or Multiple-Block Read”](#) on page 186, except for the following bits in the Command register:

```
transfer_mode= 1; //Stream transfer
cmd_index = CMD20;
```

A stream transfer is allowed for only a single-bit bus width.

7.2.8 Stream Write

A stream write is exactly like the block write mentioned in [“Single-Block or Multiple-Block Write”](#) on page 187, except for the following bits in the Command register:

```
transfer_mode= 1; //Stream transfer
cmd_index = CMD11;
```

In a stream transfer, if the byte count is 0, then the software must send the STOP command. If the byte count is not 0, then when a given number of bytes completes a transfer, the DWC_mobile_storage sends the STOP command. Completion of this AUTO_STOP command is reflected by the Auto_command_done interrupt. A response to an AUTO_STOP is stored in the RESP1 register @0x34.

A stream transfer is allowed for only a single-bit bus width.

7.2.9 Sending Stop or Abort in Middle of Transfer

The STOP command can terminate a data transfer between a memory card and the DWC_mobile_storage, while the ABORT command can terminate an I/O data transfer for only the SDIO_IOONLY and SDIO_COMBO cards.

- ❖ Send STOP command – Can be sent on the command line while a data transfer is in progress; this command can be sent at any time during a data transfer. For information on sending this command, refer to [“No-Data Command With or Without Response Sequence”](#) on page 184.

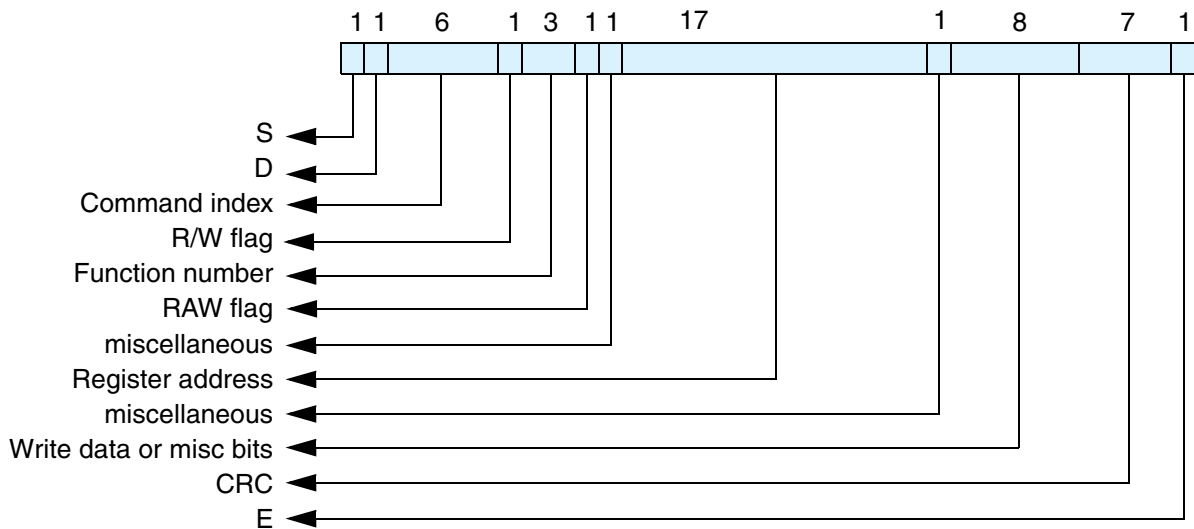
You can also use an additional setting for this command in order to set the Command register bits (5-0) to CMD12 and set bit 14 (stop_abort_cmd) to 1. If stop_abort_cmd is not set to 1, the DWC_mobile_storage does not know that the user stopped a data transfer. Reset bit 13 of the Command register (wait_prvdata_complete) to 0 in order to make the DWC_mobile_storage send the command at once, even though there is a data transfer in progress.

- ❖ Send ABORT command – Can be used with only an SDIO_IOONLY or SDIO_COMBO card. To abort the function that is transferring data, program the function number in ASx bits (CCCR register of card, address 0x06, bits (0-2) using CMD52.

This is a non-data command. For information on sending this command, refer to [“No-Data Command With or Without Response Sequence”](#) on page 184.

The command format for CMD52 is illustrated in [Figure 7-2](#).

Figure 7-2 Command format for CMD52



- Program the CMDARG register @0x28 with the appropriate command argument parameters listed in [Table 7-4](#).

Table 7-4 Parameters for CMDARG Register

CMDARG Bits	Contents	Value
31	R/W flag	1
30-28	Function Number	0, for CCCR access
27	RAW flag	1, if needed to read after write
26	Don't care	—
25-9	Register address	0x06
8	Don't care	—
7-0	Write Data	Function number to be aborted

- Program the Command register using the command index as CMD52. Similar to the STOP command described on [189](#), set bit 14 of the Command register (stop_abort_cmd) to 1, which must be done in order to inform the DWC_mobile_storage that the user aborted the data transfer. Reset bit 13 (wait_prvdata_complete) of the Command register to 0 in order to make the DWC_mobile_storage send the command at once, even though a data transfer is in progress.
- Wait for command_transfer_over.
- Check response (R5) for errors.

7.2.10 Suspend or Resume Sequence

In an SDIO card, the data transfer between an I/O function and the DWC_mobile_storage can be temporarily halted using the SUSPEND command; this may be required in order to perform a high-priority

data transfer with another function. When desired, the data transfer can be resumed using the RESUME command.

The following functions can be implemented by programming the appropriate bits in the CCCR register (Function 0) of the SDIO card. To read from or write to the CCCR register, use the CMD52 command.

1. SUSPEND data transfer – Non-data command.
 - a. Check if the SDIO card supports the SUSPEND/RESUME protocol; this can be done through the SBS bit in the CCCR register @0x08 of the card.
 - b. Check if the data transfer for the required function number is in process; the function number that is currently active is reflected in bits 0-3 of the CCCR register @0x0D. Note that if the BS bit (address 0xc::bit 0) is 1, then only the function number given by the FSx bits is valid.
 - c. To suspend the transfer, set BR (bit 2) of the CCCR register @0x0C.
 - d. Poll for clear status of bits BR (bit 1) and BS (bit 0) of the CCCR @0x0C. The BS (Bus Status) bit is 1 when the currently-selected function is using the data bus; the BR (Bus Release) bit remains 1 until the bus release is complete. When the BR and BS bits are 0, the data transfer from the selected function has been suspended.
 - e. During a read-data transfer, the DWC_mobile_storage can be waiting for the data from the card. If the data transfer is a read from a card, then the DWC_mobile_storage must be informed after the successful completion of the SUSPEND command. The DWC_mobile_storage then resets the data state machine and comes out of the wait state. To accomplish this, set abort_read_data (bit 8) in the Control register.
 - f. Wait for data completion. Get pending bytes to transfer by reading the TCBCNT register @0x5C.
2. RESUME data transfer – This is a data command.
 - a. Check that the card is not in a transfer state, which confirms that the bus is free for data transfer.
 - b. If the card is in a disconnect state, select it using CMD7. The card status can be retrieved in response to CMD52/CMD53 commands.
 - c. Check that a function to be resumed is ready for data transfer; this can be confirmed by reading the RFx flag in CCCR @0x0F. If RF = 1, then the function is ready for data transfer.
 - d. To resume transfer, use CMD52 to write the function number at FSx bits (0-3) in the CCCR register @0x0D. Form the command argument for CMD52 and write it in CMDARG @0x28; bit values are listed in [Table 7-5](#).

Table 7-5 CMDARG Bit Values

CMDARG Bits	Content	Value
31	R/W flag	1
30-28	Function Number	0, for CCCR access
27	RAW flag	1, read after write
26	Don't care	–
25-9	Register address	0x0D
8	Don't care	–
7-0	Write Data	Function number that is to be resumed

- e. Write the block size in the BLKSIZ register @0x1C; data will be transferred in units of this block size.
- f. Write the byte count in the BYTCNT register @0x20. This is the total size of the data; that is, the remaining bytes to be transferred. It is the responsibility of the software to handle the data.
- g. Program Command register; similar to a block transfer. For details, refer to [“Single-Block or Multiple-Block Read”](#) on page 186 and [“Single-Block or Multiple-Block Write”](#) on page 187.
- h. When the Command register is programmed, the command is sent and the function resumes data transfer. Read the DF flag (Resume Data Flag). If it is 1, then the function has data for the transfer and will begin a data transfer as soon as the function or memory is resumed. If it is 0, then the function has no data for the transfer.
- i. If the DF flag is 0, then in case of a read, the DWC_mobile_storage waits for data. After the data timeout period, it gives a data timeout error.

7.2.11 Read_Wait Sequence

Read_wait is used with only the SDIO card and can temporarily stall the data transfer – either from function or memory – and allow the host to send commands to any function within the SDIO device. The host can stall this transfer for as long as required. The DWC_mobile_storage provides the facility to signal this stall transfer to the card. The steps for doing this are:

1. Check if the card supports the read_wait facility; read SRW (bit 2) of the CCCR register @0x08. If this bit is 1, then all functions in the card support the read_wait facility. Use CMD52 to read this bit.
2. If the card supports the read_wait signal, then assert it by setting the read_wait (bit 6) in the CTRL register @0x00.
3. Clear the read_wait bit in the CTRL register.

7.2.12 CE-ATA Data Transfer Commands

This section describes the CE-ATA data transfer commands. For information on the basic settings and interrupts generated for different conditions, refer to [“Data Transfer Commands”](#) on page 185.

7.2.12.1 Reset and Device Recovery

Before starting CE-ATA operations, the host should perform an MMC reset and initialization procedure. The host and device should negotiate the MMC TRAN state (defined by the MultiMedia Card System Specification) before the device enters the MMC TRAN state.

The host should follow the existing MMC Card enumeration procedure in order to negotiate the MMC TRAN state. After completing normal MMC reset and initialization procedures, the host should query the initial ATA Task File values using RW_REG/CMD39.

By default, the MMC block size is 512 bytes – indicated by bits 1:0 of the srcControl register inside the CE-ATA device. The host can negotiate the use of a 1KB or 4KB MMC block size. The device indicates MMC block sizes that it can support through the srcCapabilities register; the host reads this register in order to negotiate the MMC block size. Negotiation is complete when the host controller writes the MMC block size into the srcControl register bits 1:0 of the device.

7.2.12.2 ATA Task File Transfer

ATA task file registers are mapped to addresses 0x00h-0x10h in the MMC register space. RW_REG is used to issue the ATA command, and the ATA task file is transmitted in a single RW_REG MMC command sequence.

The host software stack should write the task file image to the FIFO before setting the CMDARG and CMD registers. The host processor then sets the address and byte count in the CMDARG – offset 0x28 in the BIU register space – before setting the CMD (offset 0x2C) register bits.

For RW_REG, there is no command completion signal from the CE-ATA device.

7.2.12.3 ATA Task File Transfer Using RW_MULTIPLE_REGISTER (RW_REG)

This command involves data transfer between the CE-ATA device and the DWC_mobile_storage. To send a data command, the DWC_mobile_storage needs a command argument, total data size, and block size. Software can receive or send data through the FIFO.

Steps involved in an ATA Task file transfer (read or write) are:

1. Write the data size in bytes in the BYTCNT register @0x20.
2. Write the block size in bytes in the BLKSIZ register @0x1C; the DWC_mobile_storage expects a single block transfer.
3. Program the CMDARG register @0x28 with the beginning register address.

You should program the CMDARG, CMD, BLKSIZ, and BYTCNT registers according to the following tables.

❖ Program the Command Argument ([CMDARG](#)) register as shown below.

Bits	Value	Comment
31	1 or 0	Set to 0 (Read operation) or 1 (write operation)
30:24	0	Reserved; bits cleared to 0 by host processor
23:18	0	Starting register address for read/write; Dword aligned
17:16	0	Register address; Dword aligned
15:8	0	Reserved; bits cleared to 0 by host processor
7:2	16	Number of bytes to read/write; integral number of Dwords
1:0	0	Byte count in integral number of Dwords

❖ Program the Command ([CMD](#)) register as shown below.

Bits	Value	Comment
start_cmd	1	—
ccs_expected	0	Command Completion Signal is not expected
read_ceata_device	0 or 1	1 – If RW_BLK or RW_REG read
update_clk_regs_only	0	No clock parameters update command
card_no	nCardNo	Actual device number

Bits	Value	Comment
send_initialization	0	No initialization sequence
stop_abort_cmd	0	—
send_auto_stop	0	—
transfer_mode	0	Block transfer mode; block size and byte count should match number of bytes to read/write
read_write	1 or 0	1 – write 0 – read
data_expected	1	Data is expected
response_length	0	—
response_expect	1	—
cmd_index	Command index	—
wait_prv_data_complete	1	0 – Sends command immediately 1 – Sends command after previous data transfer over
check_response_CRC	1	0 – Not check response CRC 1 – Check response CRC

❖ Program the Block Size ([BLKSIZ](#)) register as shown below.

Bits	Value	Comment
31:16	0	Reserved bits set as zeroes (0)
15:0 (block_size)	16	For accessing entire task file (16, 8-bit registers); block size of 16 bytes

❖ Program the Byte Count ([BYTCNT](#)) register as shown below.

Bits	Value	Comment
31:0	16	For accessing entire task file (16, 8-bit registers); byte count value of 16 is used with the Block size set to 16

7.2.12.4 ATA Payload Transfer Using RW_MULTIPLE_BLOCK (RW_BLK)

This command involves data transfer between the CE-ATA device and the DWC_mobile_storage. To send a data command, the DWC_mobile_storage needs a command argument, total data size, and block size. Software can receive or send data through the FIFO.

Steps involved in an ATA payload transfer (read or write) are:

1. Write the data size in bytes in the BYTCNT register @0x20.
2. Write the block size in bytes in the BLKSIZ register @0x1C. The DWC_mobile_storage expects a single/multiple block transfer.
3. Program the CMDARG register @0x28 to indicate the Data Unit Count.

You should program the CMDARG, CMD, BLKSIZ, and BYTCNT registers according to the following tables.

- ❖ Program the Command Argument (**CMDARG**) register as shown below.

Bits	Value	Comment
31	1 or 0	Set this bit to 0 (Read operation) or 1 (write operation)
30:24	0	Reserved; bits are cleared to 0 by host processor
23:16	0	Reserved; bits are cleared to 0 by host processor
15:8	Data count	Data Count Unit [15:8]
7:0	Data count	Data Count Unit [7:0]

- ❖ Program the Command (**CMD**) register with the parameters as shown below.

Bits	Value	Comment
start_cmd	1	—
ccs_expected	1	Command Completion Signal is expected; set for RW_BLK if interrupts are enabled in CE-ATA device, nIEN = 0
read_ceata_device	0 or 1	Set to 1 if RW_BLK or RW_REG read
update_clk_regs_only	0	No clock parameters update command
card_no	nCardNo	Actual device number
send_initialization	0	No initialization sequence
stop_abort_cmd	0	—
send_auto_stop	0	—
transfer_mode	0	Block transfer mode; byte count should be integer multiple of 4KB; block size can be 512, 1K, or 4K bytes
read_write	1 or 0	1 – write 0 – read
data_expected	1	Data is expected
response_length	0	—
response_expect	1	—
cmd_index	Command index	—
wait_prv_data_complete	1	0 – Sends command immediately 1 – Sends command after previous data transfer over
check_response_CRC	1	0 – Not check response CRC 1 – Check response CRC

- ❖ Program the Block Size (**BLKSIZ**) register as shown below.

Bits	Value	Comment
31:16	0	Reserved bits set as 0's
15:0 (block_size)	512, 1024, or 4096	MMC block size can be 512, 1024, or 4096 bytes as negotiated by host

- ❖ Program the Byte Count (**BYTCNT**) register as shown below.

Bits	Value	Comment
31:0	$N \times \text{block_size}$	byte_count should be integral multiple of block size; for ATA media access commands, byte count should be multiple of 4KB. ($N \times \text{block_size} = X \times 4\text{KB}$, where N and X are integers)

7.2.12.5 Sending Command Completion Signal Disable

While waiting for the Command Completion Signal (CCS) for an outstanding RW_BLK, the host can send a Command Completion Signal Disable (CCSD).

- ❖ Send CCSD - DWC_mobile_storage sends CCSD to the CE-ATA device if the send_ccsd bit is set in the CTRL register; this bit should be set only after a response is received for the RW_BLK.
- ❖ Send internal Stop command - Send internally generated STOP (CMD12) command after sending the CCSD pattern. If send_auto_stop_ccsd bit is also set when the controller is programmed to send the CCSD pattern, the DWC_mobile_storage sends the internally generated STOP command on the CMD line. After sending the STOP command, the DWC_mobile_storage sets the Auto Command Done bit in the RINTSTS register.

7.2.12.6 Recovery after Command Completion Signal Timeout

If timeout happened while waiting for Command Completion Signal (CCS), the host needs to send Command Completion Signal Disable (CCSD) followed by a STOP command to abort the pending ATA

command. The host can program the DWC_mobile_storage to send internally generated STOP command after sending the CCSD pattern.

- ❖ Send CCSD – Set the send_ccsd bit in the CTRL register.
- ❖ Send external STOP command – Terminate the data transfer between the CEATA device and the DWC_mobile_storage. For more information on sending the stop command, refer to [“Sending Stop or Abort in Middle of Transfer”](#) on page 189.
- ❖ Send internal STOP command – Set send_auto_stop_ccsd bit in the CTRL register, which programs the host controller to send the internally generated STOP command. After sending the STOP command, the DWC_mobile_storage sets the Auto Command Done bit in the RINTSTS register. The send_auto_stop_ccsd bit should be set along with setting the send_ccsd bit.

7.2.12.7 Recovery after the N_{ACIO} timeout

- ❖ If CCS is expected from the CE-ATA device (that is, the ccs_expected bit is set in the CMD register), then follow the same steps for [“Recovery after Command Completion Signal Timeout”](#), above.
- ❖ If CCS is not expected from the CEATA device follow these steps:
 - a. Send external STOP command
 - b. Terminate the data transfer between the DWC_mobile_storage and CE-ATA device.

For more information on sending the STOP command, refer to [“Sending Stop or Abort in Middle of Transfer”](#) on page 189.

7.2.12.8 Reduced ATA Command Set

It is necessary for the CE-ATA device to support the reduced ATA command subset. The following details discuss this reduced command set.

- ❖ IDENTIFY DEVICE – Returns 512-byte data structure to the host that describes device-specific information and capabilities. The host issues the IDENTIFY DEVICE command only if the MMC block size is set to 512 bytes; any other MMC block size has indeterminate results.

The host issues RW_REG for the ATA command, and the data is retrieved through RW_BLK.

The host controller uses the following settings while sending RW_REG for the IDENTIFY DEVICE ATA command. The following lists the primary bit settings:

- ◆ CMD register setting – data_expected field set to 0
- ◆ CMDARG register settings:
 - ◇ Bit [31] set to 0
 - ◇ Bits [7:2] set to 128.
- ◆ Task file settings:
 - ◇ Command field of the ATA task file set to ECh
 - ◇ Reserved fields of the task file cleared to 0
- ◆ BLKSIZ register bits [15:0] and BYTCNT register – Set to 16

The host controller uses the following settings for data retrieval (RW_BLK):

- ◆ CMD register settings:
 - ◇ ccs_expected set to 1
 - ◇ data_expected set to 1

- ◆ CMDARG register settings:
 - ◇ Bit [31] set to 0 (Read operation)
 - ◇ Data Count set to 1 (16'h0001)
- ◆ BLKSIZ register bits [15:0] and BYTCNT register – Set to 512
- ❖ READ DMA EXT – Reads a number of logical blocks of data from the device using the Data-In data transfer protocol. The host uses RW_REG to issue the ATA command and RW_BLK for the data transfer.
- ❖ WRITE DMA EXT – Writes a number of logical blocks of data to the device using the Data-Out data transfer protocol. The host uses RW_REG to issue the ATA command and RW_BLK for the data transfer.
- ❖ STANDBY IMMEDIATE – No data transfer (RW_BLK) is expected for this ATA command, which causes the device to immediately enter the most aggressive power management mode that still retains internal device context.

For devices that do not provide a power savings mode, the STANDBY IMMEDIATE command returns a successful status indication. The host issues RW_REG for the ATA command, and the status is retrieved through CMD39/RW_REG. Only the status field of the ATA task file contains the success status; there is no error status.

The host controller uses the following settings while sending the RW_REG for STANDBY IMMEDIATE ATA command:

- ◆ CMD Register setting – data_expected field set to 0
- ◆ CMDARG register settings:
 - ◇ Bit [31] set to 1
 - ◇ Bits [7:2] set to 4
- ◆ Task file settings:
 - ◇ Command field of the ATA task file set to E0h
 - ◇ Reserved fields of the task file cleared to 0
- ◆ BLKSIZ register bits [15:0] and BYTCNT register – Set to 16
- ❖ FLUSH CACHE EXT – No data transfer (RW_BLK) is expected for this ATA command. For devices that buffer/cache written data, the FLUSH CACHE EXT command ensures that buffered data is written to the device media. For devices that do not buffer written data, FLUSH CACHE EXT returns a success status. The host issues RW_REG for the ATA command, and the status is retrieved through CMD39/RW_REG; there can be error status for this ATA command, in which case fields other than the status field of the ATA task file are valid.

The host controller uses the following settings while sending the RW_REG for STANDBY IMMEDIATE ATA command:

- ◆ CMD register setting – data_expected field set to 0
- ◆ CMDARG register settings:
 - ◇ Bit [31] set to 1
 - ◇ Bits [7:2] set to 4

- ◆ Task file settings:
 - ◇ Command field of the ATA task file set to EAh
 - ◇ Reserved fields of the task file cleared to 0
- ◆ BLKSIZ register bits [15:0] and BYTCNT register – Set to 16

7.2.13 Controller/DMA/FIFO Reset Usage

Communication with the card involves the following:

- ❖ Controller – Controls all functions of the DWC_mobile_storage.
- ❖ FIFO – Holds data to be sent or received.
- ❖ DMA – If DMA transfer mode is enabled, then transfers data between system memory and the FIFO.
- ❖ Controller reset – Resets the controller by setting the controller_reset bit (bit 0) in the CTRL register; this resets the CIU and state machines, and also resets the BIU-to-CIU interface. Since this reset bit is self-clearing, after issuing the reset, wait until this bit is cleared.
- ❖ FIFO reset – Resets the FIFO by setting the fifo_reset bit (bit 1) in the CTRL register; this resets the FIFO pointers and counters of the FIFO. Since this reset bit is self-clearing, after issuing the reset, wait until this bit is cleared.
- ❖ DMA reset – Resets the internal DMA controller logic by setting the dma_reset bit (bit 2) in the CTRL register, which abruptly terminates any DMA transfer in process. Since this reset bit is self-clearing, after issuing the reset, wait until this bit is cleared.

The following are recommended methods for issuing reset commands:

- ❖ Non-DMA transfer mode – Simultaneously sets controller_reset and fifo_reset; clears the RAWINTS register @0x44 using another write in order to clear any resultant interrupt.
- ❖ Generic DMA mode – Simultaneously sets controller_reset, fifo_reset, and dma_reset; clears the RAWINTS register @0x44 by using another write in order to clear any resultant interrupt. If a “graceful” completion of the DMA is required, then it is recommended to poll the status register to see whether the dma request is 0 before resetting the DMA interface control and issuing an additional FIFO reset.
- ❖ DW-DMA/Non-DW-DMA mode – Sets controller_reset and fifo_reset; waits until dw_dma_req goes inactive (the Status register indicates the value of this signal). Resets the FIFO again. Clears the interrupts by clearing the RAWINTS register @0x44 using another write in order to clear any resultant interrupt. You also need to reset and reprogram the channel(s) of the DW_ahb_dmac controller that are interfaced to the DWC_mobile_storage.

In external DMA transfer mode, even when the FIFO pointers are reset, if there is a DMA transfer in progress, it could push or pop data to or from the FIFO; the DMA itself completes correctly. In order to clear the FIFO, the software should issue an additional FIFO reset and clear any FIFO underrun or overrun errors in the RAWINTS register caused by the DMA transfers after the FIFO was reset.

7.2.14 Card Read Threshold

When an application needs to perform a Single or Multiple Block Read command, the application must program the CardThrCtl register with the appropriate Card Read Threshold size (CardRdThreshold) and set the Card Read Threshold Enable (CardRdThrEnable) bit to 1'b1. This additional programming ensures that the Host controller sends a Read Command only if there is space equal to the CardRdThreshold available in the Rx FIFO. This in turn ensures that the card clock is not stopped in the middle a block of data

being transmitted from the card. The Card Read Threshold can be set to the block size of the transfer, which guarantees that there is a minimum of one block size of space in the RxFIFO before the controller enables the card clock.

The Card Read Threshold is required when the Round Trip Delay is greater than 0.5cclk_in period as listed in [Table 7-6](#).

Table 7-6 Card Read Threshold for Round Trip Delay

Number	Model	Round Trip Delay (Delay_R = Delay_O + tODLY + Delay_I)	Is Stopping of Card Clock Allowed?	Card Read Threshold Required?
1	SDR104	Delay_R > 0.5cclk_in period	No	Yes
		Delay_R < 0.5cclk_in period	No	Yes
2	SDR50	Delay_R > 0.5cclk_in period	No	Yes
		Delay_R < 0.5cclk_in period	Yes	No
3	DDR50	Delay_R > 0.5cclk_in period	No	Yes
		Delay_R < 0.5cclk_in period	Yes	No
		0.5cclk_out period > Delay_R < 1cclk_out period	No	Yes
		1cclk_out period < Delay_R < 1.5cclk_out period	Yes	No
		Delay_R < 0.5cclk_out period	Yes	No
4	SDR25	Delay_R > 0.5cclk_in period	No	Yes
		Delay_R < 0.5cclk_in period	Yes	No
5	SDR12	Delay_R > 0.5cclk_in period	No	Yes
		Delay_R < 0.5cclk_in period	Yes	No

7.2.14.1 Recommended Usage Guidelines for Card Read Threshold

1. The CardThrCtl register must be programmed before the programming the CMD register for a Data Read command.
2. The CardThrCtl register should not be programmed when a data transfer command is in progress.
3. A CardRdThreshold greater than or equal to the block size of the read transfer ensures that the card clock does not stop in between a block of data.
4. If the delay from the output of the card to the first sampling flip-flop in the Host controller is greater than 0.5 UI for SDR and DDR modes, then the Card Read Threshold must be enabled and the Card Threshold must be programmed as per guideline #3 to guarantee that the Card Clock does not stop in between a block of data.

UI (Unit Interval) - 1 UI is equal to 1 card clock period

If (Delay_O + tODLY + Delay_S) > 0.5 UI, then the Host Controller samples data incorrectly if the card clock stops in between a block of data.

5. CardRdThreshold is greater than or equal to BlockSize (Recommended)

The Card Read Threshold Size (CardRdThershold) must be programmed to at the least 1*BlockSize of the multi-block transfer in order to guarantee that the Card Clock does not stop in between a block of data due to the RxFIFO becoming full during the read transfer.

6. CardRdThreshold is less than BlockSize

If the CardReadThreshold is programmed to less than the BlockSize of the transfer, then the Host Controller System must ensure that the receive FIFO never becomes full and overflows during the read transfer; this can cause the card clock from the DWC_mobile_storage to stop. The DWC_mobile_storage is not be able to guarantee that the Card Clock does not stop during a read transfer.



Note

If the CardRdThreshold, RX_WMark, and MSIZE are programmed incorrectly, then the Card Clock may stop indefinitely and no interrupts will be generated from the Host Controller.

7.2.14.2 Card Read Threshold Programming Sequence

Most cards such as SDHC or SDXC typically support block sizes that are specified in the card or are fixed to 512 bytes. For SDIO cards – standard capacity SD cards that support READ_BL_PARTIAL=1 and MMC cards – the block size is variable and can be chosen by the application.

In order to use the Card Read Threshold feature effectively and to guarantee that the Card Clock does not stop because of a FIFO Full condition in the middle of a block of data being read from the Card, follow these steps:

1. Choose Block Size

The block size must be based on the following:

◆ Rule 1 - DWORD-aligned Block Size

The block size requested by the application from the card for the read transfer card must be DWORD-aligned.

2. Enable Card Read Threshold feature

◆ Rule 2 - Block Size ≤ Total Fifo Depth

CardRdThreshold can be enabled only if the block size for the given transfer is less than the total depth of the FIFO.

$$\diamond BlkSize \leq FifoDepth$$

Where:

$BlkSize = (\text{block size in bytes}) * 8 / F_DATA_WIDTH$; that is, the number of the block size in terms of FIFO locations

$FifoDepth = \text{total number of FIFO Locations}$



Note

In order to use the Card Read Threshold for different block sizes when selecting the FIFO depth during configuration of the Host Controller, the selected FIFO depth must be greater than or equal to the maximum block size that the Host Controller is required to support. Typically the largest block size to support is 512 bytes; the corresponding FIFO depth would 128 locations in the 32-bit-wide FIFO.

If you choose a FIFO depth that is two or more times the maximum block size that the Host Controller is required to support, there will be greater performance and flexibility in choosing the MSIZE and RX_WMARK for the best throughput.

3. Choose CardRdThreshold

- ◆ If $BlkSize \geq \frac{1}{2} FifoDepth$, choose CardRdThreshold such that $CardRdThreshold \leq BlkSize$ in bytes
- ◆ If $BlkSize < \frac{1}{2} FifoDepth$, choose CardRdThreshold such that $CardRdThreshold = BlkSize$ in bytes

**Attention**

- If the Host Controller is operating in Internal DMA Mode, or if the application does not use burst transfers to read data out of the FIFO while operating in External DMA mode or Slave mode, then use steps 4a and 5a below, and skip steps 6b, 7b, and 8.
- If the application uses burst transfers to read data out of the FIFO while operating in External DMA mode or in Slave mode, then skip steps 4a and 5a, and instead follow steps 6b, 7b, and 8 below.

4a. Choose DW_DMA_Mutiple_Transaction_Size

The possible values for the DW_DMA_Mutiple_Transaction_Size (*MSIZE*) are 1, 4, 8, 16, 32, 64, 128, and 256 transfers. Choose the value of *MSIZE* from the above transfer values so that *MSIZE* is a multiple of *BlkSize*.

$$BlkSize \% (MSIZE) = 0$$

◆ **Special Cases:**

- ◇ When *MSIZE* = 1 transfer

The *MSIZE* is equal to 1 if the block size chosen in Step 1 is not a multiple of the FIFO Width (in bytes).

If *MSIZE* = 1 is not acceptable and a higher burst size is desired — that is, a higher *MSIZE* — then go back to Step 1 and recalculate the block size.

- ◇ Internal DMA (IDMAC) mode

The size of the data buffer (*BuffSize* in bytes) for each descriptor must be a multiple of $MSIZE * H_DATA_WIDTH / 8$. For example, $BuffSize = n * MSIZE * H_DATA_WIDTH / 8$, where $n = 1, 2, 3 \dots$

5a. Choose RX Watermark

- ◆ If *MSIZE* = 1, then the *RX_WMARK* = 1 or *RX_WMARK* = *BlkSize* - 1

**Note**

For slave mode when the *RX_WMARK* is reached, the application must read only *RX_WMARK* number of locations from the FIFO.

Additionally, for all DMA modes the RX Watermark (*RX_Wmark*) chosen must be a multiple of the chosen *MSIZE*.

$$RX_WMark = MSIZE * n, \text{ where } n = 1, 2, 3 \dots$$

**Attention**

- If the Host Controller is operating in Internal DMA Mode, or if the application does not use burst transfers to read data out of the FIFO while operating in External DMA mode or Slave mode, then use steps 4a and 5a above, and skip steps 6b, 7b, and 8.
- If the application uses burst transfers to read data out of the FIFO while operating in External DMA mode or in Slave mode, then skip steps 4a and 5a above, and instead follow steps 6b, 7b, and 8 below.

6b. Choose Burst Size

- In order to determine burst transfers to drain data from the FIFO in external DMA or slave mode, use the following:

$$BurstSize = number_of_beats * transfer_size \text{ in bytes per beat}$$

- b. Choose the number of beats and the transfer size per beat so that *BurstSize* is a sub-multiple of *BlkSize*:

$$BlkSize \% (BurstSize) = 0$$

Where:

$$BlkSize = \text{Block Size in bytes} * 8 / F_DATA_WIDTH$$

If *H_DATA_WIDTH* = 16 then:

$$(BlkSize * 2) \% (BurstSize) = 0$$

7b. Choose RX Watermark

- ◆ Use the following to determine RX Watermark (RX_Wmark):

$$RX_WMark = (BurstSize / F_DATA_WIDTH * 8) - 1$$



Note

For slave mode when the *RX_WMARK* is reached, the application must read only *RX_WMARK* number of locations from the FIFO.

8. Program MSIZE

The possible values for the *DW_DMA_Mutiple_Transaction_Size* (MSIZE) are 1, 4, 8, 16, 32, 64, 128 or 256 transfers (refer *FIFOH*[30:28]).

- a. Choose the following to determine the value of MSIZE:

$$MSIZE > (BurstSize / H_DATA_WIDTH * 8)$$

Example 7-1 Card Read Threshold Programming when *BlkSize* > ½ *FifoDepth*

Given:

$$H_DATA_WIDTH = 32$$

$$F_DATA_WIDTH = 32$$

Total FIFO Depth:

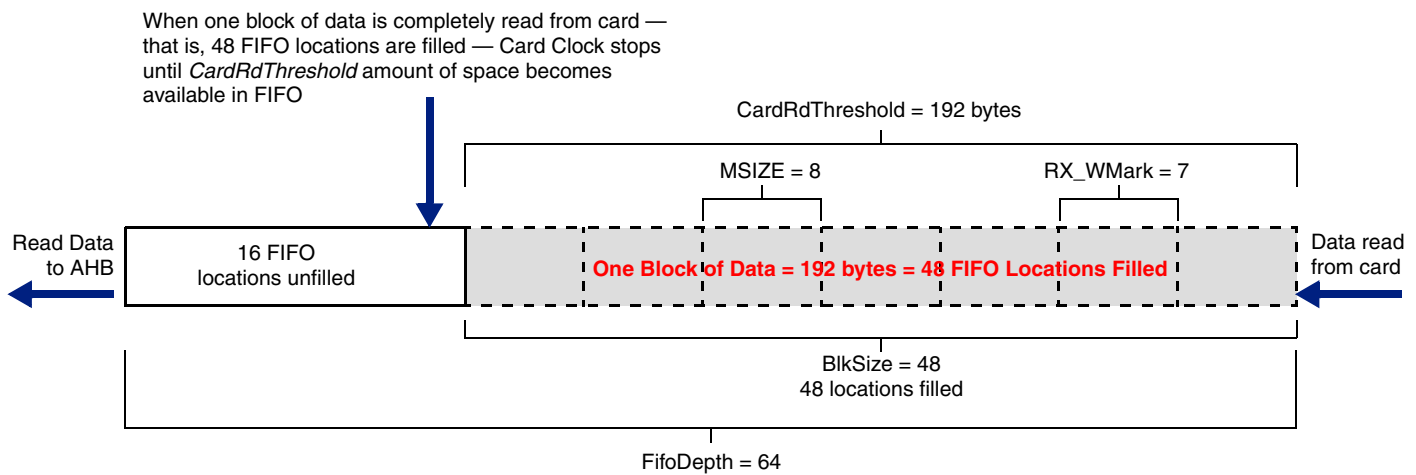
$$FifoDepth = 64 \text{ locations} = 64 * F_DATA_WIDTH / 8 = 256 \text{ bytes}$$

The following shows how to program the *CardRdThreshold* when *BlkSize* is greater than ½ *FifoDepth*.

1. Choose a DWORD-aligned *BlkSize* less than *FifoDepth*.
If Block Size is 192 bytes, then $BlkSize = 192 * 8 / F_DATA_WIDTH = 48$ FIFO locations
2. For DMA modes, choose MSIZE such that the *BlkSize* is a multiple of MSIZE.
Possible MSIZE values 1, 4, 8 and 16, such that $(48 \% MSIZE) = 0$.
Choose MSIZE = 8.
3. Choose the $RX_WMark = MSIZE - 1$.
For example, $RX_WMark \leq 8 - 1 = 7$ FIFO locations.
4. Since $BlkSize > \frac{1}{2} FifoDepth$, choose *CardRdThreshold* = Block Size.
CardRdThreshold = 192 bytes

Figure 7-3 shows the contents of the FIFO when BlkSize is greater than $\frac{1}{2} \text{FifoDepth}$.

Figure 7-3 FIFO Contents when $\text{BlkSize} > \frac{1}{2} \text{FifoDepth}$



Example 7-2 Card Read Threshold Programming when $\text{BlkSize} < \frac{1}{2} \text{FifoDepth}$

Given:

$H_DATA_WIDTH = 32$

$F_DATA_WIDTH = 32$

Total FIFO Depth:

$\text{FifoDepth} = 64 \text{ locations} = 64 * F_DATA_WIDTH / 8 = 256 \text{ bytes}$

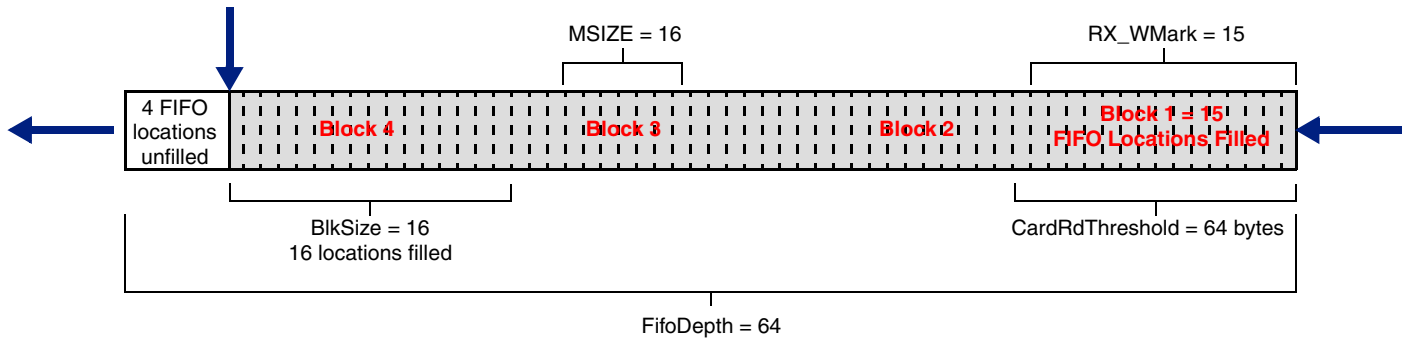
The following shows how to program the *CardRdThreshold* when *BlkSize* is less than $\frac{1}{2} \text{FifoDepth}$.

1. Choose a DWORD-aligned *BlkSize* less than *FifoDepth*.
If Block Size is 64 bytes, then $\text{BlkSize} = 64 * 8 / F_DATA_WIDTH = 64 * 8 / 32 = 16$ FIFO locations
2. For DMA modes, choose *MSIZE* such that the *BlkSize* is a multiple of *MSIZE*.
Possible *MSIZE* values are 1, 4, 8, and 16 such that $(16 \% \text{MSIZE}) = 0$.
Choose *MSIZE* = 16.
3. Choose the $\text{RX_WMark} = \text{MSIZE} - 1$.
For example, $\text{RX_WMark} = 16 - 1 = 15$ FIFO locations.
4. Since $\text{BlkSize} < \frac{1}{2} \text{FifoDepth}$, choose *CardRdThreshold* = Block Size.
CardRdThreshold = 64 bytes.

Figure 7-4 shows the contents of the FIFO when $\text{BlkSize} < \frac{1}{2} \text{FifoDepth}$.

Figure 7-4 FIFO Contents when $\text{BlkSize} < \frac{1}{2} \text{FifoDepth}$

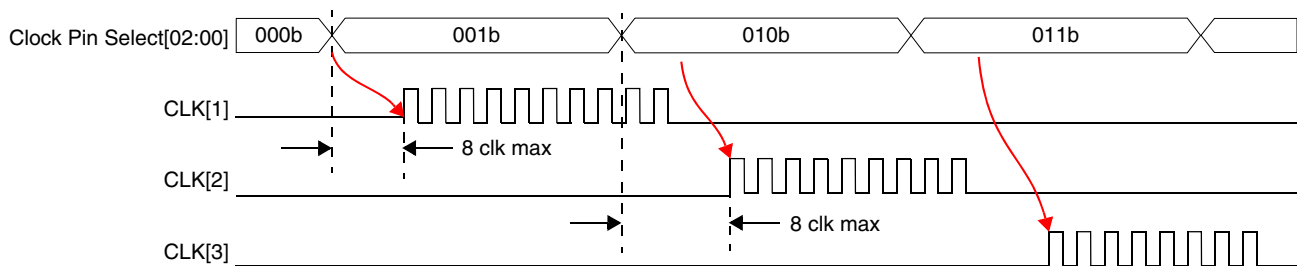
When four blocks of data are completely read from card—that is, 16 FIFO locations are filled—Card Clock stops until CardRdThreshold equals 64 bytes of space—that is, 16 FIFO locations—become available in FIFO



7.2.15 Shared bus mode

When the core is used in the shared bus mode of operation, the driver should ensure that the clock is provided to only the device that is intended to be addressed; that is, only one clock oscillates at a time without any overlap with other clocks. You can do this by switching off the clock to all other devices by programming 0 in the clk_enable register of the corresponding card; Figure 7-5 illustrates clock switching for the shared bus mode. This can be done for only cards that are connected in shared bus mode; standalone devices can have their card clock on at all times. All cards in shared bus mode can be given different divided card clock values.

Figure 7-5 Clock Switching for Shared Bus Mode



7.2.16 Dedicated Interrupts

No special programming is required for dedicated interrupt ports. The only thing required to be done by the driver is if only one interrupt port is implemented by ORing the $\text{INT\#}$ lines. In that case, the application must read all the functions connected to the interrupt in order to detect which one generated the interrupt.

More than one but less than sixteen interrupt ports can be implemented by ORing the $\text{INT\#}$ lines. In this case, the application must read all the functions connected to the interrupt that flagged an interrupt in order to detect which one generated the interrupt.

7.2.17 Asynchronous Interrupts

No special programming is required for Asynchronous interrupt mechanism. As with dedicated interrupt ports, the one requirement by the driver is if only one interrupt port is implemented by ORing the INT# lines. In that case, the application must read all the functions connected to the interrupt in order to detect which one generated the interrupt. More than one but less than sixteen interrupt ports can be implemented by ORing the INT# lines. In this case, the application must read all the functions connected to the interrupt that flagged an interrupt in order to detect which one generated the interrupt.

During this mode, the card can send interrupts using the DAT[1] line or the dedicated interrupt ports (for eSDIO only).

7.2.18 Back-End Power

The host controller needs to set the bit to enable the power sent to the back end function of the card.

7.2.19 Master Power Control

Whether a card requires greater than 720mW of power or not can be determined by reading the SMPC (Support Master Power Control) and the EMPC (Enable Master Power Control) registers in the card.

[Figure 7-6](#) illustrates the CCCR register.

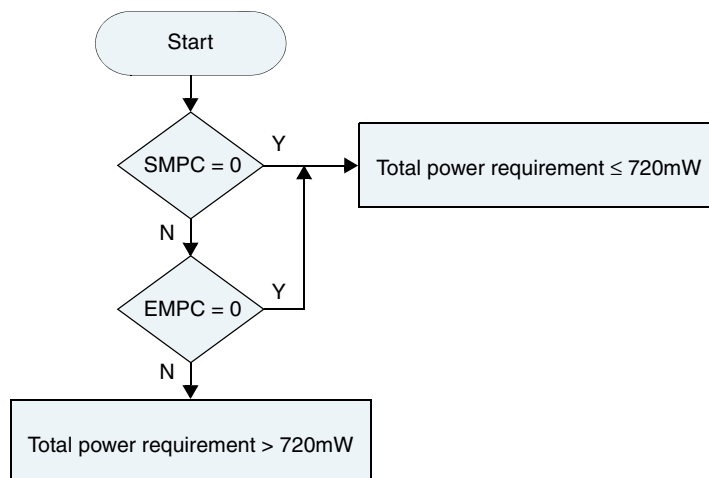
Figure 7-6 Card Common Control Registers (CCCR)

12h	Power Control	RFU	RFU	RFU	RFU	RFU	EMPC	SMPC
-----	---------------	-----	-----	-----	-----	-----	------	------

Addr: Field	Type	Description
12h: EMPC	R/W	Enable Master Power Control - EMPC = 0 (default): Total card power up to 720mW (3.6V x 200mA). Card automatically switches model of function(s) to lower current or does not allow some functions to become enabled, regardless of value of EPS, so that total card power is 720mW or less. (Card manufacturer determines which functions operate and their modes to guarantee this limit.) - EMPC = 1: Total card power can exceed 720mW and SPS and EPS are available. Host uses SPS, EPS in FBR and IOEx to enable higher current function modes based on host's ability to supply necessary current. - If PS[3:0] = 0 (Ver 1.10-compatible), two power modes can be selected by using SPS, EPS in FBR for SDIO Card supporting 2.7V-3.6V power supply range. - If PS[3:0] > 0 is defined by Ver 3.0 Power Control Extension (applicable to multiple voltages device).
12h: SMPC	R/W	Support Master Power Control These bits tell host if card supports Master Power Control. - SMPC = 0: Total card power up to 720mW (3.6V x 200mA), even if all functions are active (IOEx=1). EMPC, SPS, and EPS are zero. - SMPC = 1: Total card power can exceed 720mW (3.6V x 200mA). Controls of EMPC, SPS, and EPS are available.

The chart in [Figure 7-7](#) illustrates a basic flow that the device driver follows in order to detect the power requirements of the card.

Figure 7-7 FlowChart for Power Requirements of Card

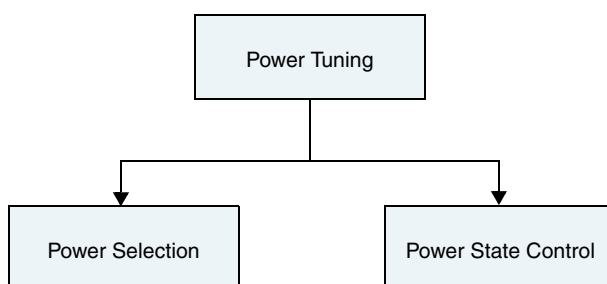


1. The driver reads the SMPC register.
2. If the value is 0, then the total power requirement is less than or equal to 720mW.
3. If the value is 1, then the driver reads the EMPC register.
 - a. If the value is 0 then the total power requirement is less than or equal to 720mW.
 - b. If the value is 1, then the total power requirement is greater than 720mW.

Once the power requirement has been determined to be more than 720mW, the driver may tune the power to match the card requirements by tuning each function in the card.

[Figure 7-8](#) illustrates the two mechanisms for power tuning to get the correct power levels of each function.

Figure 7-8 Different Power Tuning Mechanisms

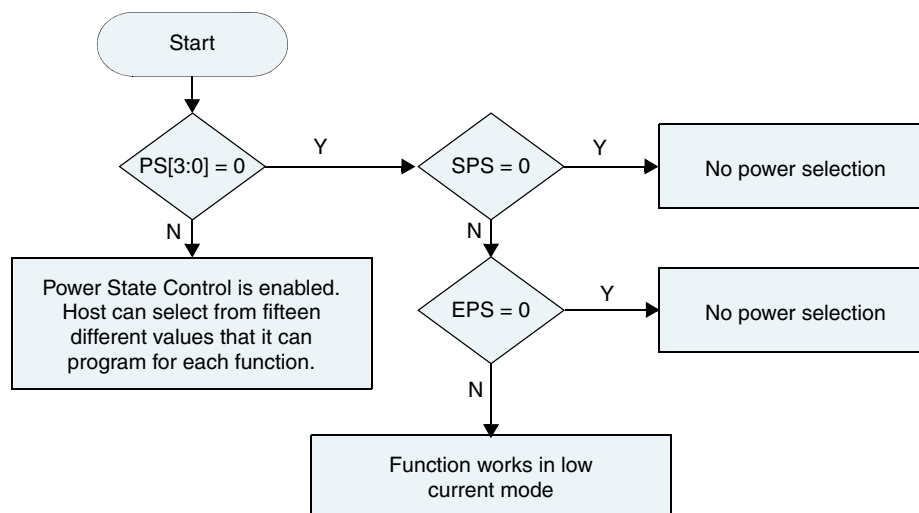


In addition to the CCCR, each supported I/O function has an FBR register.

Table 7-7 FBR Registers for Power Tuning

Addr: Field	Type	Description
SPS	R	Support Power Selection Indicates if function has Power Selection. - SPS = 0: Has no Power Selection; EPS is zero. - SPS = 1: Has two power modes selected by EPS. This bit is effective when EMPC = 1 in CCCR and PS[3:0] = 0.
EPS	R/W	Enable Power Selection - EPS = 0 (default): Operates in Higher Current Mode. Maximum current is given in TPLFE_HP_MAX_PWR_3.3V. - EPS = 1: Operates in Lower Current Mode. Maximum current is given in TPLFE_LP_MAX_PWR_3.3V. This bit is reset when IOEx = 0 and is effective when EMPC = 1 in CCR and PS[3:0] = 0.
PSx	R/W	Power State PS[3:0] If PS[3:0] is set to 0, TPL_CODE_CISTPL_FUNCCE (22h) extension 01h is used and card power is controlled by EMPC and EPS (SDIO Ver 2.0-compatible). Power State control is defined by SDIO Ver 3.0 and is effective when EMPC is set to 1 and PS[3:0] is set to greater than 0. In this case, a list of card-supported power states is determined by TPL_CODE_CISTPL_FUNCCE (22h) extension 02h (Power State Tuple). Host driver finds affordable power in Tuple and $n (>0)$ is set to this field. Total current of card is sum of selected current of each function.

The chart in [Figure 7-9](#) defines a basic flow that the device driver follows in order to detect and program the power requirements of each function in the card.

Figure 7-9 FlowChart for Detect and Program Power Requirements of Functions

The driver reads the PS[3:0] register.

1. If the value of PS[3:0] is 1, the driver can program a value in the PS[3:0] register based on its affordable power in the tuples and program a number greater than 0.
2. If the value of PS [3:0] is 0, the driver reads the SPS register.
 - ◆ If the value of SPS is 0, the function does not support power selection.
 - ◆ If the value of SPS is 1, the driver reads the EPS register.
 - ◇ If the value of EPS is 0, the function is set to High Current Mode.
 - ◇ If the value of EPS is 1, the function is set to Low Current Mode.



Note Testing this feature may not be of much value to the verification team, but will help in building the driver.

7.2.20 Error Handling

The DWC_mobile_storage implements error checking; errors are reflected in the RAWINTS register @0x44 and can be communicated to the software through an interrupt, or the software can poll for these bits. Upon power-on, interrupts are disabled (int_enable in the CTRL register is 0), and all the interrupts are masked (bits 0-31 of the INTMASK register; default is 0).

Error handling:

- ❖ Response and data timeout errors – For response timeout, software can retry the command. For data timeout, the DWC_mobile_storage has not received the data start bit – either for the first block or the intermediate block – within the timeout period, so software can either retry the whole data transfer again or retry from a specified block onwards. By reading the contents of the TCBCNT later, the software can decide how many bytes remain to be copied.
- ❖ Response errors – Set when an error is received during response reception. In this case, the response that copied in the response registers is invalid. Software can retry the command.
- ❖ Data errors – Set when error in data reception are observed; for example, data CRC, start bit not found, end bit not found, and so on. These errors could be set for any block-first block, intermediate block, or last block. On receipt of an error, the software can issue a STOP or ABORT command and retry the command for either whole data or partial data.
- ❖ Hardware locked error – Set when the DWC_mobile_storage cannot load a command issued by software. When software sets the start_cmd bit in the CMD register, the DWC_mobile_storage tries to load the command. If the command buffer is already filled with a command, this error is raised. The software then has to reload the command.
- ❖ FIFO underrun/overflow error – If the FIFO is full and software tries to write data in the FIFO, then an overflow error is set. Conversely, if the FIFO is empty and the software tries to read data from the FIFO, an underrun error is set. Before reading or writing data in the FIFO, the software should read the fifo_empty or fifo_full bits in the Status register.
- ❖ Data starvation by host timeout – Raised when the DWC_mobile_storage is waiting for software intervention to transfer the data to or from the FIFO, but the software does not transfer within the stipulated timeout period. Under this condition and when a read transfer is in process, the software

should read data from the FIFO and create space for further data reception. When a transmit operation is in process, the software should fill data in the FIFO in order to start transferring data to the card.

- ❖ **CRC Error on Command** – If a CRC error is detected for a command, the CE-ATA device does not send a response, and a response timeout is expected from the DWC_mobile_storage. The ATA layer is notified that an MMC transport layer error occurred.
- ❖ **Write operation** – Any MMC Transport layer error known to the device causes an outstanding ATA command to be terminated. The ERR bits are set in the ATA status registers and the appropriate error code is sent to the ATA Error register.

If $nIEN=0$, then the Command Completion Signal (CCS) is sent to the host.

If device interrupts are not enabled ($nIEN=1$), then the device completes the entire Data Unit Count if the host controller does not abort the ongoing transfer.



Note

During a multiple-block data transfer, if a negative CRC status is received from the device, the data path signals a data CRC error to the BIU by setting the data CRC error bit in the RINTSTS register. It then continues further data transmission until all the bytes are transmitted.

- ❖ **Read operation** – If MMC transport layer errors are detected by the host controller, the host completes the ATA command with an error status.

The host controller can issue a Command Completion Signal Disable (CCSD) followed by a STOP TRANSMISSION (CMD12) to abort the read transfer. The host can also transfer the entire Data Unit Count bytes without aborting the data transfer.

7.2.21 Transmission and Reception with Internal DMAC (IDMAC)

The general sequence of events for transmit and receive is as follows:

1. Program the required programming in the Bus Mode Register. If IDMAC enable is disabled during the middle of an IDMAC transfer, it has no effect. It will only take effect for a new data transfer command. Issuing a Software reset will immediately terminate the transfer. It is recommended that the host issue a reset to DMA interface by setting `dma_reset` bit of the CTRL register and then issue a IDMAC software reset. The PBL contents are read-only and a direct reflection of the `DW_DMA_Mutiple_Transaction_Size` contents in FIFOTH register. The Fixed burst bit has to be programmed appropriately for system performance.
2. In case a Descriptor Unavailable Interrupt is asserted, the host needs to form the descriptor, appropriately set its own bit and then write to poll demand register for the IDMAC to re-fetch the descriptor.
3. It is always appropriate for the host to enable abnormal interrupts since any errors related to the transfer are reported to the host.
4. For handling scenarios like Fatal Bus Error, Abort and FIFO overrun/under-run refer to the Interrupts section of IDMAC chapter.

For more information, refer to “[Transmission](#)” on page 78 and “[Reception](#)” on page 78.

7.3 Programming DWC_mobile_storage for Boot Operation

This section discusses details on programming the DWC_mobile_storage for Boot operation.

- ❖ “Boot Operation”
- ❖ “Alternative Boot Operation” on page 216

7.3.1 Boot Operation



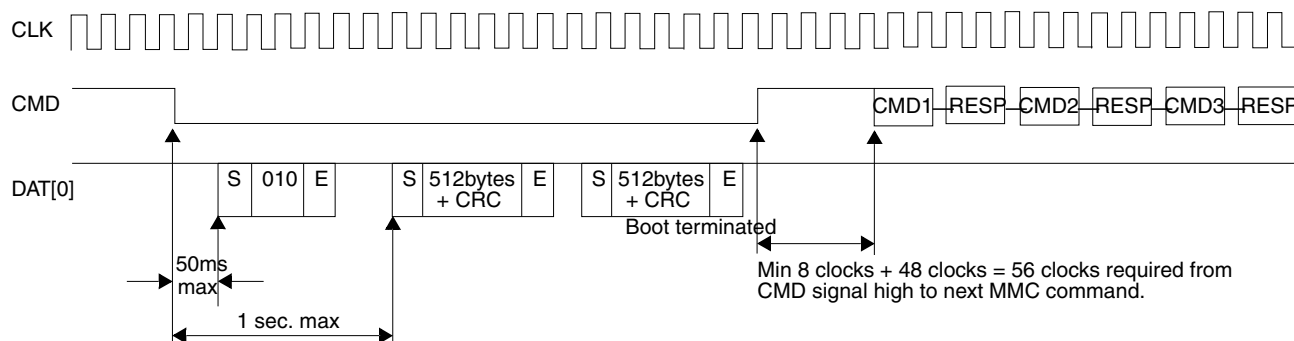
Note

Boot operation:

- Needs to be performed in SD_MMC_CEATA mode
- Is applicable to MMC4.3, MMC4.4, and MMC4.41 cards
- Needs to be performed in push-pull mode
- Does not support Boot Operation in Dual Data Rate Mode (DDR)

Figure 7-10 illustrates timing for Boot operation.

Figure 7-10 Boot Operation



Once the power and clocks are stable, reset_n should be asserted (active-low) for at least two clocks of clk or cclk_in, whichever is slower. The reset initializes the following:

- ❖ Registers
- ❖ Ports
- ❖ FIFO-pointers
- ❖ DMA interface controls
- ❖ State-machines in the design

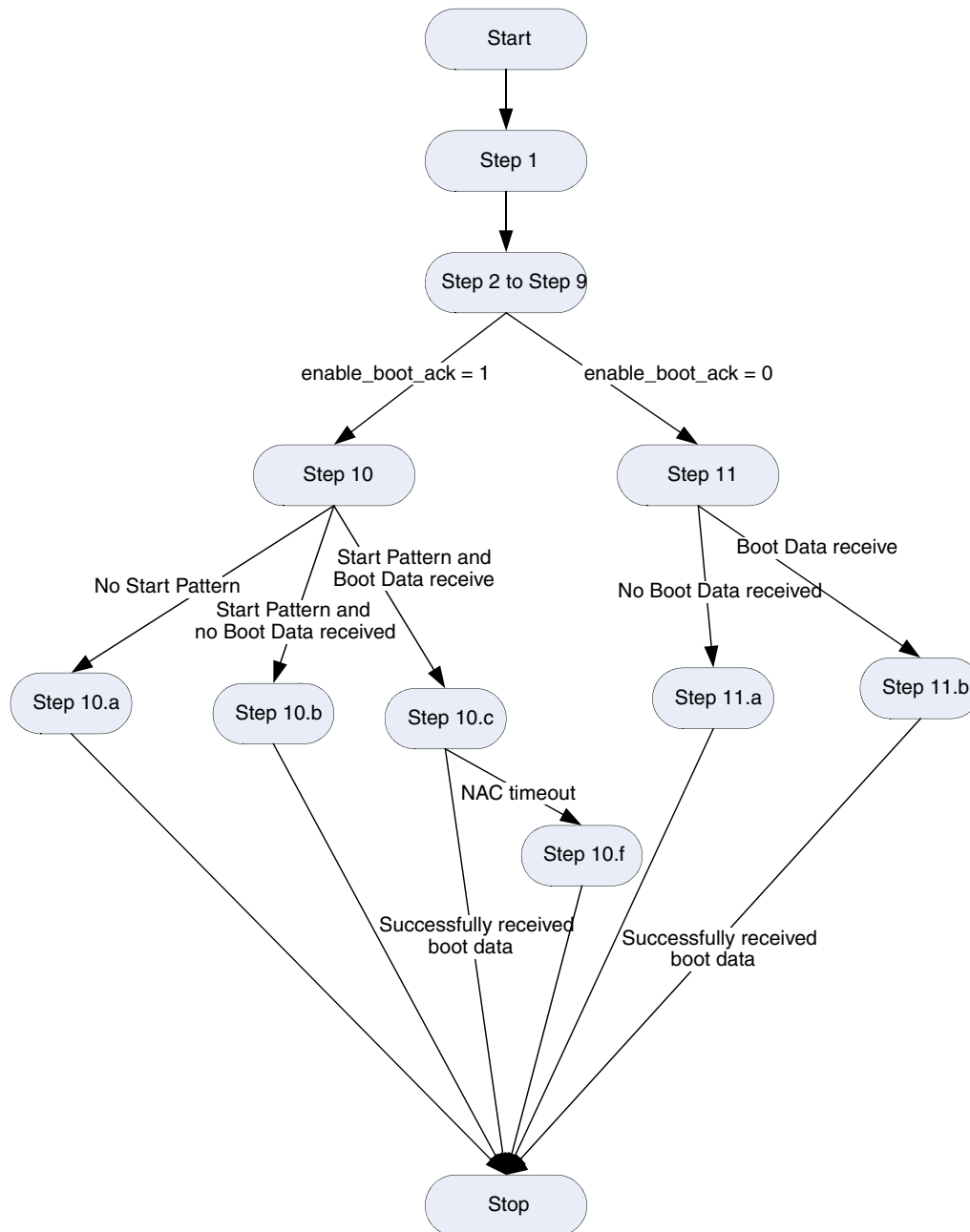
After power-on reset, the software should perform the appropriate steps described in the following sections for the respective types of cards.

Following are the steps that the software driver must follow when working with eMMC or removable MMC 4.3 cards for Boot operation.

7.3.1.1 eMMC

Figure 7-11 illustrates the flow of the steps described in this section.

Figure 7-11 Host Controller Flow for Boot Operation



1. The software driver is aware:
 - ◆ That the card supports boot operation – `BOOT_PARTITION_ENABLE` bit set in the card.
 - ◆ Of the `BOOT_SIZE_MULT` value in the card and the data bus width to use during boot operation – Extend CSD register byte[177] bit[0:1].

2. Set the following:

- ◆ Masks for interrupts by clearing appropriate bits in the Interrupt Mask register @0x024.
- ◆ Global int_enable bit of the Control register @0x00.

It is recommended that you write 0xffff_ffff to the Raw Interrupt register @0x044 and IDSTS @0x8C in order to clear any pending interrupts before setting the int_enable bit.

For Internal DMAC mode, the software driver needs to unmask all the relevant fields in the IDINTEN register.

3. Configure control register (CTRL):

- ◆ enable_OD_pullup = 1'b0
- ◆ int_enable = 1'b1
- ◆ Appropriate values for card_voltage_a and card_voltage_b
- ◆ Other fields should be 1'b0.

If the software driver needs to use the Internal DMAC to transfer the boot data received, it must also:

- ◆ Set up the descriptors as described in [“Transmission and Reception with Internal DMAC \(IDMAC\)”](#) on page 210
- ◆ Program use_internal_dmac field to 1

4. Change clock source assignment – Set the card frequency to 400 KHz using the clock-divider and clock-source registers; for details, refer to [“Clock Programming”](#) on page 183.
5. Set DataTimeOut = (10 * ((TAAC * Fop) + (100 * NSAC))); this is NAC.
6. Program the BLKSIZ register with 0x200 (512 bytes).
7. Program the BYTCNT register with multiples of 128K bytes, as indicated by the BOOT_SIZE_MULT value in the card.
8. Program the Rx FIFO threshold value in bytes in the FIFOTH register @0x04C.
Typically, the threshold value can be set to half the FIFO depth; that is,
 $RX_WMark = (FIFO_DEPTH/2) - 1$.
9. Program the CMD register with the following fields:
 - a. start_cmd = 1'b1
 - b. enable_boot = 1'b1
 - c. enable_boot_ack – depends on whether a start-acknowledge pattern is expected from the card
 - d. Card_number = *appropriate_card_number*; obtained by referring to CDETECT register
 - e. Data_expected = 1'b1
 - f. Remainder of CMD register fields = 1'b0

10. If enable_boot_ack = 1'b1, the software driver should start a timer after step #9; the terminal value is 50ms.

- a. Before this timer elapses, the BAR interrupt should be received from the DWC_mobile_storage. If this does not occur, the software driver must program the CMD register with the following fields:

- i. start_cmd = 1'b1
- ii. disable_boot = 1'b1
- iii. Card_number = *appropriate_card_number*
- iv. All other fields = 0

The DWC_mobile_storage generates a Command Done (CD) interrupt after de-asserting the CMD line of the card. In IDMAC mode:

- ✧ Descriptor is closed
- ✧ CES in the IDSTS register is set, indicating BAR timeout
- ✧ RI is not set

- b. If the BAR interrupt is received, the software driver should clear this interrupt by writing a 1 to it. The software driver should then start another timer with a terminal value of $1 - 0.05 = 0.95$ seconds. Before this timer elapses, the BDS interrupt should be received from the DWC_mobile_storage. If this does not occur, the software driver must program the CMD register with the following fields:

- i. start_cmd = 1'b1
- ii. disable_boot = 1'b1
- iii. Card_number = *appropriate_card_number*; obtained by referring to CDETECT register
- iv. All other fields = 0

The DWC_mobile_storage generates a Command Done (CD) interrupt after de-asserting the CMD line of the card. In IDMAC mode:

- ✧ Descriptor is closed
- ✧ CES in the IDSTS register is set, indicating BDS timeout
- ✧ RI is not set

- c. If the BDS interrupt is received, it indicates that the boot data is being received from the card. In non-IDMAC mode, the software driver can then initiate a data read from the DWC_mobile_storage based on the RXDR interrupt bit in the RINTSTS register.

In IDMAC mode, the IDMAC engine starts transferring the data from the FIFO to the system memory as soon as the programmed RX_WMark level is attained.

At the end of a successful boot data transfer from the card, the following interrupts are generated:

- i. Command Done (CD) in RINTSTS register
- ii. Data Transfer Over (DTO) in RINTSTS register
- iii. Receive Interrupt (RI) in IDSTS register in IDMAC mode only

- d. If an Error occurs in Boot Ack pattern (010) or an end bit Error occurs:
 - i. RTL automatically aborts boot by pulling CMD line high
 - ii. RTL generates Command done interrupt
 - iii. RTL does not generate BAR interrupt
 - iv. Application aborts boot transfer
 - e. In IDMAC mode:
 - ✧ If the software driver creates more descriptors than the boot data received, the extra descriptors are not closed by the DWC_mobile_storage.
 - ✧ If the software driver creates less descriptors than the boot data received, the DWC_mobile_storage generates a Descriptor Unavailable (DU) interrupt and does not transfer any further data to system memory.
 - f. If between data block transfers NAC is violated, DRTO (Data Read Timeout) is asserted. Apart from this, if there are errors associated with Start/End bits, SBE/EBE interrupts are also generated.
11. If enable_boot_ack = 1'b0, the software driver should start a timer after the step #9 where the terminal value is 1 second.
- a. Before this timer elapses, a BDS interrupt should be received from the DWC_mobile_storage. If this does not occur, the software driver must program the CMD register with the following fields:
 - i. start_cmd = 1'b1
 - ii. disable_boot = 1'b1
 - iii. Card_number = *appropriate_card_number*
 - iv. All other fields = 0

The DWC_mobile_storage generates a Command Done (CD) interrupt after de-asserting the CMD line of the card. In IDMAC mode, the descriptor is closed and the CES in IDSTS register is set, indicating a BDS timeout.
 - b. If a BDS interrupt is received, it indicates that the boot data is being received from the card. In non-IDMAC mode, the software driver can then initiate a data read from the DWC_mobile_storage based on the RXDR interrupt bit in the RINTSTS register. In IDMAC mode, the IDMAC engine starts transferring the data from the FIFO to the system memory as soon as the programmed RX_WMark level is hit. At the end of a successful boot data transfer from card, the following interrupts are generated.
 - i. Command Done (CD) in RINTSTS.
 - ii. Data Transfer Over (DTO) in RINTSTS.
 - iii. Receive Interrupt (RI) in IDSTS in IDMAC mode only
 - c. In IDMAC mode:
 - ✧ If the software driver created more descriptors than the boot data received, the extra descriptors are not closed by the DWC_mobile_storage.
 - ✧ If the software driver created less descriptors than the boot data received, the DWC_mobile_storage generates a Descriptor Unavailable (DU) interrupt and does not transfer any further data to system memory.

7.3.1.2 Removable MMC4.3, MMC4.4, and MMC4.41 Cards

Removable MMC4.3, MMC4.4, and MMC4.41 cards differ with respect to eMMC in that the Host Controller driver is not aware whether these cards support the Boot mode of operation when plugged in. Thus the Host controller must:

1. Enumerate these cards as it would enumerate MMC4.0/4.1/4.2 cards for the first time
2. Know the card characteristics
3. Decide whether to perform a boot operation or not

For these cards, the software driver must follow these steps:

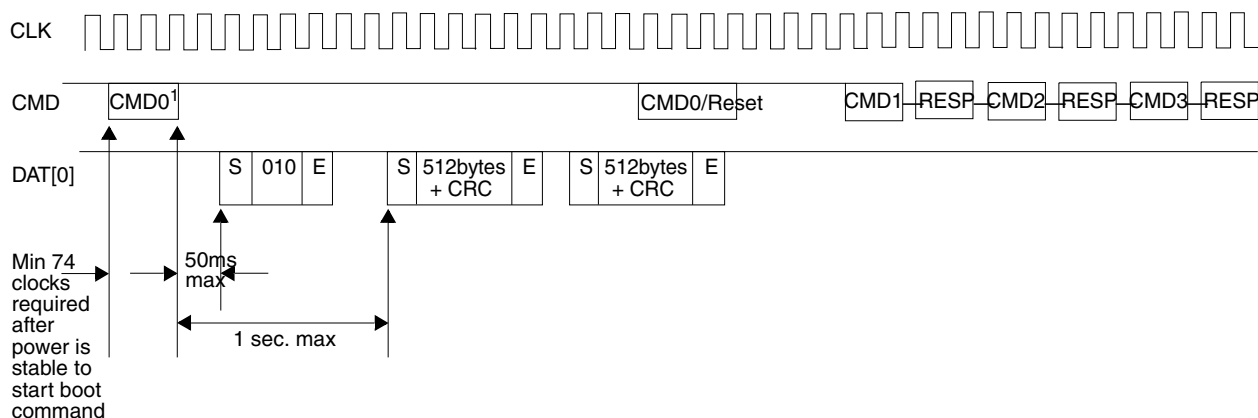
1. Enumerate the card as described in [“Programming Sequence”](#) on page 179.
2. Read the EXT_CSD register of the card and examine the following fields:
 - a. BOOT_PARTITION_ENABLE
 - b. BOOT_SIZE_MULT
 - c. BOOT_INFO
3. If necessary, the Host controller can manipulate the boot information in the card; for more information, refer to section “7.2.4 Access to Boot Partition” in the MMC4.3, MMC4.4, and MMC4.41 specifications.
4. If the Host controller needs to perform a boot operation at the next power-up cycle, it can manipulate the EXT_CSD register contents by using a SWITCH (CMD6) command.
5. After this step, the software driver must power down the card by programming the PWREN register in the DWC_mobile_storage.
6. From here on, use the same steps as in [“eMMC”](#) starting on 212.

7.3.2 Alternative Boot Operation

The Alternative Boot Operation differs from the Boot Operation in that CMD0 is used to boot the card rather than holding down the CMD-line of the card. The Alternative Boot Operation can be done only if bit 0 in the extended CSD byte[228] (BOOT_INFO) is set to 1.

[Figure 7-12](#) illustrates timing for the Alternative Boot operation.

Figure 7-12 Alternative Boot Operation



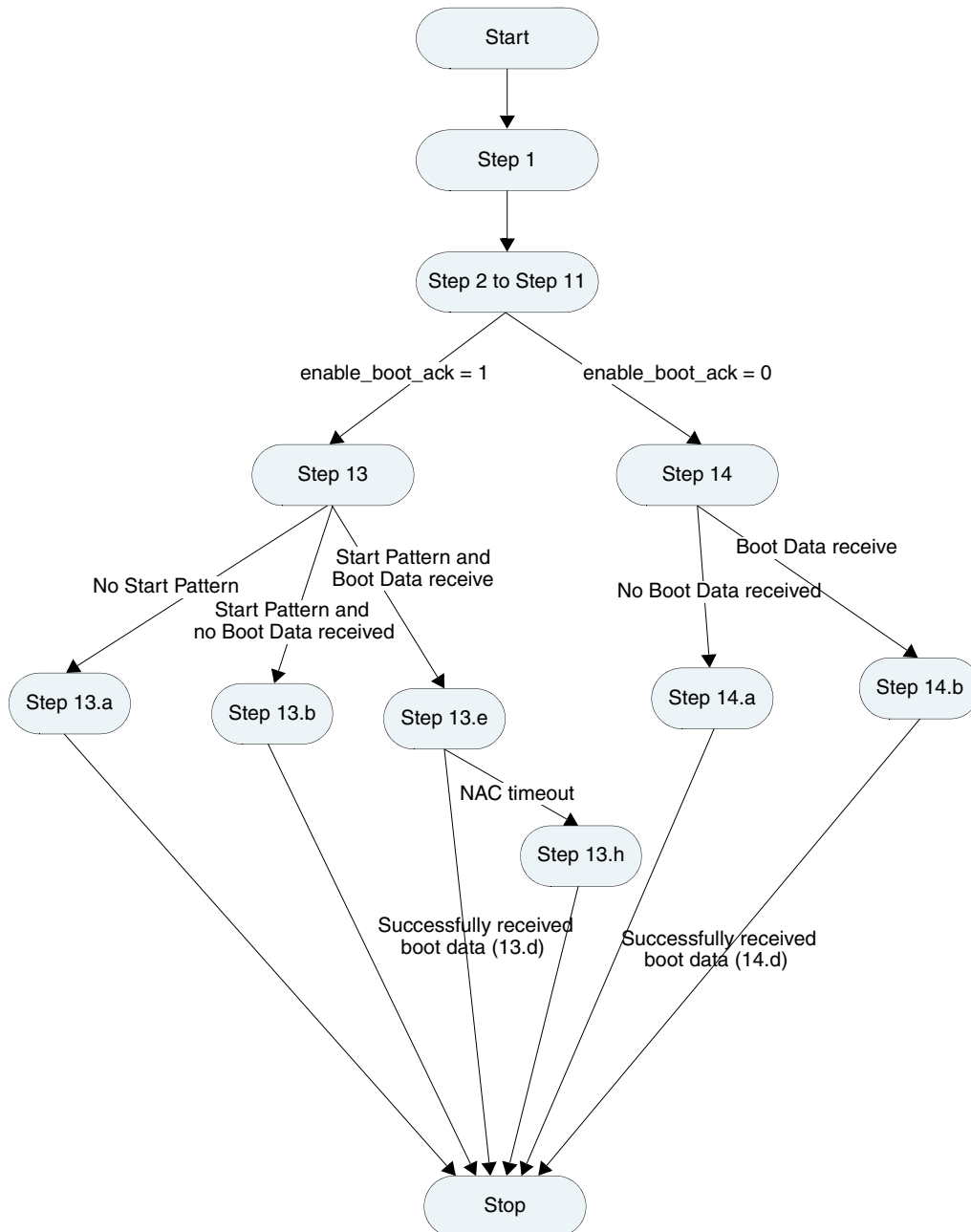
1. CMD0 with argument 0xFFFFFFFF

Following are the steps that the software driver must follow when working with eMMC or removable MMC 4.3 cards for the Alternative Boot operation.

7.3.2.1 eMMC

Figure 7-13 illustrates the flow of the steps described in this section.

Figure 7-13 Host Controller Flow for Alternative Boot Mode



1. The Software Driver is aware:
 - ◆ That the card supports the Alternative Boot operation – BOOT_INFO bit is set in the card.
 - ◆ Of the BOOT_SIZE_MULT value in the card and the data bus width to use during the boot operation – Extend CSD register byte[177] bit[0:1].

2. Set the following:

- ◆ Masks for interrupts by clearing appropriate bits in the Interrupt Mask register @0x024
- ◆ Global int_enable bit of the Control register @0x00

It is recommended that you write 0xffff_ffff to the Raw Interrupt register @0x044 and IDSTS @0x8C in order to clear any pending interrupts before setting the int_enable bit.

For Internal DMAC mode, software driver needs to unmask all the relevant fields in IDINTEN register.

3. Configure control register (CTRL):

- ◆ enable_OD_pullup = 1'b0
- ◆ int_enable = 1'b1
- ◆ Appropriate values for card_voltage_a and card_voltage_b
- ◆ Other fields should be 1'b0

If the software driver needs to use the Internal DMAC to transfer the boot data received, it must also:

- ◆ Set up the descriptors as described in [“Transmission and Reception with Internal DMAC \(IDMAC\)”](#) on page 210
- ◆ Program use_internal_dmac field to 1

4. Changing clock source assignment – Set the card frequency to 400 KHz using the clock-divider and clock-source registers; for details, refer to [“Clock Programming”](#) on page 183.

Ensure that the card clock – cclk_out – is running.

5. Wait for a time that ensures that at least 74 card clock cycles have occurred on the card interface.
6. Set DataTimeOut = (10 * ((TAAC * Fop) + (100 * NSAC))); this is NAC.
7. Program the BLKSIZ register with 0x200 – 512 bytes.
8. Program the BYTCNT register with multiples of 128K bytes, as indicated by the BOOT_SIZE_MULT value in the card.
9. Program the Rx FIFO threshold value in bytes in the FIFOTH register @0x04C.

Typically, the threshold value can be set to half the FIFO depth; that is,
 $RX_WMark = (FIFO_DEPTH/2) - 1$.

10. Program CMDARG = 0xFFFFFFFFFA.

11. Program the CMD register with the following fields.

- a. start_cmd = 1'b1
- b. boot_mode = 1'b1
- c. enable_boot_ack – depends on whether a start-acknowledge pattern is expected from the card.
- d. Card_number – *appropriate_card_number*, obtained by referring to CDETECT register
- e. Data_expected = 1'b1

- f. Cmd_index = 0
 - g. Remainder of CMD register fields = 1'b0
12. The software driver should wait for the Command Done (CD) interrupt.
13. If enable_boot_ack = 1'b1 in step 11, the software driver should start a timer after the above step with a terminal value of 50ms.
- a. Before this timer elapses, the BAR interrupt should be received from the DWC_mobile_storage. If this does not occur, the software driver needs to infer that the start-pattern has not been received and should discontinue the boot process and start with normal enumeration.
In IDMAC mode:
 - ✧ Descriptor is closed
 - ✧ CES in IDSTS register is set, indicating BAR timeout
 - ✧ RI is not set
 - b. If the BAR interrupt is received, the software driver should clear this interrupt by writing a 1 to it. The software driver should then start another timer with a terminal value of $1 - 0.05 = 0.95$ seconds. Before this timer elapses, the BDS interrupt should be received from the DWC_mobile_storage. If this does not occur, the software driver should discontinue the boot process and start with normal enumeration.
In IDMAC mode:
 - ✧ Descriptor is closed
 - ✧ CES in IDSTS register is set, indicating BDS timeout
 - ✧ RI is not set
 - c. If the BDS interrupt is received, it indicates that the boot data is being received from the card. In non-IDMAC mode, the software driver can then initiate a data read from the DWC_mobile_storage based on the RXDR interrupt bit in the RINTSTS register.
In IDMAC mode, the IDMAC engine starts transferring the data from the FIFO to the system memory as soon as the programmed RX_WMark level is hit.
 - d. It is the responsibility of the software driver to terminate the boot operation by programming the DWC_mobile_storage to send a CMD0 by programming the registers CMDARG = 0 and CMD = {start_cmd = 1, card number = *appropriate_card_number*, cmd_index = 0, *all_other_fields* = 0}.
 - e. At the end of a successful boot data transfer from the card, the following interrupts are:
 - i. Command Done (CD) in RINTSTS
 - ii. Data Transfer Over (DTO) in RINTSTS
 - iii. Receive Interrupt (RI) in IDSTS in IDMAC mode only
 - f. If an Error occurs in Boot Ack pattern (010) or an end bit Error occurs:
 - i. RTL does not generate BAR interrupt
 - ii. RTL detects Boot Data Start and generates BDS interrupt
 - iii. RTL continues to receive Boot Data
 - iv. Application must abort boot after receiving BDS interrupt

- g. In IDMAC mode:
 - ✧ If the software driver creates more descriptors than the boot data received, the extra descriptors are not closed by the DWC_mobile_storage.
 - ✧ If the software driver creates less descriptors than the boot data received, the DWC_mobile_storage generates a Descriptor Unavailable (DU) interrupt and does not transfer any further data to system memory.
 - h. If between data block transfers NAC is violated, DRTO (Data Read Timeout) is asserted. Apart from this, if there are errors associated with Start/End bits, SBE/EBE interrupts are also generated.
14. If enable_boot_ack = 1'b0 in step 11, the software driver should start a timer after step #11 with a terminal value of 1 second.
- a. Before this timer elapses, the BDS interrupt should be received from the DWC_mobile_storage. If this does not occur, the software driver should discontinue the boot process and start with normal enumeration.

In IDMAC mode:

 - ✧ descriptor is closed
 - ✧ CES in IDSTS register is set, indicating BDS timeout
 - ✧ RI is not set
 - b. If the BDS interrupt is received, it indicates that the boot data is being received from the card. In non-IDMAC mode, the software driver can then initiate a data read from the DWC_mobile_storage based on the RXDR (in RINTSTS) interrupt.

In IDMAC mode, the IDMAC engine starts transferring the data from the FIFO to the system memory as soon as the programmed RX_WMark level is hit.
 - c. It is the responsibility of the software driver to terminate the boot operation by programming the DWC_mobile_storage to send a CMD0 by programming the registers CMDARG = 0 and CMD = {start_cmd=1, card number = appropriate card number, cmd_index = 0, rest of the fields = 0}.
 - d. At the end of a successful boot data transfer from card, the following interrupts are generated.
 - i. Command Done (CD) in RINTSTS.
 - ii. Data Transfer Over (DTO) in RINTSTS.
 - iii. Receive Interrupt (RI) in IDSTS in IDMAC mode only.
 - e. In IDMAC mode:
 - ✧ If the software driver creates more descriptors than the boot data received, the extra descriptors are not closed by the DWC_mobile_storage.
 - ✧ If the software driver creates less descriptors than the boot data received, the DWC_mobile_storage generates a Descriptor Unavailable (DU) interrupt and does not transfer any further data to system memory.

7.3.2.2 Removable MMC4.3 Card

Removable MMC4.3 cards differ with respect to eMMC in that the Host Controller driver is not aware whether these cards support the Boot mode of operation. Thus the Host controller must:

1. Enumerate these cards as it would enumerate MMC4.0/4.1/4.2 cards for the first time
2. Know the card characteristics
3. Decide whether to perform a boot operation or not.

For these cards, the software driver must follow these steps:

1. Enumerate the card as described in [“Programming Sequence”](#) on page 179.
2. Read the EXT_CSD register of the card and examine the following fields:
 - a. BOOT_PARTITION_ENABLE
 - b. BOOT_SIZE_MULT
 - c. BOOT_INFO
3. If necessary, the Host controller can also manipulate the boot information in the card; for more information, refer to section “7.2.4 Access to Boot Partition” in the MMC4.3 specification.
4. If the Host controller needs to perform a boot operation at the next power-up cycle, it can manipulate the EXT_CSD register contents by using a SWITCH (CMD6) command.
5. After this step, the software driver must power down the card by programming the PWREN register in DWC_mobile_storage.
6. From here on, use the same steps as in [“eMMC”](#) starting on 217.



Note

- You should ignore the End bit error if it is generated during an abort scenario.
- If a boot acknowledge error occurs, the boot acknowledge received interrupt will time out.
- In the IDMAC mode, the application needs to depend on the descriptor close interrupt instead of the data done interrupt.

7.4 Programming DWC_mobile_storage for Voltage Switching and DDR Operations

This section discusses details on programming the DWC_mobile_storage for voltage switching and DDR operations.

- ❖ [“Voltage Switch Operation”](#)
- ❖ [“DDR Operation”](#) on page 227

7.4.1 Voltage Switch Operation

Once the power and clocks are stable, the reset_n signal should be asserted (active-low) for at least two clocks of clk or cclk_in, whichever is slower. The reset initializes the registers, ports, FIFO-pointers, DMA interface controls, and state-machines in the design. After power-on reset, the software should perform the steps discussed in the following subsections.



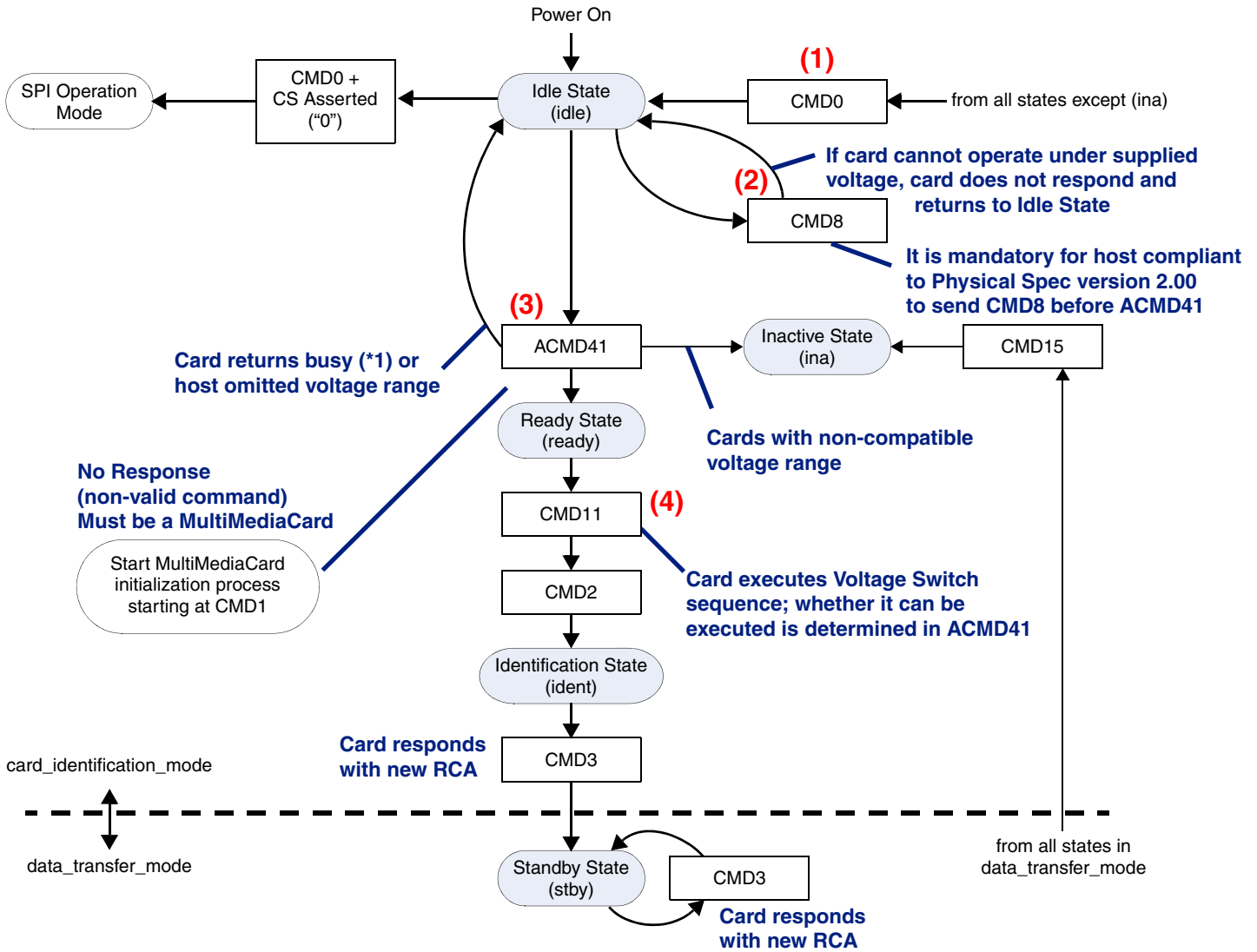
Note

- The Voltage Switch operation must be performed in SD mode only.
- You should run cclk_in at 200 MHz in order to support all UHS-1 speeds.

7.4.1.1 Programming Sequence

Figure 7-14 illustrates the voltage switch programming sequence.

Figure 7-14 Voltage Switching Command Flow Diagram



(*1) Note: Card returns busy when:

- Card executes internal initialization process
- Card is High or Extended capacity SD Memory Card and host does not support High or Extended capacity; that is, CMD8 is mandatory to initialize High capacity SD Memory Card

The following outlines the steps for the voltage switch programming sequence; refer to [Figure 7-14](#).

1. Software Driver starts CMD0, which selects the bus mode as SD.
2. After the bus is in SD card mode, CMD8 is started in order to verify if the card is compatible with the SD Memory Card Specification, Version 2.00.

CMD8 determines if the card is capable of working within the host supply voltage specified in the VHS (19:16) field of the CMD; the card supports the current host voltage if a response to CMD8 is received.

3. ACMD 41 is started.

The response to this command informs the software if the card supports voltage switching; bits 38, 36, and 32 are checked by the card argument of ACMD41; refer to [Figure 7-15](#).

Figure 7-15 ACMD41 Argument

47	46	45-40	39	38	37	36	35-33	32	31-16	15-08	07-01	00
S	D	Index	Busy 31	HCS 30	(FB) 29	XPC 28	Reserved 27-25	S18R 24	OCR 23-08	Reserved 07-00	CRC7	E
0	1	101001	0	X	0	X	000	X	xxxxh	0000000	xxxxxxx	1

Host Capacity Support
 0b: SDSC-only Host
 1b: SDHC or SDXC supported

SCXC Power Control
 0b: Power saving
 1b: Maximum performance

S18R: Switching to 1.8V Request
 0b: Use current signal voltage
 1b: Switch to 1.8V signal voltage

- a. Bit 30 informs the card if host supports SDHC/SDXC or not; this bit should be set to 1'b1.
- b. Bit 28 can be either 1 or 0.
- c. Bit 24 should be set to 1'b1, indicating that the host is capable of voltage switching; refer to [Figure 7-16](#).

Figure 7-16 ACMD41 Response (R3)

47	46	45-40	39	38	37	36-33	32	31-16	15-08	07-01	00
S	D	Index	Busy 31	CCS 30	Rsvd 29	Reserved 28-25	S18R 24	OCR 23-08	Reserved 07-00	CRC7	E
0	0	111111	X	X	0	0000	X	xxxxh	0000000	1111111	1

Busy Status
 0b: On Initialization
 1b: Initialization complete

Card Capacity Status
 0b: SDSC
 1b: SCHC or SCXC

S18R: Switching to 1.8V Accepted
 0b: Continues current voltage signalling
 1b: Ready for switching signal voltage

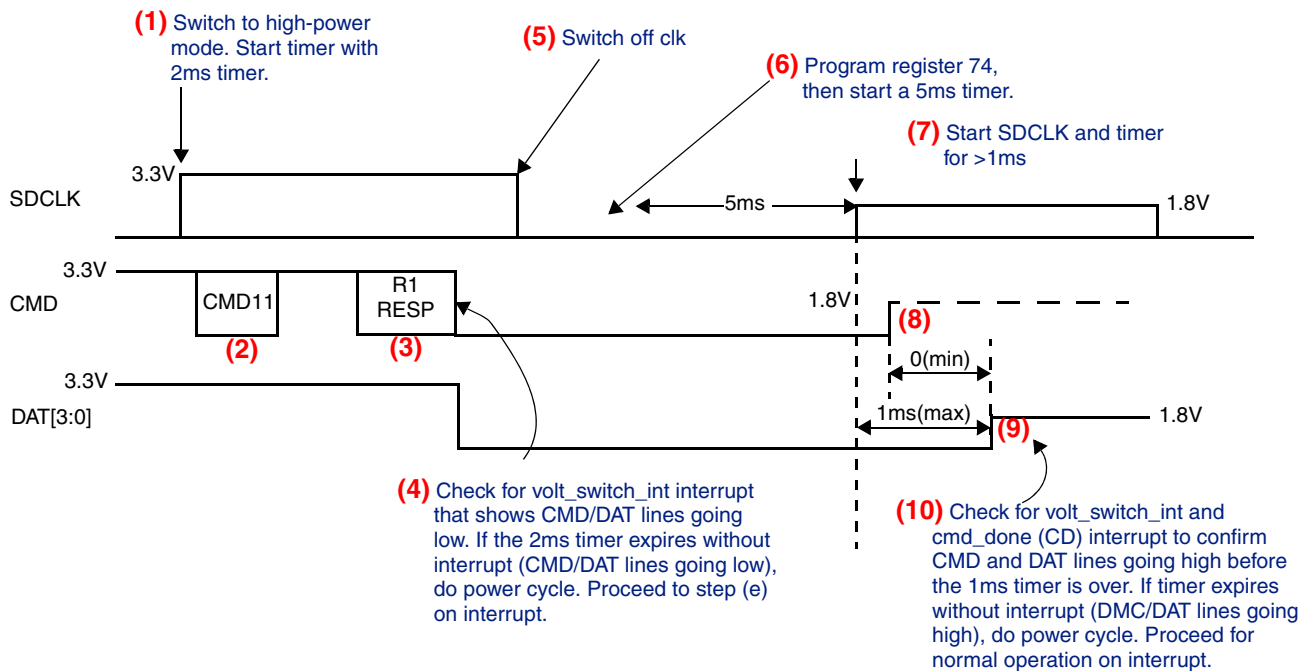
- d. Bit 30 – If set to 1'b1, card supports SDHC/SDXC; if set to 1'b0, card supports only SDSC
- e. Bit 24 – If set to 1'b1, card supports voltage switching and is ready for the switch
- f. Bit 31 – If set to 1'b1, initialization is over; if set to 1'b0, means initialization in process

- If the card supports voltage switching, then the software must perform the steps discussed for either the “Voltage Switch Normal Scenario” or the “Voltage Switch Error Scenario” on page 226.

7.4.1.2 Voltage Switch Normal Scenario

Figure 7-17 illustrates the voltage switch normal scenario.

Figure 7-17 Voltage Switch Normal Scenario



- The host programs CLKENA – cclk_low_power register – with zero (0) for the corresponding card, which makes the host controller move to high-power mode. The application should start a timer with a recommended value of 2 ms; this value of 2 ms is determined as below:
 - Total clk required for CMD11 = 48 clks
 - Total clk required for RESP R1 = 48 clks
 - Maximum clk delay between MCD11 end to start of RESP1 = 60 clks
 - Total = 48+48 + 60 = 160
 - Minimum frequency during enumeration is 100KHz; that is, 10us
 - Total time = 160 * 10us = 1600us = 1.6ms ~ 2ms
- The host issues CMD11 to start the voltage switch sequence.
 - Set bit 28 to 1'b1 in CMD when setting CMD11; for more information on setting bits, refer to “Boot Operation” on page 211.
- The card returns R1 response; the host controller does not generate cmd_done interrupt on receiving R1 response.

4. The card drives CMD and DAT [3:0] to low immediately after the response.

The host controller generates interrupt (VOLT_SWITCH_INT) once the CMD or DAT [3:0] line goes low. The application should wait for this interrupt. If the 2ms timer expires without an interrupt (CMD/DAT lines going low), do a power cycle.

**Note**

Before doing a power cycle, switch off the card clock by programming CLKENA register.

Proceed to step (5) on getting an interrupt (VOLT_SWITCH_INT).

**Note**

This interrupt must be cleared once this interrupt is received. Additionally, this interrupt should not be masked during the voltage switch sequence.

If the timer expires without interrupt (CMD/DAT lines going low), perform a power cycle. Proceed to step (5) on interrupt.

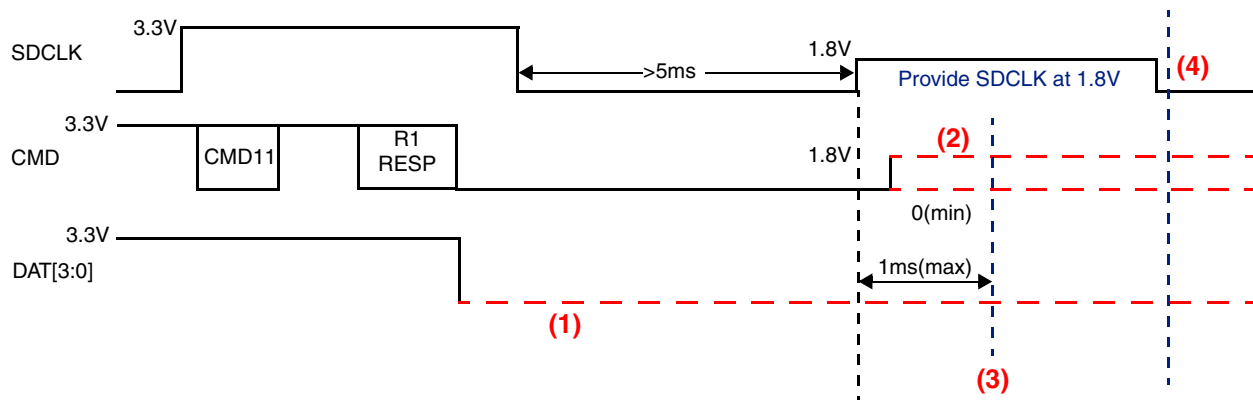
5. Program the CLKENA, cclk_enable register, with 0 for the corresponding card; the host stops supplying SDCLK.
6. Program VOLT_REG to the required values for the corresponding card. The application must program the newly-defined VOLT_REG register to assign 1 for the bit corresponding to the card number. The application should start a timer > 5ms.
7. After the 5ms timer expires, the host voltage regulator is stable.
Program CLKENA, cclk_enable register, with 1 for the corresponding card; the host starts providing SDCLK at 1.8V; this can be at zero time after VOLT_REG has been programmed. When the CLKENA register is programmed, the application should start another timer > 1ms.
8. By detecting SDCLK, the card drives CMD to high at 1.8V for at least one clock and then stops driving (tri-state); CMD is triggered by the rising edge of SDCLK (SDR timing).
9. If switching to 1.8V signaling is completed successfully, the card drives DAT [3:0] to high at 1.8V for at least one clock and then stops driving (tri-state); DAT [3:0] is triggered by the rising edge of SDCLK (SDR timing). DAT[3:0] must be high within 1ms from the start of SDCLK.
10. The host controller generates a voltage switch interrupt (VOLT_SWITCH_INT) and a command done (CD) interrupt once the CMD and DAT[3:0] lines go high. The application should wait for this interrupt to confirm CMD and DAT lines going high before the 1ms timer is done.

If the timer expires without the voltage switch interrupt (VOLT_SWITCH_INT), a power cycle should be performed. Program the CLKENA register to stop the clock for the corresponding card number. Wait for the cmd_done (CD) interrupt. Proceed for normal operation on interrupt. After the sequence is completed, the host and the card start communication in SDR12 timing.

7.4.1.3 Voltage Switch Error Scenario

Figure 7-18 illustrates the voltage switch error scenario.

Figure 7-18 Voltage Switch Error Scenario



1. If the interrupt (VOLT_SWITCH_INT) does not come, then the 2 ms timer should time out and a power cycle should be initiated.



Note

Before performing a power cycle, switch off the card clock by programming CLKENA register; no cmd_done (CD) interrupt is generated.

Additionally, if the card detects a voltage error at any point in between steps (5) and (7) in Figure 7-17, the card keeps driving DAT[3:0] to low until card power off.

2. CMD can be low or tri-state.
3. The host controller generates a voltage switch interrupt once the CMD and DAT[3:0] lines go high. The application should check for an interrupt to confirm CMD and DAT lines going high before the 1 ms timer is done.

If the 1 ms timer expires without interrupt (VOLT_SWITCH_INT) and cmd_done (CD), a power cycle should be performed. Program the CLKENA register to stop SDCLK of the corresponding card. Wait for the cmd_done interrupt. Proceed for normal operation on interrupt.

4. If DAT[3:0] is low, the host drives SDCLK to low and then stops supplying the card power.



Note

The card checks voltages of its own regulator output and host signals to ensure they are less than 2.5V. Errors are indicated by (1) and (2) in Figure 7-18.

- ❖ If voltage switching is accepted by the card, the default speed is SDR12.
- ❖ Command Done is given:
 - ◆ If voltage switching is properly done, CMD and DAT line goes high.
 - ◆ If switching is not complete, the 1ms timer expires, and the card clk is switched off.



Note

No other CMD should be driven before the voltage switching operation is completed and Command Done is received.

- ❖ The application should use CMD6 to check and select the particular function; the function appropriate-speed should be selected.

After the function switches, the application should program the correct value in the CLKDIV register, depending on the function chosen. Additionally, if Function 0x4 of the Access mode is chosen – that is, DDR50 in [Table 7-8](#) – then the application should also program 1'b1 in DDR_REG for the card number that has been selected for DDR50 mode.

Table 7-8 Function Names with Corresponding Speeds

Argument Slice	[23:20]	[19:16]	[15:12]	[11:8]	[7:4]	[3:0]
Group Number	6	5	4	3	2	1
Function Name	reserved	reserved	Current Limit	Driver Strength	Command System	Access Mode
0x0	Default		Default 200mA	Default Type B	Default	Default SDR12
0x1	Reserved	Reserved	400mA	Type A	For eC	High-Speed/ SDR25
0x2	Reserved	Reserved	600mA	Type C	Reserved	SDR50
0x3	Reserved	Reserved	800mA	Type D	OTP	SDR104
0x4	Reserved	Reserved	Reserved	Reserved	ASSD	DDR50

- ❖ When using SDR104, cclk_in should be 200Mhz and CLKDIV must be set to 0 (divide by 1).

7.4.2 DDR Operation

DDR programming should be done only after the voltage switch operation has completed.

The following outlines the steps for the DDR programming sequence:

- Once the voltage switch operation is complete, the user must program VOLT_REG to the required values for the corresponding card.
 - ◆ To start a card to work in DDR mode, the application must program a bit of the newly defined VOLT_REG[31:16] register with a value of 1'b1.
 - ◆ The bit that the user programs depends on which card is to be accessed in DDR mode.
- To move back to SDR mode, a power cycle should be run on the card – putting the card in SDR12 mode – and only then should VOLT_REG[31:16] be set back to 1'b0 for the appropriate card.

For information on VOLT_REG and DDR_REG registers, refer to “[UHS_REG](#)” on page [167](#).

7.4.2.1 Timing for DDR50

The following are timing details for DDR50:

- ❖ Data sampling is done on both edges of clk
- ❖ Data sampled on rising edge are called odd bytes
- ❖ Data sampled on falling edge are called even bytes
- ❖ Data payload size is always a multiple of two bytes
- ❖ Read and write data block length size is always 512 bytes
- ❖ Data lines are four bits wide
- ❖ Two CRC16 are computed per data line – one for odd bits and second for even bits

Sampled as full clock pulse

- ❖ CMD line remains sampled on only CLK rising edge – Host command and Card response
- ❖ Start and Stop bits remain full cycle
- ❖ CRC status and Busy signaling remains sampled on only CLK rising edge
- ❖ NCR and NAC

Sampled on both clock edges

- ❖ Data
- ❖ CRC16

Figure 7-19 illustrates the normal data packet format when the DWC Mobile Storage Host is in DDR50 mode.

Figure 7-19 Data Packet Format in DDR50 Mode – Usual Data

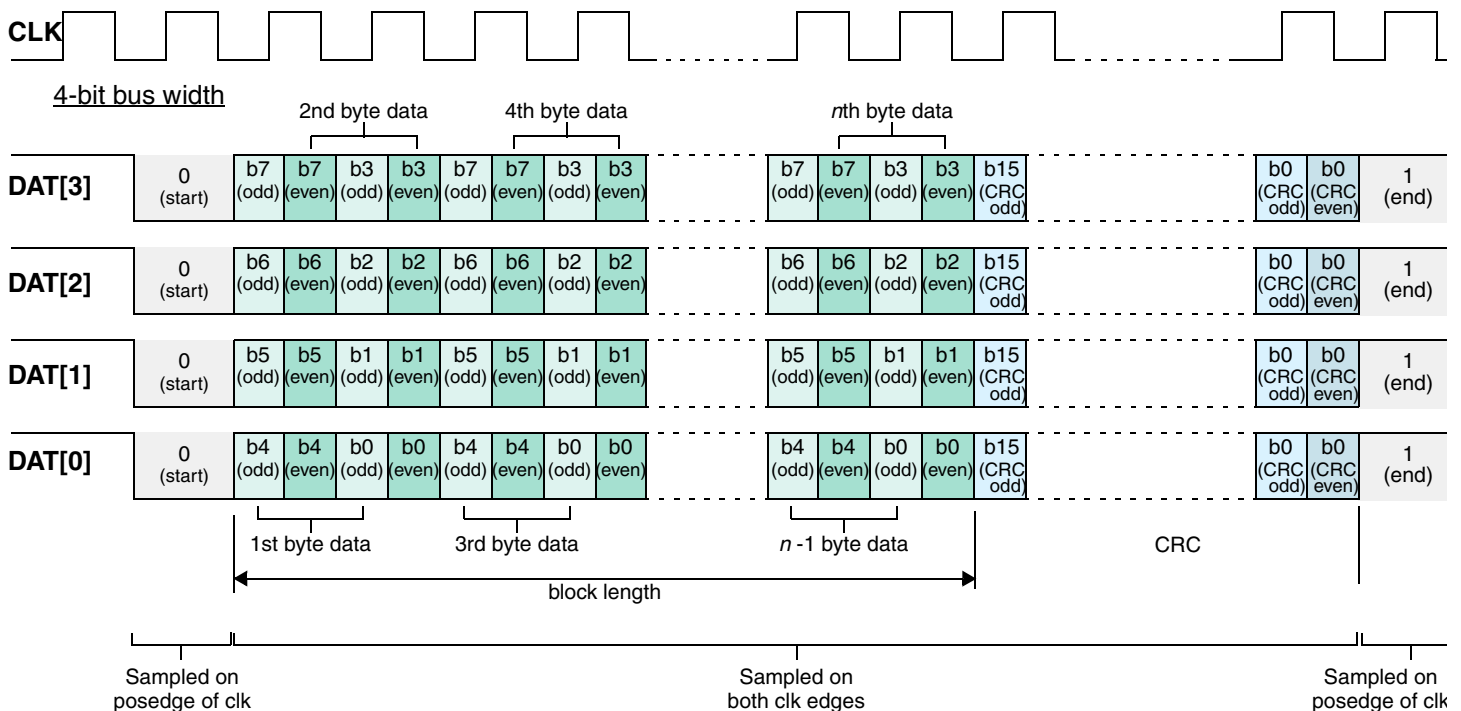


Figure 7-20 illustrates CRC16 when the DWC Mobile Storage Host is in DDR50 mode.

Figure 7-20 CRC16 in DDR50 Mode

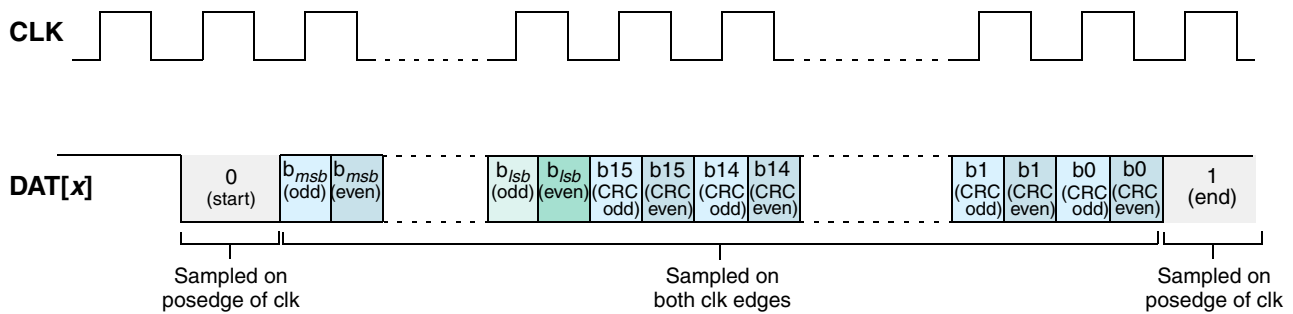
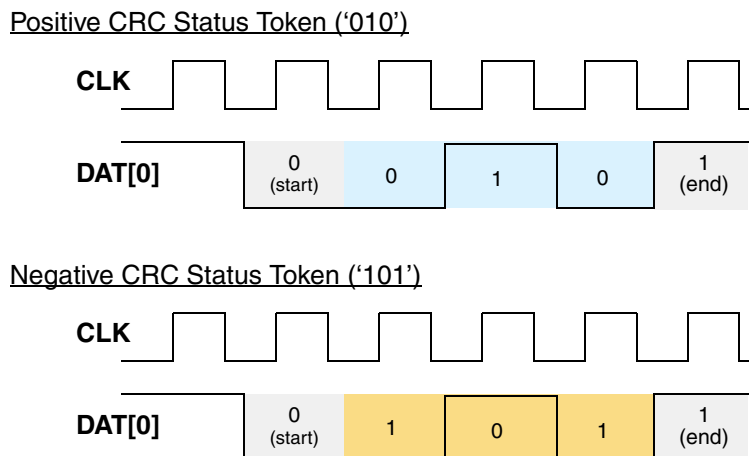


Figure 7-21 illustrates the CRC status token when the DWC Mobile Storage Host is in DDR50 mode.

Figure 7-21 CRC Status Token in DDR50 Mode



7.4.2.2 Card Clock control

The SD clock – cclk_out – can be stopped only after the clock falling edge, when the clock line is in a low logical state.

7.4.2.3 Reset Command/Moving from DDR50 to SDR12

To reset the mode of operation from DDR50 to SDR12, the following sequence of operations has to be done by the application:

1. Issue CMD0.
When CMD0 is received, the card changes from DDR50 to SDR12.
2. Program the CLKDIV register with an appropriate value.
3. Set DDR_REG[card_number] to 0.



Note The VOLT_REG register should not be programmed to 0 while switching from DDR50 to SDR12, since the card is still operating in 1.8V mode after receiving CMD0.

7.4.3 8-bit DDR Transfer



DDR mode is an optional feature for MMC 4.41 cards.

You can select an 8-bit DDR transfer, but only for MMC 4.41 cards. In this mode, the data is received and transmitted on both edges of the clock, and the data is available on all eight data lines. [Table 7-9](#) lists the DDR-mode fields in the Extended CSD register.

Table 7-9 DDR-Mode Fields in Extended CSD

Name	Field	Size (bytes)	Cell Type	CSD-Slice
Power class for 52MHz, DDR at 3.6V	PWR_CL_DDR_52_360	1	R	[239]
Power class for 52MHz, DDR at 1.9V	PWR_CL_DDR_52_195	1	R	[238]
Minimum Write Performance for 8-bit at 52MHz in DDR mode	MIN_PERF_DDR_W_8_52	1	R	[235]
Minimum Read Performance for 8-bit at 52MHz in DDR mode	MIN_PERF_DDR_R_8_52	1	R	[234]

[Table 7-10](#) lists the bits for different cards based on their card type and voltage/frequency of operation.

Table 7-10 Card Type Voltages and Frequencies in DDR Mode

Bit	Card Type
7:4	Reserved
3	High-Speed Dual Data Rate Multimedia Card @52MHz – 1.2V I/O
2	High-Speed Dual Data Rate Multimedia Card @52MHz – 1.8V or 3V I/O
1	High-Speed Multimedia Card @52MHz – rated device voltage(s)
0	High-Speed Multimedia Card @26MHz – rated device voltage(s)

The following commands are not allowed once the card is configured to operate in DDR mode:

- ◆ Bus testing (CMD19, CMD14)
- ◆ Lock-unlock (CMD42)
- ◆ Set block-length (CMD16)
- ◆ Stream transfer (CMD11, CMD20)

These commands are not executed and are regarded as illegal commands in DDR mode.

In DDR mode, the data size of a block read is always 512 bytes; a partial block data read is not supported. Two CRCs are appended at the end of each block; one for even bytes and one for odd bytes.

7.4.3.1 8-Bit DDR Programming Sequence

The following outlines the steps for the 8-bit DDR programming sequence:

1. The cclk_in signal should be twice the speed of the required cclk_out. Thus, if the cclk_out signal is required to be 50 MHz, the cclk_in signal should be 100 MHz.
2. The CLKDIV register should always be programmed with a value higher than zero (0); that is, a clock divider should always be used for 8-bit DDR mode.
3. The application must program the UHS_REG [31:16] register (DDR_REG bits) by assigning it with a value of 1 for the bit corresponding to the card number; this causes the selected card to start working in DDR mode.
4. Depending on the card number, the CTYPE [31:16] bits should be set in order to make the host work in the 8-bit mode.



Note

- The register programming should be done before the CMD register is programmed.
- The voltages of MMC cards can be 3V, 1.8V, 1.95V, or 1.2V and be working in DDR mode, which is determined by the CARD_TYPE parameter.
- The core DDR_REG bit should not be set when the card is working in the 1-bit mode; the core supports DDR in only 4-bit and 8-bit modes.

For information on the DDR_REG bits, refer to “UHS_REG” on page 167. For information on the Card Type register, refer to “CTYPE” on page 145.

7.4.3.2 Timing for 8-Bit DDR

The following are timing details for DDR50:

- ❖ Data sampling is done on both edges of clk
- ❖ Data sampled on rising edge are called odd bytes
- ❖ Data sampled on falling edge are called even bytes
- ❖ Data payload size is always a multiple of block size
- ❖ Read and write data block length size is always 512 bytes
- ❖ Data lines are eight bits wide
- ❖ Two CRC16 are computed per data line – one for odd bits and second for even bits

Sampled as full clock pulse

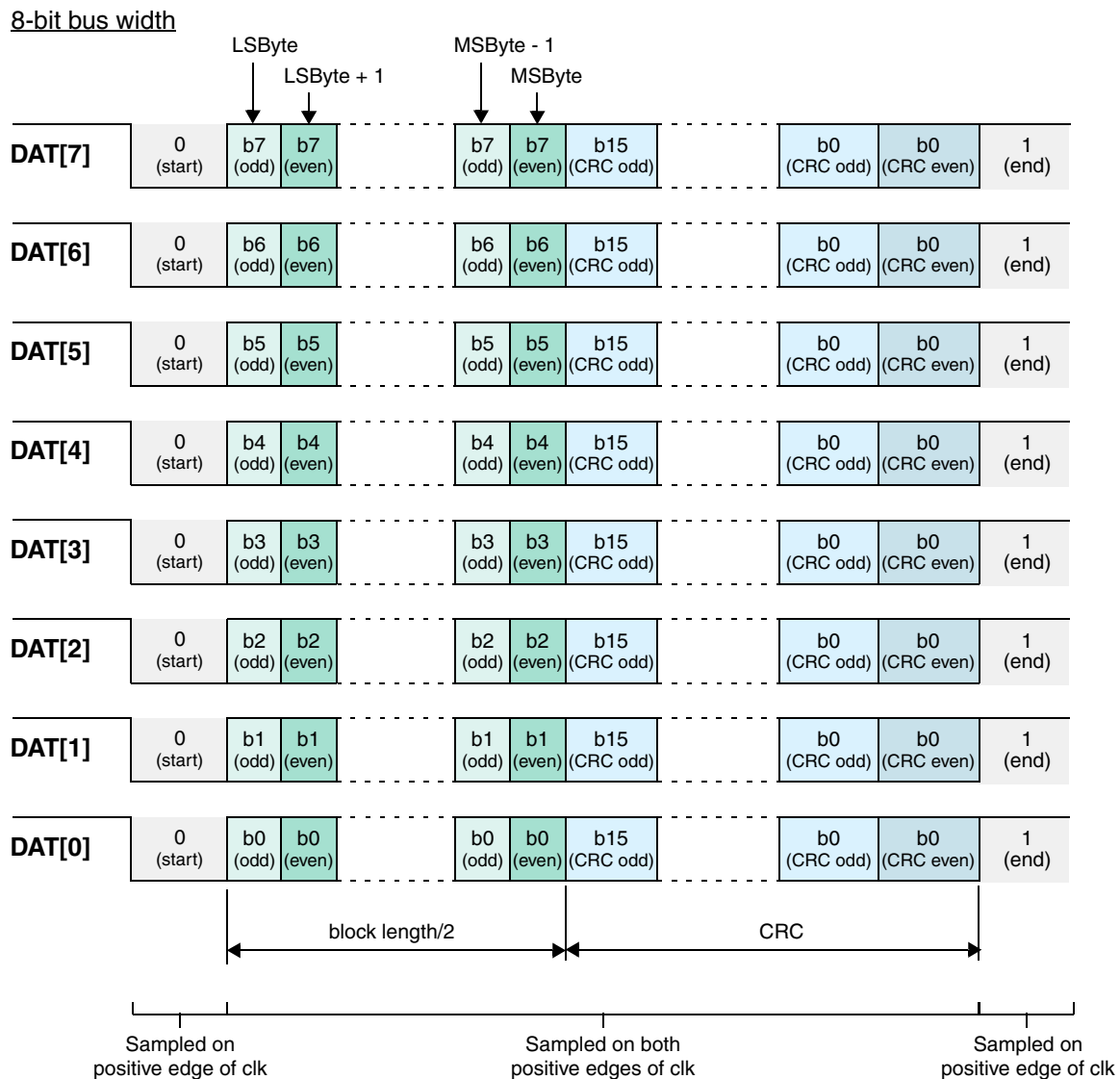
- ❖ CMD line remains sampled on only CLK rising edge – Host command and Card response
- ❖ Start and Stop bits remain full cycle
- ❖ CRC status and Busy signaling remains sampled on only CLK rising edge

Sampled on both clock edges

- ❖ Data
- ❖ CRC16

Figure 7-22 illustrates the normal data packet format when the DWC Mobile Storage Host is in 8-bit DDR mode.

Figure 7-22 Data Packet Format in 8-Bit DDR Mode – Usual Data



7.4.4 H/W Reset Operation

The RST_n signal has sixteen output pins from the host controller; one pin per card. As illustrated in Figure 7-23, when the RST_n signal goes low, the card enters a pre-idle state from any state other than the inactive state.

Figure 7-23 RST_n State Transition Inside Card



7.4.4.1 H/W Reset Programming Sequence

The following outlines the steps for the H/W reset programming sequence:

1. Program CMD12 to end any transfer in process.
2. Wait for DTO, even if no response is sent back by the card.
3. Set the following resets:
 - ◆ Software reset – BMOD[0] for IDMAC only
 - ◆ DMA reset– CTRL[2]
 - ◆ FIFO reset – CTRL[1] bits



Note

The above steps are required only if a transfer is in process.

4. Program the CARD_RESET register with a value of 0 for the bit corresponding to the card number; this can be done at any time when the card is connected to the controller. This programming asserts the RST_n signal and resets the card.
5. Wait for minimum of 1 μ s or cclk_in period, which ever is greater
6. After a minimum of 1 μ s, the application should program a value of 0 into the CARD_RESET register for the bit corresponding to the card number. This de-asserts the RST_n signal and takes the card out of reset.
7. The application can program a new CMD only after a minimum of 200 μ s after the de-assertion of the RST_n signal, as per the MMC 4.41 standard.

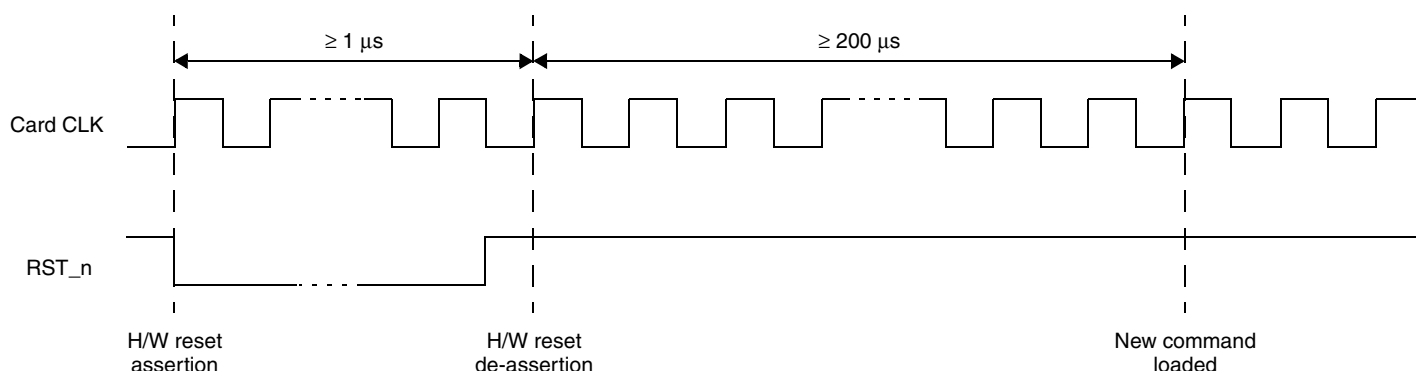


Note

For backward compatibility, the RST_n signal is temporarily disabled in the card by default. The host may need to set the signal as either permanently enabled or permanently disabled before it uses the card.

Figure 7-20 illustrates the H/W reset timing.

Figure 7-24 H/W Reset Timing in 8-Bit DDR Mode



Note

- The RST_n signal puts the card into a pre-idle state; the card must be initialized again.
- Between two H/W reset assertions, there should be a minimum of 1 μ s.

8

Verification

Synopsys supplies a complete functional verification environment for the DWC_mobile_storage. You can select and simulate the included tests, then automatically check simulation results using the coreConsultant tool described in [“Building and Verifying Your Core”](#) on page 25. Alternatively, you can perform these verification activities directly from the UNIX command line by following the procedures in this chapter.

This chapter provides the information you need to understand how the verification environment works. With this knowledge, you can perform verification activities by executing simulation scripts yourself, instead of through coreConsultant, and extend the verification environment by creating test scenarios of your own design. Additionally, you can verify DWC_mobile_storage cores after they are integrated into larger designs that use a Vera-based system testbench.

**Note**

- You must complete all of the coreConsultant procedures described in [“Building and Verifying Your Core”](#) on page 25 at least once before directly simulating the verification environment.
- The Vera files used in the verification environment are encrypted, and therefore cannot be viewed.

8.1 Overview of Vera Tests

The DWC_mobile_storage verification testbench performs tests that have been written to verify the following functionalities:

- ❖ act – AMBA certification test (ACT) compliance file.
- ❖ fifo – FIFO tests. Accesses FIFO in all AHB burst cycles with and without “busy” insertion.
- ❖ early_term – Verifies AHB accesses with early termination to both register and FIFO region.
- ❖ readcmd17 – Verifies single block reads SD/MMC cards.
- ❖ readcmd18 – Verifies multiple block reads to SD/MMC cards.
- ❖ readcmd18_clkcov – Reads multiple blocks of data from SD/MMC cards.
- ❖ writecmd24_clkcov – Writes single blocks of data into SD/MMC card.
- ❖ writecmd24 – Verifies single block writes to SD/MMC card.
- ❖ writecmd25 – Verifies multiple block writes to SD/MMC cards.
- ❖ rd_blk_1, rd_blk_2 – Verify block reads to SD/MMC card.
- ❖ wr_blk_1, wr_blk_2, wr_blk_3 – Verify block writes to SD/MMC card.
- ❖ rd_wr_blk_1, rd_wr_blk_2 – Verify block read/writes to SD/MMC card.

- ❖ `io_rd_wr_1`, `io_rd_wr_2` – Verify block read/writes to SDIO card.
- ❖ `rd_wr_strm_1`, `rd_wr_strm_2`, `rd_wr_strm_3` – Verify stream read/writes to MMC card.
- ❖ `misc_1`, `misc_2` – Verify other SD/MMC commands.
- ❖ `io_rd_intr_1`, `io_rd_intr_2` – Verify SDIO interrupts during read.
- ❖ `io_wr_intr_1`, `io_wr_intr_2` – Verify SDIO interrupts during write.
- ❖ `io_rd_wait_1`, `io_rd_wait_2` – Verify SDIO read wait operation.
- ❖ `io_suspend_1`, `io_suspend_2` – Verifies SDIO suspend operation.
- ❖ `mmc_intr_1` – Verifies MMC Interrupt.
- ❖ `rd_wr_blk_dat_err`, `rd_wr_blk_resp_err` – Tests for error conditions.
- ❖ `basic_clock` – Verifies clock generation logic.
- ❖ `basic_misc` – Verify other SD/MMC commands.
- ❖ `gedma_cardRd` – Verifies Generic DMA interface during card read.
- ❖ `gedma_cardWr` – Verifies Generic DMA interface during card write.
- ❖ `basic_wrdata_cov`, `basic_cov` – Write blocks of data into SD/MMC card to improve coverage and tests to improve basic coverage.
- ❖ `io_suspend_intr` – Verifies SDIO suspend operation with different variations like FIFO threshold, auto stop, block size, and so on.
- ❖ `rd_wr_starv_1`, `rd_wr_starv_2` – Verifies SD/MMC Block read/write operation with data starvation.
- ❖ `basic_cov_2` – Tests to improve coverage.
- ❖ `basic_rd_wait` – Verifies SD/MMC Block read operation for read wait functionality.
- ❖ `err_inject` – Verifies error injection and detection; CRC error, CMD index error, and so on.
- ❖ `fifo_cov` – Verifies block read/write to improve the coverage.
- ❖ `ceata_ccsd_col` – Verifies Host is driving the CCSD and device driving the CCS on the CMD line. Another loop tests the condition where `send_ccsd` is asserted on the same clock cycle of CCS sampling inside the CIU module.
- ❖ `ceata_basic_err_inject` – Verifies the basic error injection scenarios, which include Response Transmit Bit error, CRC status error, No CRC status, Data CRC error, and CMD index error in the response.
- ❖ `ceata_allcmds` – Verifies randomly generated ATA commands issued to the CEATA device for validation of the complete functionality of the ATA commands. Each ATA command consists of `RW_REG` for task file and `RW_BLK` for the data transfer, if any. The `RW_BLK` transfers are with either CCS enabled or CCS disabled.
- ❖ `ceata_ccsd` – Verifies randomly generated CCSD and STOP commands given during the data transfer. The STOP commands can be either internally generated auto stop or externally generated stop.
- ❖ `ceata_err_inject` – Verifies randomly generated error scenarios. The error scenarios include Response timeout and ACIO timeout.

8.2 Verification Environment Overview

You should ensure that Synopsys tools, Vera, coreConsultant, and your simulator are installed. As appropriate for your simulation environment, and in addition to the path and license, you should set the following environment variables in your .cshrc file:

- ❖ SYNOPSIS - \$SYNOPSIS
- ❖ Vera - \$VERA_HOME
- ❖ VCS - \$VCS_HOME (or)
- ❖ NC-Verilog - \$CDS_INST_DIR

The following is an example .cshrc file - note that for MTL, there is no \$MTI_HOME requirement:

```
# Synopsys Setup
setenv SYNOPSIS /remote/release/2003.03-3
set path=( $SYNOPSIS/sparcOS5/syn/bin $path)

#vera setup
setenv VERA_HOME /remote/dw104/tools/vera-latest
set path = ( $VERA_HOME/bin $path )

#VCS setup
setenv VCS_HOME /remote/drg17/tools/VCS_7.0
setenv VCS_CC /opt/SUNWspro/bin/cc
set path = ( ${VCS_HOME}/bin $path )

#NC setup
setenv CDS_INST_DIR /remote/insync4/cadence/LDV3.2
set path = ($CDS_INST_DIR/tools/verilog/bin \
    $CDS_INST_DIR/tools/bin $path)
setenv LD_LIBRARY_PATH $CDS_INST_DIR/tools/lib:$LD_LIBRARY_PATH

#MTI setup
set path = ( /remote/insync4/MTI/5.5b/sunOS4/modeltech/sunos5 $path )

#license setup
setenv SNPSLMD_LICENSE_FILE 27000@pasta
setenv LM_LICENSE_FILE \
    /remote/insync2/cadenceIC4.42QSR1/share/license/license.dat: \
    /remote/dw104/tools/lib/flexlm/synopsys_snpslmd_keys.dat: \
    /remote/dw104/tools/lib/flexlm/license.dat: \
    /remote/dw104/tools/vss_v6.100_solaris/pgm/lmgr/license.dat
```

8.3 Testbench Structure

The DWC_mobile_storage top-level testbench file provides the top-level testbench module; it is located at *workspace/sim/testbench/DWC_mobile_storage_test.v*.

The main features of the verification environment are as follows:

- ❖ Configurable
- ❖ Able to issue constrained, random-card command stimulus
- ❖ Able to issue directed (highly constrained) command stimulus
- ❖ On-the-fly, automated self checking of the transactions performed by the DWC_mobile_storage
- ❖ Functional coverage objects

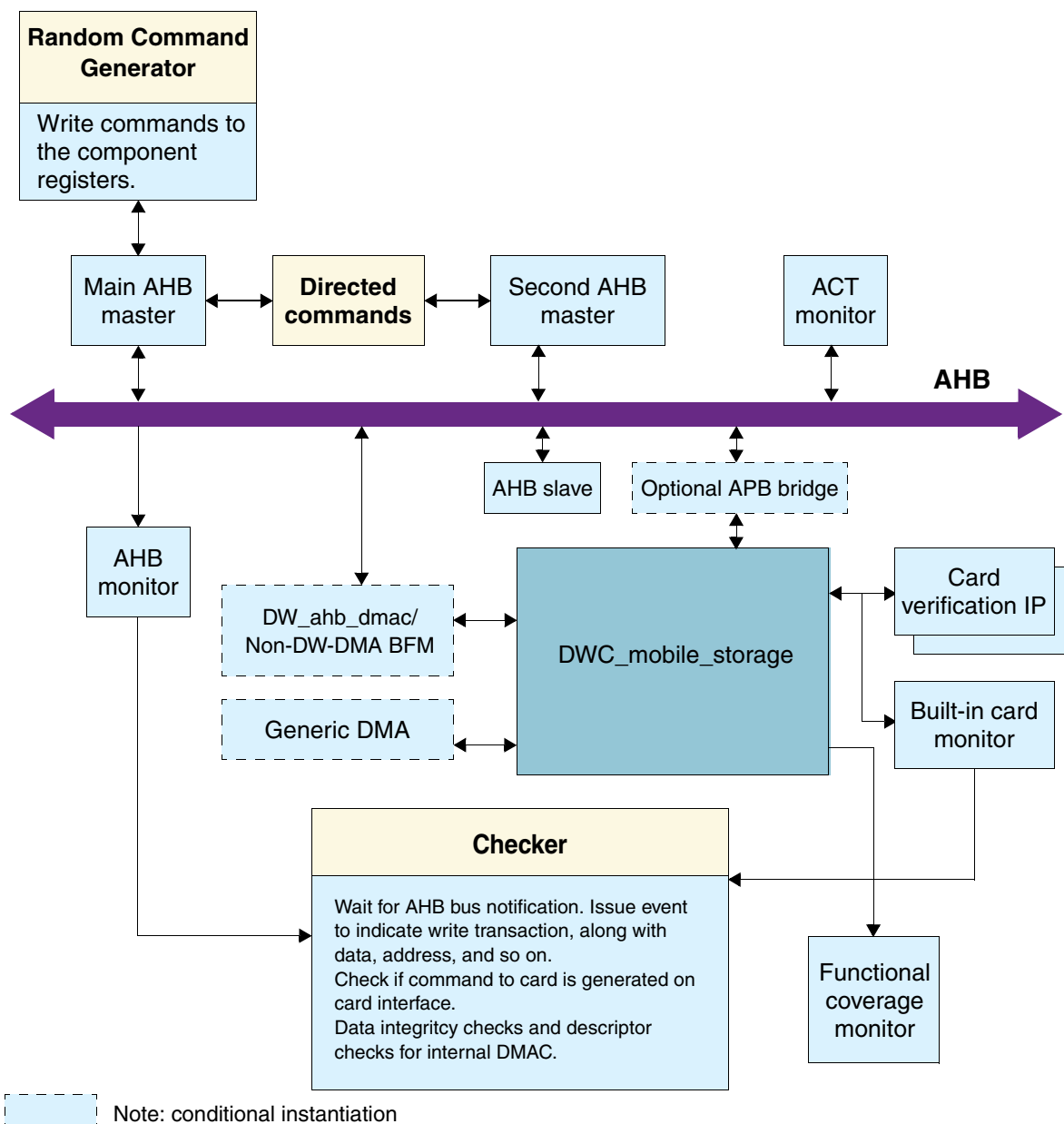
8.3.1 System-Test Architecture

DWC_mobile_storage system testing uses a configurable, constrained, random-testing methodology to achieve the targeted functional coverage. The random command generator can be highly constrained to generate all possible directed testing; later on, the constraints also can be set to cover corner cases.

Directed, constrained random-command stimulus (legal and illegal) is applied, and the checker in the system checks for the expected transaction completion by the DWC_mobile_storage. The checker also checks for data integrity on the data flowing in and out of the DWC_mobile_storage. The coverage objects built into the system provide feedback on which scenarios need to be generated in order to maximize DWC_mobile_storage testing.

Figure 8-1 illustrates DWC_mobile_storage system verification.

Figure 8-1 Verification of DWC_mobile_storage



To understand the roles of the components in the system in [Figure 8-1](#), consider an example scenario of reading n blocks of data from the card through the DW DMA controller.

1. To achieve this, the random command generator initiates several AHB write commands on the AHB bus to the DWC_mobile_storage register; the “read n blocks from the card” is programmed into the DWC_mobile_storage registers.
2. At the same time, when these write cycles are completed on the AHB bus, the AHB master sends message notifications to the checker along with the write cycle address and data details. The checker decodes these AHB cycles in a manner similar to the DWC_mobile_storage in order to infer a that a “read n blocks from the card” is requested. Based on this, it queues expected responses from the card BFM.
3. The DWC_mobile_storage issues the read command to the card and reads the data into its internal FIFO.
4. The checker determines the input requests received and compares them with expected responses from the card BFM. The checker generates errors or warnings if there is a mismatch in the expected responses. Depending upon the error count set, the simulation exits when errors occur.
5. When the internal FIFO of the DWC_mobile_storage is filled with data from the card, it generates the appropriate requests to the DW DMA controller BFM to read the blocks of data from the card.
6. The DW-DMA/non-DW-DMA controller BFM reads n blocks of data from the DWC_mobile_storage (source) into its internal channel FIFO.
7. The random command generator continues issuing new transactions for the card.
8. During all the above steps, the AHB Card monitors are monitoring the respective busses.
9. A coverage module and the Card BFM maintain functional coverage objects.
10. Various coverage objects are used to generate the required command stimulus and corner cases.
11. As shown in [Figure 8-1](#), the entire environment is configurable according to the different configurations of the DWC_mobile_storage.

The following are the DWC_mobile_storage system components illustrated in [Figure 8-1](#):

- ❖ AHB master – Generates cycles on the AHB bus; VMT-based BFM.
 - ◆ Commands for these AHB read/write cycles are fed from an external random command generator to which constraints are applied in order to control how variations of these commands are generated for the card.
 - ◆ The checker then receives message notifications.
- ❖ AHB monitor – Monitors all AHB bus transactions and logs them; VMT-based BFM.
- ❖ SD Card, MMC, and CE-ATA BFM –Receive commands, send command responses, and send or receive data with regard to SD_MMC_CEATA protocols; these VMT-based BFM are targets for all initiated accesses, and they:
 - ◆ Can also be configured to generate protocol errors or inject errors in the response/data tokens that are transmitted to the DWC_mobile_storage.
 - ◆ Have constrained random testing features.
 - ◆ Maintain cumulative functional coverage objects and also monitor the SD_MMC_CEATA bus, to a limited extent, to check for protocol errors.

- ❖ Checker – Waits for transaction notifications (that have details of a transaction completed on the AHB bus) from the AHB master.
 - ◆ Checks the transactions requested at the AHB bus and the transactions that occurred on the card bus interface and other host-side interfaces, such as the DW_ahb_dma, generic DMA, Non-DW-DMA, and interrupts.
 - ◆ Does not check for any AHB, APB, or card bus-specific protocol violations.

8.4 Testharness

The TestHarness class in the DWC_mobile_storage verification environment assists with instantiating and initializing the various components in a system.

- ❖ AHB VMT components – These are ahb_master_vmt, ahb_slave_vmt, ahb_monitor_vmt, and ahb_bus_vmt. The TestHarness configures these components according to the specified DWC_mobile_storage configuration.
- ❖ Card VMT components – These are mmc_card_vmt, sd_card_vmt, and ceata_card_vmt. The card models are initialized and connected to the DWC_mobile_storage ports, according to your specifications. Any of the card types – such as CEATA, MMC, SD Memory, SDIO and Combo – can be connected to different ports of the DWC_mobile_storage.
- ❖ DMA BFM – These are verification models of the DW DMA Controller BFM, generic DMA BFM, and Non-DW-hDMA controller BFM. Only one of these, if any, can be connected and initialized according to the DWC_mobile_storage configuration.
- ❖ Other verification components that can be selectively initialized are:
 - ◆ Random command generator (RCG)
 - ◆ Checker
 - ◆ Functional coverage monitor

The following sections outline the commands that define the TestHarness-class user interface. Other coverage-related methods assist in reporting and obtaining a cumulative coverage of the functional-coverage monitor and card models.

8.4.1 disableModelMsg

Disables VMT model messages.

Syntax

```
disableModelMsg(modelNameMacro, msgType);
```

Arguments

<i>modelNameMacro</i>	An integer that defines a macro specifying the models whose messages are to be enabled. SDMMC_TEST_CARDS_MSG SDMMC_TEST_AHB_MSTR_MSG SDMMC_TEST_AHB_MON_MSG
<i>msgType</i>	An optional 32-bit vector passed directly to the VMT model in disable_msg_type command. By default, the value is VMT_MSG_ALL.

Description

The disableModelMsg command disables message generation of VMT models to the simulator transcript window.

8.4.2 driveReset

Drives AHB system reset.

Syntax

```
driveReset( );
```

Description

The driveReset task drives the AHB system reset.

8.4.3 enableModelMsg

Enables VMT model messages.

Syntax

enableModelMsg(*modelNameMacro*, *msgType*)

Arguments

<i>modelNameMacro</i>	An integer that defines a macro specifying the models whose messages are to be enabled. SDMMC_TEST_CARDS_MSG SDMMC_TEST_AHB_MSTR_MSG SDMMC_TEST_AHB_MON_MSG
<i>msgType</i>	An optional 32-bit vector passed directly to the VMT model by the enable_msg_type command. By default, the value is VMT_MSG_DEFAULT.

Description

The enableModelMsg command enables message generation of VMT models to the simulator transcript window.

8.4.4 initialize

Initializes instances of all Vera verification components.

Syntax

initialize(*initChecker*, *initRcg*, *initCoverage*, (*simTimeOut*));

Arguments

<i>initChecker</i>	An integer that optionally instantiates the checker. SDMMC_TEST_INIT_CHECKER SDMMC_TEST_DO_NOT_INIT_CHECKER
<i>initRcg</i>	An integer that optionally instantiates the Random Command Generator. SDMMC_TEST_INIT_RCG SDMMC_TEST_DO_NOT_INIT_RCG
<i>initCoverage</i>	An integer that optionally instantiates the coverage monitor. SDMMC_TEST_INIT_COV SDMMC_TEST_DO_NOT_INIT_COV
<i>simTimeOut</i>	An integer that decides when to exit the simulation when the simulation goes into a loop. This is used to avoid simulation “hangs” during an unrecoverable

situation. This argument is an optional argument; if no count is specified, a default count of 50,000 AHB clocks is used.

Description

The initialize command creates instances of all Vera verification components, performs their initialization according to the `DWC_mobile_storage` configuration, and interconnects them in the environment.

8.4.5 setTopConfig

Gets top configuration of environment from user.

Syntax

```
setTopConfig(endianness, logFilePrefix, exitErrorCount, setRCA, msgLevel);
```

Arguments

<i>endianness</i>	An integer that affects configuration of AHB components and how the <code>DWC_mobile_storage</code> <code>hbig_endian</code> pin is driven; will be overridden by the simulation parameter. SDMMC_TEST_LITTLE_ENDIAN SDMMC_TEST_BIG_ENDIAN
<i>logFilePrefix</i>	A string that contains names of all log files generated by the checker; RCG is prefixed with this string.
<i>exitErrorCount</i>	An integer set as the exit error count in the checker.
<i>setRCA</i>	An integer that enables initialization of the card model state machine so that initialization commands can be bypassed.
<i>msgLevel</i>	A 4-bit vector that sets message levels for the checker and RCG logs.

Description

The `setTopConfig` command gets the top configuration of the environment from the user.

8.5 Simulation Scripts



Before running the scripts described in this section, you *must* execute at least once the coreConsultant configuration and simulation activities described in “Building and Verifying Your Core” on page 25.

All simulation activities are controlled by a set of scripts, whether you indirectly execute them through coreConsultant or manually from the UNIX command line.

- ❖ If you use the coreConsultant GUI, you use the Simulate activity to select the tests that you want to run. When you click OK, commands are written into testlist scripts that create a simulator executable and execute the tests you selected; the testlist script is then executed by the top-level script, `run.scr`.
- ❖ If you run scripts from the UNIX command line, you can edit and run the top-level `run.scr` script in the same way that coreConsultant does, or you can simply execute the scripts that `runtest` calls.

8.5.1 run.scr

The run.scr script is the top-level simulation script that coreConsultant generates to execute the AMBA certification test (ACT) compliance and corner case tests you selected from the list of supported tests for your chosen configuration. In coreConsultant, this script is executed when you click OK in the Simulate activity.

Use run.scr when you want to run multiple tests, such as for regression testing. Also, you can run this script – with the default core configuration and default tests enabled – to confirm that your simulation environment is set up correctly.

To execute this script from the command line, go to the *workspace/sim* directory and enter:

```
% run.scr
```

This script does the following:

1. Checks to see that no previous invocation of run.scr is currently executing.
2. Executes “sim/compile,” which creates a simulator executable if a non-Verilog-XL simulator is chosen; additionally recompiles sim/vera/src/get_params.vr.
3. Executes runtest.sh, which runs “runtest” for each of the selected tests:
 - a. Compiles the Vera test there
 - b. Invokes the simulation

For several simulators, the PLIs must be created before running the tests using the appropriate scripts:

- ◆ build_pli_mti.sh – MTI
- ◆ build_pli_nc.sh – NC-Verilog

You can use the Netscape browser on the .../sim/runtest.log file to see the results of these scripts.

4. Sends e-mail to inform you that all testcases in the testlist have executed, if this option is enabled.

The message includes the following for the coreConsultant Simulate activity:

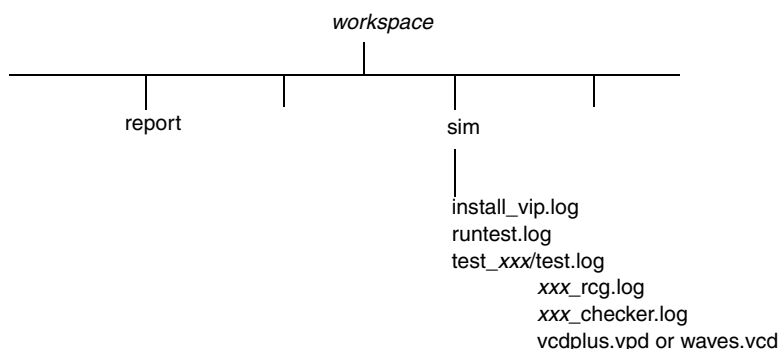
- ◆ Workspace directory
- ◆ Design name
- ◆ Status (for example, Completed)
- ◆ Path of the runtest.log file
- ◆ Contents of the runtest.log file

8.6 Simulation Output Files

Figure 8-2 shows the simulation output files that exist in your `DWC_mobile_storage` workspace at the end of a test case simulation. If the `run.scr` script was invoked, a new `runtest.log` was generated.

Additionally for each command file in the testlist, `test.log`, `xxx_rcg.log`, and `xxx_checker.log` files are generated in the `sim/test_xxx` directory. In addition, the VCD/VCDPLUS dump file is also created if the `VCD_ON` parameter is enabled in the coreConsultant configuration.

Figure 8-2 Simulation Output File Directories



When you simulate the testbench, either through coreConsultant or directly from the UNIX system prompt, the testbench writes the following files to your `workspace/sim` directory:

- ❖ `install_vip.log` – Shows log generated during VIP model installation
- ❖ `sim/runtest.log` – Shows log generated during the simulator executable generation
- ❖ `sim/test_xxx/test.log` – Log file generated by the test, containing captured simulator output; contains INFO, WARNING, ERROR messages from BFM's, the monitor, the `DWC_mobile_storage`, or the checker
- ❖ `sim/test_xxx/xxx_rcg.log` – Log file generated by the Random Command Generator
- ❖ `sim/test_xxx/xxx_checker.log` – Log file generated by the checker
- ❖ `sim/test_xxx/vcdplus.vpd` (or `waves.vcd`) – Optional VCD/VCDPLUS dump file

8.7 Waveform Output

You can enable this option in coreConsultant by selecting “Generate Waves File.”

If you enable this option, simulations produce a VCD/VCDPLUS format file called `workspace/sim/test_xxx/waves.vcd` (`vcdplus.vpd`), which you can display using a waveform viewer. When the VCS simulator is chosen, the dump file is in VCDPLUS format (`vcdplus.vpd`); for other simulators, the dump file is in VCD format (`waves.vcd`). The VCD/VCDPLUS capture initial blocks are located at the end of the `sim/testbench/DWC_mobile_storage_test_top.v` file and can be modified, if necessary, in order to capture a different vector supported by your simulator.

Every node that is internal or external to the configuration under test will have its waveform captured in the `waves.vcd/vcdplus.vpd` file. The VCDPLUS is a more compact format than the VCD file. You can view VCD/VCDPLUS files by using the VCS graphical interface tool (`vcs -RPP`). Similarly, NC-Verilog has a waveform viewer called SignalScan, which can be used to view VCD files.

8.8 Executing Individual Tests or Batch Jobs

To run just one test and automatically verify your simulation results, perform the following steps:

1. Go to your *workspace/sim* directory and edit the *runtest.sh* to include only the tests you want to run.
2. Run the “*run.scr*” command.

8.9 Constrained Random Testing

Constrained random testing covers all legal and illegal protocol variations at the different interfaces of the *DWC_mobile_storage*. Both directed and random test cases are achieved by constraining the random command data generator. The entire verification environment needs to have the *DWC_mobile_storage* configuration set and the constraints selected for a random simulation run.

8.9.1 Random Command Data Generator

Constrained random testing facilitates:

- ❖ Generating a random sequence of card transactions with random attributes for each transaction.
- ❖ Imposing constraints, in the form of weightages and value ranges, on the types of card transactions and their associated attributes.

The AHB master VIP BFM is used as a vehicle to issue commands to the DW Card Controller.

The random command generator model currently generates its own random binary data to be written into the card.

8.9.2 Constraint Selection for Random Simulation Runs

Generating random stimuli to the *DWC_mobile_storage* consists of:

- ❖ Random programming of the *DWC_mobile_storage* registers.
- ❖ Random command sequences and card BFM responses with respect to card protocols.
- ❖ Random arguments, and data supplied to the *DWC_mobile_storage* or generated by the card BFMs.

8.9.3 ConnectNewCard

Specifies the cards connected to the DWC_mobile_storage.

Syntax

```
ConnectNewCard(cardType, rca, cardBlkLen, dataWidth, cardState);
```

Arguments

<i>cardType</i>	A 2-bit vector that specifies the type of connected card. SDMMC_TEST_CARD_TYPE_MMC SDMMC_TEST_CARD_TYPE_SDMEM SDMMC_TEST_CARD_TYPE_SDIO SDMMC_TEST_CARD_TYPE_COMBO SDMMC_TEST_CARD_TYPE_CEATA
<i>rca</i>	A 16-bit vector that specifies the relative card address in case of MMC cards.
<i>cardBlkLen</i>	A 32-bit vector that specifies the default block length of the connected card.
<i>dataWidth</i>	A bit that specifies whether the card is 1-bit or 4-bits wide.
<i>cardState</i>	A 4-bit vector that specifies the state the card is in when it is connected.

Description

The ConnectNewCard command informs the random command generator of the cards that are connected across the DWC_mobile_storage.

8.9.4 setRcgMsgLevel

Sets level of messages.

Syntax

```
setMsgLevel(msgLevel);
```

Arguments

<i>msgLevel</i>	A 4-bit vector that specifies the level of the messages that will be generated. SDMMC_TEST_WARNING SDMMC_TEST_NOTE
-----------------	--

Description

The setRcgMsgLevel command sets the level on the messages to be generated.

8.9.5 setBaseAddress

Sets the base address of the DWC_mobile_storage.

Syntax

```
setMsgLevel(baseAddr);
```

Arguments

baseAddr A 55-bit vector that specifies the base address of the DWC_mobile_storage.

Description

The setBaseAddress command sets the base address of the DWC_mobile_storage. This address is used to issue read/write commands to the DWC_mobile_storage registers.

8.9.6 issueXactions

Issues randomly generated transactions to the DWC_mobile_storage.

Syntax

```
issueXactions(totalNumXactions);
```

Arguments

totalNumXactions An integer that specifies the total number of card transactions to be issued.

Description

The issueXactions command issues card command requests to the DWC_mobile_storage. The selection and setting of the constraints on the attributes decides whether the commands will be issued randomly or in a directed fashion. The total number of card transactions to be issued can be specified. A random seed can also be specified if transactions need to be run in different sequences each time.

8.9.7 setAttrRange

Sets the range of values for the specified attribute.

Syntax

```
setAttrRange(randAttr, lowValue, highValue);
```

Arguments

randAttr An integer that specifies the attribute for which the range will be set.
lowValue A 32-bit vector that specifies the lower value of the range.
highValue A 32-bit vector that specifies the higher value of the range.

Description

The setAttrRange command sets the range of values for the specified attribute. This range is used to generate a value randomly within the range with equal weight to all the values.

8.9.8 setAttrWeight

Sets the weight on the specified attribute.

Syntax

```
setAttrRange(randAttr, weight);
```

Arguments

<i>randAttr</i>	An integer that specifies the attribute for which the weight will be set.
<i>weight</i>	An integer that specifies the weight of the attribute.

Description

The setAttrWeight command sets the weight on the specified attribute. This is used to randomly select an attribute type.

8.9.9 Testcases Using Random Generator

The random command generator issues random test cases that constrain attributes according to desired test scenarios. The following Vera code shows an example of single-block reads (CMD17) and multiple-block reads (CMD18).

```
harness.buildEnv(SDMMC_TEST_SET_CARD_RCA, "tb_rd_blk_1");

////////////////////////////////////////
//Example of generating random test scenarios
////////////////////////////////////////

// set ranges for clock divider value to be selected
// for each of the clock divider
harness.m_rcg.setAttrRange(SDMMC_RCG_CLK_DIV0, 1, 5);
harness.m_rcg.setAttrRange(SDMMC_RCG_CLK_DIV1, 2, 8);
harness.m_rcg.setAttrRange(SDMMC_RCG_CLK_DIV2, 3, 10);
harness.m_rcg.setAttrRange(SDMMC_RCG_CLK_DIV3, 4, 20);

harness.m_rcg.setAttrWeight(SDMMC_RCG_CMD13, 50);
harness.m_rcg.setAttrWeight(SDMMC_RCG_CMD17, 70);
harness.m_rcg.setAttrWeight(SDMMC_RCG_CMD18, 70);

harness.m_rcg.setAttrWeight(SDMMC_RCG_WAIT_PREVDAT_COMPL_OFF, 50);
harness.m_rcg.setAttrWeight(SDMMC_RCG_WAIT_PREVDAT_COMPL_ON, 50);

harness.run_t();
```

8.9.9.1 Supported Randomization Attributes

For the CardCommandTransaction class, there is a set of attributes that you can randomize; these are listed in [Table 8-1](#). Each attribute has an attribute ID string associated with it.

You can use the attribute ID as a string in its default form without a leading or ending white space. If the attribute is of an enumerated type, each enumerated item is referenced as an enum ID. Enum IDs are of the type “string” when used in setConstraints, but are integer macros when used in get/set EnumAttr. The

returned integer macro eases the work of a testbench because there is no need to convert a string into an integer. All strings are case-insensitive.

Table 8-1 DWC_mobile_storage Randomization Attributes

Attribute ID	XACT_TYPE	
Description	Decides the type of transactions to be issued.	
Type	Integer	
Integer macros	Reset Cards	SDMMC_RCG_CMD0
	Set DSR	SDMMC_RCG_CMD4
	Select/Deselect Card	SDMMC_RCG_CMD7
	Send CSD	SDMMC_RCG_CMD9
	Send CID	SDMMC_RCG_CMD10
	Stream read	SDMMC_RCG_CMD11
	Send Status	SDMMC_RCG_CMD13
	Go inactive	SDMMC_RCG_CMD15
	Read single block	SDMMC_RCG_CMD17
	Read multiple block	SDMMC_RCG_CMD18
	Stream write	SDMMC_RCG_CMD20
	Write single block	SDMMC_RCG_CMD24
	Write multiple block	SDMMC_RCG_CMD25
	Program CID	SDMMC_RCG_CMD26
	Program CSD	SDMMC_RCG_CMD27
	Clear write protect	SDMMC_RCG_CMD29
	Send write protect	SDMMC_RCG_CMD30
	Erase	SDMMC_RCG_CMD38
	Fast IO	SDMMC_RCG_CMD39_READ
	Fast IO	SDMMC_RCG_CMD39_WRITE

Table 8-1 DWC_mobile_storage Randomization Attributes

Integer macros (cont.)	Go IRQ state	SDMMC_RCG_CMD40
	Lock/Unlock	SDMMC_RCG_CMD42
	General cmd	SDMMC_RCG_CMD56
	Set bus width	SDMMC_RCG_ACMD6
	Send SD status	SDMMC_RCG_ACMD13
	Send num of write blks	SDMMC_RCG_ACMD22
	SD send OCR	SDMMC_RCG_ACMD41
	Set/Clr Card detect	SDMMC_RCG_ACMD42
	Send SCR	SDMMC_RCG_ACMD51
	SDIO read multiple	SDMMC_RCG_CMD53_READ
	SDIO write multiple	SDMMC_RCG_CMD53_WRITE
	FIFO reset	SDMMC_RCG_FIFO_RESET
	DMA reset	SDMMC_RCG_DMA_RESET
	Controller reset	SDMMC_RCG_CONTR_RESET
	Change card clocks	SDMMC_RCG_CARDCLK_CHANGE
	ATA command READ DMA	CEATA_RCG_READ_DMA_EXT
	ATA command WRITE DMA	CEATA_RCG_WRITE_DMA_EXT
	ATA command IDENTIFY DEVICE	CEATA_RCG_IDENTIFY_DEVICE
	ATA command STANDBY IMMEDIATE	CEATA_RCG_STANDBY_IMMEDIATE
	ATA command FLUSH CACHE EXT	CEATA_RCG_FLUSH_CACHE_EXT
	RW_REG Register read	CEATA_RCG_CMD60_READ
	RW_REG Register write	CEATA_RCG_CMD60_WRITE
Default distribution	All transactions get equal weightage.	
Attribute ID	CARD_ADDRESS	
Description	Bit vector attribute that determines the address for different accesses to the card	
Type	Bit vector	
Integer macro	SDMMC_RCG_CARDADDRESS	
Value range	Any value of 32 bit vector. Address represents the byte address for Standard Memory SD card. Address represents the block address for High Capacity SD Card and MMC 4.2.	
Default distribution	One range from 0 to 32'h0000FFFF	

Table 8-1 DWC_mobile_storage Randomization Attributes

Attribute ID	HOST_INIT_DATA_STOP	
Description	Decides when (after how many bytes) the external STOP command is to be issued for open-ended data transfer abort.	
Type	Integer	
Integer macro	SDMMC_RCG_HOST_INIT_DATA_STOP	
Value range	>= 0	
Default distribution	No stop command is issued.	
Attribute ID	SUSPEND_XACT	
Description	Decides when (after how many bytes) a suspend command is to be issued. This attribute setting is only valid for SDIO and SDCombo type of cards connected.	
Type	Integer	
Integer macro	SDMMC_RCG_SUSPEND_XACT	
Value range	>= 0	
Default distribution	No suspend command is issued.	
Attribute ID	XACT_READWAIT	
Description	Controls when the Card read transactions need to be 'read waited'. This attribute setting is only valid for SDIO read type of transactions. This will specify the number of bytes after which the read-wait will be issued.	
Type	Integer	
Integer macro	SDMMC_RCG_XACT_READWAIT	
Value range	>= 0	
Default distribution	No read-wait command is issued.	
Attribute ID	LOAD_BIU_CMDS	
Description	Controls the manner in which new commands are programmed into the bus interface unit of the DW Card Controller.	
Type	Integer	
Integer macro	Load the new command only after the data transfer of previous command is done	SDMMC_RCG_LD_CMD_NORMAL
	Load the new command after the response of the previous command is received but data transfer is in progress	SDMMC_RCG_LD_CMD_AFTER_CMD_DONE
Default distribution	New commands are loaded normally; that is, only the data transfer of the previous command is completed.	

Table 8-1 DWC_mobile_storage Randomization Attributes

Attribute ID	RESP_TIMEOUT
Description	Determines the value of the response time out to be set in the DW Card Controller.
Type	Integer
Integer macro	SDMMC_RCG_RESP_TIMEOUT
Value range	≥ 2 and ≤ 64
Default distribution	Response timeout is set to 64.
Attribute ID	DATA_TIMEOUT
Description	Determines the value of the read data time out to be set in the DW Card Controller.
Type	Integer
Integer macro	SDMMC_RCG_DATA_TIMEOUT
Value range	≥ 2
Default distribution	Read data timeout is set to 4095.
Attribute ID	BLK_SIZE
Description	Determines the block size of the data to be set in the DW Card Controller and also for the Card.
Type	Integer
Integer macro	SDMMC_RCG_BLOCK_SIZE
Value range	≥ 1 and ≤ 2048 if SD card is Standard Memory SD Card, and $= 512$ if SD card is High Capacity SD Card
Default distribution	Block size is set as per the block size of the card connected to the DWC_mobile_storage.
Attribute ID	BYT_COUNT
Description	Determines the total number of bytes to be transferred to the Card. This value is set in the DW Card Controller register.
Type	Integer
Integer macro	SDMMC_RCG_BYTE_COUNT
Value range	≥ 0
Default distribution	0

Table 8-1 DWC_mobile_storage Randomization Attributes

Attribute ID	TX_FIFO_THRSH	
Description	Controls the fifo transmit threshold setting in the DW Card Controller. This setting is dependent on the depth of the FIFO set during configuration.	
Type	Integer	
Integer macro	SDMMC_RCG_TX_FIFO_THRSH	
Value range	≥ 2 and ≤ 1024	
Default distribution		
Attribute ID	RX_FIFO_THRSH	
Description	Controls the fifo receive threshold setting in the DW Card Controller. This setting is dependent on the depth of the FIFO set during configuration.	
Type	Integer	
Integer macro	SDMMC_RCG_RX_FIFO_THRSH	
Value range	≥ 2 and ≤ 1024	
Default distribution		
Attribute ID	RESET	
Description	Decides the type of reset to be issued to the DW Card Controller.	
Type	Integer	
Integer macros	FIFO reset to be issued	SDMMC_RCG_FIFO_RESET
	DMA reset to be issued	SDMMC_RCG_DMA_RESET
	Controller reset to be issued	SDMMC_RCG_CONTR_RESET
Default distribution	None of the reset types have any weightage.	
Attribute ID	CARD_NUM	
Description	Decides the card to be selected for the issuance of command. This is dependent on the type (SD_MMC_CEATA) of the card connected and number of cards configured for the DWC_mobile_storage.	
Type	Integer	
Integer macro	SDMMC_RCG_CARD_NUM	
Value range	≥ 0 and ≤ 30	
Default distribution		

Table 8-1 DWC_mobile_storage Randomization Attributes

Attribute ID	CLK_DIV	
Description	Decides the range of divisor values for each of the clock dividers. This selection is dependent on the number of clock dividers selected in the DWC_mobile_storage configuration.	
Type	Integer	
Integer macro	SDMMC_RCG_CLK_DIV0 SDMMC_RCG_CLK_DIV1 SDMMC_RCG_CLK_DIV2 SDMMC_RCG_CLK_DIV3	
Value range	>= 0 and <= 255	
Default distribution	Value of 1 is set	
Attribute ID	CLK_DIV_SRC	
Description	Decides the weightage for four clock dividers to be the source for card clock. This selection is dependent on the number of clock dividers selected in the DWC_mobile_storage configuration.	
Type	Integer	
Integer macro	SDMMC_RCG_CLK_DIV0_SRC SDMMC_RCG_CLK_DIV1_SRC SDMMC_RCG_CLK_DIV2_SRC SDMMC_RCG_CLK_DIV3_SRC	
Value range	>= 0	
Default distribution		
Attribute ID	CARD_WIDTH	
Description	Decides weightage for card widths (1bit/4bit/8bit) while changing card widths in simulation.	
Type	Integer	
Integer macros	1-bit card data width	SDMMC_RCG_CARD_1BIT
	4-bit card data width	SDMMC_RCG_CARD_4BIT
	8-bit card data width	SDMMC_RCG_CARD_8BIT
Default distribution	1-bit card data width has all the weightage.	

Table 8-1 DWC_mobile_storage Randomization Attributes

Attribute ID	WAIT_PREV_DAT_COMPL	
Description	Decides if commands will be loaded with wait_prev_dat_comp set or not for every command.	
Type	Integer	
Integer macros	Set wait_prev_dat_compl	SDMMC_RCG_WAIT_PREVDAT_COMPL_ON
	Do not set wait_prev_dat_compl	SDMMC_RCG_WAIT_PREVDAT_COMPL_OFF
Default distribution	wait_prev_dat_compl is not set.	
Attribute ID	BLK_BYT_MODE	
Description	Decides if the new SDIO extended read/write commands(cmd53) will be in block or byte mode.	
Type	Integer	
Integer macros	read/write commands in block mode	SDMMC_RCG_BLOCK_MODE
	read/write commands in byte mode	SDMMC_RCG_BYTE_MODE
Default distribution	Block mode is given more weightage than byte mode(7:3)	
Attribute ID	LDIRQ_COUNT	
Description	Decides after how many clocks the send_irg bit (in the control register of DWC_mobile_storage) will be set after CMD40 is loaded in the DWC_mobile_storage.	
Type	Integer	
Integer macro	SDMMC_RCG_LDIRQ_COUNT	
Value range	>= 0	
Default distribution		
Attribute ID	SECTOR_COUNT	
Description	Sector count value to be programmed into the CEATA device through the taskfile.	
Type	Integer	
Integer macro	CEATA_RCG_SECTOR_COUNT	
Value range	>= 1	
Default distribution	1 – One sector count.	

Table 8-1 DWC_mobile_storage Randomization Attributes

Attribute ID	CEATA_INTERRUPT_BIT
Description	The value to be programmed into the nIEN bit of the control register inside the CEATA device.
Type	Integer
Integer macro	CEATA_RCG_INTERRUPT_BIT
Value range	0 to 1
Default distribution	0 – Enable the interrupt bit.
Attribute ID	SEND_CCSD
Description	Controls whether to send the CCSD pattern to the CEATA device while waiting for the CCS from the CEATA device.
Type	Integer
Integer macro	CEATA_RCG_SEND_CCSD
Value range	0 to 1
Default distribution	0 – Do not send CCSD
Attribute ID	SEND_AUTO_STOP
Description	Decides whether to send the internally generated auto stop after the CCSD is send to the CEATA device.
Type	Integer
Integer macro	CEATA_RCG_SEND_AUTO_STOP
Value range	0 to 1
Default distribution	0 – Do not send auto stop after CCSD
Attribute ID	CCS_TIMEOUT
Description	If set, the CEATA VIP is configured not to send CCS at the end of the data transmission
Type	Bit Vector
Integer macro	CEATA_RCG_CCS_TIMEOUT
Value range	0 to 1
Default distribution	0 – Disable the CCS Timeout

Table 8-1 DWC_mobile_storage Randomization Attributes

Attribute ID	BURST_ENABLED
Description	If set the AHB Burst transactions are enabled else only single transfers.
Type	Bit Vector
Integer macro	CEATA_RCG_BURST_ENABLED
Value range	0 to 1
Default distribution	0 – Disable the Burst
Attribute ID	BURST_TRANSFER_SIZE
Description	Determines the HSIZE of the AHB transaction in a given burst.
Type	Bit Vector
Integer macro	CEATA_RCG_BURST_TRANSFER_SIZE
Value range	Transfer Sizes of 8, 16, 32, 64
Default distribution	16
Attribute ID	NUM_BEATS
Description	Determines the Number of Beats in a given AHB Burst.
Type	Bit Vector
Integer macro	CEATA-RCG_NUM_BEATS
Value range	Number of Beats equal to 4, 8, 16
Default distribution	4
Attribute ID	SKIP_LENGTH
Description	Determines the number of locations to be skipped to get the next descriptor address which is used by internal DMA controller.
Type	Bit Vector
Integer macro	SDMMC_RCG_SKIP_LENGTH
Value range	0 to 31
Default distribution	0

Table 8-1 DWC_mobile_storage Randomization Attributes

Attribute ID	DMA_BUFFER_SIZE
Description	Determines the Buffer Size for the DMA transfers.
Type	Bit Vector
Integer macro	SDMMC_RCG_DMA_BUFFER_SIZE
Value range	64, 128, 256, 512, 1024
Default distribution	512
Attribute ID	DMA_SUSPEND_ENABLE
Description	If set, the own bit of the descriptor is corrupted.
Type	Bit Vector
Integer macro	SDMMC_RCG_DMA_SUSPEND_ENABLE
Value range	0 to 1
Default distribution	0
Attribute ID	ABORT_BOOT_ACK
Description	If set, enables the environment to issue abort command (CMD12) to abort the boot acknowledgement from MMC4.3 card.
Type	Bit Vector
Integer macro	SDMMC_RCG_ABORT_BOOT_ACK
Value range	0 to 1
Default distribution	0
Attribute ID	ABORT_BOOT_DATA
Description	If set, enables the environment to issue abort cmd (CMD12) to abort the boot data from MMC4.3 card.
Type	Bit Vector
Integer macro	SDMMC_RCG_ABORT_BOOT_DATA
Value range	0 to 1
Default distribution	0

Table 8-1 DWC_mobile_storage Randomization Attributes

Attribute ID	SPEED_MODE
Description	Defines the speed mode selected for SD3.0 card.
Type	Integer
Integer macro	SDMMC_RCG_SPEED_MODE
Value range	<i>default_speed, high_speed, sdr50_speed, sdr104_speed, ddr50_speed</i>
Default distribution	<i>default_speed</i>

8.9.9.2 Verification Constraint Limitations

The following are verification environment constraint limitations:

1. If a host-initiated abort or suspend is to be issued, weightage for “wait for previous data complete” should be set to OFF completely.
2. If auto-stop is set for a data transfer command, a host-initiated abort should not be issued in a way that causes it to occur close to an auto-stop generated by the DWC_mobile_storage.
3. The following commands are not issued by the rcg, even if independent weightages are set for them, for following reasons:
 - ◆ CMD12, CMD16, CMD55, ACMD6, DMA_RESET, CONTR_RESET are issued in a directed manner. Issuing them randomly requires more of verification environment space and does not particularly help in DWC_mobile_storage testing.
 - ◆ CMD42 is equivalent to a write command in DWC_mobile_storage testing.
 - ◆ CMD56 is not supported by rcg, since it is similar to a single-block write command.
4. The Rx and Tx FIFO threshold constraints should be set so that:
 - ◆ Tx threshold value can range from 1 to $fifo_depth/2$
 - ◆ Rx threshold value ranges from $fifo_depth/2$ to $(fifo_depth-1)$
5. Host initiated aborts, and suspends should not be set together.
6. The response-time range set for card BFMs should be less than the response-time out set in the DWC_mobile_storage for that card. Similarly, the data-access time range set for card BFMs should be less than the data-access time out set in the DWC_mobile_storage for that card.
7. Block sizes should not be set to less than 25.
8. CMD53 read and write data transfers can be suspended and then resumed. Suspend and resume for CMD18 and CMD25 are supported, but they are not thoroughly tested with regard to the verification environment.
9. Random verification environment does not support configurations that have a Generic DMA interface.

8.10 Generic DMA BFM

You can use the generic DMA BFM to verify the generic DWC_mobile_storage DMA interface. The generic DMA BFM:

- ❖ Is a Vera class that can be instantiated in a Vera testbench
- ❖ Supports generic DMA data widths of 16, 32, and 64 bits
- ❖ Performs generic DMA read and write cycles
- ❖ Checks for a `ge_dma_done` pulse after data transfer completes
- ❖ Checks the `ge_dma_card_num` signal

You can use the following commands of the generic DMA BFM through a Vera testbench.

8.10.1 `checkDmaDone`

Checks for `ge_dma_done` pulse of one clock duration.

Syntax

```
checkDmaDone(timeoutValue);
```

Arguments

<i>timeoutValue</i>	An integer that specifies the timeout value in terms of clocks for <code>ge_dma_done</code> checking. After the timeout count, an error message is output.
---------------------	--

Description

The `checkDmaDone` command checks for a `ge_dma_done` pulse of one clock duration. This command should be called after the data transfer completes.

Example

```
ge_dma.geDmaDone(100);
```

8.10.2 `ChkGeDMAErrCnt`

Generates error if error count is non-zero.

Syntax

```
ChkGeDMAErrCnt();
```

Arguments

None

Description

The `ChkGeDMAErrCnt` command generates an error if the error count is non-zero. This command should be called at the end of the test case.

Example

```
ge_dma.ChkGeDMAErrCnt();
```

8.10.3 GeDmaRead

Performs generic DMA read cycle during write to card.

Syntax

```
GeDmaRead(byte_count, card_num, req_ack_delay, ack_assert_count, req_timeout);
```

Arguments

<i>byte_count</i>	An integer that specifies the number of bytes to be transferred during a DMA read cycle.
<i>card_num</i>	A 4-bit vector that specifies the expected card number on the <code>ge_dma_card_num</code> bus.
<i>req_ack_delay</i>	An integer that specifies the delay between <code>ge_dma_req</code> assertion and <code>ge_dma_ack</code> assertion; default value is 0.
<i>ack_assert_count</i>	An integer that specifies the number of clocks that <code>ge_dma_ack</code> should be asserted in response to <code>ge_dma_req</code> being asserted. The default count is 0; that is, <code>ge_dma_req</code> de-asserts only when <code>ge_dma_req</code> becomes de-asserted.
<i>req_timeout</i>	An integer that specifies the timeout value of <code>ge_dma_req</code> sampling; default value is 10000 clocks.

Description

The `GeDmaRead` command performs a generic DMA read cycle while data is written to the card.

Example

```
ge_dma.geDmaRead(64, 2, 0, 0, 5000);
```

8.10.4 geDmaWrite

Performs generic DMA write cycle during read from card.

Syntax

```
geDmaWrite(byte_count, card_num, req_ack_delay, ack_assert_count, req_timeout);
```

Arguments

<i>byte_count</i>	An integer that specifies the number of bytes to be transferred during a DMA write cycle.
<i>card_num</i>	A 4-bit vector that specifies the expected card number on the <code>ge_dma_card_num</code> bus.
<i>req_ack_delay</i>	An integer that specifies the delay between <code>ge_dma_req</code> assertion and <code>ge_dma_ack</code> assertion; default value is 0.
<i>ack_assert_count</i>	An integer that specifies the number of clocks <code>ge_dma_ack</code> should be asserted in response to assertion of <code>ge_dma_req</code> . The default count is 0; that is, <code>ge_dma_req</code> de-asserts only when <code>ge_dma_req</code> becomes de-asserted.

req_timeout

An integer that specifies the timeout value of `ge_dma_req` sampling; default value is 10000 clocks.

Description

The `geDmaWrite` command performs a generic DMA write cycle while data is read from the card.

Example

```
ge_dma.geDmaWrite(64, 2, 0, 0, 5000);
```

8.10.5 New

Instantiates `GeDma` class similar to normal `Vera` class.

Syntax

```
New(GeDma_port_instance_handle, error_count, log_file_handle);
```

Arguments

<i>GeDma_port_instance_handle</i>	An integer of value <code>TestGenDmaPort</code> that specifies the instance handle of the generic DMA test port.
<i>error_count</i>	An integer that specifies the exit error count.
<i>log_file_handle</i>	An integer that specifies the handle of the generic DMA logfile.

Description

The `New` command instantiates the `GeDma` class similar to the normal `Vera` class.

Example

```
Ge_dma=new(direct_test.m_genDmaPort, 100, LogFileHandle);
```

8.10.6 setConfigParam

Configures generic DMA data width.

Syntax

SetConfigParam(*ConfigParam*, *ParamValue*)

Arguments

<i>ConfigParam</i>	Configuration parameter of generic DMA. SDMMC_TEST_GE_DMA_DATA_WIDTH_PARAM
<i>ParamValue</i>	An integer that specifies a valid generic DMA data width, which can be 16, 32, or 64.

Description

The setConfigParam command configures the generic DMA data width.

Example

```
ge_dma.setConfigParam(SDMMC_TEST_GE_DMA_DATA_WIDTH_PARAM, 32);
```

8.10.7 Resetting the generic DMA BFM

In order to reset the generic DMA BFM, do the following:

1. Reset the DMA interface of the DWC_mobile_storage.
2. Set the m_term_gedma_write flag from the testbench.

8.10.8 Data Buffer

The generic DMA BFM has an 8-bit data buffer implemented as an 8-bit associative array. You can copy the data into the buffer while writing to the card, and you can read the data while reading from the card. Use m_data_buffer directly from the testbench for accessing the data buffer.

9

DWC_mobile_storage Design Integration Requirements

This chapter provides information on instantiating and integrating the DWC_mobile_storage into a design.

9.1 Instantiating DWC_mobile_storage

Once you have configured, tested, and synthesized the DWC_mobile_storage with coreConsultant, you can integrate the output files from the coreKit into your design environment.

- ❖ The RTL source files at *workspace/src* – The top-level file is *DWC_mobile_storage.v*, an unencrypted Verilog file; for pin details and RTL hierarchy, refer to “[Functional Description](#)” on page 43.
- ❖ Vera files – For the verification environment, an AHB monitor and BFM, and SD_MMC_CEATA BFM are shipped with the DWC_mobile_storage; the object and include files are in *workspace/sim/testbench/designware_home* and *workspace/sim/testbench/installed_vip* directories. For more details, refer to “[Verification](#)” on page 235.

If you run synthesis through the coreKit, the report and db files are in the *workspace/syn/final* directory. You can view the synthesis scripts in the *syn/constrain/script* directory.

Depending on the configuration that you choose, you can instantiate the DWC_mobile_storage in the next higher-level design. [Example 9-1](#) shows an instantiation with the following main configurations:

```
32-bit AHB Bus interface
No-DMA support,
SD_MMC controller
3 cards support
```

Example 9-1 32-bit AHB, No-DMA, SD_MMC, 3-Cards

```
DWC_mobile_storage dut(
    .clk (clk),
    .reset_n (reset_n),
    .biu_volt_reg (biu_volt_reg),
    .haddr (haddr),
    .hsel (hsel),
    .hwrite (hwrite),
    .htrans (htrans),
    .hsize (hsize),
    .hburst (hburst),
    .hready (hready),
```

```
.hwd_data (hwd_data),
.hr_data (hr_data ),
.hready_resp (hready_resp ),
.hresp (hresp ),
.hbig_endian (hbig_endian),
.int (int ),
.raw_ints (raw_ints ),
.int_mask_n (int_mask_n ),
.int_enable (int_enable ),
.cclk_in (cclk_in),
.cclk_in_drv (cclk_in_drv),
.cclk_in_sample (cclk_in_sample),
.card_detect_n (card_detect_n),
.card_write_prt (card_write_prt),
.cdata_in (cdata_in),
.cdata_out (cdata_out ),
.cdata_out_en (cdata_out_en ),
.ccmd_in (ccmd_in),
.ccmd_out (ccmd_out ),
.ccmd_out_en (ccmd_out_en ),
.ccmd_od_pullup_en_n (ccmd_od_pullup_en_n ),
.card_power_en (card_power_en ),
.card_volt_a (card_volt_a ),
.card_volt_b (card_volt_b ),
.cclk_out (cclk_out ),
.scan_mode (scan_mode),
.gp_in (gp_in),
.gp_out (gp_out ),
.debug_status (debug_status )
);
```

Figure 9-1 illustrates a typical AMBA AHB-based system environment using the DWC_mobile_storage in SD_MMC_CE-ATA mode.

Figure 9-1 DWC_mobile_storage Integrated in SD_MMC_CE-ATA mode in a Typical AMBA AHB System

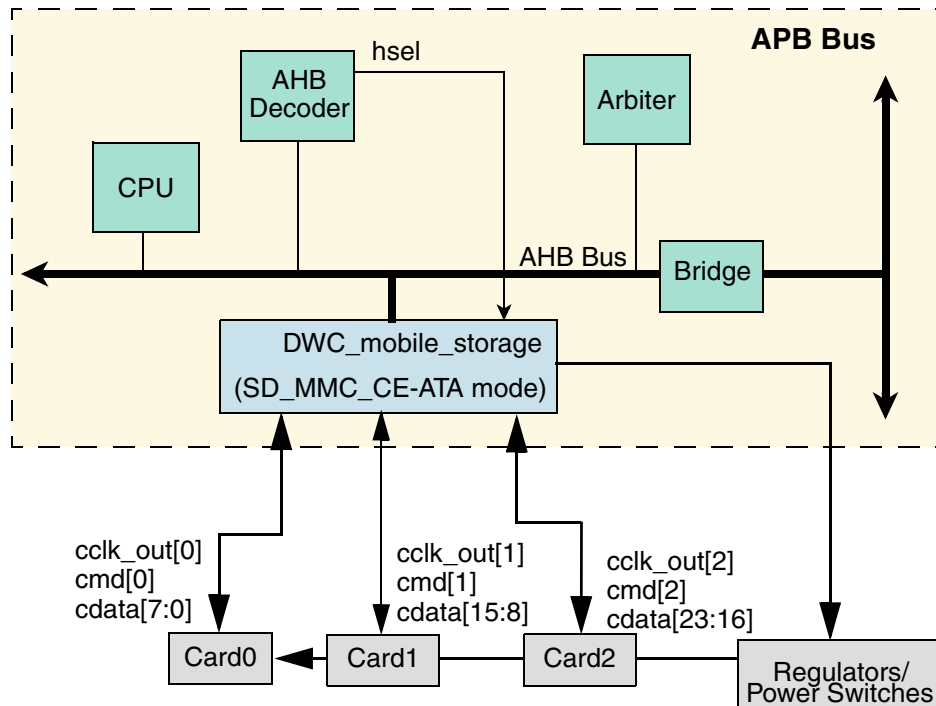
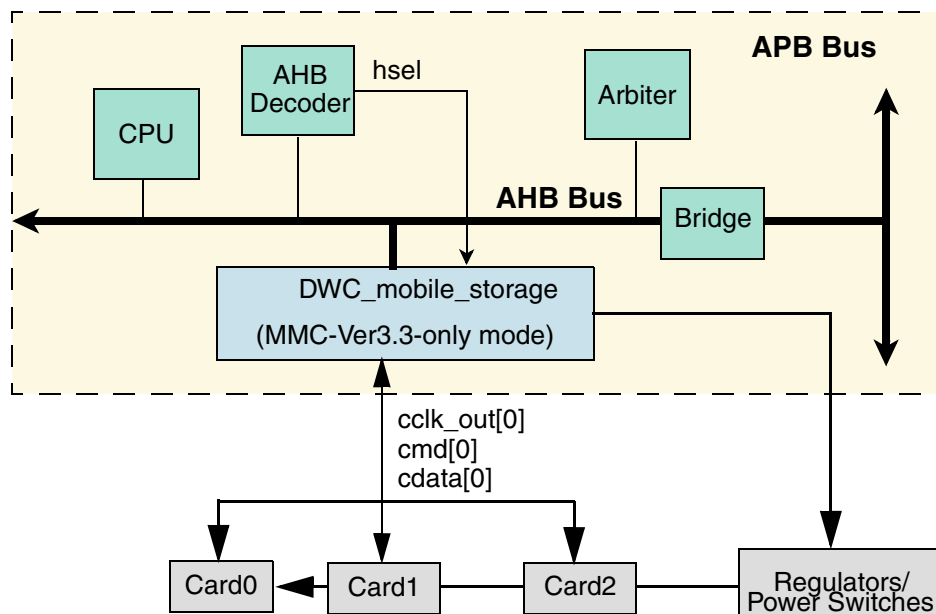


Figure 9-2 shows the DWC_mobile_storage instantiated in MMC-Ver3.3-only mode (MMC 3.31 cards); the data bus is 1-bit wide, and the bus is configured in a bus topology.

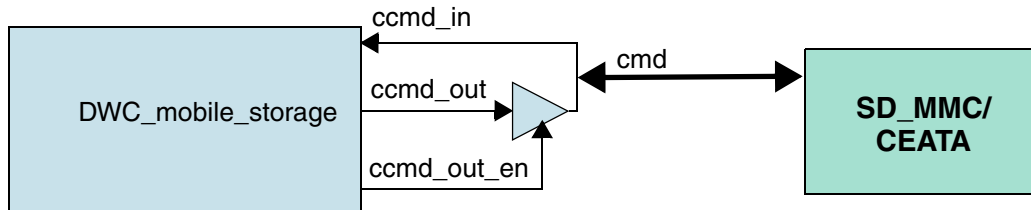
Figure 9-2 DWC_mobile_storage in MMC-Ver3.3-Only Integration – Typical AHB System



9.2 Bi-directional Buses

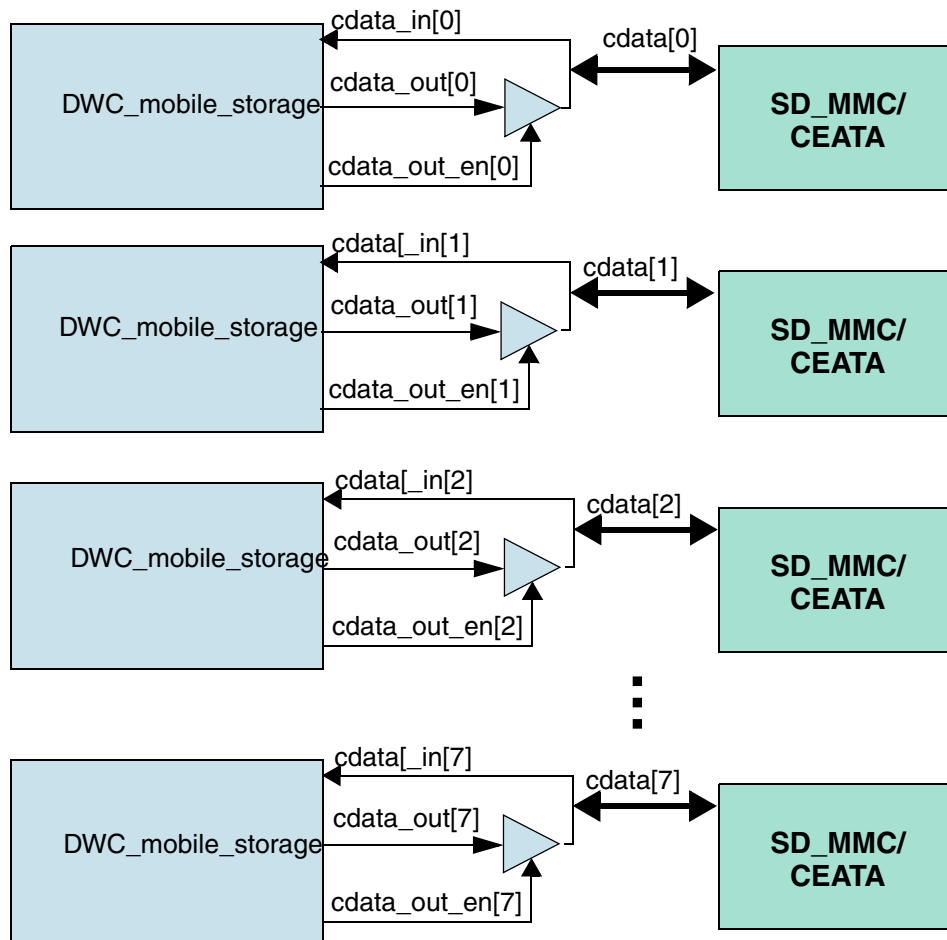
The DWC_mobile_storage provides only uni-directional ports. It provides output-enable signals for connecting uni-directional ports to bi-directional ports. Figure 9-3 shows the connection between the uni-directional command out and the command bus to the bi-directional card command bus.

Figure 9-3 Bi-directional Command Pin Connection



Each **cdata_out** bit has a corresponding **cdata_out_en** enable bit. A separate enable bit for each data bit is provided in order to support 1-bit or 4-bit data modes and SDIO interrupts. Figure 9-4 shows the connection between the uni-directional **cdata_out** and **cdata_in** to the bi-directional card data bus.

Figure 9-4 Bi-directional Data Pin Connection



9.3 System Reset

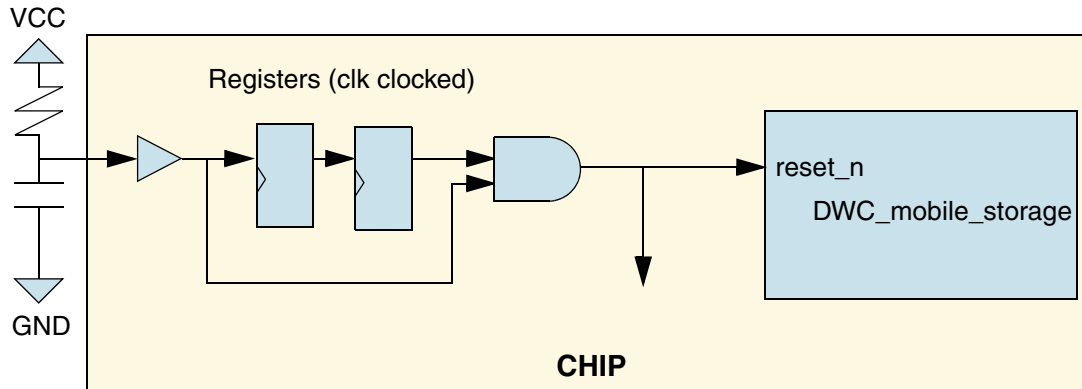
In RTL, all the “clocked-always blocks” are coded for asynchronous reset, as shown in the following code – the reset is an active-low signal.

```
always @ (posedge clk or negedge reset_n)
begin
    if (~reset_n)
    begin
        .....
    end
else
    begin
        .....
    end
end
end
```

In order to meet timing requirements and correct operation, the reset_n signal must be synchronously de-asserted with regard to the clk signal and can be asynchronously asserted. Asynchronous reset assertion guarantees that all signals and drivers are in a safe state during power-up when the clocks are typically not active or are unstable. The synchronous reset de-assertion guarantees that there is no timing or metastability violations.

Figure 9-5 illustrates a typical system reset interface.

Figure 9-5 Typical System Reset Interface

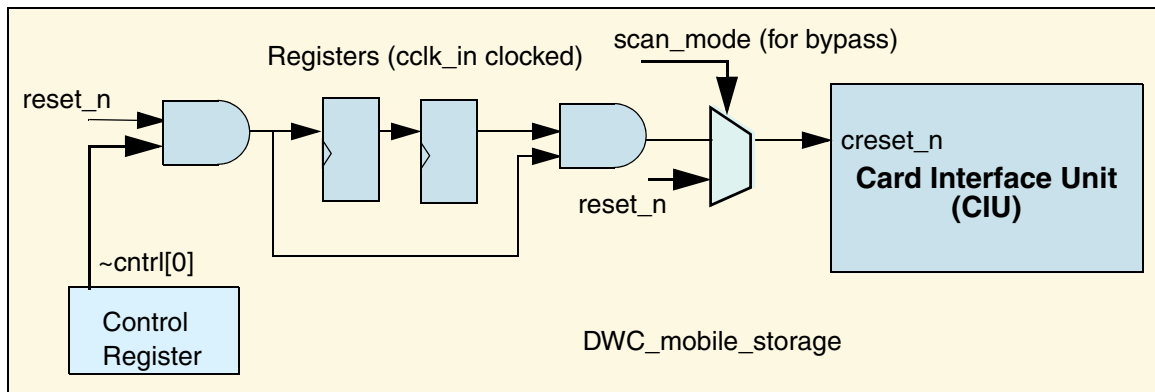


Since the card-interface unit works on a different clock than the host-bus interface, the host reset is internally double-synchronized by the card clock (cclk_in) to create the card domain reset. Since firmware can also reset the card-interface unit by programming the control register, the control register output is ANDed with the host reset (reset_n) before being double-synchronized.

When controlling the card-reset bit of the control register, firmware must ensure that the card reset is active for more than two cclk_in clocks. Since there are two cclk_in delays between when reset_n de-asserts and when reset_n de-asserts, you should not attempt any transactions to the SD_MMC_CEATA cards for at least two cclk_in clocks after reset_n de-asserts.

Figure 9-6 illustrates the card-interface unit reset.

Figure 9-6 Internal Card-Interface Unit reset



9.4 Clock Requirements and Recommendations

There are several clock requirements and recommendations when interfacing the DWC_mobile_storage with cards.

9.4.1 Clock Requirements

DWC_mobile_storage uses the following clocks to achieve a reliable communication with interfaced cards:

- ❖ **cclk_in** – Clocks logic in CIU clock domain.
- ❖ **cclk_in_drv** – Has the same frequency as **cclk_in**, but is phase shifted. Satisfies the minimum hold time requirement of 5 ns at the input to the cards while operating in SDR12 or identification modes.
- ❖ **cclk_in_sample** – Has the same frequency as **cclk_in**, but is phase-shifted. Provides a “best sampling window” for DWC_mobile_storage while reading data from the card.

9.4.2 Clock Generation Recommendations

The following are recommendations for clock generation:

- ❖ The **cclk_in** frequency input to DWC_mobile_storage can be switched, depending on the data rate.
 - ◆ If the interfaced card is communicating in SDR50 mode, then **cclk_in** = 50 Mhz.
 - ◆ If the interfaced card is communicating in SDR104 mode, then **cclk_in** = 200 Mhz.



Note

You should ensure that there are no glitches in the clock inputs while switching clock frequencies. The **cclk_in_drv** and **cclk_in_sample** clock frequencies must switch in relation to **cclk_in** frequencies.

- ❖ The **cclk_in_drv** and **cclk_in_sample** clocks are phase-shifted versions of **cclk_in**.

The value of the phase shift is selectable, based on the enumerated data rate. The phase shift can have a resolution of 90 degrees with respect to the **cclk_in** clock period.



Note

Minimum phase-shift resolution of 90 degrees is not compulsory. The more the resolution, the better it is.

You should instantiate the above structure described outside the `DWC_mobile_storage`. The clock speed for `cclk_in` and the phase shifts for `cclk_in_drv` and `cclk_in_sample` should be changed according to the mode of operation—that is, the different data rates—chosen at the end of enumeration.

The system software must perform the frequency switch based on the enumerated data speeds.

9.4.2.1 Simulation-Based Analysis for Timing and Clocking

The following assumptions pertain to the timing and clocking recommendations:

- ❖ The `DWC_mobile_storage` in the design is instantiated with one port.
- ❖ The following delays are assumed:
 - ◆ Host-controller output path:
 - ◇ $\text{Delay_O} = \text{cclk_in to cclk_out delay} = 1.4\text{ns}$ (Typical value)
 - ◆ Host-controller input path:
 - ◇ $\text{tODLY} = \text{cclk_out to cdata_in delay} = \text{max value}$ (specified in [Table 9-4](#))
 - ◇ $\text{Delay_I} = \text{IO_pad delay} + \text{routing delays within host controller} = 2.35\text{ ns}$ (typical value)
 - ◇ $\text{Delay_S} = \text{Delay_O} + \text{Delay_I} = 3.75\text{ ns}$
 - ◇ $\text{Delay}_{\text{total turnaround}}$ between Host Controller output and input = $\text{tODLY} + \text{Delay_S}$ ($\text{Delay_O} + \text{Delay_I}$)



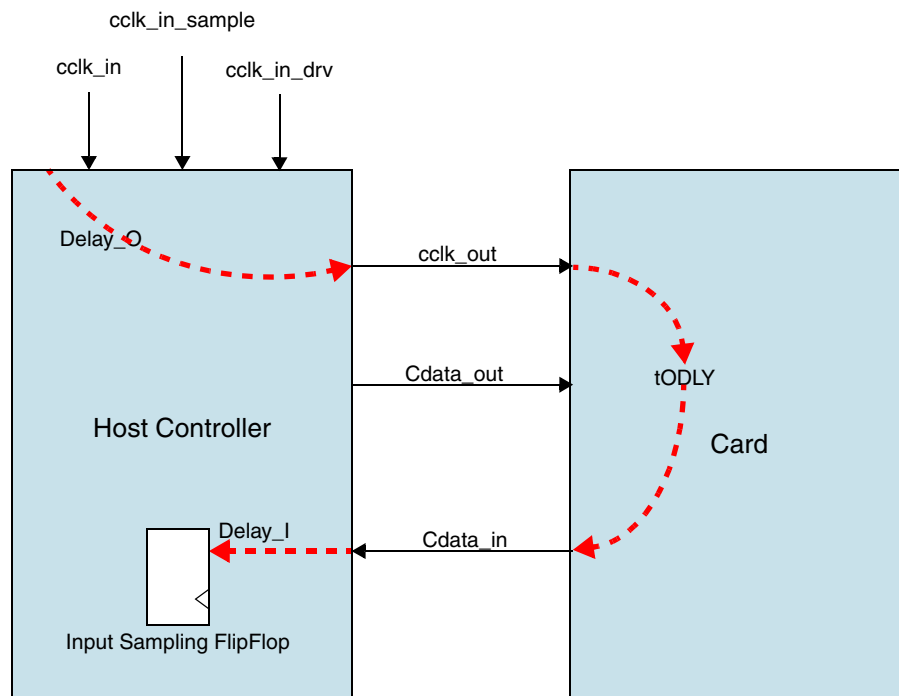
Note

Setup and hold times have been computed from exact `Delay_O`, `Delay_I` and `tODLY` values.

- ◆ Synthesis is done in 65nm; setup and hold times for the flip-flops within the host controller are assumed to be 1 ns.

Figure 9-16 illustrates how the host controller and card delays relate to each other.

Figure 9-7 Host Controller and Card Delays

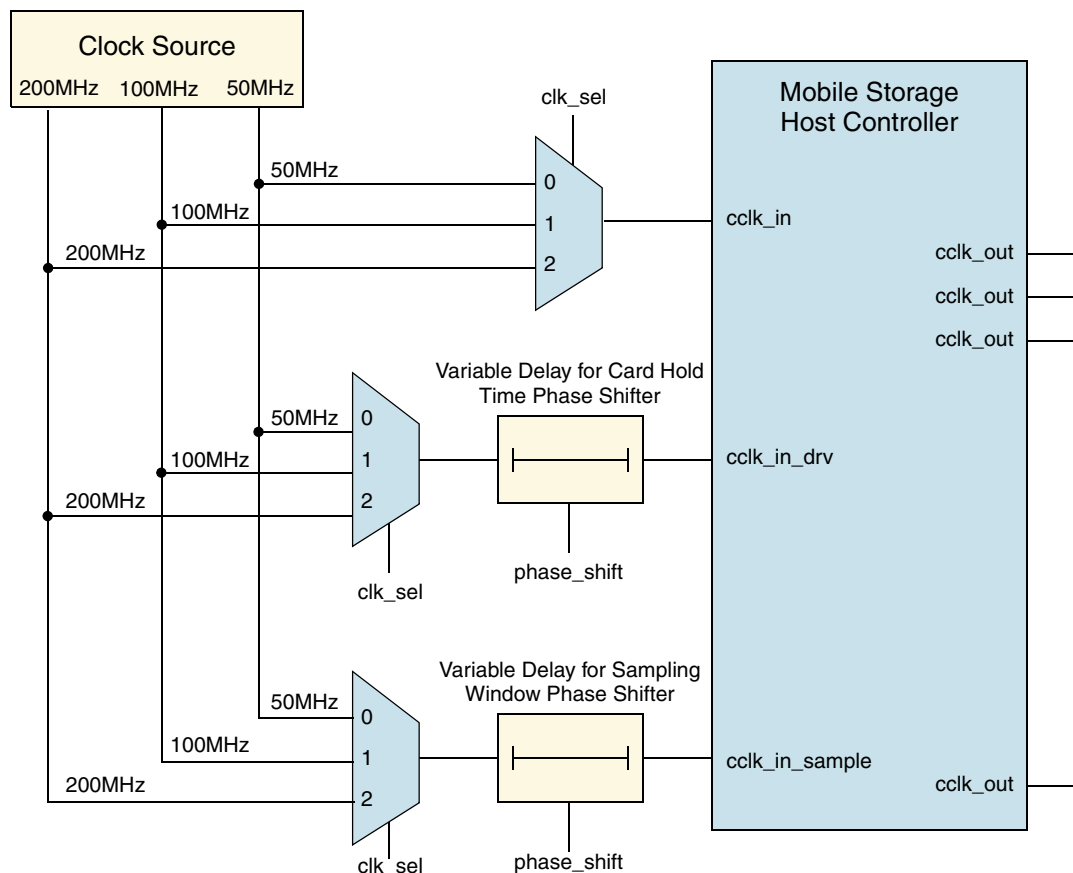


Clock frequency set at 50MHz or 200 MHz, depending on card type

9.5 Card Clock Input Structure

Figure 9-8 illustrates the DWC_mobile_storage clock structure for the Host controller and shows how the `cclk_in`, `cclk_in_drv`, and `cclk_in_sample` card clock input signals should be supplied.

Figure 9-8 Clock Structure for Host Controller



Note

- A single clock source is recommended.
- Figure 9-8 shows maximum required clocks.
- Clock selection is based on bus speed mode of card.
- Variable delay must be implemented between `cclk_in` and `cclk_in_sample`.
- Variable delay must be implemented between `cclk_in` and `cclk_in_drv`.

9.5.1 Clock Selection

The `clk_sel` input must be controlled by the Host Application Layer and be used to select the `cclk_in` and `cclk_in_sample` frequency supplied to the Host Controller. The value of `clk_sel` is based on the speed modes listed in [Table 9-1](#).

Table 9-1 `clk_sel` Values Based on Speed Modes

Speed Mode	<code>clk_sel</code>
DDR50, SDR12 and SDR25	2'b00
DDR50 (8 Bit Mode)	2'b01
SDR104 and SDR50	2'b10

9.5.2 Variable Delay Implementation

Variable Delay Implementation for the sampling path clock is required to achieve the best sampling window for data. Depending on the speed mode, the maximum output delay from the card can be up to 2 UI. In order to sample the data correctly, the best sampling window must be found using CMD19 tuning, and the application must use the tuning information to adjust the delay between `cclk_in` and `cclk_in_sample`; the delay can be achieved using a phase shifter with a minimum phase-shift resolution of 90 degrees. The value of the `phase_shift` input must be provided by the Host Controller Application based on CMD19 tuning.

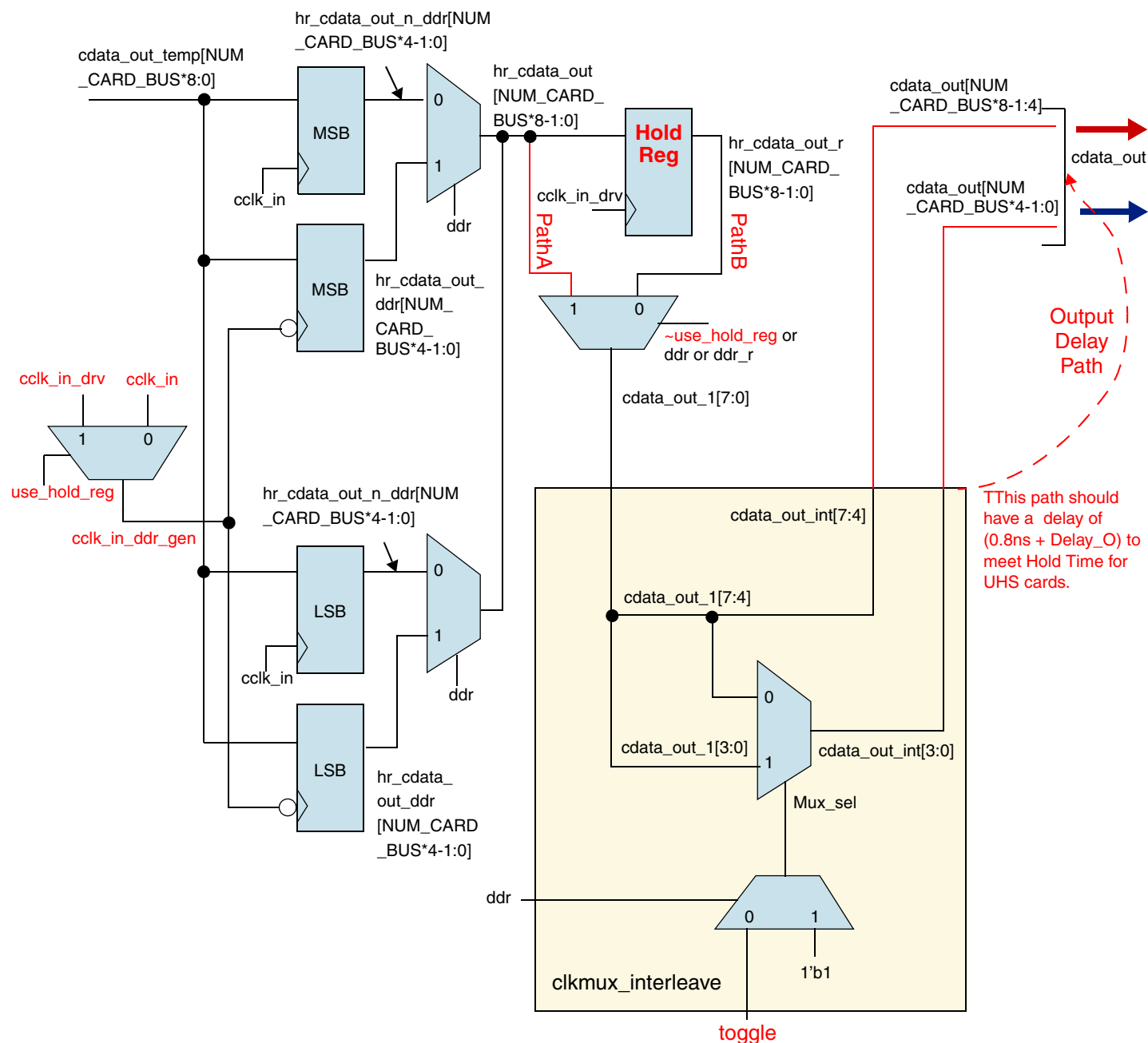
**Note**

Minimum phase-shift resolution of 90 degrees is not compulsory. The more the resolution, the better it is.

9.6 Host Controller Output Path Timing

Figure 9-9 illustrates the DWC_mobile_storage clock connection.

Figure 9-9 Host Controller Output Path Structure



The SDR12 and SDR25 speed modes have a relatively higher Input Hold Time requirement of 5ns and 2ns, respectively, whereas the higher speed modes for SDR104, SDR50 and DDR50 have a much smaller Input Hold Time requirement of 0.8ns.

To meet the relatively high Input Hold Time requirement for SDR12 and SDR25 speed modes, you should program bit[29]use_hold_Reg of the CMD register to 1'b1; the output data is then registered again in the cclk_in_drv domain by using the Hold Register as shown in Path B of Figure 9-9. However, for the higher

speed modes of SDR104, SDR50 and DDR50, you can meet the much smaller Input Hold Time requirement of 0.8ns by bypassing the Hold Register (Path A in [Figure 9-9](#), programming `CMD.use_hold_reg = 1'b0`) and then adding delay elements on the output path as indicated.

SDR104, SDR50, and DDR50 are usually able to meet their hold time of 0.8ns with the minimum delay being set between the `cdata_out_int` and the `cdata_out`. However, if more HOLD time is required, `CMD.use_hold_reg` should be set to `1'b1` and the `cclk_in_drv` clock should be phase shifted, just as with SDR12 and SDR25. The total hold time becomes (`Delay_O` + 0.8ns + phase shift of `cclk_in_drv`).

**Note**

The `set_min_delay` constraint must be set on the following path as shown below :

```
set_output_delay -min -0.8 -clock vcclk_out [get_ports cdata_out]
set_output_delay -min -0.8 -clock vcclk_out [get_ports cdata_out_en]
set_output_delay -min -0.8 -clock vcclk_out [get_ports ccmd_out]
set_output_delay -min -0.8 -clock vcclk_out [get_ports ccmd_out_en]
```

A minimum hold time of 0.8ns + `Delay_O` is required for SDR50, SDR104, and DDR50 speed modes; note that `vcclk_out` is a virtual clock imitating `cclk_out`.

9.6.1 Recommended Usage

Assumptions:

- ❖ Path delay of (0.8ns+`Delay_O`) between `cdata_out_int` and `cdata_out`
- ❖ `cclk_in_drv` phase shifted with respect to `cclk_in`

Table 9-2 Host Controller Output Path Timing

No.	Speed Mode	CMD.use_hold_reg	cclk_in (MHz)	cclk_in_drv (MHz)	clk_divider	cclk_in to cclk_in_drv delay to be used	Effective Minimum Hold Time at card input
1	SDR104	1'b0	200	200	0	0 degrees	0.8ns
2	SDR104	1'b1	200	200	0	Tunable > 0	0.8 + phase shift on cclk_in_drv
3	SDR50	1'b0	200	200	1	0 degrees	0.8ns
4	SDR50	1'b1	200	200	1	Tunable > 0	0.8 + phase shift on cclk_in_drv
5	DDR50 (8 bit)	1'b0	100	100	1	0 degrees	0.8ns
6	DDR50 (8 bit)	1'b1	100	100	1	Tunable > 0	0.8 + phase shift on cclk_in_drv
7	DDR50 (4 bit)	1'b0	50	50	0	0 degrees	0.8ns
8	DDR50 (4 bit)	1'b1	50	50	0	Tunable > 0	0.8 + phase shift on cclk_in_drv

Table 9-2 Host Controller Output Path Timing

No.	Speed Mode	CMD.use_hold_reg	cclk_in (MHz)	cclk_in_drv (MHz)	clk_divider	cclk_in to cclk_in_drv delay to be used	Effective Minimum Hold Time at card input
9	SDR25	1'b1	50	50	0	Tunable > 0 (should meet at least 4.2ns)	0.8 + phase shift on cclk_in_drv
10	SDR12	1'b1	50	50	1	Tunable > 0 (should meet at least 4.2ns)	0.8 + phase shift on cclk_in_drv

**Note**

The effective hold time is computed by considering that the correct Delay_O value is used while setting the Path delay of (0.8ns+Delay_O) between cdata_out_int and cdata_out.

**Attention**

Never set CMD.use_hold_reg = 1 and cclk_in_drv phase shift to 0 at the same time.

9.7 FIFO Size Requirements

The delay on CMD and Data Output from a card (tODLY) can vary from 0 to 2 Unit Intervals (UI) — where 1 UI equals one card clock period. Added to the tODLY, there is an I/O Pad delay at the Host Controller input.

Due to this variable delay, it is recommended that the Host Controller should not stop the clock during the data phase; that is, the card clock should be stopped only between blocks of data. This can be achieved by providing a sufficiently large FIFO to ensure that the FIFO does not become full during a Read Transfer; as a result, the Card Clock will not stop.

This requirement can be guaranteed by enabling the Card Read Threshold feature and by programming Card Read Threshold Size, RX_WMark, and MSIZE correctly; for details, refer to [“Card Read Threshold Programming Sequence”](#) on page 201.

It is recommended that the minimum FIFO size be equal to the largest block size that can be supported by the Host Controller. For example, if the largest block size that can be supported is 512 bytes and the FIFO width is 32 bits, then the minimum FIFO size is 128 locations.

**Note**

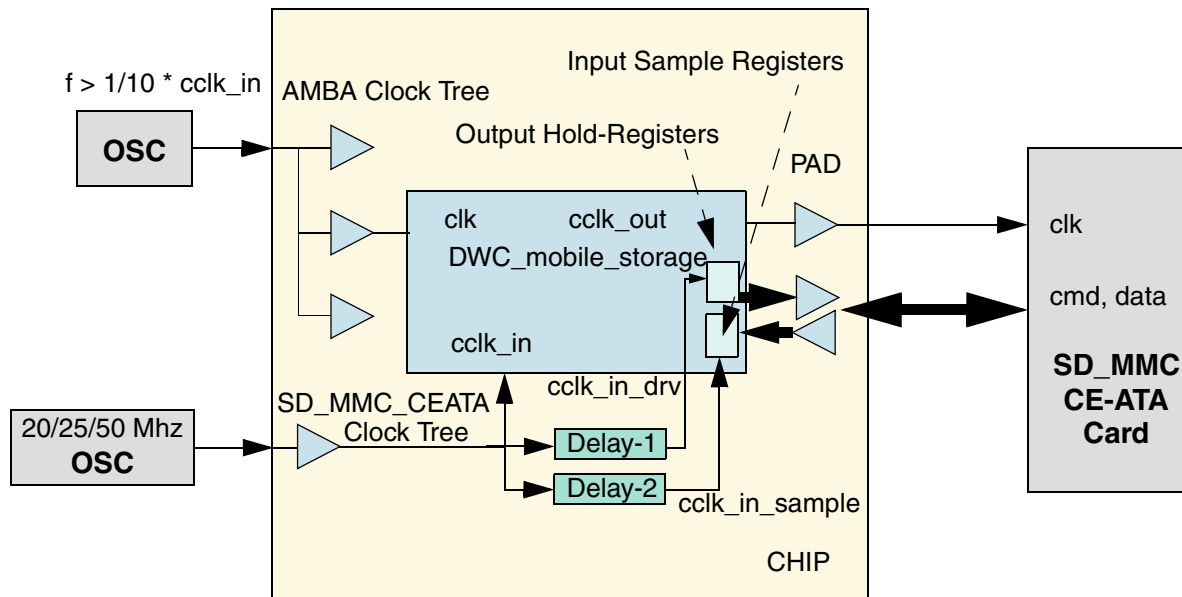
An important point to remember when selecting the FIFO Depth while configuring the DWC_mobile_storage Host Controller is that, in order to use the Card Read Threshold feature for different block sizes, the selected FIFO depth must be greater than or equal to the maximum block size that the Host Controller is required to support; this is a minimum requirement in order to use the feature.

Choosing a FIFO depth that is two or more times the maximum Block Size that the Host Controller is required to support gives greater performance and flexibility when choosing the Burst Size (MSIZE) and Rx Threshold (RX_WMARK) in order to get the best throughput. For more information, refer to [“Card Read Threshold”](#) on page 199.

9.8 System Clock Topology

Figure 9-10 illustrates the DWC_mobile_storage clock connection.

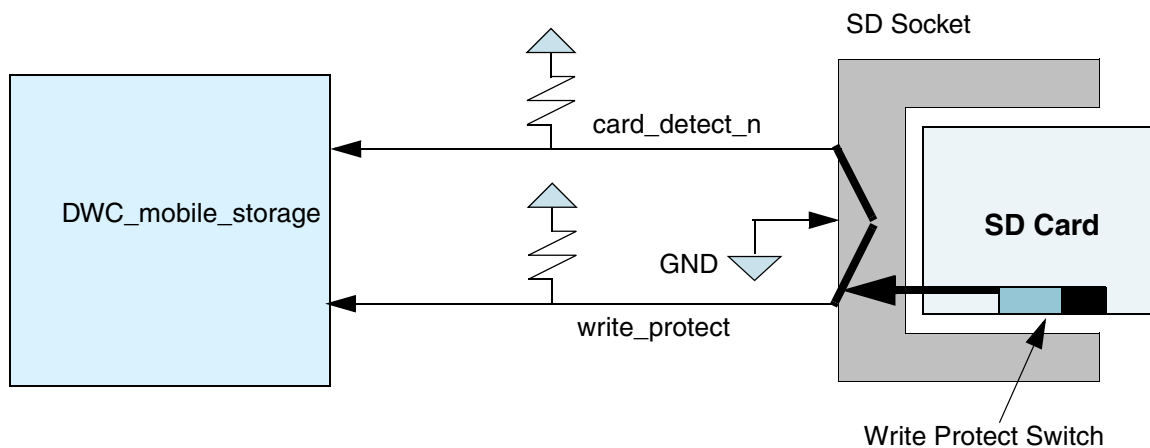
Figure 9-10 DWC_mobile_storage and SD_MMC_CEATA Card Clock Connection



9.9 Card-Detect and Write-Protect Mechanism

Figure 9-11 illustrates how the DWC_mobile_storage card detection and write-protect signals are connected. Most of the SD_MMC sockets have card-detect pins. When no card is present, `card_detect_n` is 1 due to the pull-up. When the SD_MMC card is inserted, the card-detect pin is shorted to ground, which makes `card_detect_n` go to 0. Similarly in SD cards, when the write-protect switch is toward the left, it shorts the write_protect port to ground.

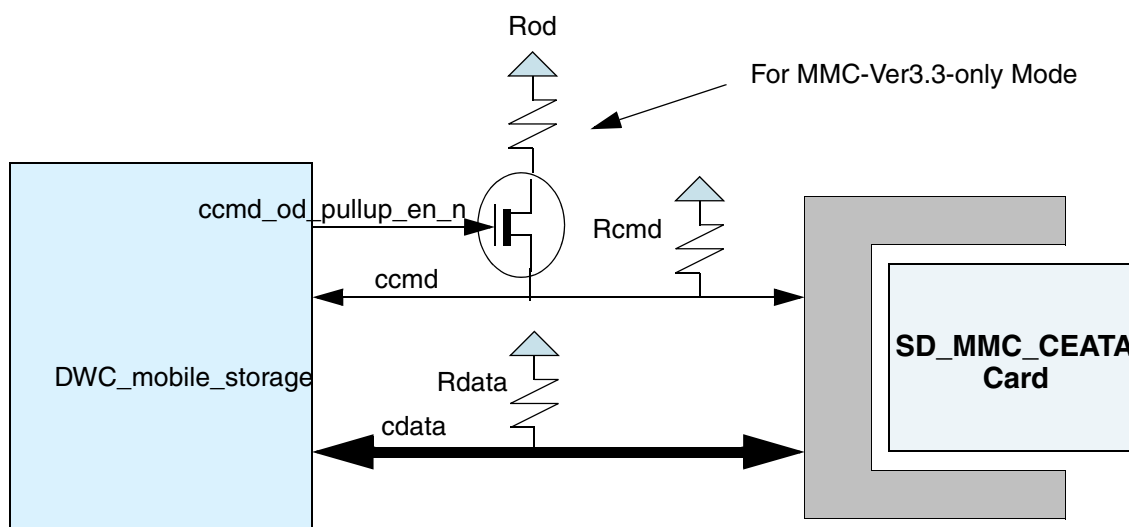
Figure 9-11 Card-Detect and Write-Protect



9.10 Termination Requirement

Figure 9-12 illustrates the DWC_mobile_storage termination requirements, which is required to pull up ccmd and cdata lines on the SD_MMC_CEATA bus. The recommended specification for pull-up on the ccmd line (Rcmd) is 4.7K - 100K for MMC, and 10K - 100K for an SD. The recommended pull-up on the cdata line (Rdata) is 50K - 100K. Additionally, in MMC-Ver3.3-only mode, a switchable open-drain pull-up (Rod) is recommended.

Figure 9-12 Termination



9.10.1 Rcmd and Rod Calculation

The SD and MMC card enumeration happens at a very low frequency – 100-400KHz. Since the MMC bus is a shared bus between multiple cards, during enumeration open-drive mode is used to avoid bus conflict. Cards that drive 0 win over cards that drive “z.” The pull-up in the command line pulls the bus to 1 when all cards drive “z.” MMC interrupt mode also uses the pull-up. During normal data transfer, the host chooses only one card and the card driver switches to push-pull mode.

In MMC-Ver3.3-only mode, since multiple cards share the same bus, the total capacitance can be in the order of 200-300pf. During enumeration, the pull-up should be able to pull the command (cmd) bus from “z” to 1 at the enumeration speed.

For example, if enumeration is done at 400KHz and the total bus capacitance is 200 pf, the pull-up needed during enumeration is:

$$\begin{aligned}
 2.2 RC &= \text{rise-time} = 1/400\text{KHz} \\
 R &= 1/(2.2 * C * 100\text{KHz}) \\
 &= 1/(2.2 \times 200 \times 10^{-12} \times 400 \times 10^3) \\
 &= 1/(17.6 \times 10^{-5}) \\
 &= 5.68\text{K}
 \end{aligned}$$

The Rod and Rcmd should be adjusted in such a way that the effective pull-up is at the maximum 5.68K during enumeration. If there are only a few cards in the bus, a fixed Rcmd resistor is sufficient and there is no need for an additional Rod pull-up during enumeration. You should also ensure the effective pull-up will not violate the I_{OL} rating of the drivers.

In SD mode, since each card has a separate bus, the capacitance is less, typically in the order of 20-30pf (host capacitance + card capacitance + trace + socket capacitance). For example, if enumeration is done at 400KHz and the total bus capacitance is 20pf, the pull-up needed during enumeration is:

$$\begin{aligned}
 2.2 RC &= \text{rise-time} = 1/400\text{KHz} \\
 R &= 1/(2.2 * C * 100\text{KHz}) \\
 &= 1/(2.2 \times 20 \times 10^{-12} \times 400 \times 10^3) \\
 &= 1/(1.76 \times 10^{-5}) \\
 &= 56.8\text{K}
 \end{aligned}$$

Therefore, a fixed 56.8K permanent Rcmd is sufficient in SD mode to enumerate the cards.

The driver of the DWC_mobile_storage on the “command” port needs to be only a push-pull driver. During enumeration, the DWC_mobile_storage emulates an open-drain driver by driving only a 0 or a “z” by controlling the ccmd_out and ccmd_out_en signals.

9.11 Interfacing to SD Memory, SDIO, and MMC Card

Figure 9-13 on page 281 illustrates the connection between the DWC_mobile_storage controller and the MMC (3.31) cards in MMC-Ver3.3-only mode, which supports only MMC (3.31) cards. Figure 9-14 on page 282 illustrates the connection between the DWC_mobile_storage and SD memory, SDIO, and MMC cards in SD_MMC_CE-ATA mode, which supports all three types of cards in the same controller. The primary differences between the MMC-Ver3.3-only and SD_MMC_CE-ATA modes are:

- ❖ In MMC-Ver3.3-only mode, one of the following can be used:
 - ◆ MMC (3.3) cards can be connected in a bus topology where only one data pin is used because cards are in 1-bit mode
 - ◆ Only one MMC (4.0) high speed MMC card can be connected where it operates in 1-bit, 4-bit, or 8-bit modes.
- ❖ In SD_MMC_CE-ATA mode, an SD card can be either a 1-bit data or 4-bit data card; MMC (3.31) cards are always 1-bit only, while MMC (4.0) cards could be either in 1-bit, 4-bit, or 8-bit mode.
- ❖ In MMC-Ver3.3-only mode, if multiple MMC (3.31) cards are connected, then the command, data, and clocks are bussed between all the cards and the DWC_mobile_storage controller.
- ❖ In SD_MMC_CE-ATA mode, each card has a separate command, data, and clock interface to the DWC_mobile_storage controller.

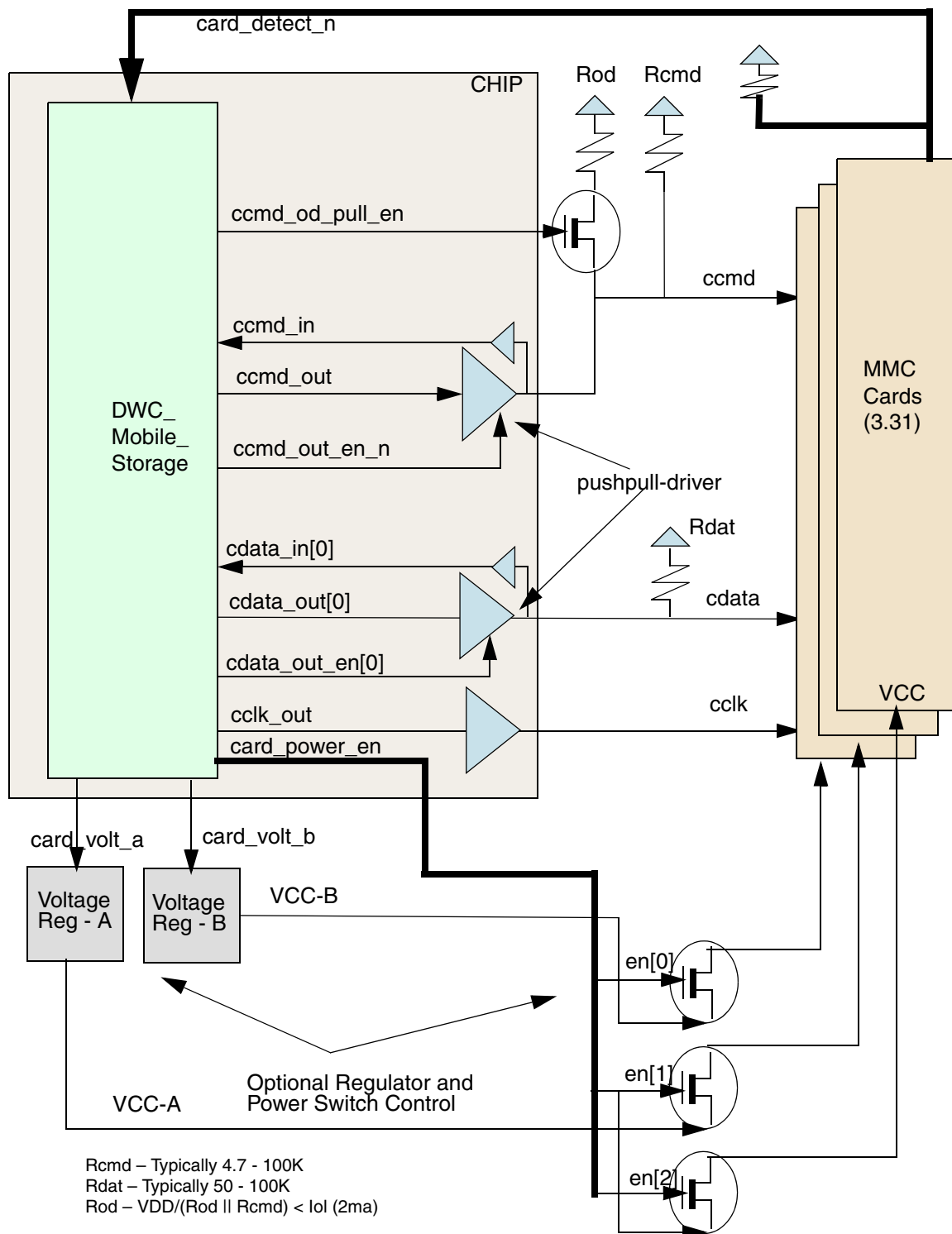
Figure 9-13 DWC_mobile_storage Controller/MMC (3.31) Card Connection – MMC-Ver3.3-only Mode

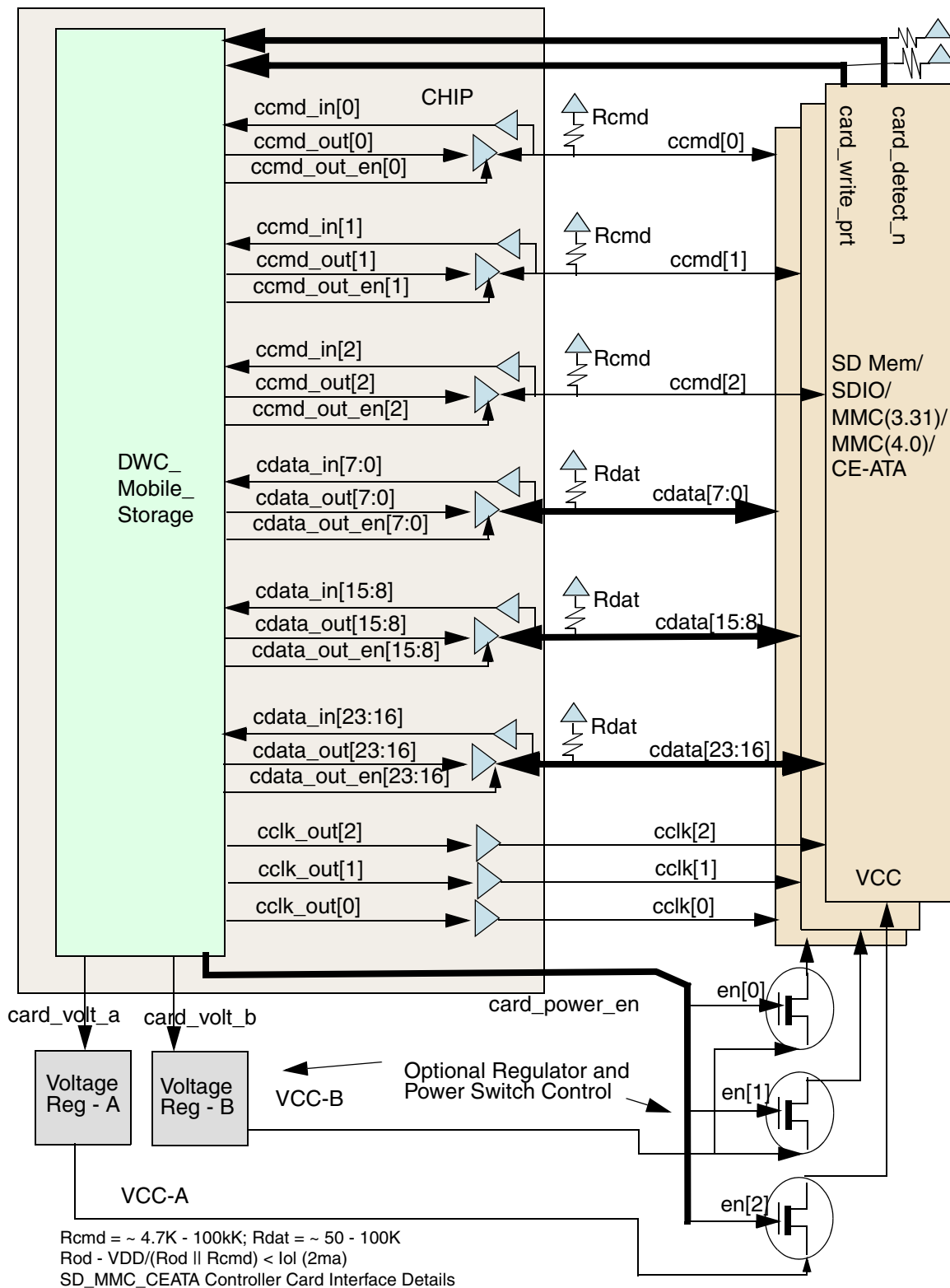
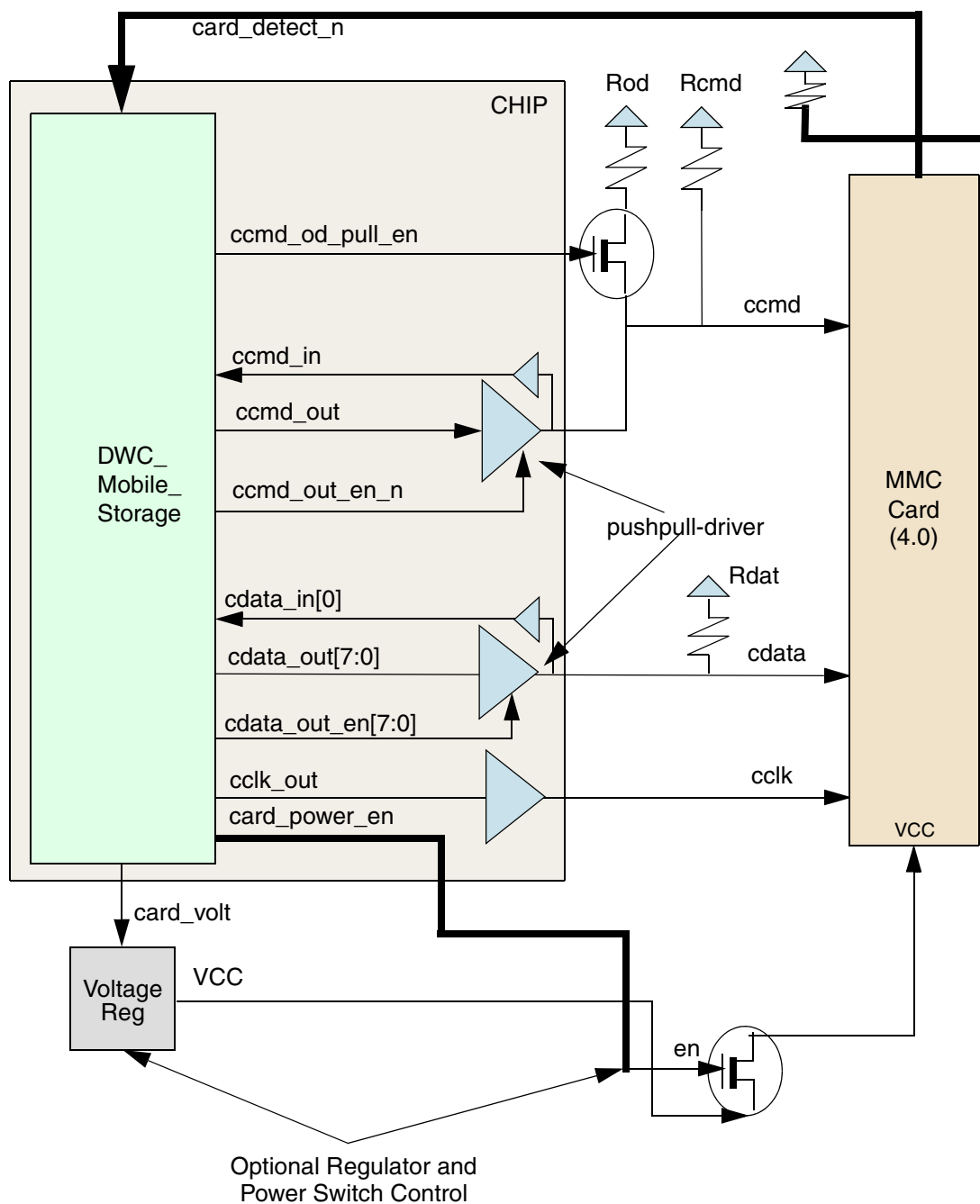
Figure 9-14 DWC_mobile_storage/SD Memory, SDIO, MMC (3.31), and MMC (4.0) Cards – SD_MMC_CE-ATA Mode

Figure 9-15 DWC_mobile_storage Controller/MMC (4.0) Single-Card Connection

Rcmd – Typically 4.7 - 100K
 Rdat – Typically 50 - 100K
 Rod – $VDD / (Rod \parallel Rcmd) < I_{OL} (2ma)$

9.12 Host Controller Clock Structure and Card Timing Requirements

An external clock multiplexor structure is needed to meet timing requirements across different card operating modes described in the SD Memory Card Specification, Version 3.0.

Phase shift values must be determined after place and route and then used for back annotation. This chapter describes the structure with typical delays of 65nm. I/O pad delays should be kept to a minimum in order to reduce stress on timing at both the host controller and card interface.

9.12.1 Timing Requirements for SD Cards

[Table 9-3](#) shows card input timing requirements that act as constraints for the Host Controller output path to a card.

Table 9-3 Card Input Timing Requirements

Mode	Maximum Frequency of Operation (MHz)	Hold Time (ns)	Setup Time (ns)
SDR104	200	0.8	1.4
SDR50	100	0.8	3.0
DDR50 (CMD line)	50	0.8	6.0
DDR50 (DAT line)	50	0.8	3.0
SDR25	50	2.0	6.0
SDR12	25	5.0	5.0
Identification Mode	400	5.0	5.0

[Table 9-4](#) shows card output timing requirements that act as constraints for the Host Controller input path from a card.

Table 9-4 Card Output Timing Requirements

Mode	Maximum Frequency of Operation (MHz)	Maximum Card Output Delay (ns)
SDR104	200	10.0
SDR50	100	7.5
DDR50 (CMD line)	50	13.7
DDR50 (DAT line)	50	7.0
SDR25	50	14.0
SDR12	25	14.0
Identification Mode	400	50.0



Note For SDR104, the SD Memory Card Specification requires 2UI as the maximum card delay before tuning. The post-tuning timing uncertainty—due to temperature variation—is specified as 1.9 ns (350 ps to +1550 ps).

where UI—Unit Interval is one bit nominal time, SDCLK nominal period

CMD19 is used for tuning in order to resolve this 2UI ambiguity, which is listed in [Table 9-4](#) as 10 ns; one card clock period (one UI) is equivalent to 5 ns. Since 2UI is a worse-case ambiguity compared to a 1.5 ns ambiguity after tuning, 10 ns (that is, 2UI) is used as the maximum card output delay in [Table 9-4](#).



Attention

If the user wants to use SDR104, then they must select the FIFO size based on the number of blocks to be used. If one block of data is to be read or written, then the FIFO size should be at least 512 bytes. If four blocks(512 bytes * 4) are to be used, then the user must select a FIFO depth of at least 2K bytes while configuring the DWC_mobile_storage.

In these circumstances, for SDR104 the maximum number of blocks that can be programmed in one transfer are:

- 64 when the AHB data width is selected as 64
- 32 when the AHB data width is selected as 32

This selection must be done during configuration.

9.12.2 Guidelines for Using Mobile Storage Host Controller in SDR104 Mode

According to the SD Memory Card Specification, Version 3.0, SDR104 mode supports Variable Output Delay on CMD and Data lines with a delay from 0 to 2 Unit Intervals (UI)—where 1 UI equals one card clock period. Due to this variable delay, it is recommended that the host should not stop the clock during the data phase; that is, the clock should be stopped between the blocks of data only.

You should use the following RTL configuration requirements and programming flow for software drivers in order to use the Mobile Storage Host Controller in SDR104 mode applications:

1. FIFO size should be selected as multiple of 512 bytes at time of RTL configuration.
2. Program BYTCNT register to value equal to multiple of 512 bytes and less than or equal to FIFO size.
3. Program BLKSIZ register to 512—block length fixed to 512 in SDR104 mode.
4. If transfer size is more than FIFO size, then it should be split into multiple transfers—size of each transfer can be equal to FIFO size; a new transfer is programmed after data done is received for previous transfer.

9.12.3 Clock Requirements and Recommendations

There are several clock requirements and recommendations when interfacing the DWC_mobile_storage with cards.

9.12.3.1 Clock Requirements

DWC_mobile_storage uses the following clocks to achieve a reliable communication with interfaced cards:

- ❖ `cclk_in` – Clocks logic in CIU clock domain.
- ❖ `cclk_in_drv` – Has the same frequency as `cclk_in`, but is phase shifted. Satisfies the minimum hold time requirement of 5 ns at the input to the cards while operating in SDR12 or identification modes.
- ❖ `cclk_in_sample` – Has the same frequency as `cclk_in`, but is phase shifted. Provides a “best sampling window” for DWC_mobile_storage while reading data from the card.

9.12.3.2 Clock Generation Recommendations

The following are recommendations for clock generation:

- ❖ The `cclk_in` frequency input to DWC_mobile_storage can be switched, depending on the data rate. For example, if the interfaced card is communicating in SDR50 mode, then `cclk_in` = 50 Mhz; if the interfaced card is communicating in SDR104 mode, then `cclk_in` = 200 Mhz.



Note

You should ensure that there are no glitches in the clock inputs while switching clock frequencies.

The `cclk_in_drv` and `cclk_in_sample` clock frequencies must switch in relation to `cclk_in` frequencies.

- ❖ The `cclk_in_drv` and `cclk_in_sample` clocks are phase-shifted versions of `cclk_in`.

The value of the phase shift should be selectable, based on the enumerated data rate. The phase shift can have a resolution of 90 degrees with respect to the `cclk_in` clock period.

You should instantiate the above structure described outside the DWC_mobile_storage. The clock speed for `cclk_in` and the phase shifts for `cclk_in_drv` and `cclk_in_sample` should be changed according to the mode of operation—that is, the different data rates—chosen at the end of enumeration.

The system software must perform the frequency switch based on the enumerated data speeds.

9.12.3.2.1 Simulation-Based Analysis for Timing and Clocking

The following assumptions pertain to the timing and clocking recommendations.

- ❖ The DWC_mobile_storage in the design is instantiated with one port.
- ❖ The following delays are assumed:
 - ◆ Host-controller output path:
 - ◇ $\text{Delay_O} = \text{cclk_in to cclk_out delay} = 1.4\text{ns}$ (Typical value)
 - ◆ Host-controller input path:
 - ◇ $\text{tODLY} = \text{cclk_out to cdata_in delay} = \text{max value}$ (specified in [Table 9-4](#))
 - ◇ $\text{Delay_I} = \text{IO_pad delay} + \text{routing delays within host controller} = 2.35\text{ ns}$ (typical value)
 - ◇ $\text{Delay_S} = \text{Delay_O} + \text{Delay_I} = 3.75\text{ ns}$
 - ◇ $\text{Delay}_{\text{total turnaround between Host Controller output and input}} = \text{tODLY} + \text{Delay_S} (\text{Delay_O} + \text{Delay_I})$

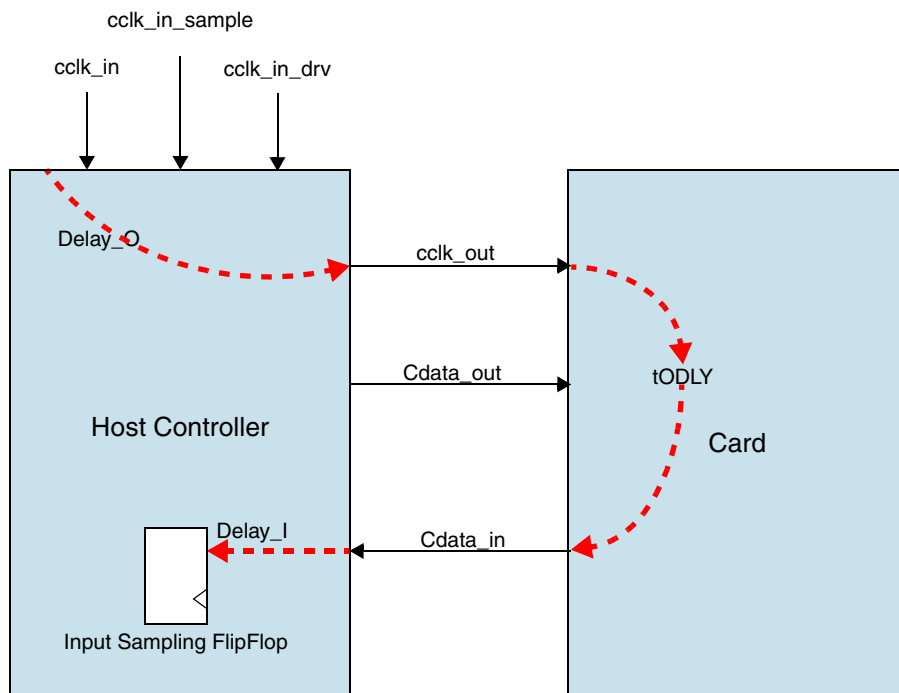


Setup and hold times have been computed from exact Delay_O, Delay_I and tODLY values.

- ◆ Synthesis is done in 65nm; setup and hold times for the flip-flops within the host controller are assumed to be 1 ns.

Figure 9-16 illustrates how the host controller and card delays relate to each other.

Figure 9-16 Host Controller and Card Delays



Clock frequency set at 50MHz or 200 MHz, depending on card type

9.12.4 Testing Results

Table 9-5 lists the simulation-based test results for DWC_mobile_storage host controller output paths. All operating modes have at least one phase shift value that can be used to communicate with the card.

Table 9-5 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Hold time delays (ns) introduced (clk_in_drv) through phase shifters	Hold time delays (ns) checked	Resultant hold time (ns)	Setup time (ns) to be checked	Resultant setup time (ns)	Final Result	Comments
SDR104	200	200	1	1.25	0.8	4.85	1.4	0.15	Fail	Keep resultant hold time greater than 1 ns
				2.5		1.1		3.9	Pass	
				3.75		2.35		2.65	Pass	
SDR50	100	200	2	1.25	0.8	9.85	3.0	0.15	Fail	Keep resultant hold time greater than from 1 ns
				2.5		1.1		8.9	Pass	
				3.75		2.35		7.65	Pass	
DDR50 (CMD line)	50	50	1	5.0	0.8	3.6	6.0	16.4	Pass	Keep same as DAT line
				10.0		8.6		11.4	Pass	
				15.0		13.6		6.4	Pass	
DDR50 (DAT line)	50	50	1	5.0	0.8	3.6	3.0	6.4	Pass	Only 90 degree phase shift matched setup and hold times
				10.0		8.6		1.4	Fail	
				15.0		3.6		6.4	Fail	Start bit for half cycle

Dark-green Pass: Data integrity pass results with best timings for *both* Setup and Hold.

Light-green Pass: Hold time *or* Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing marked in dark green; others marked as white pass.

Red: Margins below 1ns.

Table 9-5 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Hold time delays (ns) introduced (clk_in_drv) through phase shifters	Hold time delays (ns) checked	Resultant hold time (ns)	Setup time (ns) to be checked	Resultant setup time (ns)	Final Result	Comments
SDR25	50	50	1	5.0	2.0	3.6	6.0	16.4	Pass	Better margin for setup and hold times
				10.0		8.6		11.4	Pass	
				15.0		13.6		6.4	Pass	
SDR12	25	50	2	5.0	5.0	3.6	5.0	36.4	Fail	Better margin for setup and hold times
				10.0		8.6		31.4	Pass	
				15.0		13.6		26.4	Pass	
Identification on mode	400	50	125	5.0	5.0	3.6	5.0	2516.4	Fail	Better margin for hold time; setup time very large
				10.0		8.6		2511.4	Pass	
				15.0		13.6		2506.4	Pass	

Dark-green Pass: Data integrity pass results with best timings for *both* Setup and Hold.

Light-green Pass: Hold time *or* Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing marked in dark green; others marked as white pass.

Red: Margins below 1ns.

Table 9-6 lists the simulation-based test results for DWC_mobile_storage host controller input paths. All operating modes have at least one phase shift value that can be used to communicate with the card.

Table 9-6 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Sampling delays (ns) from phase shifter	Delays on cdata_in (ns) from card	Sampling result data integrity checks	Available hold time	Available setup time	Final Result	Comments
SDR104	200	200	1	0.0	0.0	Pass	3.75	1.25	Pass	
					4.8	Pass	3.75	1.45		
					5.0	Pass	3.75	1.25		
					9.6	Pass	3.75	1.65		
					10.0	Pass	3.75	1.25		
				1.25	0.0	Pass	2.5	2.5	Pass	
					4.8	Pass	2.3	2.7		
					5.0	Pass	2.5	2.5		
					9.6	Pass	2.1	2.9		
					10.0	Pass	2.5	2.5		
				2.5	0.0	Pass	1.25	3.75	Fail	
					4.8	Pass	1.05	3.95		
					5.0	Pass	1.25	3.75		
					9.6	Pass	0.85	4.15		
					10.0	Pass	1.25	3.75		
				3.75	0.0	Pass	0.0	5.0	Fail	
					4.8	Pass	4.8	0.2		
					5.0	Hangs	—	—		
					9.8	Pass	4.6	0.4		
					10.0	Hangs	—	—		

Light-green Pass: Hold time or Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing is marked in green; others marked as white pass.

Yellow: Margins below 2ns, but not as good as margins marked in green.

Red: Margins below 1ns.

Table 9-6 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Sampling delays (ns) from phase shifter	Delays on cdata_in (ns) from card	Sampling result data integrity checks	Available hold time	Available setup time	Final Result	Comments
SDR50	100	200	2	0.0	0.0	Pass	3.75	6.25	Pass	
					3.6	Pass	7.35	2.65		
					7.5	Pass	1.25	8.75		
				1.25	0.0	Pass	2.5	7.5	Fail	
					3.6	Pass	6.1	3.9		
					7.5	Pass	9.85	0.15		
				2.5	0.0	Pass	1.25	8.75	Pass	
					3.6	Pass	4.85	5.15		
					7.5	Pass	8.75	1.25		
				3.75	0.0	Pass	5.0	5.0	Pass	
					3.6	Pass	8.6	1.4		
					7.5	Pass	2.5	7.5		

Light-green Pass: Hold time *or* Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing is marked in green; others marked as white pass.

Yellow: Margins below 2ns, but not as good as margins marked in green.

Red: Margins below 1ns.

Table 9-6 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Sampling delays (ns) from phase shifter	Delays on cdata_in (ns) from card	Sampling result data integrity checks	Available hold time	Available setup time	Final Result	Comments
DDR50 (DAT)	50	50	1	0.0	0.0	Pass	3.75	6.25	Fail	All tests fail because total delay is more than 10ns: 1.4 + 7 + 2.35 = 10.75 which is greater than allowed delay of 10ns
					3.5	Pass	7.25	2.75		
					7.0	Fail	0.75	9.25		
				5.0	0.0	Fail	8.75	1.25	Fail	
					3.5	Pass	2.25	7.75		
					7.0	Pass	6.4	3.6		
				10.0	0.0	Fail	3.75	6.25	Fail	
					3.5	Fail	7.25	2.75		
					7.0	Hangs	—	—		
				15.0	0.0	Hangs	—	—	Fail	
					3.5	Fail	2.25	7.75		
					7.0	Fail	5.75	4.25		

Light-green Pass: Hold time *or* Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing is marked in green; others marked as white pass.

Yellow: Margins below 2ns, but not as good as margins marked in green.

Red: Margins below 1ns.

Table 9-6 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Sampling delays (ns) from phase shifter	Delays on cdata_in (ns) from card	Sampling result data integrity checks	Available hold time	Available setup time	Final Result	Comments
SDR25. DDR50 (CMD)	50	50	1	0.0	0.0	Pass	3.75	16.25	Pass	
					7.0	Pass	10.75	9.25		
					14.0	Pass	17.75	2.25		
				5.0	0.0	Pass	18.75	1.25	Pass	
					7.0	Pass	5.75	14.25		
					14.0	Pass	12.75	7.25		
				10.0	0.0	Pass	13.75	6.25	Fail	
					7.0	Pass	0.75	19.25		
					14.0	Pass	7.75	12.25		
				15.0	0.0	Pass	8.75	11.25	Pass	
					7.0	Pass	15.75	4.25		
					14.0	Pass	2.75	17.25		

Light-green Pass: Hold time *or* Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing is marked in green; others marked as white pass.

Yellow: Margins below 2ns, but not as good as margins marked in green.

Red: Margins below 1ns.

Table 9-6 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Sampling delays (ns) from phase shifter	Delays on cdata_in (ns) from card	Sampling result data integrity checks	Available hold time	Available setup time	Final Result	Comments
SDR12	25	50	2	0.0	0.0	Pass	3.75	36.25	Pass	
					7.0	Pass	10.75	29.25		
					14.0	Pass	17.75	22.25		
				5.0	0.0	Pass	38.75	1.25	Pass	
					7.0	Pass	5.75	34.25		
					14.0	Pass	12.75	27.25		
				10.0	0.0	Pass	33.75	6.25	Fail	
					7.0	Pass	0.75	39.25		
					14.0	Pass	7.75	32.25		
				15.0	0.0	Pass	8.75	31.25	Pass	
					7.0	Pass	15.75	24.25		
					14.0	Pass	22.75	17.25		

Light-green Pass: Hold time *or* Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing is marked in green; others marked as white pass.

Yellow: Margins below 2ns, but not as good as margins marked in green.

Red: Margins below 1ns.

Table 9-6 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Sampling delays (ns) from phase shifter	Delays on cdata_in (ns) from card	Sampling result data integrity checks	Available hold time	Available setup time	Final Result	Comments
Identification Mode	400	50	125	0.0	0.0	Pass	3.75	2516.3	Pass	
					25.0	Pass	28.75	2491.3		
					50.0	Pass	53.75	2466.3		
				5.0	0.0	Pass	2518.8	1.25	Pass	
					25.0	Pass	23.75	2496.3		
					50.0	Pass	48.75	2471.3		
				10.0	0.0	Pass	2513.8	6.25	Fail	
					25.0	Pass	28.75	2491.3		
					50.0	Pass	43.75	2476.3		
				15.0	0.0	Pass	8.75	2511.3	Pass	
					25.0	Pass	33.75	2486.3		
					50.0	Pass	58.75	2461.3		

Light-green Pass: Hold time *or* Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing is marked in green; others marked as white pass.

Yellow: Margins below 2ns, but not as good as margins marked in green.

Red: Margins below 1ns.

The timing in the DDR50 (DAT) fails because the total delay is greater than 10ns – that is:

$$1.4 + 7 + 2.35 = 10.75\text{ns}$$

The following can meet the DDR50 (DAT) timing needs:

- ❖ The delay on the cdata_in from the card has a delay greater than 0.75 ns, where the cclk_in_sample has a 90 degree (5 ns) phase shift.
- ❖ Within DWC_mobile_storage_ciu, delay the cclk_in to other modules by 1.4 ns except for DWC_mobile_storage_clkcntl.v. This eliminates phase shifts on cclk_out and the cclk_in.

A

Timing Requirements for SD 3.0 Cards

This appendix describes timing requirements necessary for interfacing the DWC_mobile_storage to SD 3.0 cards that have different modes of operation.

An external clock multiplexor structure is needed to meet timing requirements across different card operating modes described in the SD Memory Card Specification, Version 3.0.

Phase shift values must be determined after place and route and then used for back annotation. This chapter describes the structure with typical delays of 65nm. I/O pad delays should be kept to a minimum in order to reduce stress on timing at both the host controller and card interface.

A.1 Timing Requirements for SD 3.0 Cards

[Table A-1](#) shows card input timing requirements that act as constraints for the Host Controller output path to a card.

Table A-1 Card Input Timing Requirements

Mode	Maximum Frequency of Operation (MHz)	Hold Time (ns)	Setup Time (ns)
SDR104	200	0.8	1.4
SDR50	100	0.8	3.0
DDR50 (CMD line)	50	0.8	6.0
DDR50 (DAT line)	50	0.8	3.0
SDR25	50	2.0	6.0
SDR12	25	5.0	5.0
Identification Mode	400	5.0	5.0

Table A-2 shows card output timing requirements that act as constraints for the Host Controller input path from a card.

Table A-2 Card Output Timing Requirements

Mode	Maximum Frequency of Operation (MHz)	Maximum Card Output Delay (ns)
SDR104	200	10.0
SDR50	100	7.5
DDR50 (CMD line)	50	13.7
DDR50 (DAT line)	50	7.0
SDR25	50	14.0
SDR12	25	14.0
Identification Mode	400	50.0



Note

For SDR104, the SD Memory Card Specification requires 2UI as the maximum card delay before tuning. The post-tuning timing uncertainty—due to temperature variation—is specified as 1.9 ns (350 ps to +1550 ps).

where UI—Unit Interval is one bit nominal time, SDCLK nominal period

CMD19 is used for tuning in order to resolve this 2UI ambiguity, which is listed in Table A-2 as 10 ns; one card clock period (one UI) is equivalent to 5 ns. Since 2UI is a worse-case ambiguity compared to a 1.5 ns ambiguity after tuning, 10 ns (that is, 2UI) is used as the maximum card output delay in Table A-2.



Attention

If the user wants to use SDR104, then they must select the FIFO size based on the number of blocks to be used. If one block of data is to be read or written, then the FIFO size should be at least 512 bytes. If four blocks(512 bytes * 4) are to be used, then the user must select a FIFO depth of at least 2K bytes while configuring the DWC_mobile_storage.

In these circumstances, for SDR104 the maximum number of blocks that can be programmed in one transfer are:

- 64 when the AHB data width is selected as 64
- 32 when the AHB data width is selected as 32

This selection must be done during configuration.

A.2 Guidelines for Using Mobile Storage Host Controller in SDR104 Mode

According to the SD Memory Card Specification, Version 3.0, SDR104 mode supports Variable Output Delay on CMD and Data lines with a delay from 0 to 2 Unit Intervals (UI) — where 1 UI equals one card clock period. Due to this variable delay, it is recommended that the host should not stop the clock during the data phase; that is, the clock should be stopped between the blocks of data only.

You should use the following RTL configuration requirements and programming flow for software drivers in order to use the Mobile Storage Host Controller in SDR104 mode applications:

1. FIFO size should be selected as multiple of 512 bytes at time of RTL configuration.
2. Program BYTCNT register to value equal to multiple of 512 bytes and less than or equal to FIFO size.
3. Program BLKSIZ register to 512 – block length fixed to 512 in SDR104 mode.
4. If transfer size is more than FIFO size, then it should be split into multiple transfers – size of each transfer can be equal to FIFO size; a new transfer is programmed after data done is received for previous transfer.

A.3 Testing Results

Table A-3 lists the simulation-based test results for DWC_mobile_storage host controller output paths. All operating modes have at least one phase shift value that can be used to communicate with the card.

Table A-3 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Hold time delays (ns) introduced (clk_in_drv) through phase shifters	Hold time delays (ns) checked	Resultant hold time (ns)	Setup time (ns) to be checked	Resultant setup time (ns)	Final Result	Comments
SDR104	200	200	1	1.25	0.8	4.85	1.4	0.15	Fail	Keep resultant hold time greater than 1 ns
				2.5		1.1		3.9	Pass	
				3.75		2.35		2.65	Pass	
SDR50	100	200	2	1.25	0.8	9.85	3.0	0.15	Fail	Keep resultant hold time greater than from 1 ns
				2.5		1.1		8.9	Pass	
				3.75		2.35		7.65	Pass	

Dark-green Pass: Data integrity pass results with best timings for *both* Setup and Hold.

Light-green Pass: Hold time *or* Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing marked in dark green; others marked as white pass.

Red: Margins below 1ns.

Table A-3 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Hold time delays (ns) introduced (clk_in_drv) through phase shifters	Hold time delays (ns) checked	Resultant hold time (ns)	Setup time (ns) to be checked	Resultant setup time (ns)	Final Result	Comments
DDR50 (CMD line)	50	50	1	5.0	0.8	3.6	6.0	16.4	Pass	Keep same as DAT line
				10.0		8.6		11.4	Pass	
				15.0		13.6		6.4	Pass	
DDR50 (DAT line)	50	50	1	5.0	0.8	3.6	3.0	6.4	Pass	Only 90 degree phase shift matched setup and hold times
				10.0		8.6		1.4	Fail	
				15.0		3.6		6.4	Fail	Start bit for half cycle
SDR25	50	50	1	5.0	2.0	3.6	6.0	16.4	Pass	Better margin for setup and hold times
				10.0		8.6		11.4	Pass	
				15.0		13.6		6.4	Pass	
SDR12	25	50	2	5.0	5.0	3.6	5.0	36.4	Fail	Better margin for setup and hold times
				10.0		8.6		31.4	Pass	
				15.0		13.6		26.4	Pass	
Identification on mode	400	50	125	5.0	5.0	3.6	5.0	2516.4	Fail	
				10.0		8.6		2511.4	Pass	

Dark-green Pass: Data integrity pass results with best timings for *both* Setup and Hold.

Light-green Pass: Hold time *or* Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing marked in dark green; others marked as white pass.

Red: Margins below 1ns.

Table A-3 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Hold time delays (ns) introduced (clk_in_drv) through phase shifters	Hold time delays (ns) checked	Resultant hold time (ns)	Setup time (ns) to be checked	Resultant setup time (ns)	Final Result	Comments
				15.0		13.6		2506.4	Pass	Better margin for hold time; setup time very large

Dark-green Pass: Data integrity pass results with best timings for *both* Setup and Hold.
Light-green Pass: Hold time *or* Setup time pass independently; does not guarantee both.
Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.
Among final results that pass, best timing marked in dark green; others marked as white pass.
Red: Margins below 1ns.

Table A-4 on page 302 lists the simulation-based test results for DWC_mobile_storage host controller input paths. All operating modes have at least one phase shift value that can be used to communicate with the card.

Table A-4 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Sampling delays (ns) from phase shifter	Delays on cdata_in (ns) from card	Sampling result data integrity checks	Available hold time	Available setup time	Final Result	Comments
SDR104	200	200	1	0.0	0.0	Pass	3.75	1.25	Pass	
					4.8	Pass	3.75	1.45		
					5.0	Pass	3.75	1.25		
					9.6	Pass	3.75	1.65		
					10.0	Pass	3.75	1.25		
				1.25	0.0	Pass	2.5	2.5	Pass	
					4.8	Pass	2.3	2.7		
					5.0	Pass	2.5	2.5		
					9.6	Pass	2.1	2.9		
					10.0	Pass	2.5	2.5		
				2.5	0.0	Pass	1.25	3.75	Fail	
					4.8	Pass	1.05	3.95		
					5.0	Pass	1.25	3.75		
					9.6	Pass	0.85	4.15		
					10.0	Pass	1.25	3.75		
				3.75	0.0	Pass	0.0	5.0	Fail	
					4.8	Pass	4.8	0.2		
					5.0	Hangs	—	—		
					9.8	Pass	4.6	0.4		
					10.0	Hangs	—	—		

Light-green Pass: Hold time or Setup time pass independently; does not guarantee both.
Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.
Among final results that pass, best timing is marked in green; others marked as white pass.
Yellow: Margins below 2ns, but not as good as margins marked in green.
Red: Margins below 1ns.

Table A-4 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (clk_in)	Divider	Sampling delays (ns) from phase shifter	Delays on cdata_in (ns) from card	Sampling result data integrity checks	Available hold time	Available setup time	Final Result	Comments
SDR50	100	200	2	0.0	0.0	Pass	3.75	6.25	Pass	
					3.6	Pass	7.35	2.65		
					7.5	Pass	1.25	8.75		
				1.25	0.0	Pass	2.5	7.5	Fail	
					3.6	Pass	6.1	3.9		
					7.5	Pass	9.85	0.15		
				2.5	0.0	Pass	1.25	8.75	Pass	
					3.6	Pass	4.85	5.15		
					7.5	Pass	8.75	1.25		
				3.75	0.0	Pass	5.0	5.0	Pass	
					3.6	Pass	8.6	1.4		
					7.5	Pass	2.5	7.5		

Light-green Pass: Hold time or Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing is marked in green; others marked as white pass.

Yellow: Margins below 2ns, but not as good as margins marked in green.

Red: Margins below 1ns.

Table A-4 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Sampling delays (ns) from phase shifter	Delays on cdata_in (ns) from card	Sampling result data integrity checks	Available hold time	Available setup time	Final Result	Comments
DDR50 (DAT)	50	50	1	0.0	0.0	Pass	3.75	6.25	Fail	All tests fail because total delay is more than 10ns: 1.4 + 7 + 2.35 = 10.75 which is greater than allowed delay of 10ns
					3.5	Pass	7.25	2.75		
					7.0	Fail	0.75	9.25		
				5.0	0.0	Fail	8.75	1.25	Fail	
					3.5	Pass	2.25	7.75		
					7.0	Pass	6.4	3.6		
				10.0	0.0	Fail	3.75	6.25	Fail	
					3.5	Fail	7.25	2.75		
					7.0	Hangs	—	—		
				15.0	0.0	Hangs	—	—	Fail	
					3.5	Fail	2.25	7.75		
					7.0	Fail	5.75	4.25		

Light-green Pass: Hold time or Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing is marked in green; others marked as white pass.

Yellow: Margins below 2ns, but not as good as margins marked in green.

Red: Margins below 1ns.

Table A-4 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cc1k_in)	Divider	Sampling delays (ns) from phase shifter	Delays on cdata_in (ns) from card	Sampling result data integrity checks	Available hold time	Available setup time	Final Result	Comments
SDR25. DDR50 (CMD)	50	50	1	0.0	0.0	Pass	3.75	16.25	Pass	
					7.0	Pass	10.75	9.25		
					14.0	Pass	17.75	2.25		
				5.0	0.0	Pass	18.75	1.25	Pass	
					7.0	Pass	5.75	14.25		
					14.0	Pass	12.75	7.25		
				10.0	0.0	Pass	13.75	6.25	Fail	
					7.0	Pass	0.75	19.25		
					14.0	Pass	7.75	12.25		
				15.0	0.0	Pass	8.75	11.25	Pass	
					7.0	Pass	15.75	4.25		
					14.0	Pass	2.75	17.25		

Light-green Pass: Hold time or Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing is marked in green; others marked as white pass.

Yellow: Margins below 2ns, but not as good as margins marked in green.

Red: Margins below 1ns.

Table A-4 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Sampling delays (ns) from phase shifter	Delays on cdata_in (ns) from card	Sampling result data integrity checks	Available hold time	Available setup time	Final Result	Comments
SDR12	25	50	2	0.0	0.0	Pass	3.75	36.25	Pass	
					7.0	Pass	10.75	29.25		
					14.0	Pass	17.75	22.25		
				5.0	0.0	Pass	38.75	1.25	Pass	
					7.0	Pass	5.75	34.25		
					14.0	Pass	12.75	27.25		
				10.0	0.0	Pass	33.75	6.25	Fail	
					7.0	Pass	0.75	39.25		
					14.0	Pass	7.75	32.25		
				15.0	0.0	Pass	8.75	31.25	Pass	
					7.0	Pass	15.75	24.25		
					14.0	Pass	22.75	17.25		

Light-green Pass: Hold time or Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing is marked in green; others marked as white pass.

Yellow: Margins below 2ns, but not as good as margins marked in green.

Red: Margins below 1ns.

Table A-4 Test Results for Simulation-Based Host Controller Output Path

Mode	Frequency of operation (MHz)	Input frequency (MHz) (cclk_in)	Divider	Sampling delays (ns) from phase shifter	Delays on cdata_in (ns) from card	Sampling result data integrity checks	Available hold time	Available setup time	Final Result	Comments
Identification Mode	400	50	125	0.0	0.0	Pass	3.75	2516.3	Pass	
					25.0	Pass	28.75	2491.3		
					50.0	Pass	53.75	2466.3		
				5.0	0.0	Pass	2518.8	1.25	Pass	
					25.0	Pass	23.75	2496.3		
					50.0	Pass	48.75	2471.3		
				10.0	0.0	Pass	2513.8	6.25	Fail	
					25.0	Pass	28.75	2491.3		
					50.0	Pass	43.75	2476.3		
				15.0	0.0	Pass	8.75	2511.3	Pass	
					25.0	Pass	33.75	2486.3		
					50.0	Pass	58.75	2461.3		

Light-green Pass: Hold time *or* Setup time pass independently; does not guarantee both.

Note: When Hold time or Setup time pass with data integrity pass, final result is written as pass.

Among final results that pass, best timing is marked in green; others marked as white pass.

Yellow: Margins below 2ns, but not as good as margins marked in green.

Red: Margins below 1ns.

The timing in the DDR50 (DAT) fails because the total delay is greater than 10ns – that is:

$$1.4 + 7 + 2.35 = 10.75\text{ns}$$

The following can meet the DDR50 (DAT) timing needs:

- ❖ The delay on the cdata_in from the card has a delay greater than 0.75 ns, where the cclk_in_sample has a 90 degree (5 ns) phase shift.
- ❖ Within DWC_mobile_storage_ciu, delay the cclk_in to other modules by 1.4 ns except for DWC_mobile_storage_clkcntl.v. This eliminates phase shifts on cclk_out and the cclk_in.

B

Atomic VIP Command Verilog Testbench

This appendix describes the provided Atomic VIP Command Verilog Testbench (AVTB) files and explains how to create your own test scenarios and verify the DWC Mobile Storage Host Controller core after integrating it into your design.

B.1 AVTB Overview

The Atomic VIP Command Verilog Testbench (AVTB) is packaged in the DWC Mobile Storage installation and provides a starting point for understanding how to use DesignWare Verification IP (VIP) and the configured DWC Mobile Storage Host Controller core together in the verification environment.

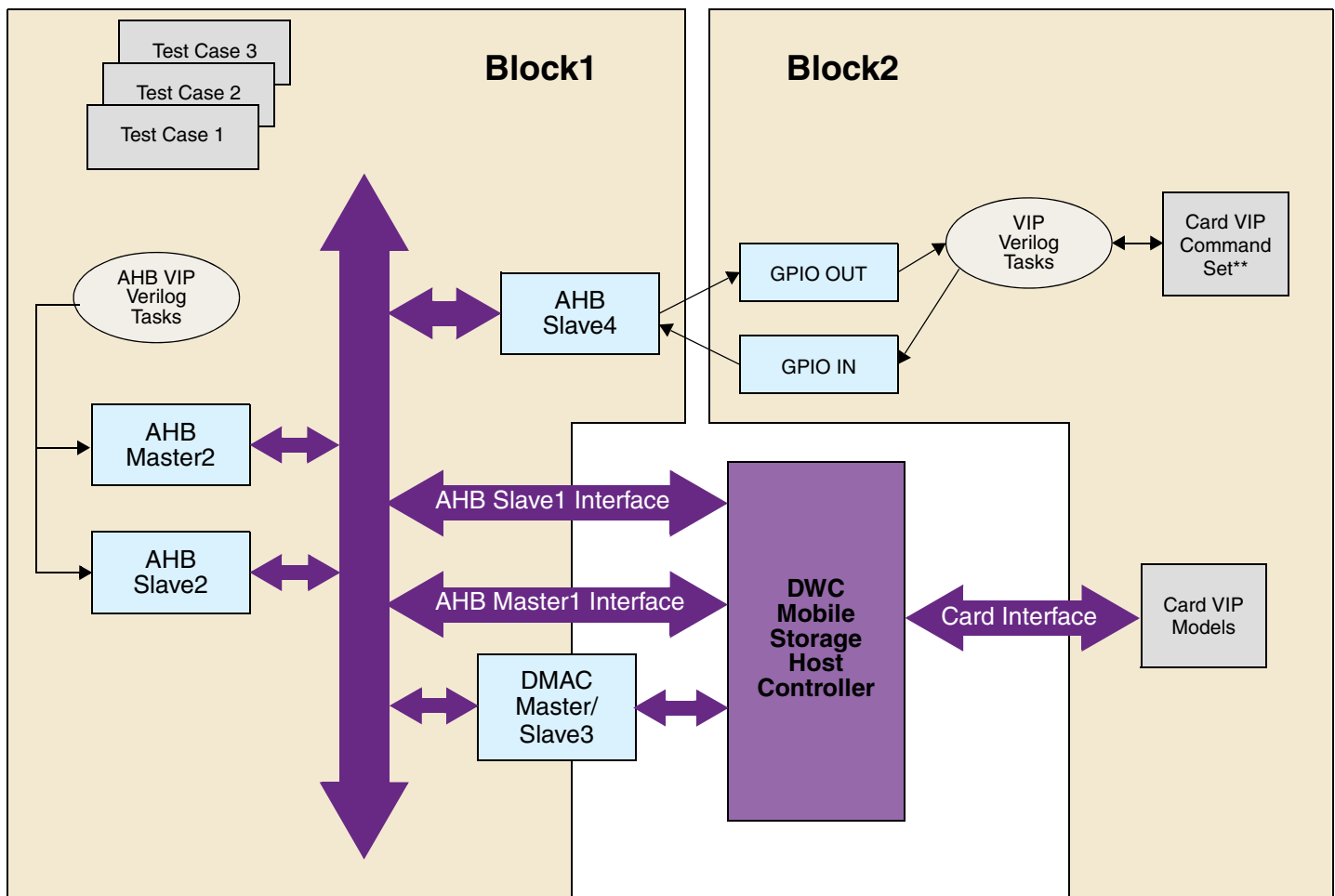
The VTB environment is organized in such a way that tasks related to AHB transactions – on the application side – and tasks related to CARD transactions – CARD side – are split to ease System-on-Chip (SoC) integration testing. The application side communicates with the card side through a GPIO. The card side maintains a VIP command set, which the application side uses choosing specific commands via the GPIO to configure cards before performing data transfers.

The VTB demonstrates:

- ❖ How to connect the following in a Verilog-based testbench:
 - ◆ DesignWare AMBA
 - ◆ Mobile Storage card VIPs
 - ◆ Mobile Storage host controller core synthesizable (RTL) component
- ❖ How to initialize and program the DWC Mobile Storage Host controller core – using Verilog VIP commands – to perform basic operating functions

On the CARD side, the DWC Mobile Storage Host controller provides a card interface. On the application side, it interfaces to an AHB bus. The VIPs are used in the VTB as regular Verilog instances. These VIPs have their own built-in tasks that can be used just as any regular Verilog task in the VTB.

[Figure B-1](#) illustrates the block diagram of the testbench.

Figure B-1 Atomic VIP Command Verilog Testbench Block Diagram

**Only initialization of cards is supported in the current release. More commands will be supported in future releases.

The VTB architecture (mmc_vtb_top) consists of two distinct blocks – block1 and block2.:

- ❖ block1 contains AHB tasks and AHB component (AHB Master, Slave and Bus) instantiations.
- ❖ block2 contains card VIP tasks and card VIP instantiations.

Controlling test cases are present in block1 – verilog test case1, verilog test case2, and so on. These test cases not only control the activities of block1, but also that of block2 through the GPIO_OUT and GPIO_IN interfaces. Block2 includes a card VIP command set that every Verilog test case in block1 uses.

The handshake protocol between block1 and block2 is as follows:

- ❖ block2 includes a VIP command set accessed by block1 via a GPIO_OUT request specifying a particular command to be executed.
- ❖ It is always block1 that initiates a request to block2.
- ❖ When there is no request to be serviced, the value on GPIO_OUT is 8'h00 and the value of GPIO_IN is 8'hFF.
- ❖ block1 programs the GPIO_OUT register in AHB Slave4 with a valid command number in the command set. AHB Slave4 drives the GPIO_OUT with the programmed number.

- ❖ On seeing a value of GPIO_OUT != 8'h00, block2 drives a value of GPIO_IN==8'h00.
- ❖ After block2 services the request, it drives GPIO_IN==8'hFF.
- ❖ As long as block2 is servicing a request, block1 keeps polling for the completion of the request by reading GPIO_IN register in AHB Slave 4.
- ❖ On seeing a value of GPIO_IN==8'hFF, AHB Slave4 drives a value of GPIO_OUT==8'h00, thus completing the handshake.
- ❖ When block1 reads a value of 8'hFF from GPIO_IN, it assumes that the requested transfer is completed by block2.

B.2 Reset, Start, and Initialization

The block1 (ahb_blk) is the primary control block of simulation. The following is the sequence of events in block1:

1. RESET asserted
2. AMBA VIPs are initialized
3. RESET de-asserted
4. All cards are initialized
5. Host Controller is programmed (see details below)
6. Cards enumerated
7. Cards ready for data transfer

The following is the Host Controller programming sequence:

1. CTRL register is programmed with the value 32'h0000_0010 to set the global interrupt enable bit.
2. PWREN register is programmed with the value 32'h3fff_ffff to enable power to all cards.
3. INTMASK register is programmed with the value 32'hffff_ffff to enable all interrupts.
4. CLKDIV register is programmed with the value 32'h0000_0000 to bypass the card clock frequency.
5. CLKSRC register is programmed with the value 32'h0000_0000 to select clock_divider_0 for all connected cards.
6. CLKENA register is programmed with the value 32'h0000_ffff to enable clock for all connected cards.
7. CMD register is programmed with the value 32'h8020_0000 to set the update_clock_registers_only bit.
8. TMOUT register is programmed with the value 32'hffff_ff40 (0xffff for data_timeout and 0x40 for response_timeout).
9. CMD register is programmed with the value 32'h8000_8000 to set the send_initialization bit.
10. CTYPE register is programmed with the value 32'h0000_0000 to set all cards to 1-bit mode.
11. FIFOTH register is programmed to set the threshold watermark level for both transmission and reception of data.

The block2 (mmc_blk) is activated whenever block1 sends a card VIP request via the GPIO_OUT pins. Different card VIP requests are serviced in block2.

B.3 DWC Mobile Storage Data Transfer Tests

The VTB testbench has the following features:

- ❖ The testbench instantiates appropriate number of cards for simulation following card definitions (see below).
- ❖ The type of card for each port is user-selectable via the coreConsultant “Testbench Parameter Setup” activity.
- ❖ The testbench instantiates the correct VIP model for each card as per the user selected card type in the test configuration file.

The tests listed in [Table B-1](#) are examples of data transfer tests for different SD and MMC cards.

Table B-1 Example Data Transfer Tests for SD and MMC Cards

Data Transfer Test	Description
VTB_SBW	Does single-block write to all connected cards on 1-bit, 4-bit, and 8-bit data width
VTB_SBR	Does single-block read to all connected cards on 1-bit, 4-bit, and 8-bit data width
VTB_SBW_RANDOM	Does single-block write to random ports on 1-bit, 4-bit, and 8-bit data width
VTB_SBR_RANDOM	Does single-block read to random ports on 1-bit, 4-bit, and 8-bit data width
VTB_SW_PREDEF_BYTCNT	Does stream write with predefined byte count to all connected cards on 1-bit, 4-bit, and 8-bit data width
VTB_SR_PREDEF_BYTCNT	Does stream read with predefined byte count to all connected cards on 1-bit, 4-bit, and 8-bit data width
VTB_SW_BYTCNT_ZERO	Does stream write with byte count set to zero to all connected cards on 1-bit, 4-bit, and 8-bit data width
VTB_SR_BYTCNT_ZERO	Does stream read with byte count set to zero to all connected cards on 1-bit, 4-bit, and 8-bit data width
VTB_PREDEFINED_MBW	Does predefined multiple-block write to all connected cards on 1-bit, 4-bit, and 8-bit data width
VTB_PREDEFINED_MBR	Does predefined multiple-block read to all connected cards on 1-bit, 4-bit, and 8-bit data width
VTB_OE_MBW_PREDEF_BYTCNT	Does open-end multiple-block write with predefined byte count to all connected cards on 1-bit, 4-bit, and 8-bit data width
VTB_OE_MBR_PREDEF_BYTCNT	Does open-end multiple-block read with predefined byte count to all connected cards on 1-bit, 4-bit, and 8-bit data width
VTB_OE_MBW_BYTCNT_ZERO	Does open-end multiple-block write with byte count set to zero to all connected cards on 1-bit, 4-bit, and 8-bit data width

Table B-1 Example Data Transfer Tests for SD and MMC Cards

Data Transfer Test	Description
VTB_OE_MBR_BYTCNT_ZERO	Does open-end multiple-block read with byte count set to zero to all connected cards on 1-bit, 4-bit, and 8-bit data width
VTB_MMC_INTR	Verifies interrupt mode in MMC low speed cards in MMC_ONLY mode where card responds to exit IRQ state
VTB_MMC_INTR_HOST_RESPONSE	Verifies interrupt mode in MMC low speed cards in MMC_ONLY mode where host responds to exit IRQ state
VTB_SDIO_SUSPEND_RESUME_WRITE	Verifies suspend resume operation on SDIO and COMBO cards during write operation
VTB_SDIO_SUSPEND_RESUME_READ	Verifies suspend resume operation on SDIO and COMBO cards during read operation
VTB_SDIO_READ_WAIT	Verifies read wait operation on SDIO and COMBO cards
VTB_SDIO_INTERRUPT	Monitors the SDIO interrupt and clears the interrupt using CLEAR_INTERRUPT vip command
VTB_SDIO_INVALID_INTERRUPT_IN_4BIT	Programs VIP to generate SDIO interrupt in invalid interrupt period and checks if core detects it (in 4-bit data bus width)
VTB_SDIO_ASYNC_INTERRUPT_IN_4BIT	Programs VIP to generate SDIO interrupt in asynchronous interrupt period in following two scenarios: • Interrupt from selected card with and without card clock running • Interrupt from non-selected card with and without card clock running
VTB_BOOT_TEST_MANDATORY_WITH_ACK	Verifies mandatory boot mode with acknowledge enabled
VTB_BOOT_TEST_MANDATORY_ACK_ENDBIT_ERR	Verifies mandatory boot mode with acknowledge endbit error
VTB_BOOT_TEST_MANDATORY_ACK_PATTERN_ERR	Verifies mandatory boot mode with acknowledge pattern error
VTB_BOOT_TEST_MANDATORY_CRC16_ERR_WITH_ACK	Verifies mandatory boot mode with data CRC error with acknowledge enabled
VTB_BOOT_TEST_MANDATORY_CRC16_ERR_NO_ACK	Verifies mandatory boot mode with data CRC error with acknowledge disabled
VTB_BOOT_TEST_MANDATORY_DATA_TIMEOUT_WITH_ACK	Verifies mandatory boot mode with data timeout with acknowledge enabled
VTB_BOOT_TEST_MANDATORY_DATA_TIMEOUT_NO_ACK	Verifies mandatory boot mode with data timeout with acknowledge disabled
VTB_BOOT_TEST_MANDATORY_NO_ACK	Verifies mandatory boot mode with acknowledge disabled
VTB_BOOT_TEST_ALTERNATE_WITH_ACK	Verifies alternate boot mode with acknowledge enabled

Table B-1 Example Data Transfer Tests for SD and MMC Cards

Data Transfer Test	Description
VTB_BOOT_TEST_ALTERNATE_ACK_ENDBIT_ERR	Verifies alternate boot mode with acknowledge endbit error
VTB_BOOT_TEST_ALTERNATE_ACK_PATTERN_ERR	Verifies alternate boot mode with acknowledge pattern error
VTB_BOOT_TEST_ALTERNATE_CRC16_ERR_WITH_ACK	Verifies alternate boot mode with data CRC error with acknowledge enabled
VTB_BOOT_TEST_ALTERNATE_CRC16_ERR_NO_ACK	Verifies alternate boot mode with data CRC error with acknowledge disabled
VTB_BOOT_TEST_ALTERNATE_DATA_TIMEOUT_WITH_ACK	Verifies alternate boot mode with data timeout with acknowledge enabled
VTB_BOOT_TEST_ALTERNATE_DATA_TIMEOUT_NO_ACK	Verifies alternate boot mode with data timeout with acknowledge disabled
VTB_BOOT_TEST_ALTERNATE_NO_ACK	Verifies alternate boot mode with acknowledge disabled
VTB_SBW_DDR	Verifies single-block write in DDR mode
VTB_SBR_DDR	Verifies single-block read in DDR mode
VTB_PREDEFINED_MBW_DDR	Verifies multiple-block write in DDR mode for a predefined byte count
VTB_PREDEFINED_MBR_DDR	Verifies multiple-block read in DDR mode for a predefined byte count
VTB_VOLTAGE_SWITCH_END_ERR	Verifies the voltage switching sequence end error scenario for SD3.0
VTB_VOLTAGE_SWITCH_START_ERR	Verifies the voltage switching sequence start error scenario for SD3.0

Using the provided generic transfer task, any type of single and multiple-block transfers can be performed to any type of card – MMC, SD_MEM, SD_IO, SD_COMBO – on any port following the example test cases

B.4 Using VTB

This section explains how to operate the VTB.



VTB does not support eSDIO cards.

The VTB implements a method to take in the design configuration – the various design parameters – and the simulation configuration – the different cards types of interest and the slots where they sit – to run a simulation.

The design parameters are in the `DWC_mobile_storage_deriverd_params.v` file, which is included as part of the design compile. All defined design parameters are available in a simulation and are used to control the simulation in the VTB.

The simulation parameters—as well as the different card types and the slots where they sit—are selected using the core Consultant “Testbench Parameter Setup” activity.

B.4.1 Directory Structure and Files

Table B-2 lists the files in the directory structure.

Table B-2 Directory Structure and Files

Source File	Description
block1 Source Files	
<code>ahb_vip_tasks.v</code>	AHB VIP tasks
<code>block1.v</code>	<code>block1.v</code>
<code>ahb_vip_init.v</code>	AMBA VIP initialization
<code>intr_serv_routine.v</code>	Interrupt check routine
<code>master_driver.v</code>	Application read and write tasks built on top of AHB Master VIP commands
<code>ctrl_reg_prgm.v</code>	Host Controller Register programming
block2 Source Files	
<code>block2.v</code>	Mobile Storage Block(block2) top level instantiations
<code>mmc_vip_tasks.v</code>	Tasks to initialize the cards connected
<code>test_definitions.v</code>	Card VIP tasks definition file
<code>testdefn.v</code>	Executes the defined tasks based on selected test case
VTB Source Files	
<code>gpio_top.v</code>	gpio top file
<code>mmc_vtb_top.v</code>	testbench top file
<code>testcontrol.v</code>	List of test names
<code>DWC_mobile_storage_2clk_dssram_tst.v</code>	FIFO RAM module used in external FIFO configuration
VTB Simulation Output Files	
<code>VCS.log</code>	Output log file generated from simulation
<code>mmc.vpd</code>	.vpd dump file
<code>simv, simv.daidir, vcs.key</code>	VCS residue files

B.4.2 Tool and Setup Requirements

Before running a simulation, \$DESIGNWARE_HOME must be set as follows:

```
setenv DESIGNWARE_HOME DWC_mobile_storage_base_installation_dirctory
```

B.5 Running VCS Simulations

All testcases are included in Block1 with an include file called testcases.v. However, a testcase is turned on only if it is defined. The testcases to be run are defined using parameters and are available in the file called testcontrol.v, as shown below.

```
parameter VTB_SBW = 1
parameter VTB_SBR = 1
```

VTB_SBR and VTB_SBW are the testcases. Setting a parameter to 1 enables the testcase to run; setting it to 0 disables the testcase from running. The user can select multiple testcases by enabling the tests accordingly.

The corresponding test definition for a testcase is included in Block2. All test definitions are included with an include file called testdefn.v. Again, a test definition is turned on only if its corresponding testcase is defined in the testcontrol.v file, which is shown below.

```
*****TRANSFER testcases*****
parameter TRANSFER = 1
parameter VTB_SBW = 1
parameter VTB_SBR = 1
parameter VTB_SBW_RANDOM = 1
*****BOOT testcases*****
parameter BOOT = 1
parameter VTB_BOOT_TEST_MANDATORY_WITH_ACK = 1
parameter VTB_BOOT_TEST_MANDATORY_ACK_ENDBIT_ERR = 0
parameter VTB_BOOT_TEST_MANDATORY_ACK_PATTERN_ERR = 0
```

The following restrictions apply while selecting testcases:

- ❖ To select a data transfer testcase, both the TRANSFER parameter and the *TEST_NAME* parameter must be set to 1.
- ❖ More than one data transfer testcase can be selected at a time
- ❖ To select a boot testcase, both the BOOT parameter and the *TEST_NAME* parameter must be set to 1.

Simulations are run in the directory *workspace/split_VTB/VTB* as shown below:

```
runsim -simChoice VCS
```



Note The other simulator choices are NC_Verilog and MTI_Verilog.

While a simulation is in progress, the card VIP model prints messages to the console highlighting the state changes and data transactions occurring on the card. These messages can be monitored to follow the progress of the data transfer test being run on the card.

B.6 Working with Testcases

The following subsections describe how to run and debug testcases.

B.6.1 Configuring the Testbench

Using coreConsultant, configure the testbench:

1. Select the cards that need to be tested from the Testbench Parameter Setup activity:
 - ◆ Testbench Parameter Setup – Set card type for each slot in design



VTB does not support eSDIO cards.

2. Select the VIP and VMT versions to be used from the Setup and Run Simulations activity:
 - ◆ Setup and Run Simulations:
 - ◇ VIP – Specify VMT and AMBA VIP versions
 - ◇ Execution options – Select “Do not launch Simulations” check box
 - ◆ Click Apply.



This step is required when the VTB is configured for the first time, but not subsequently to run testcases.

B.6.2 Running a Testcase



You should read the instructions given before selecting the testcases. Certain combinations may lead to failure/hanging of the testcase, so make sure to select a valid combination of the options.

To run a testcase in VTB, run the test shown below (according to [“Running VCS Simulations”](#)):

```
runsim -simChoice VCS
```



The other simulator choices are NC_Verilog and MTI_Verilog.

If a single test is selected, the simulation runs the test for all the selected cards. If multiple tests are selected, the simulation runs all the tests for all the selected cards.

A file called VCS.log is created at the end of the run. A combined single log file VCS.log is created for all the testcases when the simulation completes. The user can look for messages in the log file while debugging the test.

For more runtime options, use the following command:

```
runsim -help
```

B.6.3 Changing Selected Card Types

Use the following procedure to change the selected card type.

1. Select the cards using coreConsultant:
 - ◆ Testbench Parameter Setup — set the type of card for each socket in the design.
2. In the splilt_VTB/VTB/ directory:
3. Select the tests to be run according to [“Running VCS Simulations”](#) on page 316.
4. Run the test using the following command:

```
runsim -simChoice VCS
```

B.6.4 Debugging a Testcase

The following example show debug messages for a testcase named VTB_OE_MBW_BYTCNT_ZERO.

Debug Message	Description
1. Testcase VTB_OE_MBW_BYTCNT_ZERO Started	Comes at the beginning of every test case.
2. BEGIN : OpenEnded Multiple Block Write Transfer with Bytcnt Zero at [7356909000]	Gives a description of testcase; comes after first message.
3. Card in Port 1 is MMC	Gives connection details (card type to port number); comes after “BEGIN” message.
4. PASS : OpenEnded Multiple Block Write Transfer with Bytcnt Zero at [7742049000]	Displayed if the test passes for the selected card.
5. FAIL: OpenEnded Multiple Block Write Transfer with Bytcnt Zero at [7742049000]	Displayed if the test fails for the selected card.
6. Transfer has not taken place	Displayed if the transfer has not taken place because the transfer is not valid for the selected card.
7. Testcase VTB_OE_MBW_BYTCNT_ZERO Ended	Displayed at the end of the testcase.



Note

- As the same test runs for all the cards, one of messages 4, 5, or 6 will be displayed in the log file. If multiple cards are connected, you will see messages 2 through 6 displayed for all the cards because the same testcase is run for all of them.
- If a test fails, as a part of the debug procedure it should be ensured that the VTB test matches selected configuration parameters; it is explained with an example.

C

Database Description

This appendix lists the deliverables and other reference files that are generated from the coreConsultant flow.

C.1 Design/HDL Files

The following sections describe the design and HDL files that are produced by coreConsultant when configuring and verifying a DesignWare AMBA component.

C.1.1 RTL-Level Files

The following table describes the RTL files that are generated by the Create RTL activity of the coreConsultant GUI. They are encrypted except where otherwise noted.

 **Note**

Any Synopsys synthesis tool or simulator can read encrypted RTL files.

Table C-1 RTL-Level Files

Files	Encrypted?	Purpose
<code>./src/component_params.v</code>	No	Includes definitions and values of all configuration parameters that you have specified for the component.
<code>./src/component.lst</code>	No	Lists the order in which the RTL files should be read into tools, such as simulators or dc_shell. For example, use the following option to read the design into VCS: <code>vcs -f component.lst</code>

C.1.2 Simulation Model Files

The following table includes the simulation model files generated for the component during the Generate GTECH Simulation activity in coreConsultant. These files are needed when you are using a non-Synopsys simulator (when you can not use the encrypted RTL).

Table C-2 Simulation Model Files

Files	Encrypted?	Purpose
<code>./gtech/final/db/component.v</code>	No	Simulation model of the component for use with non-Synopsys simulators. A technology-independent, gate-level netlist. VHDL and Verilog versions are generated. When you use this simulation model in your simulation, you must include the DesignWare libraries by using the following options in your simulator invocation: -y \${SYNOPSYS}/packages/gtech/src_ver -y \${SYNOPSYS}/dw/sim_ver

C.2 Register Map Files

These files only pertain to DW_ahb and DW_apb slaves, basically components that have a programming interface. The DesignWare AMBA components that do not have register map files are the DW_apb, DW_ahb_icm, and DW_ahb_h2h components. These files include address definitions (memory map) for the component. The following table includes a description of the C and Verilog header files generated for components with programming interfaces.

Table C-3 Header Files

Files	Encrypted?	Purpose
<code>./c_headers/component_defs.h</code>	No	For use when programming the component in a C environment.
<code>./verilog_headers/component_defs.v</code>	No	For use when programming the component in a Verilog environment.

C.3 Synthesis Files

The following table includes the files that are generated after the Create Gate-Level Netlist activity in coreConsultant is performed on a component.

Table C-4 Synthesis Files

Files	Encrypted?	Purpose
<code>./syn/final/db/component.db</code>	Binary format	Synopsys .db files (gate level) that can be read into dc_shell for further synthesis, if desired.
<code>./syn/final/db/component.v</code>	No	Gate-level netlist that is mapped to technology libraries that you specify.
<code>./syn/constrain/script/*.*</code>	No	Constraint files for the components.
<code>./syn/final/report/*.*</code>	No	Synthesis result files.

C.4 Verification Reference Files

The files described in the following table include information pertaining to the component's operation so that you can verify installation and configuration of the component has been successful. These files are not for re-use during system-level verification.

Table C-5 **Verification Reference Files**

Files	Encrypted?	Purpose
./sim/runtest	No	Perl script that runs the coreConsultant Verify Component activity from the command line.
./sim/runtest.log	No	The overall result of simulation, including pass/fail results.
./sim/test_ <i>testname</i> /test.result	No	Pass/fail of individual test.
./sim/test_ <i>testname</i> /test.log	No	Log file for individual test.

For more information about performing verification on your component, see the chapter titled [“Verification”](#) on page 235.

D

Area, Speed, Power, and Quality Matrix

All the synthesis results in this appendix are based on an industry-standard 65nm library. The area is calculated by dividing the total cell area by the area of the smallest NAND gate in the library. [Table D-1](#) lists the areas for some synthesis configurations.

Table D-1 Synthesis – Area For a Few Configurations

Configuration	Tool	Speed (AHB/Card) MHz	Area (Nand)
CARD_TYPE=1 NUM_CARDS=3 H_BUS_TYPE=0 H_DATA_WIDTH=16 H_ADDR_WIDTH=10 INTERNAL_DMAC=0 DMA_INTERFACE=0 GE_DMA_DATA_WIDTH=32 FIFO_DEPTH=16	Design Compiler	100/200	33K
FIFO_RAM_INSIDE=1 NUM_CLK_DIVIDERS=3 UID_REG=0x0 SET_CLK_FALSE_PATH=1 HCLK_PERIOD=10 CCLKIN_PERIOD=5 AREA_OPTIMIZED=0 IMPLEMENT_SCAN_MUX=0			
CARD_TYPE=1 NUM_CARDS=8 H_BUS_TYPE=1 H_DATA_WIDTH=32 H_ADDR_WIDTH=10 INTERNAL_DMAC=0 DMA_INTERFACE=1 GE_DMA_DATA_WIDTH=32 FIFO_DEPTH=64	Design Compiler	100/200	32K
FIFO_RAM_INSIDE=0 NUM_CLK_DIVIDERS=2 UID_REG=0x40 SET_CLK_FALSE_PATH=0 HCLK_PERIOD=10 CCLKIN_PERIOD=5 AREA_OPTIMIZED=0 IMPLEMENT_SCAN_MUX=1			

Table D-1 Synthesis – Area For a Few Configurations

Configuration	Tool	Speed (AHB/Card) MHz	Area (Nand)
CARD_TYPE=1 NUM_CARDS=7 H_BUS_TYPE=0 H_DATA_WIDTH=32 H_ADDR_WIDTH=15 INTERNAL_DMAC=1 DMA_INTERFACE=0 GE_DMA_DATA_WIDTH=32 FIFO_DEPTH=4096	FIFO_RAM_INSIDE=0 NUM_CLK_DIVIDERS=1 UID_REG=0x0 SET_CLK_FALSE_PATH=0 HCLK_PERIOD=10 CCLKIN_PERIOD=5 AREA_OPTIMIZED=0 IMPLEMENT_SCAN_MUX=0	Design Compiler	100/200 38K
CARD_TYPE=1 NUM_CARDS=1 H_BUS_TYPE=1 H_DATA_WIDTH=32 H_ADDR_WIDTH=20 INTERNAL_DMAC=1 DMA_INTERFACE=0 GE_DMA_DATA_WIDTH=32 FIFO_DEPTH=512	FIFO_RAM_INSIDE=0 NUM_CLK_DIVIDERS=3 UID_REG=0x0 SET_CLK_FALSE_PATH=1 HCLK_PERIOD=10 CCLKIN_PERIOD=5 AREA_OPTIMIZED=0 IMPLEMENT_SCAN_MUX=1	Design Compiler	100/200 32K

Table D-2 lists the speeds and corresponding areas for some synthesis configurations.

Table D-2 Synthesis – Speed and Corresponding Area

Configuration	Tool	Speed (AHB/Card) MHz	Area (Nand)
CARD_TYPE=1 NUM_CARDS=7 H_BUS_TYPE=0 H_DATA_WIDTH=32 H_ADDR_WIDTH=15 INTERNAL_DMAC=1 DMA_INTERFACE=0 GE_DMA_DATA_WIDTH=32	FIFO_DEPTH=4096 FIFO_RAM_INSIDE=0 NUM_CLK_DIVIDERS=1 UID_REG=0x0 SET_CLK_FALSE_PATH=0 AREA_OPTIMIZED=0 IMPLEMENT_SCAN_MUX=0	Design Compiler	166/200 40K
		Design Compiler	200/50 38K
		Design Compiler	250/200 41K

**Note**

SD_MMC_CEATA Protocol Max speed is 52MHz. The 100MHz was used for synthesis timing analysis only in order to ensure SD_MMC_CEATA works at full-speed (that is, 52Mhz) in FPGA systems.

Table D-3 lists the area differences between different scan-ready and clock gating scenarios.

Table D-3 Area Difference Due to Scan Ready and Clock Gating

Configuration			Area (Nand)
CARD_TYPE=1 NUM_CARDS=3 H_BUS_TYPE=0 H_DATA_WIDTH=16 H_ADDR_WIDTH=10 INTERNAL_DMAC=0 DMA_INTERFACE=0 GE_DMA_DATA_WIDTH=32 FIFO_DEPTH=16	FIFO_RAM_INSIDE=1 NUM_CLK_DIVIDERS=3 UID_REG=0x0 SET_CLK_FALSE_PATH=1 HCLK_PERIOD=6 CCLKIN_PERIOD=5 AREA_OPTIMIZED=0 IMPLEMENT_SCAN_MUX=0	No Clock-Gating, No Scan-Ready	33K
		Clock Gating, No Scan-Ready	29K
		No Clock Gating, Scan-Ready	38K
		Clock Gating, Scan-Ready	34K
CARD_TYPE=1 NUM_CARDS=8 H_BUS_TYPE=1 H_DATA_WIDTH=32 H_ADDR_WIDTH=10 INTERNAL_DMAC=0 DMA_INTERFACE=1 GE_DMA_DATA_WIDTH=32 FIFO_DEPTH=64	FIFO_RAM_INSIDE=0 NUM_CLK_DIVIDERS=2 UID_REG=0x40 SET_CLK_FALSE_PATH=0 HCLK_PERIOD=6 CCLKIN_PERIOD=5 AREA_OPTIMIZED=0 IMPLEMENT_SCAN_MUX=1	No Clock-Gating, No Scan-Ready	34K
		Clock Gating, No Scan-Ready	30K
		No Clock Gating, Scan-Ready	39K
		Clock Gating, Scan-Ready	34K
CARD_TYPE=1 NUM_CARDS=7 H_BUS_TYPE=0 H_DATA_WIDTH=32 H_ADDR_WIDTH=15 INTERNAL_DMAC=1 DMA_INTERFACE=0 GE_DMA_DATA_WIDTH=32 FIFO_DEPTH=4096	FIFO_RAM_INSIDE=0 NUM_CLK_DIVIDERS=1 UID_REG=0x0 SET_CLK_FALSE_PATH=0 HCLK_PERIOD=6 CCLKIN_PERIOD=5 AREA_OPTIMIZED=0 IMPLEMENT_SCAN_MUX=0	No Clock-Gating, No Scan-Ready	40K
		Clock Gating, No Scan-Ready	35K
		No Clock Gating, Scan-Ready	45K
		Clock Gating, Scan-Ready	40K
CARD_TYPE=1 NUM_CARDS=1 H_BUS_TYPE=1 H_DATA_WIDTH=32 H_ADDR_WIDTH=20 INTERNAL_DMAC=1 DMA_INTERFACE=0 GE_DMA_DATA_WIDTH=32 FIFO_DEPTH=512	FIFO_RAM_INSIDE=0 NUM_CLK_DIVIDERS=3 UID_REG=0x0 SET_CLK_FALSE_PATH=1 HCLK_PERIOD=6 CCLKIN_PERIOD=5 AREA_OPTIMIZED=0 IMPLEMENT_SCAN_MUX=1	No Clock-Gating, No Scan-Ready	32K
		Clock Gating, No Scan-Ready	29K
		No Clock Gating, Scan-Ready	37K
		Clock Gating, Scan-Ready	33K



Scan Coverage = 99.5%

Table D-4 summarizes the Power Compiler clock gating.

Table D-4 Power Compiler Clock-Gating Summary

Configuration			
CARD_TYPE=1 NUM_CARDS=3 H_BUS_TYPE=0 H_DATA_WIDTH=16 H_ADDR_WIDTH=10 INTERNAL_DMAC=0 DMA_INTERFACE=0 GE_DMA_DATA_WIDTH=32 FIFO_DEPTH=16	FIFO_RAM_INSIDE=1 NUM_CLK_DIVIDERS=3 UID_REG=0x0 SET_CLK_FALSE_PATH=1 HCLK_PERIOD=6 CCLKIN_PERIOD=5 AREA_OPTIMIZED=0 IMPLEMENT_SCAN_MUX=0	Number of clock gating elements	182
		Number of gated registers	2922 (83.87%)
		Number of ungated registers	562 (16.13%)
		Total number of registers	3484
CARD_TYPE=1 NUM_CARDS=8 H_BUS_TYPE=1 H_DATA_WIDTH=32 H_ADDR_WIDTH=10 INTERNAL_DMAC=0 DMA_INTERFACE=1 GE_DMA_DATA_WIDTH=32 FIFO_DEPTH=64	FIFO_RAM_INSIDE=0 NUM_CLK_DIVIDERS=2 UID_REG=0x40 SET_CLK_FALSE_PATH=0 HCLK_PERIOD=6 CCLKIN_PERIOD=5 AREA_OPTIMIZED=0 IMPLEMENT_SCAN_MUX=1	Number of clock gating elements	157
		Number of gated registers	2225 (72.57%)
		Number of ungated registers	841 (27.43%)
		Total number of registers	3066
CARD_TYPE=1 NUM_CARDS=7 H_BUS_TYPE=0 H_DATA_WIDTH=32 H_ADDR_WIDTH=15 INTERNAL_DMAC=1 DMA_INTERFACE=0 GE_DMA_DATA_WIDTH=32 FIFO_DEPTH=4096	FIFO_RAM_INSIDE=0 NUM_CLK_DIVIDERS=1 UID_REG=0x0 SET_CLK_FALSE_PATH=0 HCLK_PERIOD=6 CCLKIN_PERIOD=5 AREA_OPTIMIZED=0 IMPLEMENT_SCAN_MUX=0	Number of clock gating elements	173
		Number of gated registers	2582 (72.79%)
		Number of ungated registers	965 (27.21%)
		Total number of registers	3547
CARD_TYPE=1 NUM_CARDS=1 H_BUS_TYPE=1 H_DATA_WIDTH=32 H_ADDR_WIDTH=20 INTERNAL_DMAC=1 DMA_INTERFACE=0 GE_DMA_DATA_WIDTH=32 FIFO_DEPTH=512	FIFO_RAM_INSIDE=0 NUM_CLK_DIVIDERS=3 UID_REG=0x0 SET_CLK_FALSE_PATH=1 HCLK_PERIOD=6 CCLKIN_PERIOD=5 AREA_OPTIMIZED=0 IMPLEMENT_SCAN_MUX=1	Number of clock gating elements	162
		Number of gated registers	2356 (78.17%)
		Number of ungated registers	658 (21.83%)
		Total number of registers	3014

Table D-5 lists average and peak power data.

Table D-5 Synthesis – Speed and Corresponding Area

Configuration		Power	
CARD_TYPE=1	SET_CLK_FALSE_PATH=1	Average Power	Switch Power = 2.12e-04
NUM_CARDS=3	HCLK_PERIOD=6		Internal Power = 3.48e-03
H_BUS_TYPE=1	CCLKIN_PERIOD=5		Leak Power = 8.99e-06
H_DATA_WIDTH=32	AREA_OPTIMIZED=0		Total Power = 3.70e-03
H_ADDR_WIDTH=20	IMPLEMENT_SCAN_MUX=0		
INTERNAL_DMACH=1	ENABLE_LONG_REGRESSION=0	Peak Power	Peak power = 9.32e-03
DMA_INTERFACE=0			Peak Time = 84645-84650
GE_DMA_DATA_WIDTH=32			Glitch power= 1.78e-05
FIFO_DEPTH=1024			X-Tran Power = 1.04e-08
FIFO_RAM_INSIDE=1			
NUM_CLK_DIVIDERS=3			
UID_REG=0x0			
SET_CLK_FALSE_PATH=1			

Table D-6 lists the code coverage.

Table D-6 Code Coverage

Type of Coverage	Percent Coverage
Line Coverage	99.79%
Conditional Coverage	95.6%
Toggle Coverage	93%
FSM State	98.3%

D.0.0.1 Inter-operability Test Status

- ❖ SD Mem
 - ◆ SanDisk 64MB SD, 256MB SD, 256MB MiniSD
 - ◆ Panasonic 128MB SD
 - ◆ PNY 256MB SD
 - ◆ Memorex 32MB SD
 - ◆ SimpleTech 64MB SD
- ❖ MMC
 - ◆ SanDisk 64MB MMC
 - ◆ SimpleTech 128MB MMC
 - ◆ Lexar 32MB MMC
- ❖ SDIO (Verified only Basic CMD5, CMD52, and CMD53 I/O commands)
 - ◆ PALM Bluetooth
 - ◆ Toshiba Bluetooth

- ❖ HSMMC (Verified 1-bit, 4-bit and 8-bit modes; verified CMD6, CMD8, CMD14, CMD19, CMD17, CMD18, CMD24, and CMD25 commands)
 - ◆ Pretec
 - ◆ Skymedi
- ❖ CEATA
 - ◆ Hitachi Microdrive 3K8 hard drive

Index

A

ACT compliance test [235](#), [244](#)

AMBA

bus interface unit (BIU) [19](#), [45](#)

AMBA certification test (ACT) [235](#), [244](#)

Architecture

DWC_Mobile_Storage, general discussion [43](#)

B

Batch jobs, executing [246](#)

BFMs

generic DMA [261](#)

BIU. See Bus interface unit

BLKSIZ register [135](#), [146](#)

Block size register [146](#)

BMOD register [136](#)

BUFADDR register [137](#)

Bus interface unit (BIU) [45](#)

Bus interface unit BIU) [19](#)

Buses

data, bi-direction [268](#)

BYTCNT register [135](#), [146](#)

Byte count register [146](#)

C

C header files [320](#)

card detect register [161](#)

Card interface unit (CIU) [19](#), [83](#)

Card type register [145](#)

Card-detect mechanism [278](#)

CDETECT register [136](#), [161](#)

Certification test, AMBA [235](#), [244](#)

certification, AMBA [235](#), [244](#)

CIU. See Card interface unit

CLKDIV register [134](#), [142](#)

CLKENA register [135](#), [143](#)

CLKSRC register [134](#), [143](#)

Clock divider register [142](#)

Clock enable register [143](#)

CMD register [135](#), [149](#)

CMDARG register [135](#), [148](#)

Command argument register [148](#)

Command register [149](#)

Commands

ConnectNewCard [247](#)

disableModelMsg [241](#)

driveReset [241](#)

enableModelMsg [242](#)

initialize [242](#)

issueXactions [248](#)

setBaseAddress [248](#)

setRcgMsgLevel [247](#)

setTopConfig [243](#)

Connect

creating gate-level netlist [30](#)

ConnectNewCard command [247](#)

Constrained random testing [246](#)

Control

register [138](#)

Control registers, summary [134](#)

coreConsultant

creating a batch script [41](#)

formal verification [41](#)

starting [26](#)

Creating

batch script of workspace [41](#)

gate-level netlist in Connect [30](#)

CTRL register [138](#)

CTYPE register [135](#), [145](#)

D

Data buses

bi-directional [268](#)

DATA register [137](#)

DBADDR register [137](#)

dc_shell [30](#)

DEBNCE register [136](#), [164](#)

Debounce count register [164](#)

Design for Test, synthesis options [31](#)

Design hierarchy [43](#)

Designs

integration [265](#)

Directories

simulations output file [245](#)

disableModelMsg command [241](#)

DMA

BFM, generic [261](#)

driveReset command [241](#)

DSCADDR register [137](#)

DW_ahb

parameters [115](#)

DW_apb_rap

I/O description [119](#)

I/O signal description [121](#)

interfacing to [119](#)

DWC_Mobile_Storage

architecture, general discussion [43](#)

definition [13](#)

design hierarchy [43](#)

instantiation [265](#)

integration in designs [265](#)

interfaces, general discussion [43](#)

testbench [34](#)

DWC_mobile_storage

synthesis

output files [33](#)

E

enableModelMsg command [242](#)

Environment, verification [34](#)

F

FIFO threshold watermark register [159](#)

FIFOTH register [136](#), [159](#)

Files

directories, simulation output [245](#)

simulation output [245](#)

fm_shell [31](#)

Formal verification, in coreConsultant [41](#)

G

General purpose input/output register [162](#)

Generic DMA BFM [261](#)

GPIO register [136](#), [162](#)

Guide, programmer [134](#)

H

Hardware configuration register [166](#)

HCON register [136](#), [166](#)

Hierarchy

design [43](#)

I

I/O signals

description of [121](#)

I/O signals, description of [119](#)

IDINTEN register [137](#)

IDSTS register [137](#)

initialize command [242](#)

Instantiation

DWC_Mobile_Storage [265](#)

Integration

DWC_Mobile_Storage in designs [265](#)

Interfaces

bus interface unit (BIU)

card interface unit (CIU)

DWC_Mobile_Storage, general discussion [43](#)

Interfacing

to DW_apb_rap [119](#)

Interrupt mask register [147](#)

INTMASK register [147](#)

INTMSK register [135](#)

issueXactions command [248](#)

M

Masked interrupt status register [155](#)

Mechanisms

card-detect [278](#)

write-protect [278](#)

MINTSTS register [135](#), [155](#)

O

Output

waveform [245](#)

Output files

GTECH [320](#)header files [320](#)register map [320](#)RTL-level [319](#)simulation [245](#)Simulation model [320](#)synthesis [320](#)verification [321](#)**P**Parameters, of DW_ahb [115](#)PLDMND register [136](#)Power enable register [141](#)Programmer's guide [134](#)

Programming DW_apb_rap

registers [138](#)pt_shell [30](#)PWREN register [141](#)**R**

Random testing

constrained [246](#)raw interrupt status register [156](#)

Registers

BLKSIZ [135, 146](#)block size [146](#)BMOD [136](#)BUFADDR [137](#)BYTCNT [135, 146](#)byte count [146](#)card detect [161](#)card type [145](#)CDETECT [136, 161](#)CLKDIV [134, 142](#)CLKENA [135, 143](#)CLKSRC [134, 143](#)clock divider [142](#)clock enable [143](#)CMD [135, 149](#)CMDARG [135, 148](#)command [149](#)command argument [148](#)control [138](#)control, summary [134](#)CTRL [138](#)CTYPE [135, 145](#)DATA [137](#)DBADDR [137](#)DEBNCE [136, 164](#)debounce count [164](#)DSCADDR [137](#)FIFO threshold watermark [159](#)FIFOTH [136, 159](#)general purpose input/output [162](#)GPIO [136, 162](#)hardware configuration [166](#)HCON [136, 166](#)IDINTEN [137](#)IDSTS [137](#)interrupt mask [147](#)INTMASK [147](#)INTMSK [135](#)masked interrupt status [155](#)MINTSTS [135, 155](#)PLDMND [136](#)power enable [141](#)PWREN [141](#)raw interrupt status [156](#)RESP0 [135, 153](#)RESP1 [135, 153](#)RESP2 [135, 154](#)RESP3 [135, 154](#)response0 [153](#)response1 [153](#)response2 [154](#)response3 [154](#)RINTSTS [135, 156](#)RST_n [136, 168](#)SD clock source [143](#)STATUS [135, 157](#)status [157](#)TBBCNT [136, 164](#)TCBCNT [136, 163](#)

timeout [144](#)
 TMOUT [135](#), [144](#)
 transferred CIU card byte count [163](#)
 transferred hose to BIU-FIFO byte count [164](#)
 UHS [167](#)
 UHS_REG [136](#)
 user ID [165](#)
 USRID [136](#), [165](#)
 VERID [136](#), [165](#)
 version ID [165](#)
 write protect [162](#)
 WRTprt [136](#), [162](#)

Requirements

termination [279](#)

Reset

system [269](#)

RESP0 register [135](#), [153](#)
 RESP1 register [135](#), [153](#)
 RESP2 register [135](#), [154](#)
 RESP3 register [135](#), [154](#)
 response0 register [153](#)
 Response1 register [153](#)
 Response2 register [154](#)
 Response3 register [154](#)
 RINTSTS register [135](#), [156](#)
 RST_n register [136](#), [168](#)
 run.scr [33](#)
 run.scr script [244](#)

S

Scripts

run.scr [244](#)
 simulation [243](#)

SD clock source register [143](#)

setBaseAddress command [248](#)

setRcgMsgLevel command [247](#)

setTopConfig command [243](#)

Signals, description of [119](#)

Simulation

results [41](#)
 status [41](#)

Simulations

output files [245](#)
 scripts [243](#)

Starting

coreConsultant [26](#)

STATUS register [135](#), [157](#)

Status register [157](#)

Synthesis

output files [33](#)

results [33](#)

running from command line [33](#)

target technology, specifying [31](#)

Synthesizing subsystem [30](#)

System reset [269](#)

Systems

topologies [278](#)

T

Target technology, specifying [31](#)

TBBCNT register [136](#), [164](#)

TCBCNT register [136](#), [163](#)

Termination requirement [279](#)

Testbench

DWC_Mobile_Storage [34](#)

Testbenches

structure [237](#)

Testharness [240](#)

Testing

constrained random [246](#)

random, constrained [246](#)

Tests

individual, executing [246](#)

tests [235](#), [244](#)

compliance [235](#), [244](#)

Timeout register [144](#)

TMOUT register [135](#), [144](#)

Topologies

system [278](#)

Transferred CIU card byte count register [163](#)

Transferred host to BIU-FIFO byte count register [164](#)

U

UHS_REG register [136](#), [167](#)

User ID register [165](#)

USRID register [136](#), [165](#)

V

VERID register [136](#), [165](#)

Verification

- environment [34](#)

Verilog header files [320](#)Version ID register [165](#)**W**Waveform output [245](#)Write protect register [162](#)Write-protect mechanism [278](#)WRTPRT register [136](#), [162](#)

